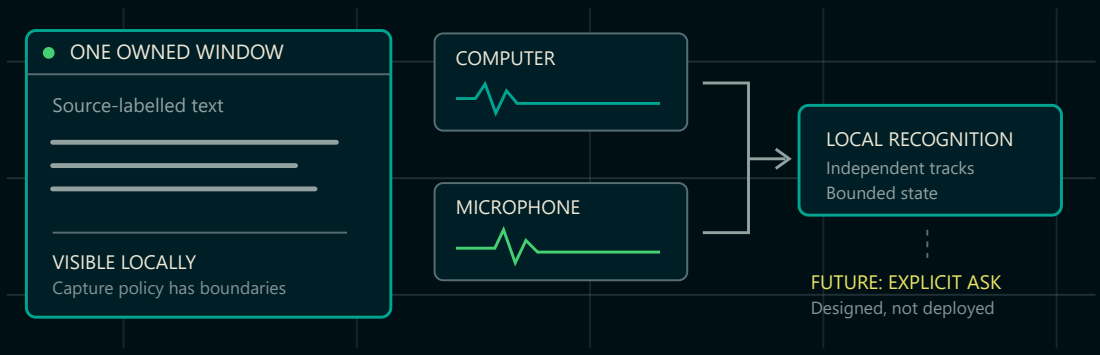
ENGINEERING EDITION 2026-10-08



Phantom Fudge Engineering Handbook

A local-first conversation companion, from Windows surfaces to source-labelled speech

Implemented local preview. Measured evidence is bounded. Azure Ask is planned, not deployed.

> This book distinguishes implemented behavior, measured observations, proposals and unknowns. No universal screen-capture or cursor invisibility is promised. Documentation images use synthetic content.

Contents

00 - Reading Map and Current Reality

01 - Principal Engineer Guide

02 - Zero-to-Hero Learning Path

03 - Product Boundaries and Decisions

04 — Windows Capture, Cursors, and Focus

05 — Desktop UI and Semantic Themes

06 — Notes, Transcripts, and Explicit Storage

7. Dual-input capture: two sources, one owned session

8. Streaming protocol: preserving identity under backpressure

9. Python recognition: shared model, independent histories

10. Native service lifecycle: readiness without accidental capture

11 — State Machines and Failure Containment

12. Performance and Benchmarks

13. Proposed Azure Answer Flow

14. Answer Latency, Identity, and Privacy

15 - Development and Testing

16 - Distribution and Troubleshooting

17 - Privacy and Assurance

18. Extending the System

19. API and configuration reference

20 — Glossary and Repository Map

21. Evidence and Roadmap

Development Journal: The Journey of Phantom Fudge

Source: docs/handbook/00-reading-map.md

Reading map and current reality

> **TL;DR:** Phantom Fudge is a Windows conversation companion with local speech recognition, not an invisible process and not yet an Azure question-answering product. Read this book as an explanation of boundaries: between presentation and capture, samples and text, a request and its completion, and a researched proposal and working software.

Overview: what this book is for

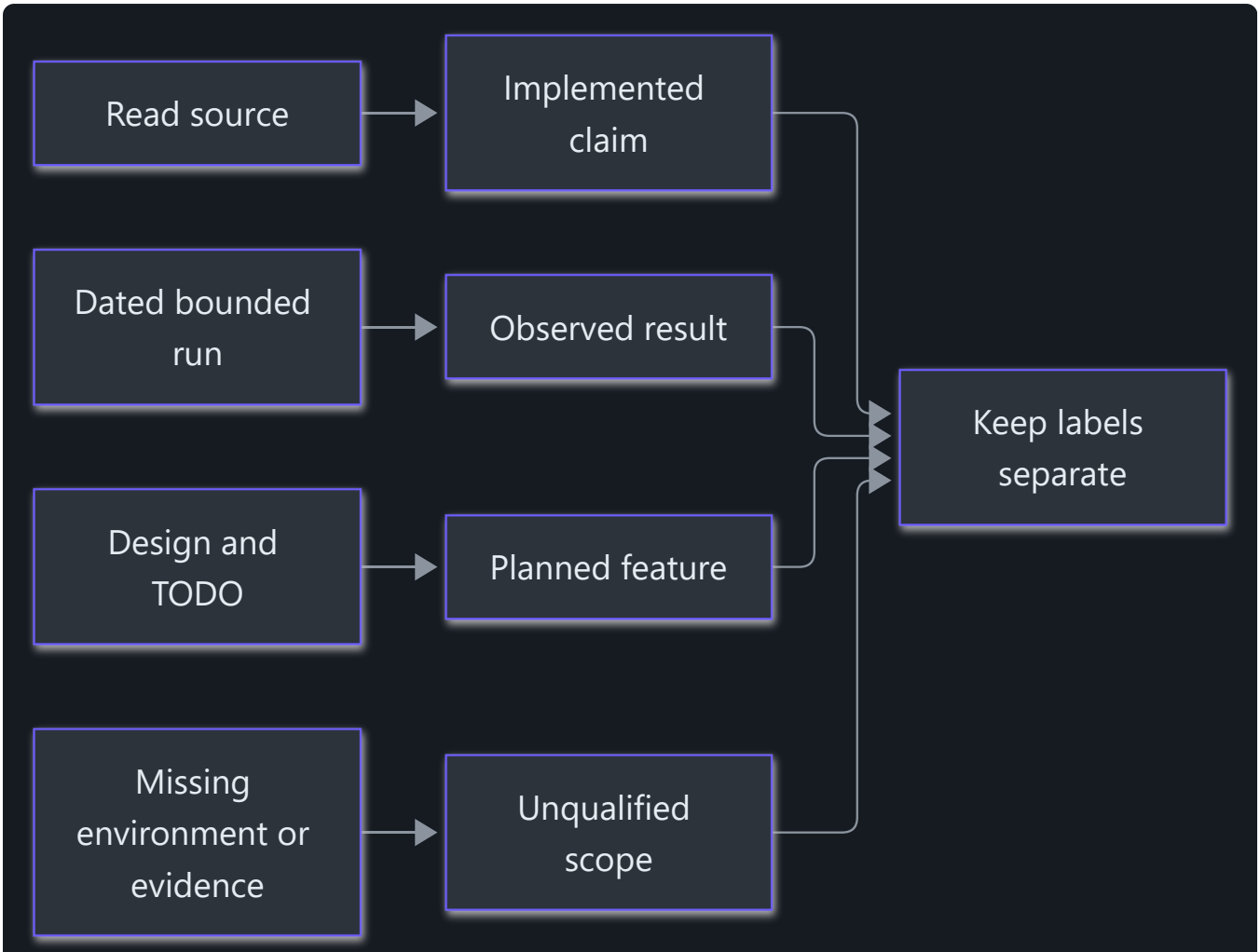
The original problem sounded like a small notepad: show private speaker notes while presenting. That problem expanded into a local transcription-first interface, selectable Computer/Mic/Both inputs, and a researched proposal for explicitly sending a question to an Azure model. The important engineering work is not drawing a rectangle. It is keeping each promise narrow enough to implement and each state transition strong enough to preserve data. The current product contract records those distinct requirements rather than collapsing them into a claim of universal invisibility (`docs\PRODUCT.md:6-29`).

This handbook is intended to support three activities. A new contributor should be able to run and navigate the application without guessing. A senior engineer should be able to review its invariants and tradeoffs. A future book editor should be able to assemble the chapters, figures, screenshots, glossary and development journal into a coherent narrative. The source chapters are Markdown; the generated HTML/PDF is an edition of those sources, not a separate authoritative implementation.

The four labels used throughout

Label	What it means	What it does not mean
Implemented	The cited source contains the behavior	The behavior works on every device/backend
Observed	A dated local run or owner report supports a bounded result	A universal guarantee or independent reproduction of an owner report
Planned	A design/TODO describes future behavior	The feature exists in the executable
Unqualified	Required evidence is missing or incomplete	The implementation is necessarily broken

These distinctions are repository requirements, not editorial niceties. `AGENTS.md` explicitly separates API configuration from captured pixels, technical passes from human acceptance, and research from a qualified release (`AGENTS.md:14-21`). In particular, a successful `SetWindowDisplayAffinity` call is not a recorder certification.



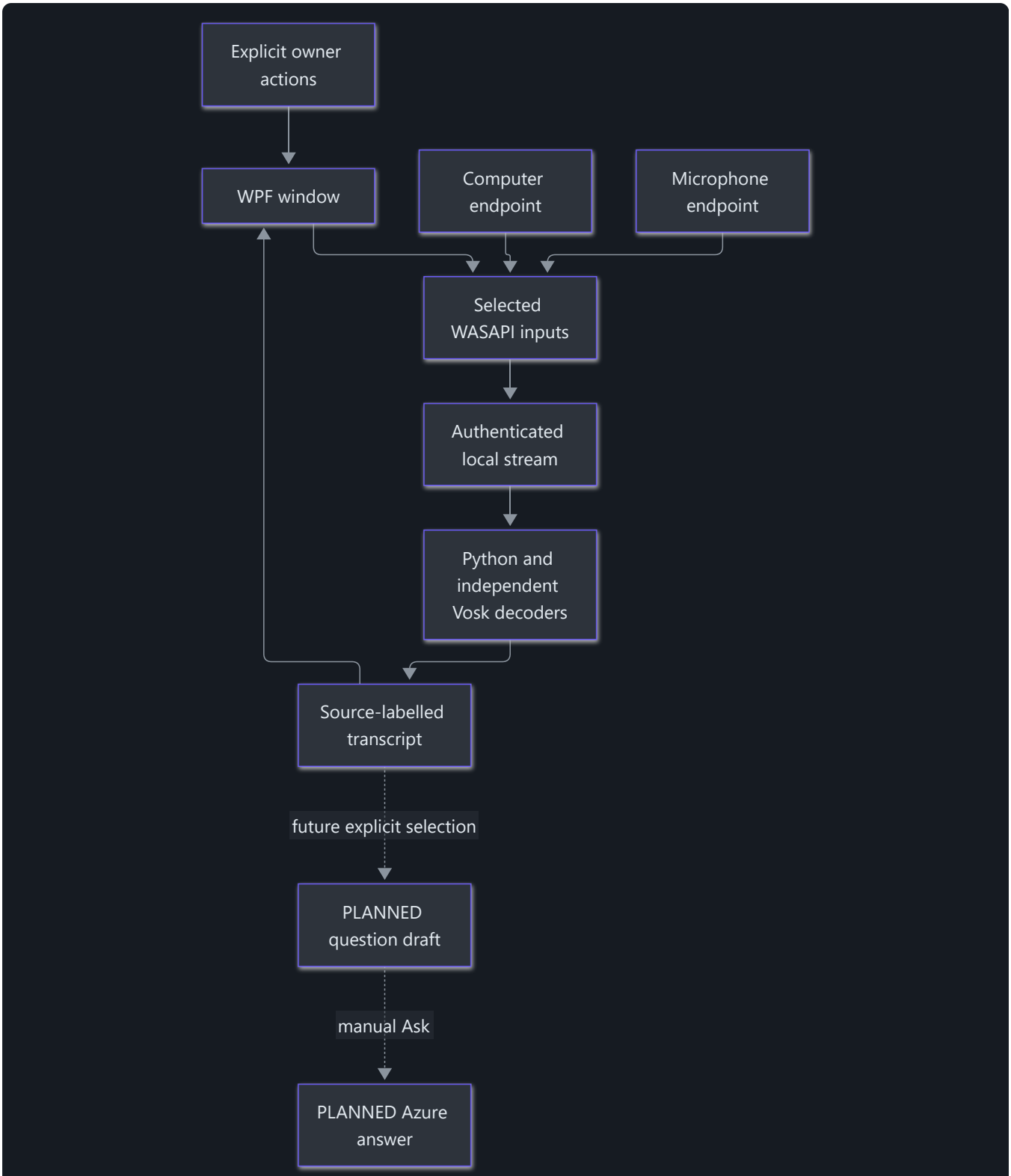
00 - Reading Map and Current Reality - diagram 1. Source-linked; status follows the chapter.

The implemented system in one page

The WPF application owns its native window and starts without an ordinary taskbar/Alt-Tab entry. Its startup path distinguishes synthetic diagnostics from normal single-instance operation, creates `MainWindow`, installs recovery callbacks, and calls `ShowProtected` (`src\PhantomFudge.App\App.xaml.cs:13-58`). The UI is not a browser or a remote webpage.

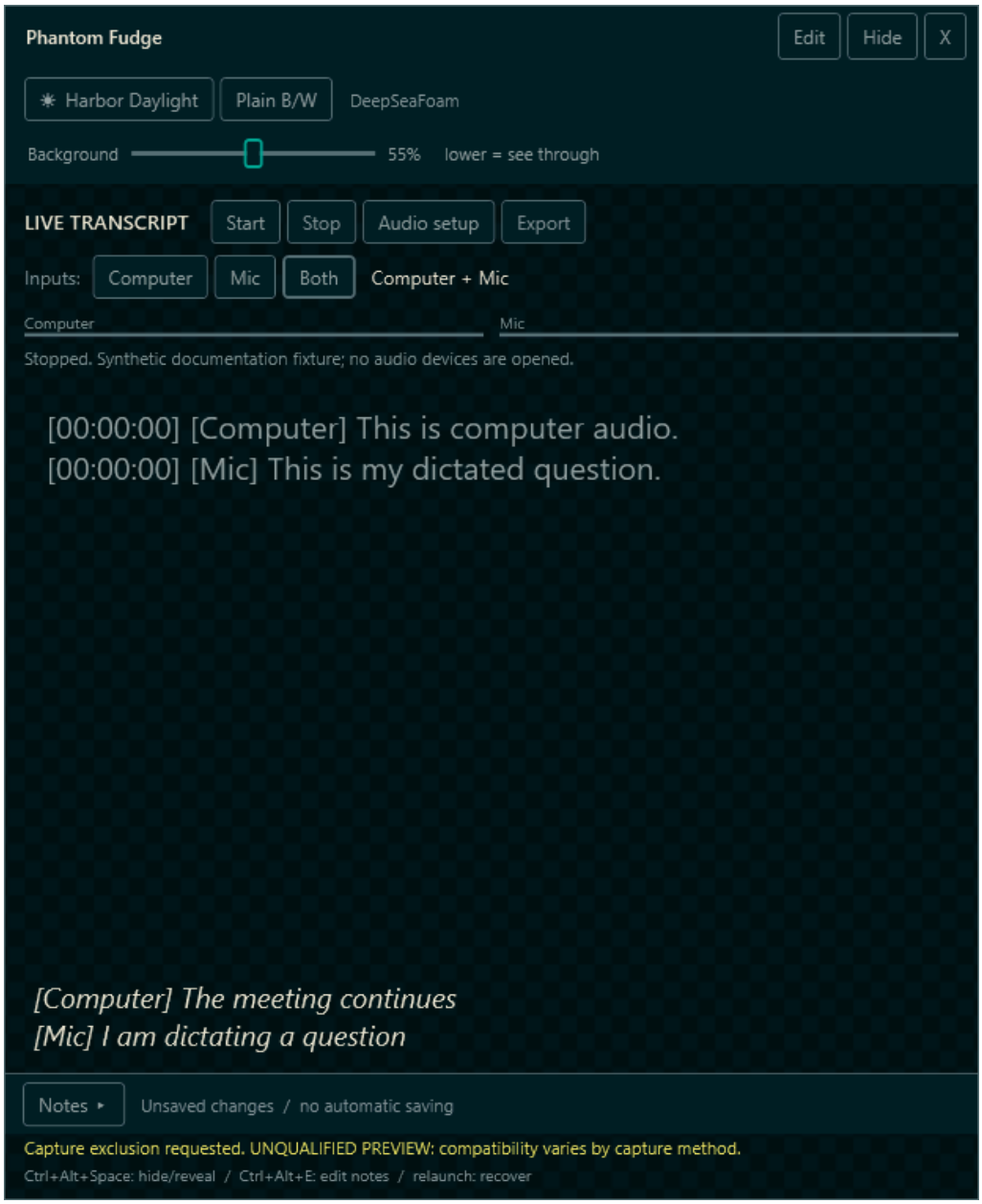
The speech path remains local. The native adapter selects explicit audio endpoints, while the Python server accepts one authenticated session and uses separate decoder state for declared sources. `AudioSource` names Computer and Microphone; those are input identities, not detected people (`src\PhantomFudge.Core\AudioSource.cs:3-30`). `TranscriptBuffer` preserves source-specific partials and labelled final lines (`src\PhantomFudge.Core\TranscriptBuffer.cs:6-97`).

The launch script can start/reuse the local Python service, but that is not permission to start audio capture. `-NotesOnly` avoids service startup, and an existing app is revealed without rebuilding it (`scripts\Run-PhantomFudge.ps1:17-41`). Docker recipes are a possible later deployment of the headless service, not the normal runtime dependency.



00 - Reading Map and Current Reality - diagram 2. Source-linked; status follows the chapter.

Visual baseline



Actual WPF visual-tree render with synthetic notes, transcript and device labels. It is not a recording of a user's desktop or evidence that a meeting recorder excludes the window.

The screenshot fixture is a separate application entry path. It invokes the same window and its rendering diagnostics, but supplies documentation-safe examples; it does not start the speech service or open a capture endpoint (`src\PhantomFudge.App\App.xaml.cs:24-36`). The accompanying screenshot manifest identifies source and image hashes.

Choose a reading path

Reader	Suggested path	Outcome
New to the stack	Zero to Hero, then Setup and Testing	Understand enough C#/Python/audio terminology to run and trace a path
Principal reviewer	Principal Guide, Failure States, Assurance	Identify the invariants and unresolved acceptance gates
Desktop contributor	Windows, UI, Storage	Change a window without losing data or weakening the stated policy
Audio contributor	Capture, Protocol, Recognition	Follow bytes from the selected device to labelled text
Operator	Native Lifecycle, Distribution, Reference	Start, recover, stop and diagnose the local system
AI-feature implementer	Benchmark, Proposed Answer Flow, Latency and Identity	Preserve the boundary between current local ASR and future explicit Azure requests

What to keep beside the book

The TODO index is the living acceptance map. The public journal explains why direction changed. The implementation status records bounded receipts. The book explains mechanisms and reasoning; it should not become a second, inconsistent task database.

The current executable does not contain the planned Ask pipeline. The design explicitly says that structured question drafts, a source commit, Azure identity and a streamed-answer adapter are future work (`docs\AI-ANSWER-ARCHITECTURE.md:1-37`). A diagram with a planned box cannot be used as evidence that a model was deployed.

Reading source citations

A citation such as `src\PhantomFudge.Core\AudioSource.cs:3-14` names a repository file and the line interval inspected for this edition. Prefer the named class/function when navigating a later revision: line numbers move. The build checks path/range validity and source freshness; it cannot prove that prose is semantically correct. Human source review remains necessary.

Checkpoint

Before moving on, explain these differences in your own words:

1. A local ASR service being ready versus a microphone recording.
2. An excluded window versus an undiscoverable process.
3. A labelled Mic track versus speaker identification.
4. A rendered documentation screenshot versus a recipient-side capture test.
5. A researched Azure architecture versus an implemented Ask feature.

If any pair sounds equivalent, revisit the relevant boundary before changing the code.

References

- `README.md:1-50` — current product and qualification summary.
- `docs\PRODUCT.md:6-29` — requirement and impossibility boundary.
- `AGENTS.md:14-43` — engineering distinctions and privacy rules.
- `src\PhantomFudge.App\App.xaml.cs:13-58` — real startup paths.
- `src\PhantomFudge.Core\AudioSource.cs:3-30` — source identity.
- `src\PhantomFudge.Core\TranscriptBuffer.cs:6-97` — partial/final text representation.
- `scripts\Run-PhantomFudge.ps1:17-41` — launch versus audio/service behavior.
- `docs\AI-ANSWER-ARCHITECTURE.md:1-37` — planned, not implemented, answer extension.

Principal engineer guide

> **TL;DR:** The central insight is **identity and completion must survive every asynchronous boundary**. Source identity survives audio transport, request identity will survive future model streams, and completion means drained ownership rather than a convenient boolean. Presentation privacy is a separate, bounded contract; it cannot repair lifecycle mistakes.

Overview: the one idea

It is tempting to describe the application as "audio in, text out, hidden window." That description omits the information required to make it dependable. Computer and Mic can produce interleaved packets; recognizers revise hypotheses; Stop clears visible running state before all final text has drained; a hidden window still owns data; and an excluded HWND remains an ordinary operating-system object. Each boundary needs explicit identity and completion semantics.

The source enum assigns different wire identities to Computer and Microphone (`src\PhantomFudge.Core\AudioSource.cs:3-22`). The transcript store holds separate partials, but final IDs are session-global (`src\PhantomFudge.Core\TranscriptBuffer.cs:9-18`, `src\PhantomFudge.Core\TranscriptBuffer.cs:48-72`). The native adapter separately protects its lifecycle and invalidates queued starts when Stop is requested (`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:71-152,182-189`). These are concrete mechanisms, not an assumed generic event-driven architecture.

The insight in a different language

This Python-like pseudocode explains the C# contract; it is not a production implementation:

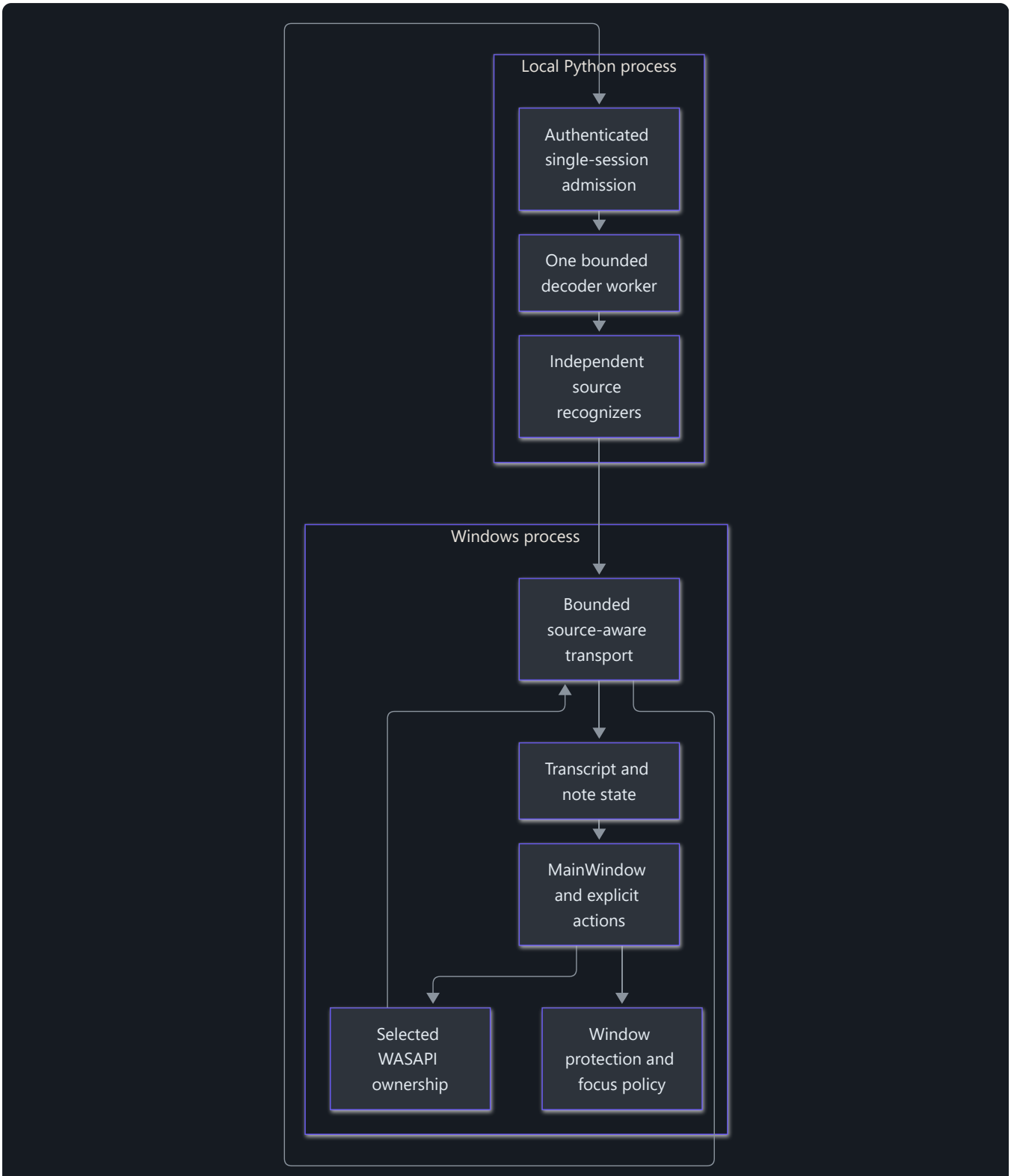
```
def on_packet(source, pcm):
    require(source in selected_sources)
    require(pcm_is_valid(pcm))
    if not queue.try_append(source, pcm):
        fault_entire_session("backpressure")  # do not silently lose a question

async def stop():
    state = STOPPING
    cancel_pending_start()
    await release_selected_inputs()
    await drain_accepted_audio_and_receive_source_finals()
    state = STOPPED

def begin_next_session():
    require(state == STOPPED)                # not merely is_running == False
    preserve_unfinished_text()
    reset_session_global_final_id()
```

The application implements bounded queues and source-aware frames in `TranscriptionSocket`; the server validates each track and finalizes each declared decoder before its stopped event (`src\PhantomFudge.App\Audio\TranscriptionSocket.cs:11-71`, `services\transcription\server.py:235-338`). The pseudocode's "drain" is therefore a chain of ownership transitions, not a delay inserted to make a race less likely.

Architecture at review depth



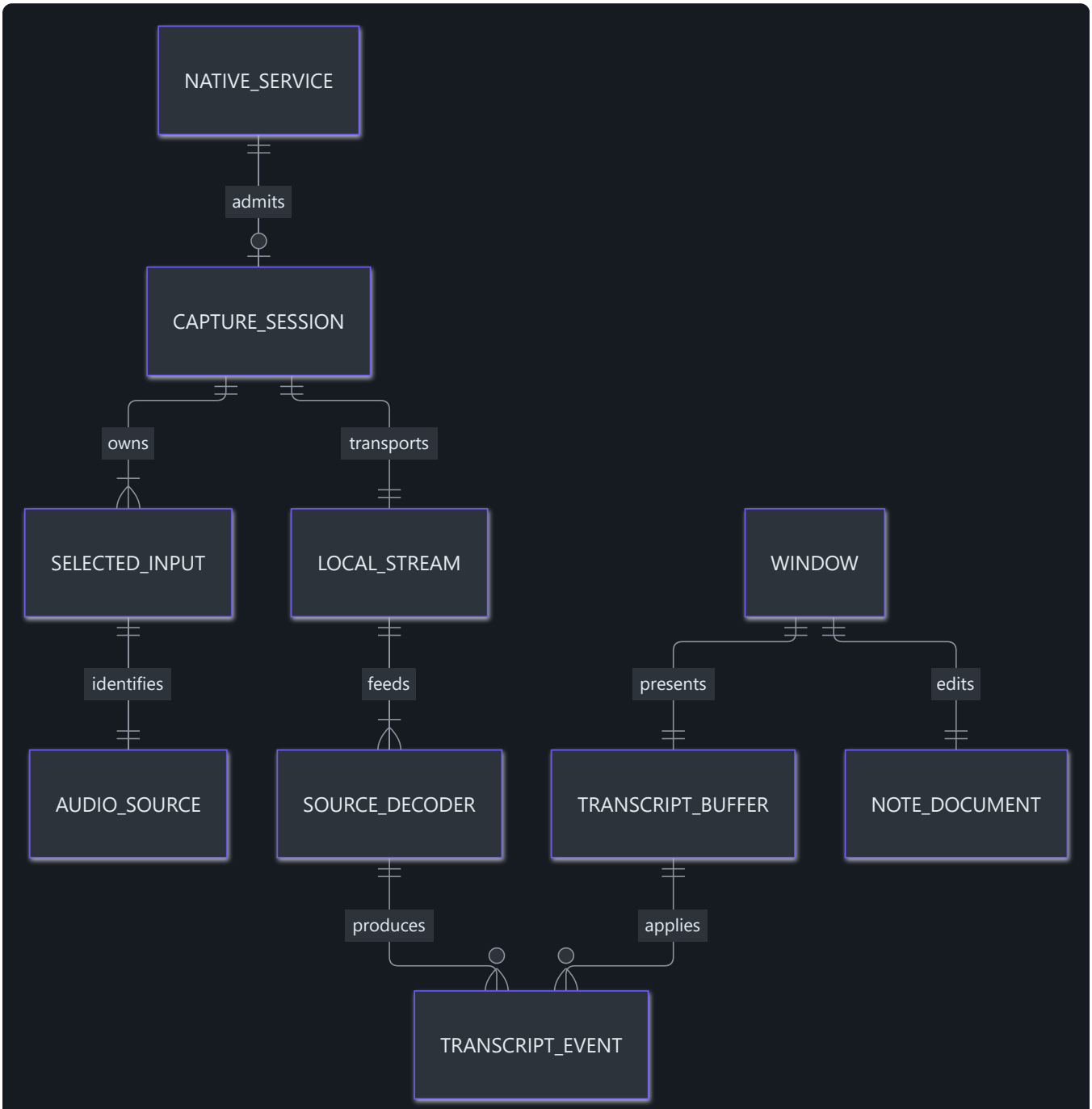
01 - Principal Engineer Guide - diagram 1. Source-linked; status follows the chapter.

The separation is intentionally modest. C# is chosen for the native desktop and input integration; Python hosts the recognizer and its protocol. The model does not become more accurate because Python calls it, nor does C# imply Windows' built-in speech engine. The measured engine comparison is explicit about that

distinction (`docs\SPEECH-BENCHMARK.md:10-35`).

The local process boundary is useful for dependency isolation and future packaging, but it adds protocol, authentication, backpressure and service-lifecycle work. The manager does not identify ownership by a PID alone: it authenticates runtime metadata and checks the saved instance ID before reuse or shutdown (`services\transcription\manage.py:112-144`).

Domain model: implemented concepts, not a database



01 - Principal Engineer Guide - diagram 2. Source-linked; status follows the chapter.

This ER diagram is a conceptual ownership map, not a claim that SQL tables exist. `Input`, `Session`, `TranscriptEvent`, `SourceState` and `TranscriptBuffer` are the source concepts; the note document's identity is currently maintained by `MainWindow` plus `NoteFileStore` rather than a separate persistent entity class. No invented repository abstraction is needed to discuss the boundary (`src\PhantomFudge.App\Audio\TranscriptEvent.cs:1-15`, `src\PhantomFudge.Core\NoteFileStore.cs:6-15`).

The strategic tradeoff table

Decision	Benefit	Cost or boundary
One layered WPF HWND	One main capture-policy surface and native controls	Auxiliary OS/auth/file surfaces still need separate reasoning
Non-activating Present default	Reduces unnecessary foreground changes	Editing deliberately requires a different interaction mode
Arrow cursor policy	Removes the app's I-beam shape cue	Cannot dictate another recorder's cursor stream
Separate Computer/Mic tracks	Source labels and independent recognizer state	More lifecycle/queue accounting; no automatic echo cancellation
Fail on sustained backlog	Avoids silently dropping words while claiming completeness	Session stops instead of degrading invisibly
One local decoder worker	Simple serialized mutable-engine access	Competing sources share CPU; it is not sample-synchronous inference
Explicit local saves	Owner controls persistence	Unsaved content can be lost on a crash; no vault or recovery guarantee
Native Python default	No container prerequisite for a desktop utility	Interpreter/venv/model setup remains an operational responsibility
Proposed direct Azure text call	Smallest initial cloud path for manual Q&A	Identity, data policy, latency and deployment remain unimplemented

These decisions come from the implementation and current requirements, not a universal design pattern checklist. `WindowProtection.Configure` requests affinity on an owned HWND and validates readback when available (`src\PhantomFudge.App\Native\WindowProtection.cs:18-45`). `NoteFileStore` bounds UTF-8 data and checks prior fingerprints around replacement; it is not a filesystem transaction manager or protection against every race (`src\PhantomFudge.Core\NoteFileStore.cs:16-91`).

Review the dangerous boundaries first

Running is not completion

The native adapter can cease reporting Running while cleanup/final delivery continues. The UI must therefore block Start and destructive transcript actions during Stopping, not reset the final-ID counter as soon as a boolean flips. Otherwise an old final can arrive with a large ID after a new session starts, and a valid new final ID1 is rejected as a duplicate. The buffer's global-ID rule is correct only when its session boundary is correct (`src\PhantomFudge.Core\TranscriptBuffer.cs:23-38,48-57`).

Input identity is not speaker identity

Mic may capture the owner's voice, room sound, or speaker echo. Computer may include all output on a selected endpoint, not only the meeting. The correct labels are source names. Turning those labels into verified people would be a new, unsupported claim. The public contract states this boundary explicitly (`docs\DUAL-SOURCE-AUDIO.md:8-26`).

Privacy is not one boolean

Window exclusion, source authorization, local token checks, no automatic cloud upload, and unsaved-data behavior answer different questions. A protected window can still coexist with an OS microphone indicator. A successful local GDI fixture does not establish every Teams/Zoom/Meet/ OBS path. The qualification matrix keeps these observations separate (`docs\QUALIFICATION.md:1-26`).

A plan is not a runtime component

Azure Ask currently exists in design documents and TODOs. A principal-level diagram should not quietly insert an operational cloud box into the implemented path. The planned architecture requires immutable question snapshots, source-scoped commit and request-bound streaming; none is implied by the current text buffer (`docs\AI-ANSWER-ARCHITECTURE.md:1-37`).

Where to go deep

Start with the native lifecycle and transport when reviewing concurrency. Read the server's source state and worker loop when reviewing completeness. Read UI stop coordination and buffer mutation together when reviewing user data preservation. Read the capture policy beside its qualifications—not alone—when reviewing privacy language.

For a product decision, ask whether it changes data origin, lifetime, visibility, authority or completion. A new color mostly changes presentation. A new input changes all five. An Azure answer provider additionally crosses an egress and identity boundary.

Review exercise

Consider a user who selects Both, starts capture, hides the window during a slow final decode, then immediately asks to start again. Draw the ownership/state sequence. Identify where the native capture, socket, decoder, UI and buffer can legitimately disagree about "running." The correct design does not require those components to share one boolean; it requires that the coordinator not start a new logical session until the old completion boundary is satisfied.

References

- `src\PhantomFudge.Core\AudioSource.cs:3-30` — source identity.
- `src\PhantomFudge.Core\TranscriptBuffer.cs:6-97` — source partials and session-global IDs.

- `src\PhantomFudge.Core\NoteFileStore.cs:6-91` — bounded local file behavior.
- `src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:71-189` — native lifecycle ownership.
- `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:11-71` — transport state/bounds.
- `services\transcription\server.py:235-338` — source queue and completion.
- `services\transcription\manage.py:112-144` — authenticated process ownership.
- `docs\SPEECH-BENCHMARK.md:10-35` — engines versus host languages.

Source: docs/handbook/02-zero-to-hero.md

Zero-to-hero learning path

> **TL;DR:** Learn enough of each technology to trace one real packet and one real user action. Do not try to master all of Windows, C#, Python, networking and machine learning before opening the project. The learning path is a sequence of explanations and bounded exercises.

Overview: two languages, one product

The native project targets `net10.0-windows`, uses WPF and references the Core project plus NAudio (`src\PhantomFudge.App\PhantomFudge.App.csproj:1-16`). The Core project has no WPF dependency, which makes text/identity policies easier to test independently (`src\PhantomFudge.Core\PhantomFudge.Core.csproj:1-5`). Python hosts the local recognition service, with dependencies explicitly pinned (`services\transcription\requirements.txt:1-20`).

This is a technology boundary, not a competition between language quality. C# controls the Windows integration; Vosk's native model does recognition through Python bindings. The benchmark compares actual recognizers, not whether braces or indentation are better.

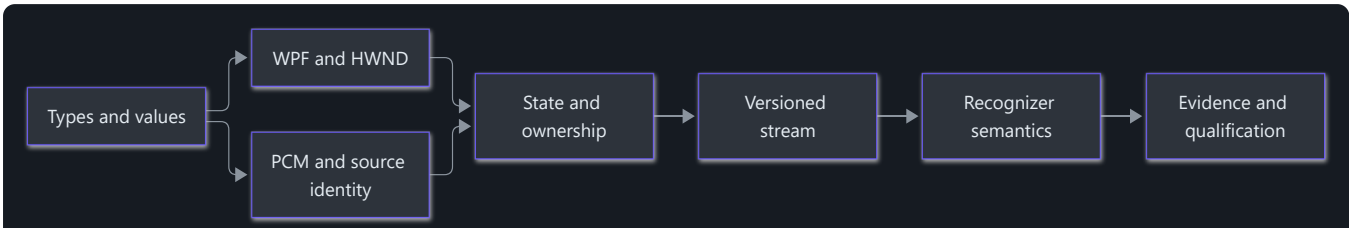
Part I: foundations in a useful order

Concept	Learn first	Project anchor
C# types and records	Values, nullable references, enums, immutable messages	`AudioSource` and `TranscriptEvent`
WPF	XAML, dependency properties, visual tree, routed events, dispatcher	`App.xaml` and `MainWindow`
Native interop	HWND, P/Invoke, return values, last-error timing	`WindowProtection.Configure`
PCM audio	Sample rate, channels, bit depth, bytes, duration	v2 protocol
Asynchrony	Tasks, cancellation, queues, callbacks and ownership	native adapter and socket
Python asyncio	Event loop versus CPU worker; await versus blocking	server worker loop
Recognition	Partial versus final, endpoint versus explicit flush	Vosk wrapper
Verification	Unit, protocol, synthetic integration, hardware and recipient evidence	separate test/qualification layers

`AudioSource` is a small, concrete introduction to strong types: Computer and Microphone have stable byte values and separate wire/display names. An unknown enum value throws instead of silently becoming a source (`src\PhantomFudge.Core\AudioSource.cs:3-30`).

```
# Teaching pseudocode: an enum carries more meaning than an unexplained integer.
source = AudioSource.MICROPHONE
packet = AudioPacket(source=source, pcm=bytes_from_that_input)
```

In Python a dictionary can express the same idea, but the compiler cannot prove that every caller supplied the right key or type. That is why the wire protocol validates fields even when the C# side has strong types: a process boundary can receive malformed bytes.



02 - Zero-to-Hero Learning Path - diagram 1. Source-linked; status follows the chapter.

Part II: trace two paths

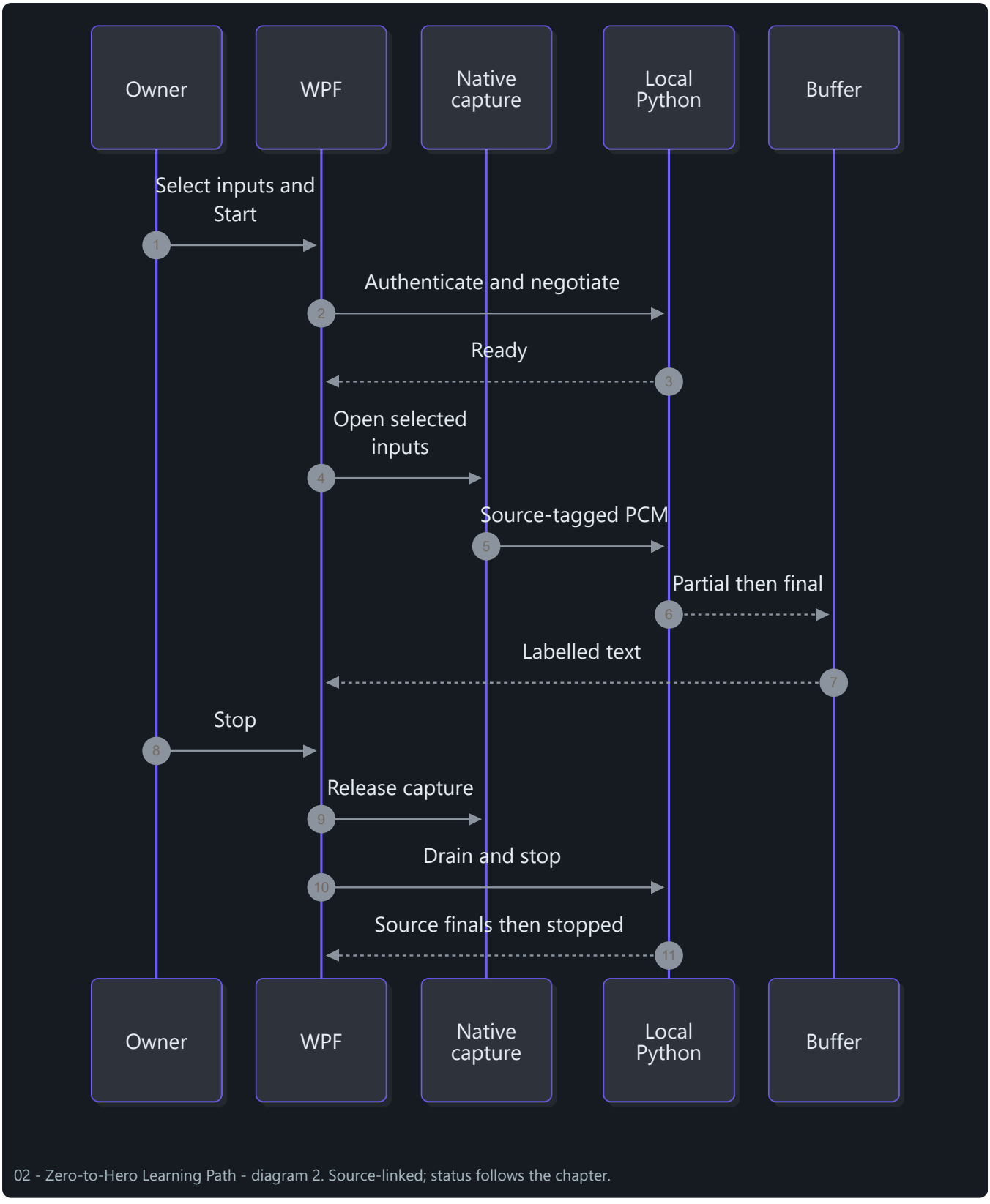
Path A: opening the application

Follow `Run-PhantomFudge.ps1`. It chooses a source-build or packaged executable, detects an existing instance, optionally builds, and starts the local service unless NotesOnly was chosen. It passes the token through the child environment rather than printing it (`scripts\Run-PhantomFudge.ps1:8-57`).

Then follow `App.OnStartup`: a normal launch establishes per-user ownership and either reveals the existing process or constructs a new protected window (`src\PhantomFudge.App\App.xaml.cs:39-58`). This explains why double-clicking an updated EXE while an old instance is running does not necessarily replace the old UI.

Path B: one utterance

An explicit Start selects inputs. PCM bytes are tagged with their source, sent over the local socket, decoded independently, and returned as partial/final events. The buffer must clear only the partial corresponding to a final's source. Do not read "partial" as a promise that every character is settled (`src\PhantomFudge.Core\TranscriptBuffer.cs:14-19`, `src\PhantomFudge.Core\TranscriptBuffer.cs:41-72`; `docs\DUAL-SOURCE-AUDIO.md:28-76`).



02 - Zero-to-Hero Learning Path - diagram 2. Source-linked; status follows the chapter.

Part III: learn the byte arithmetic

The wire format is16,000 samples per second, one channel,16 bits per sample. Divide bits by eight to get bytes:16,000 x1 x2 =32,000 PCM bytes per second. A100ms payload is3,200 bytes. V2 adds one source byte per WebSocket message, not per audio sample. Thus a3,200-byte PCM payload becomes a3,201-byte message; it is not a WAV file and has no WAV header (`docs\DUAL-SOURCE-AUDIO.md:37-53`).

This arithmetic helps diagnose errors. An odd-length PCM payload is invalid because a16-bit sample needs two bytes. A source prefix belongs outside the PCM passed to the recognizer. Counting that prefix as sound would shift every subsequent sample boundary.

The phrase "two channels" can be misleading here. The application uses two independent mono source tracks, not one interleaved stereo stream. Each recognizer gets mono audio; the protocol carries source identity separately.

Part IV: asynchronous thinking

Compare a synchronous function with a lifecycle:

```
result = recognize(audio)       # one call; ownership is simple
```

```
start_requested = True
await service_ready()
open_inputs()
# callbacks and network events now arrive independently
await stop_requested()
await all_inputs_released()
await accepted_audio_drained()
await final_text_received()
```

An `await` does not mean "nothing else can happen." The WPF dispatcher can process another user action while an operation awaits I/O. Conversely, doing CPU recognition on an event loop can prevent other requests from being handled. The server therefore uses a bounded worker for native recognition, while native startup is kept away from a blocked WPF dispatcher (`services\transcription\server.py:109-142`, `src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:106-124`).

Three questions prevent many mistakes:

1. Who owns this resource while the operation is incomplete?
2. What identifies this result as belonging to the current source/session?
3. What event proves completion, rather than merely intent to stop?

Part V: safe first exercises

Exercise 1: source labels, no devices

Read `SourceTranscriptTests`. Explain why a Computer final must leave a Mic partial visible. Then sketch a table of two partials, one Computer final, and one Mic final. This needs no microphone or backend (`tests\PhantomFudge.Core.Tests\SourceTranscriptTests.cs:1-80`).

Exercise 2: a rejected save

Read a test that changes a file outside the app, then attempts to save with an old fingerprint. Identify which version should survive. Do not use a real note as the experiment; the test creates an isolated directory (`tests\PhantomFudge.Core.Tests\NoteFileStoreTests.cs:1-110`).

Exercise 3: protocol without hardware

Run the native synthetic harness only after reading its entry mode. It uses fake transport/input objects for its normal test groups. The separate playback command is deliberately not part of the default suite (`tests\PhantomFudge.Audio.Tests\Program.cs:18-71`).

Exercise 4: a documentation-only screenshot

The documentation command renders actual WPF controls with synthetic text/device labels. It is not a Snipping Tool workaround for capturing someone else's private window (`src\PhantomFudge.App\App.xaml.cs:31-36`, `scripts\Export-HandbookScreenshots.ps1:1-76`).

How to avoid learning the wrong lesson

Do not conclude that a successful test suite proves hardware support, that the fastest batch recognizer has the lowest live partial latency, or that a private window is a security boundary. Those are different experiments. The repository deliberately leaves broader acceptance work open.

The glossary provides more than forty terms. Use it when a word becomes a blocker, then return to the code path. A good first contribution is a small test/documentation improvement that follows one real invariant end to end.

References

- `src\PhantomFudge.App\PhantomFudge.App.csproj:1-16` — native target and dependencies.
- `src\PhantomFudge.Core\PhantomFudge.Core.csproj:1-5` — platform-independent policy layer.
- `src\PhantomFudge.Core\AudioSource.cs:3-30` — typed input identity.
- `scripts\Run-PhantomFudge.ps1:8-57` — launch/reuse/token boundary.
- `src\PhantomFudge.App\App.xaml.cs:13-58` — entry paths.
- `services\transcription\requirements.txt:1-20` — Python dependency baseline.
- `tests\PhantomFudge.Audio.Tests\Program.cs:18-71` — synthetic test entry.
- `docs\DUAL-SOURCE-AUDIO.md:28-76` — byte contract.

Product boundaries and decisions

> **TL;DR:** Scope changes are not embarrassing historical debris; they are design inputs. The important discipline is to record what superseded what, update the affected contracts, and avoid treating an earlier assumption as a permanent requirement.

Overview: from a private notepad to a conversation companion

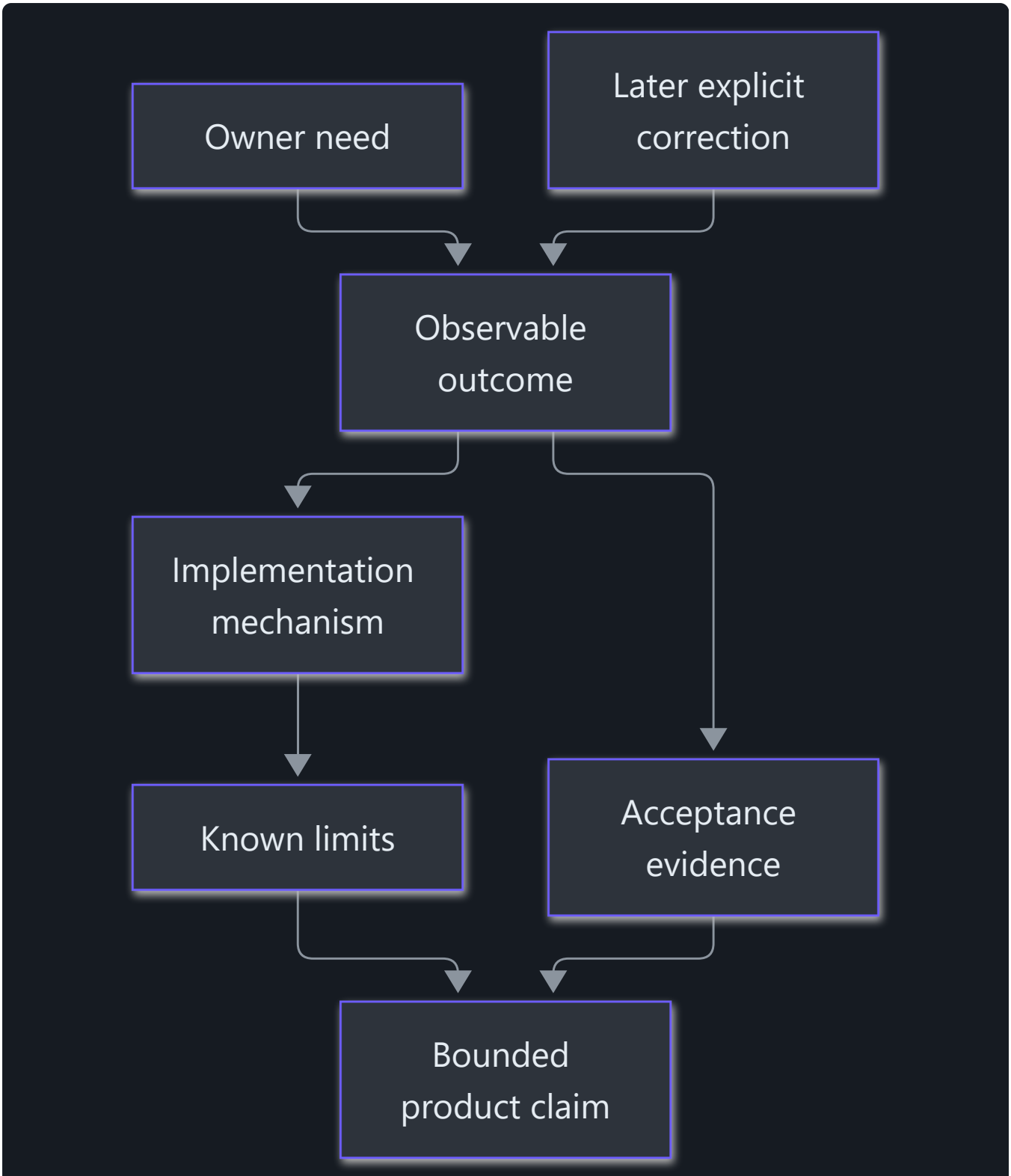
The initial request was a draggable speaker-notes window that would not appear in a screen share. The useful part of that request is implementable on supported Windows capture paths. The absolute version—no software can ever detect or record the surface—is not. The product contract therefore separates plain-text editing, movement, shell presentation, captured pixels, platform support and actual product qualification (`docs\PRODUCT.md:6-29`).

That separation made later changes tractable. Transcription became the main view. Notes became collapsible. The owner removed repetitive permission/rehearsal acknowledgments. Translucency became a real rendering requirement, rather than a hypothetical visual effect. A microphone track became explicitly requested after the original computer-only implementation. Finally, Azure answering became a researched future extension, not an automatic cloud replacement.

The development ledger records these changes as dated decisions and evidence, not a fictional linear plan that was correct from the beginning (`TODO\16_DEVELOPMENT_LEDGER.todo:1-90`). The public journal tells the fuller chronological story.

Architecture of a requirement

A requirement should identify its observable outcome and the evidence needed to claim it. For example, "no ordinary taskbar button" has a shell observation. "Computer audio becomes text" requires audio/protocol/recognizer evidence. "The recipient never sees the notes" needs the actual capture path; neither of the first two observations supplies it.



03 - Product Boundaries and Decisions - diagram 1. Source-linked; status follows the chapter.

The native mechanism uses an owned HWND and display affinity. It does not make the process cease to exist (`src\PhantomFudge.App\Native\WindowProtection.cs:18-45`). Similarly, the source enum means "where samples came from", not "who was speaking" (`src\PhantomFudge.Core\AudioSource.cs:3-30`). Precision in nouns prevents precision failures in code and documentation.

Decisions that shape the current product

Decision	Current outcome	Important cost
Windows11 first	Native WPF/Win32 behavior rather than a lowest-common-denominator overlay	Other operating systems need independent designs
Transcript first	Large reading area; notes collapsed until needed	Editing and reading modes must remain understandable
Explicit inputs	Computer, Mic or Both, followed by Start	Device selection/permissions can fail and must be visible
No repeated acknowledgments	Owner handles permission/recipient check outside the app	This does not create technical evidence that a recorder is safe
Background-only alpha	See through content while keeping text opaque	Underlying content can reduce contrast
Native Python by default	No Docker prerequisite for ordinary use	Local environment/model setup must be reproducible
Explicit future Ask	Owner chooses text crossing the Azure boundary	Structured question identity and cloud configuration are new work

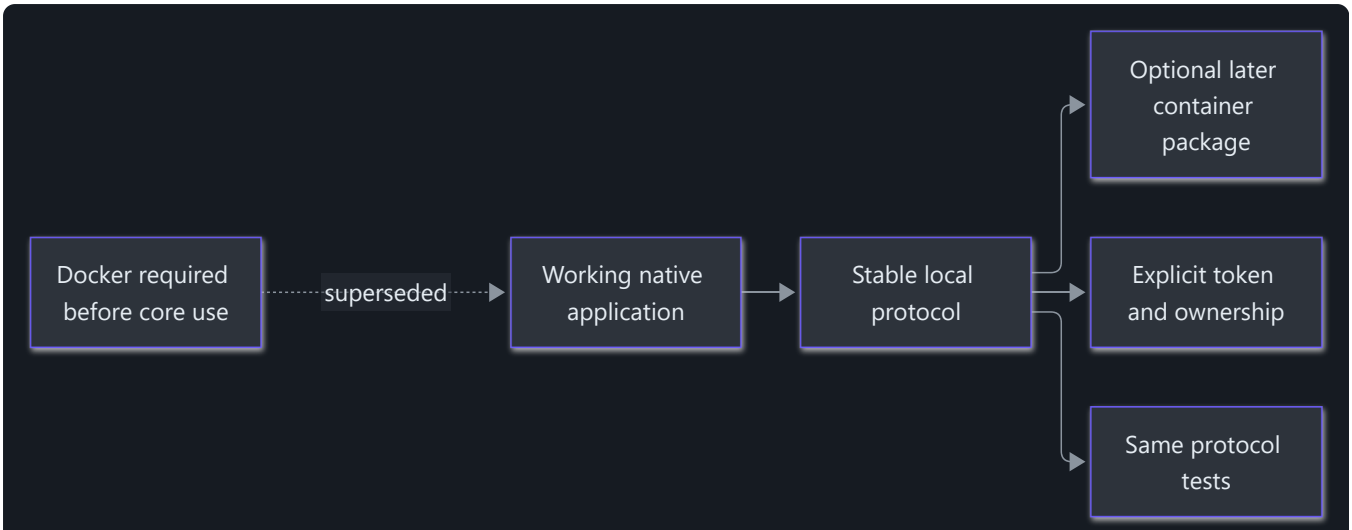
The requirements make the microphone change explicit: the later request supersedes the old never-microphone product scope, while tests still must not open live inputs implicitly (`docs\DUAL-SOURCE-AUDIO.md:1-26`). A repository that keeps both statements as current truth would eventually produce either a broken feature or an unauthorized test.

Why Docker moved to the end

Containerization and container dependence are different design decisions. A headless Python service with environment-based configuration and a versioned protocol is straightforward to package later. Requiring Docker before a Windows desktop tool can transcribe creates a larger operational dependency without proving an inference benefit.

The current launcher starts native Python unless a separate option is chosen (`scripts\Start-Transcription.ps1:1-24`). `Run-PhantomFudge` can also avoid service startup entirely with NotesOnly (`scripts\Run-PhantomFudge.ps1:1-41`). The Docker TODO now explicitly follows native release acceptance, rather than blocking it (`TODO\20_DOCKER_LOCAL_SERVICE.todo:1-42`).

The lesson is not "containers are bad." It is "portability should preserve an interface before it imposes infrastructure." A service boundary is useful only when its responsibilities are clear: Python recognizes framed PCM; C# controls Windows devices and presentation.



03 - Product Boundaries and Decisions - diagram 2. Source-linked; status follows the chapter.

Interaction choices are privacy choices

The owner observed an I-beam over text and a disappearing cursor in a shared view. The app now uses Arrow and a non-activating default, but those local changes do not control every recorder's cursor stream. The design deliberately records that remaining uncertainty rather than renaming it success (`docs\CURSOR-AND-THEMES.md:1-45`).

At the same time, normal functionality must not be sacrificed to a privacy slogan. Notes must still be editable after an explicit Edit action. Device rows must still be selectable without unnecessarily activating the window. Unsaved-content warnings remain even though permission checkboxes were removed. Those controls answer different questions.

Defining done without freezing learning

The TODO vocabulary separates NOT_STARTED, IN_PROGRESS, BLOCKED and DONE. DONE applies to the scope of a task: a research document can be complete while its architecture is not implemented. An authored workflow can be complete while execution is not started (`docs\TODO-CONVENTIONS.md:1-63`).

This is especially important for the Azure plan. The proposed path sends a frozen, explicit question to a streamed model request. It does not authorize automatic meeting uploads, resource creation or model calls simply because the diagram exists (`docs\AI-ANSWER-ARCHITECTURE.md:1-37`).

A worked change-impact exercise

Suppose a new request says, "Also transcribe my microphone." A superficial implementation adds a second device object. A complete change asks:

1. Which device direction is valid, and when may it open?
2. Are samples mixed or kept distinct?

3. How do frame formats, source labels and time offsets change?
4. What happens if only one input starts or disconnects?
5. How does Stop prove both inputs are released and both tails finalized?
6. What does the UI show before recording and during faults?
7. Which old privacy statements are now historical?
8. Which tests can run without activating real hardware?

That checklist explains why the dual-input change involved native capture, protocol, Python state, transcript storage, UI, tests and documentation rather than one checkbox. The contract is written down before pretending that all those surfaces agree (`docs\DUAL-SOURCE-AUDIO.md:28-108`).

References

- `docs\PRODUCT.md:6-29` — distinct requirements and limits.
- `TODO\16_DEVELOPMENT_LEDGER.todo:1-90` — dated scope/evidence history.
- `src\PhantomFudge.App\Native\WindowProtection.cs:18-45` — actual window mechanism.
- `src\PhantomFudge.Core\AudioSource.cs:3-30` — input identities.
- `scripts\Start-Transcription.ps1:1-24` — native default.
- `scripts\Run-PhantomFudge.ps1:1-41` — launch options.
- `docs\DUAL-SOURCE-AUDIO.md:1-108` — microphone supersession and protocol.
- `docs\AI-ANSWER-ARCHITECTURE.md:1-37` — proposed answer boundary.

04 — Windows Capture, Cursors, and Focus

Overview: four questions, not one invisibility switch

A presentation aid has two audiences: the person reading its local window and the people receiving a captured presentation. Those audiences do not necessarily receive the same pixels. Phantom Fudge asks Windows to exclude its window from supported capture paths; it does not make the process nonexistent, encrypt the desktop, or control another program's recording implementation. `WindowProtection.Configure` performs a native configuration operation, not a recipient-side test. This distinction is the foundation of the feature, rather than a disclaimer added after it. (src\PhantomFudge.App\Native\WindowProtection.cs:18-46)

Keep four observations separate:

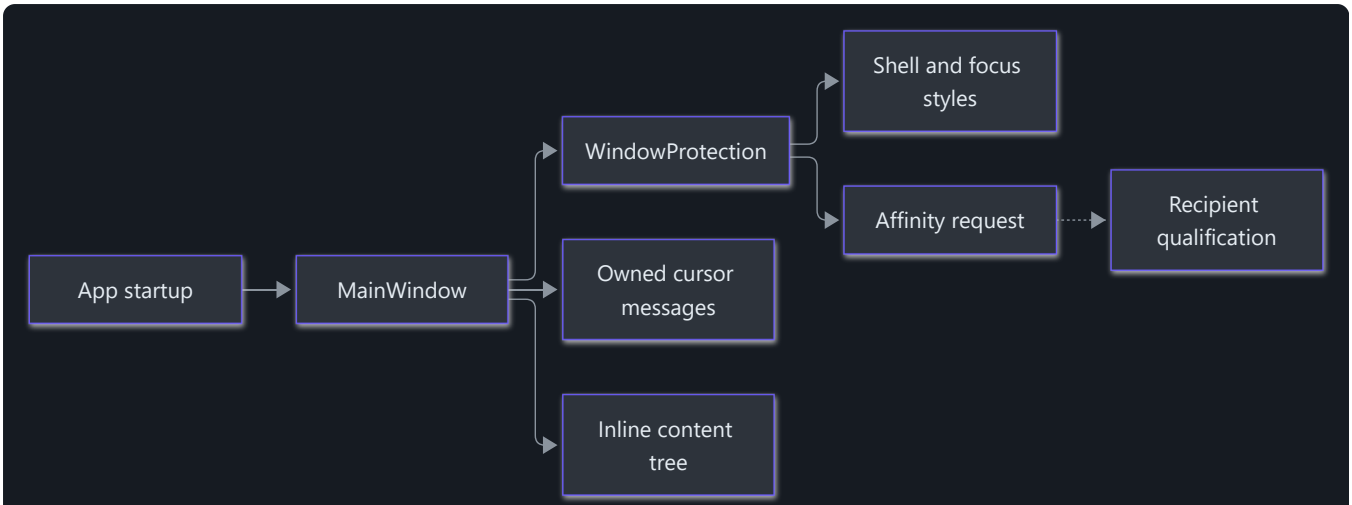
Question	Mechanism or evidence
Was capture exclusion requested successfully?	Setter result, optional getter readback, composition flush
Does the window appear in ordinary shell navigation?	WPF settings and native tool-window styles, followed by observation
Does a sharing picker list it?	The particular picker's behavior; not established by tool-window style
What does the audience actually receive?	Independent recipient frames and recordings for that exact configuration

The qualification plan explicitly separates these dimensions. A successful API call cannot fill in the other three columns. Nor does a visible local Arrow establish that a recorder includes the pointer. (docs\QUALIFICATION.md:9-26; docs\CURSOR-AND-THEMES.md:21-37)

Newcomer path: follow startup, then read the cursor section. **Principal-engineer path:** inspect the native error semantics, invalidation ordering, and evidence limits. Chapter 11 connects this window lifecycle to asynchronous audio shutdown.

Architecture: one content surface, several independent policies

The production surface is a topmost, layered WPF window with a generic title, no ordinary taskbar entry, transparent window background, and custom move/resize chrome. The root content is not click-through. Capture affinity belongs to the native window handle, whereas text controls and inline panels belong to its WPF visual tree. A panel inside that tree is not equivalent to a separate native dialog. (src\PhantomFudge.App\MainWindow.xaml:1-15,70-85,105-141)



04 — Windows Capture, Cursors, and Focus - diagram 1. Source-linked; status follows the chapter.

The dotted edge means “requires separate evidence,” not “automatically proves.” The implementation behind the solid edges is `OnStartup`, `ShowProtected`, and `WindowMessage`. (src\PhantomFudge.App\App.xaml.cs:39-52; src\PhantomFudge.App\MainWindow.xaml.cs:79-107,157-187)

Components and ownership

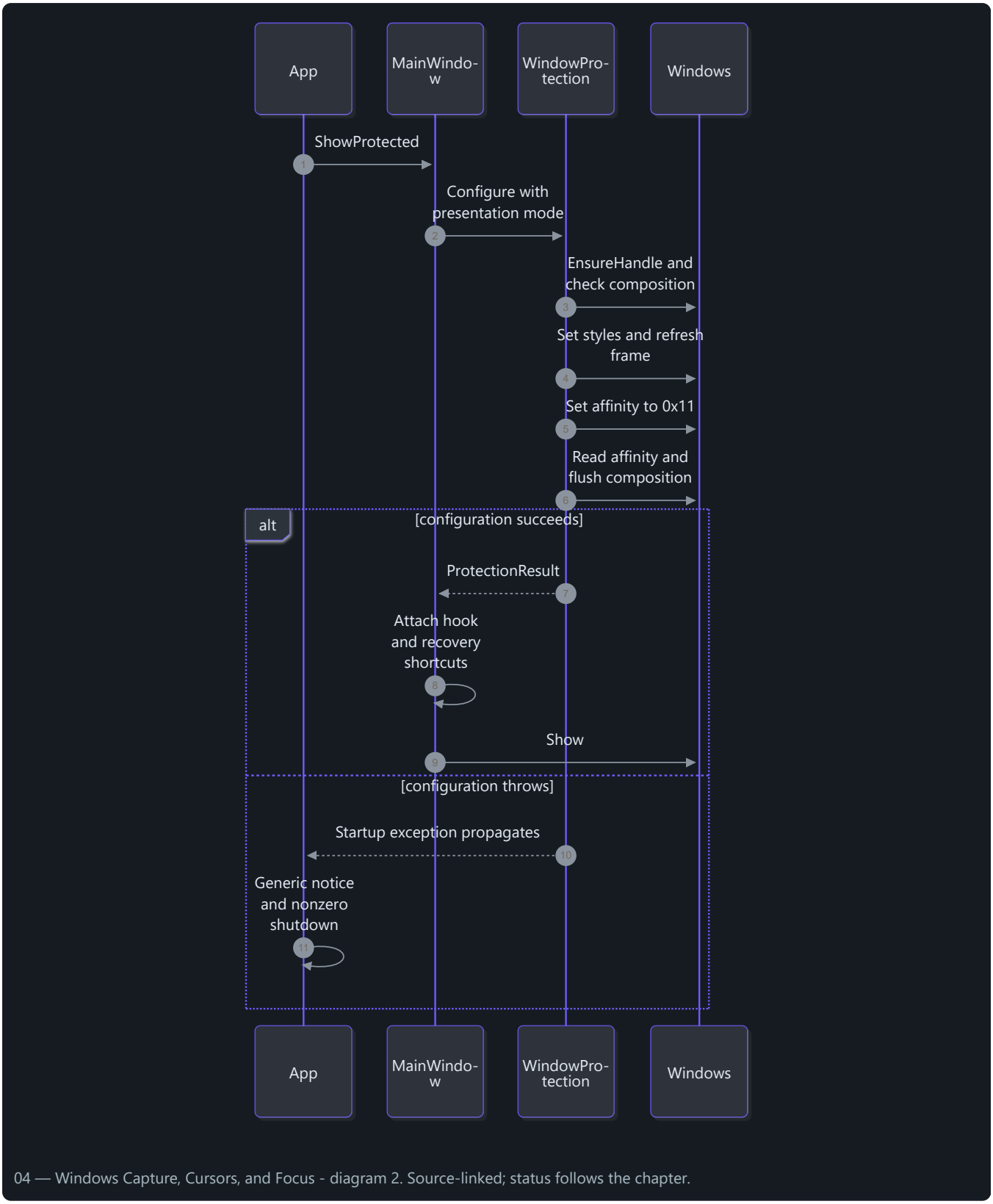
Component	Responsibility and important boundary
`App.OnStartup`	Chooses the owning instance and calls `ShowProtected`; ordinary startup does not use `StartupUri`. (src\PhantomFudge.App\App.xaml.cs:39-59; src\PhantomFudge.App\App.xaml:1-4)
`WindowProtection`	Checks Windows 11/DWM, adjusts native styles, requests affinity, and interprets readback. (src\PhantomFudge.App\Native\WindowProtection.cs:18-46)
`CursorPolicy`	Loads the standard Windows Arrow and sets the current cursor shape. (src\PhantomFudge.App\Native\CursorPolicy.cs:6-24)
`WindowHotkeys`	Registers fixed Ctrl+Alt shortcuts with no-repeat behavior; does not hook arbitrary keystrokes. (src\PhantomFudge.App\Native\WindowHotkeys.cs:6-25)
`SingleInstance`	Uses per-user instance naming and a narrow current-user pipe command. (src\PhantomFudge.App\Native\SingleInstance.cs:18-59)
`LocalFailureNotice`	Writes a fixed generic failure message instead of opening a content-bearing error window. (src\PhantomFudge.App\LocalFailureNotice.cs:5-29)

The architecture document names additional target abstractions, including a surface registry. They should not be mistaken for classes already implemented: current orchestration lives principally in `MainWindow`. (docs\ARCHITECTURE.md:35-75; src\PhantomFudge.App\MainWindow.xaml.cs:19-46)

Data flow: configuring before first show

`App.OnStartup` constructs the window explicitly. `ShowProtected` marks protection unready, calls `Configure`, and only then marks it ready and invokes `Show`. The constructor prepares resources, handlers, and device enumeration; it does not start audio. Thus window creation, device discovery, visible presentation, and

recording are distinct operations. (src\PhantomFudge.App\App.xaml.cs:47-52; src\PhantomFudge.App\MainWindow.xaml.cs:48-107,589-625)



04 — Windows Capture, Cursors, and Focus - diagram 2. Source-linked; status follows the chapter.

`Configure` calls `EnsureHandle`, which obtains an HWND without requiring an already-visible content window. It checks Windows build 22000 or later and requires desktop composition. It preserves unrelated extended-style bits, adds `WS_EX_TOOLWINDOW`, removes `WS_EX_APPWINDOW`, and conditionally sets `WS_EX_NOACTIVATE`.

`SetWindowPos` refreshes the frame without moving, resizing, reordering, or activating it. (src\PhantomFudge.App\Native\WindowProtection.cs:12-36)

The affinity setter requests `0x11`, `WDA_EXCLUDEFROMCAPTURE`. A failed setter throws. A successful getter returning anything else also throws. An unavailable getter is recorded in `ProtectionResult`; it is not treated identically to contradictory readback. Finally, `DwmFlush` checks submitted composition work. None of these calls creates an independently observed capture-frame barrier. (src\PhantomFudge.App\Native\WindowProtection.cs:35-45,70; docs\ARCHITECTURE.md:110-113)

Why the native wrapper checks zero carefully

`GetWindowLong` / `SetWindowLong` can return zero without implying failure. The wrapper clears the last-error slot first, performs the appropriate 32- or 64-bit call, then throws only when a zero result accompanies a nonzero error. This preserves the distinction between a legitimate zero value and an API failure. C# `nint` represents a native-sized integer; it is not the HWND's logical identity across arbitrary future replacements. (src\PhantomFudge.App\Native\WindowProtection.cs:48-68)

Implementation: focus policy is not pointer capture policy

Present starts with `presenting = true`. The window uses `ShowActivated=false`; native setup adds `WS_EX_NOACTIVATE`; `WM_MOUSEACTIVATE` returns `MA_NOACTIVATE` (`3`). These mechanisms reduce accidental foreground takeover while retaining deliberate pointer interaction. They do not forward clicks through the app. Edit reverses the interaction policy, enables note focus and input methods, and deliberately focuses the editor after successful reveal. (src\PhantomFudge.App\MainWindow.xaml.cs:31,177-181,223-257)

Cursor shape is handled at two levels:

1. WPF declares Arrow and `ForceCursor` on the window and text controls.
2. `WindowMessage` handles `WM_SETCURSOR` for the owned HWND by calling `CursorPolicy.SetArrow`.

The native policy loads resource `32512`, the standard Arrow. It does not replace system cursor resources or manipulate global visibility counters. Arrow is also used in Edit; editable text and an I-beam are not inseparable concepts. (src\PhantomFudge.App\MainWindow.xaml:6-9; src\PhantomFudge.App\App.xaml:48-76; src\PhantomFudge.App\MainWindow.xaml.cs:159-163; src\PhantomFudge.App\Native\CursorPolicy.cs:8-24)

A recorder may include, exclude, or separately composite cursor imagery. Consequently, the reported remote cursor disappearance remains unresolved until tested in the actual sharing product and mode. Local foreground preservation, native Arrow equality, and content-exclusion samples answer different questions. (docs\CURSOR-AND-THEMES.md:23-37)

Invalidation hides before waiting

Display/DPI messages, selected session changes, and suspend/resume notifications dispatch `InvalidateAsync`. It clears `protectionReady`, hides immediately, then awaits audio Stop. That ordering avoids leaving content intentionally visible while cleanup drains. It cannot revoke frames already captured or make

OS notifications instantaneous. Reveal deliberately re-runs `Configure`. (src\PhantomFudge.App\MainWindow.xaml.cs:182-209,131-143)

Ordinary Hide also stops transcription, but does not destroy the document or exit the process. Mode switching hides, stops audio, changes editability, and reconfigures before showing again. Theme, opacity, and Notes expansion are different operations: they update existing controls rather than changing this native policy. (src\PhantomFudge.App\MainWindow.xaml.cs:212-242,442-478)

If reveal setup fails, `TryReveal` starts invalidation and records a fixed failure notice. Unsaved text remains in the running process, not in an automatic recovery file. `LocalFailureNotice.Record` overwrites one timestamped generic notice under `%LOCALAPPDATA%\PhantomFudge\last-window-error.txt`; it does not serialize the exception, note, transcript, or filename. Failure to write that notice produces another fixed console message. (src\PhantomFudge.App\MainWindow.xaml.cs:139-143; src\PhantomFudge.App\LocalFailureNotice.cs:9-29)

Recovery without a taskbar button

The shortcuts are Ctrl+Alt+Space for visibility and Ctrl+Alt+E for deliberate editing. Registration failures produce an inline status while preserving second-launch recovery. These are fixed defaults, not a currently implemented configurable-hotkey UI. (src\PhantomFudge.App\MainWindow.xaml.cs:94-118,165-175; src\PhantomFudge.App\Native\WindowHotkeys.cs:12-17)

`SingleInstance` derives a name from the current Windows SID, creates a session-local mutex, and exposes a `CurrentUserOnly` named pipe. A second launch sends six bytes, `REVEAL`, with a five-second connection deadline. The listener reads a six-byte command with a two-second request deadline; it does not deserialize notes, paths, scripts, or executable commands. This is narrow recovery IPC, not a general command server. (src\PhantomFudge.App\Native\SingleInstance.cs:18-75)

`RevealAndReset` sizes and centers against `SystemParameters.WorkArea`, then uses the protected reveal path. That is useful recovery, but not persisted, mixed-monitor placement restoration. Minimum window dimensions still constrain what fits on a small work area. (src\PhantomFudge.App\MainWindow.xaml.cs:121-128; docs\IMPLEMENTATION-STATUS.md:149-157)

Why auxiliary surfaces change the problem

Capture policy is applied to the HWND passed to the native setter, not to an abstract application identity. Consequently, moving a find box from an inline panel into a separate popup would change the surface inventory even if it looked almost identical. The current find and unsaved-confirmation controls are elements of `MainWindow.xaml`; text-box context menus and spell checking are disabled in the application style. These choices reduce additional app-generated surfaces, but do not establish that every framework helper, input-method candidate, or accessibility window has been qualified. (src\PhantomFudge.App\Native\WindowProtection.cs:23-36; src\PhantomFudge.App\MainWindow.xaml:70-85,205-210; src\PhantomFudge.App\App.xaml:48-50; docs\QUALIFICATION.md:120-122,137-138)

Native Open/Save dialogs are a deliberate exception available through Edit. Their behavior must not be described as inheriting the main window's capture assurance merely because `ShowDialog(this)` supplies an owner. The current UI explicitly refuses saving/exporting in Present and reports the dialog boundary after

Open. This is a concrete example of preferring a narrower, understandable workflow over an unsupported "everything this process draws is private" promise. (src\PhantomFudge.App\MainWindow.xaml.cs:270-275,333-350,359-365,733-739)

Worked example: a display change during transcription

The following Python-shaped pseudocode explains ordering only; it is not production code, a Win32 binding, or a proposal to port the WPF application:

```
async def invalidate():
    state.protection_ready = False
    window.hide()                  # Do not wait for recognizer drain first.
    await stop_transcription()
    status.set_generic_reason()

def reveal():
    try:
        configure_native_window(nonactivating=state.presenting)
        state.protection_ready = True
        window.show()
    except KnownProtectionFailure:
        schedule(invalidate())
        record_generic_local_notice()
```

In C#, `Task` represents eventual completion, while `Dispatcher.BeginInvoke` schedules work on the WPF UI thread. The code must arrange both: a native notification must not directly manipulate controls on an arbitrary thread, and waiting for Stop must not precede Hide. `Dispatch` and `InvalidateAsync` express those responsibilities separately. (src\PhantomFudge.App\MainWindow.xaml.cs:149-153,182-209)

Tests, limitations, and review questions

`CaptureProof.RunAsync` establishes an unprotected synthetic positive control and an underlying baseline, then samples four show cycles with four protected frames each. Its outcome is deliberately named `PASS_LOCAL_GDI_SAMPLES_ONLY`; `Grab` uses cropped GDI `BitBlt`. This is not continuous first-frame coverage or qualification of Teams, Zoom, Meet, OBS, or their cursor streams. Do not run this visible positive-control fixture during an actual presentation merely to read the chapter. (src\PhantomFudge.App\Diagnostics\CaptureProof.cs:19-114,168-199)

`RunUiSmokeAsync` separately checks initial foreground preservation, passive styles, `MA_NOACTIVATE`, and the Arrow policy. Its rendered PNGs are visual-tree diagnostics, not independent capture evidence. All meeting-product qualification rows remain open in the maintained matrix. (src\PhantomFudge.App\MainWindow.xaml.cs:824-866,1085-1098; docs\QUALIFICATION.md:28-53)

Question: Does successful getter value `17` establish universal exclusion?
Answer: No. It confirms a configuration readback; recorder behavior still needs independent evidence.

Question: Should a protection failure display the note inside an error dialog so it can be recovered?
Answer: No. The implemented failure path hides and preserves memory, emitting only a generic notice.

Question: Can a local Arrow test close the remote-cursor issue?
Answer: No. Record the sharing backend, source type, pointer-capture setting, and actual recipient frames. These answers follow the configuration, failure, and qualification contracts above. (src\PhantomFudge.App\Native\WindowProtection.cs:38-45; src\PhantomFudge.App\LocalFailureNotice.cs:11-24; docs\QUALIFICATION.md:132-138)

References and maintenance checkpoints

- Native configuration: `Configure`, style wrappers, and `FlushComposition`.
- Window lifecycle: `ShowProtected`, `WindowMessage`, invalidation, and recovery.
- Cursor policy, hotkeys, and instance recovery.
- Failure notice and synthetic capture proof.
- Qualification matrix and cursor investigation.

Revisit this chapter whenever HWND construction, native styles, popups, focus behavior, or the rendering framework changes. Such changes require new evidence, not inheritance of an old compatibility claim. (docs\QUALIFICATION.md:24-26,113-130)

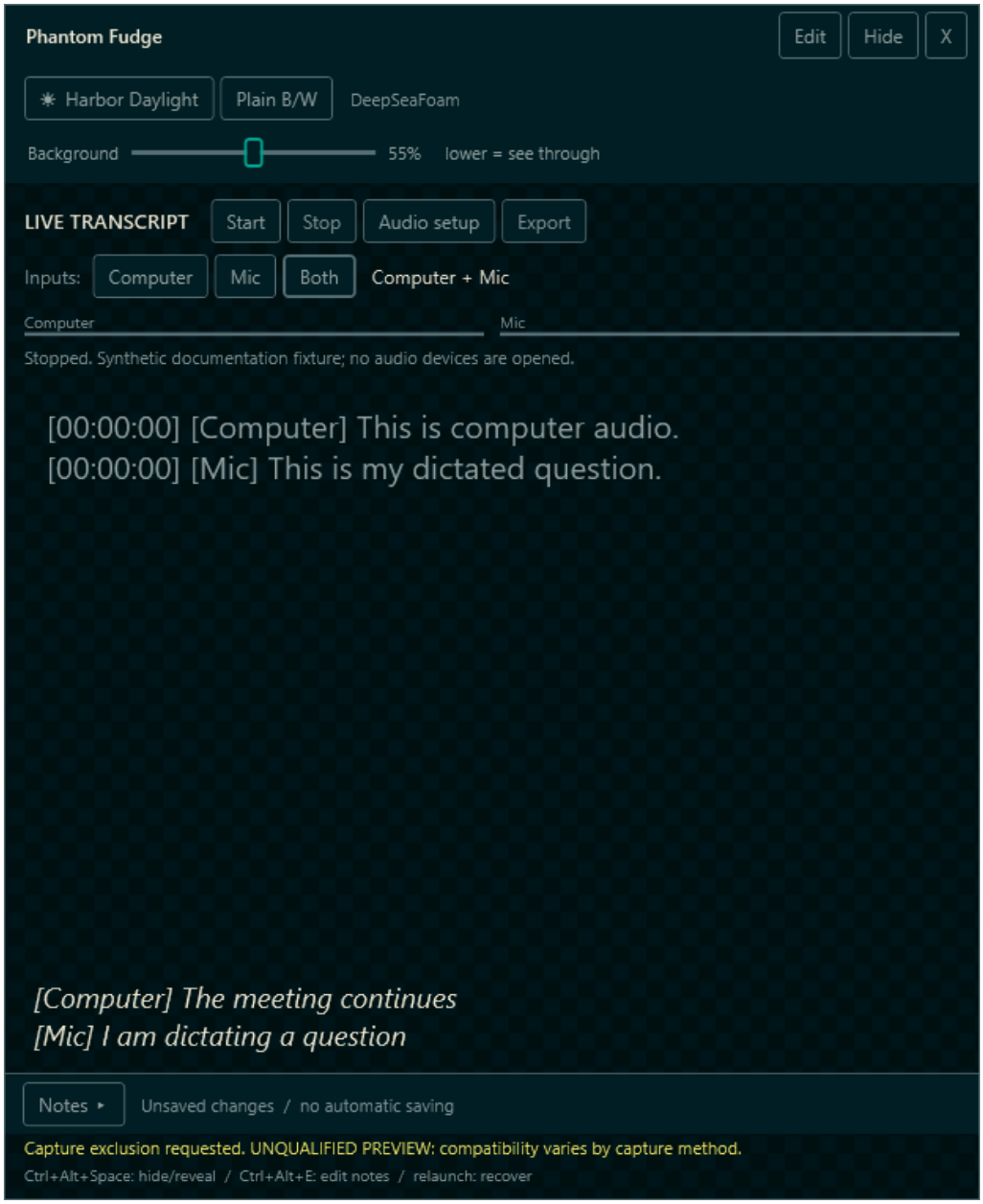
05 — Desktop UI and Semantic Themes

Overview: a reading surface before an editor

The most important UI decision is not a color. It is which activity receives space and focus by default. Phantom Fudge starts as a passive transcript-reading surface: the transcript occupies the growing central row, Notes is collapsed, and typing requires deliberate Edit. This separates “I need to see the meeting” from “I intend to take keyboard focus.” The initial fields and XAML implement that decision directly; it is not inferred from screenshots. (src\PhantomFudge.App\MainWindow.xaml.cs:31-33,48-76; src\PhantomFudge.App\MainWindow.xaml:163-188)

The UI is native C#/WPF. WASAPI capture begins only after explicit Start, for Computer, Mic, or Both. Both is the initial selection, not an instruction to record immediately. Labels describe input sources, not identified speakers. The planned Azure Ask experience is not an existing control or hidden mode. (src\PhantomFudge.App\MainWindow.xaml.cs:33,589-625; src\PhantomFudge.Core\AudioSource.cs:3-30; docs\IMPLEMENTATION-STATUS.md:5-21)

Newcomer path: read the visual-tree map, then the worked theme change. **Principal-engineer path:** inspect resource replacement, alpha compositing, passive selection, and the limits of the contrast tests. Chapter 04 explains the native capture and cursor policy; Chapter 06 explains the content retained beneath the controls.

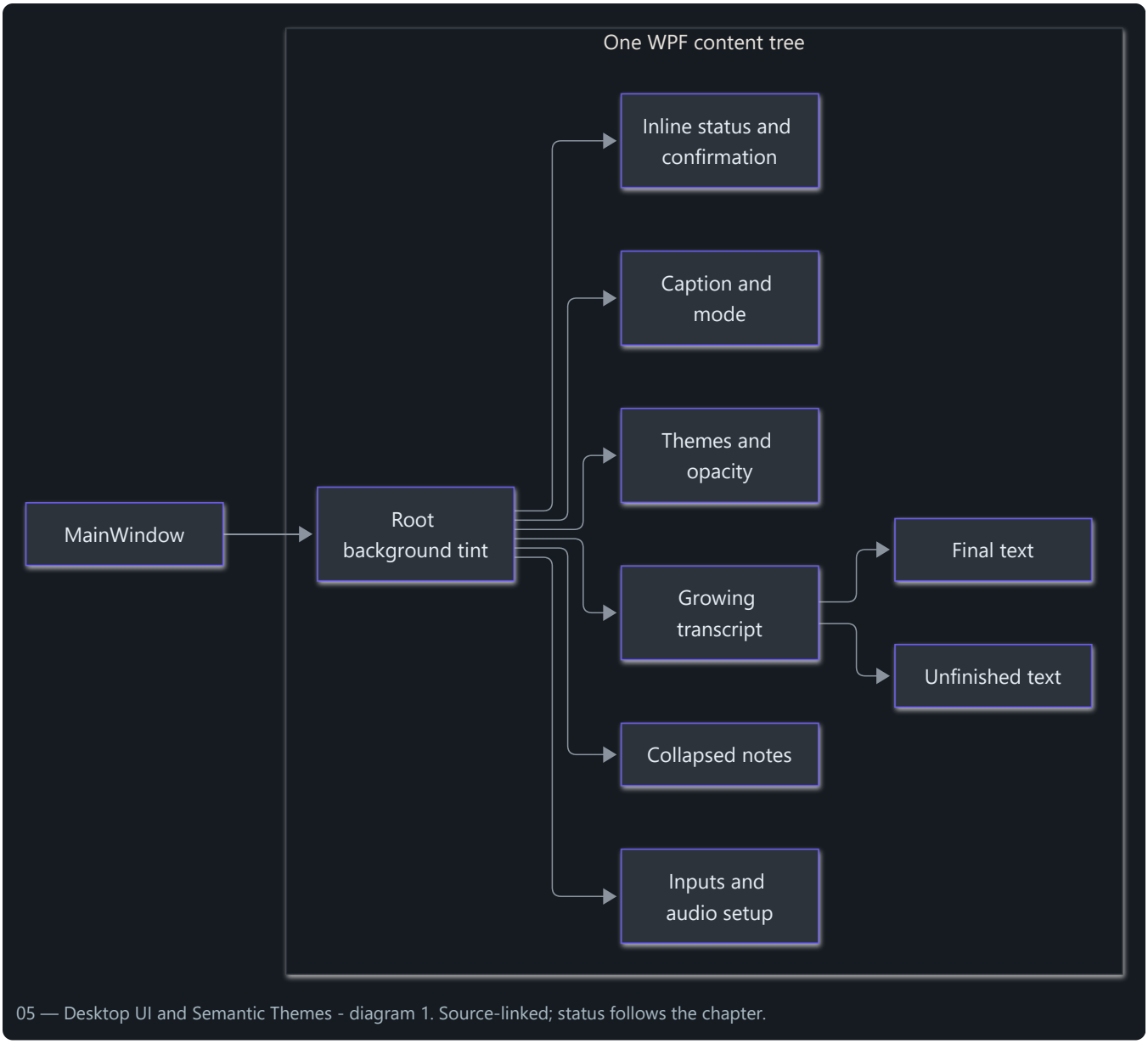


Actual WPF visual-tree render with synthetic content, not a screen-sharing capture. The fixture inserts source-labelled transcript examples without starting transcription. It demonstrates layout, not recognition accuracy, remote exclusion, or reader acceptance. (src\PhantomFudge.App\MainWindow.xaml.cs:818-859,1085-1098)

All documentation device labels are synthetic. Later fixture images may show "Wait until transcription has fully stopped before clearing": the fixture deliberately exercises that guard without opening audio inputs, then retains its status text while rendering opacity and Notes. It is not a report of a live recording. (src\PhantomFudge.App\MainWindow.xaml.cs:951-985)

Architecture: one visual tree, explicit priorities

`MainWindow.xaml` uses an eight-row grid. The caption is fixed-height; controls and optional panels size to content; the transcript row takes the remaining space with a minimum height. Expanded Notes has a 140-unit editor row rather than displacing transcription as the primary model. The default window is 620 by 760 WPF units, with minimum dimensions 470 by 560. (src\PhantomFudge.App\MainWindow.xaml:5,17-26,188-217)



05 — Desktop UI and Semantic Themes - diagram 1. Source-linked; status follows the chapter.

The caption uses `WindowChrome` with a 38-unit caption region and six-unit resize border. Its buttons opt into hit-testing inside chrome, while the title text does not intercept pointer input. The transcript and editor remain normal content hit regions; transparency is not an instruction to send clicks to another application. (src\PhantomFudge.App\MainWindow.xaml:11-40)

Components: separate meaning from painting

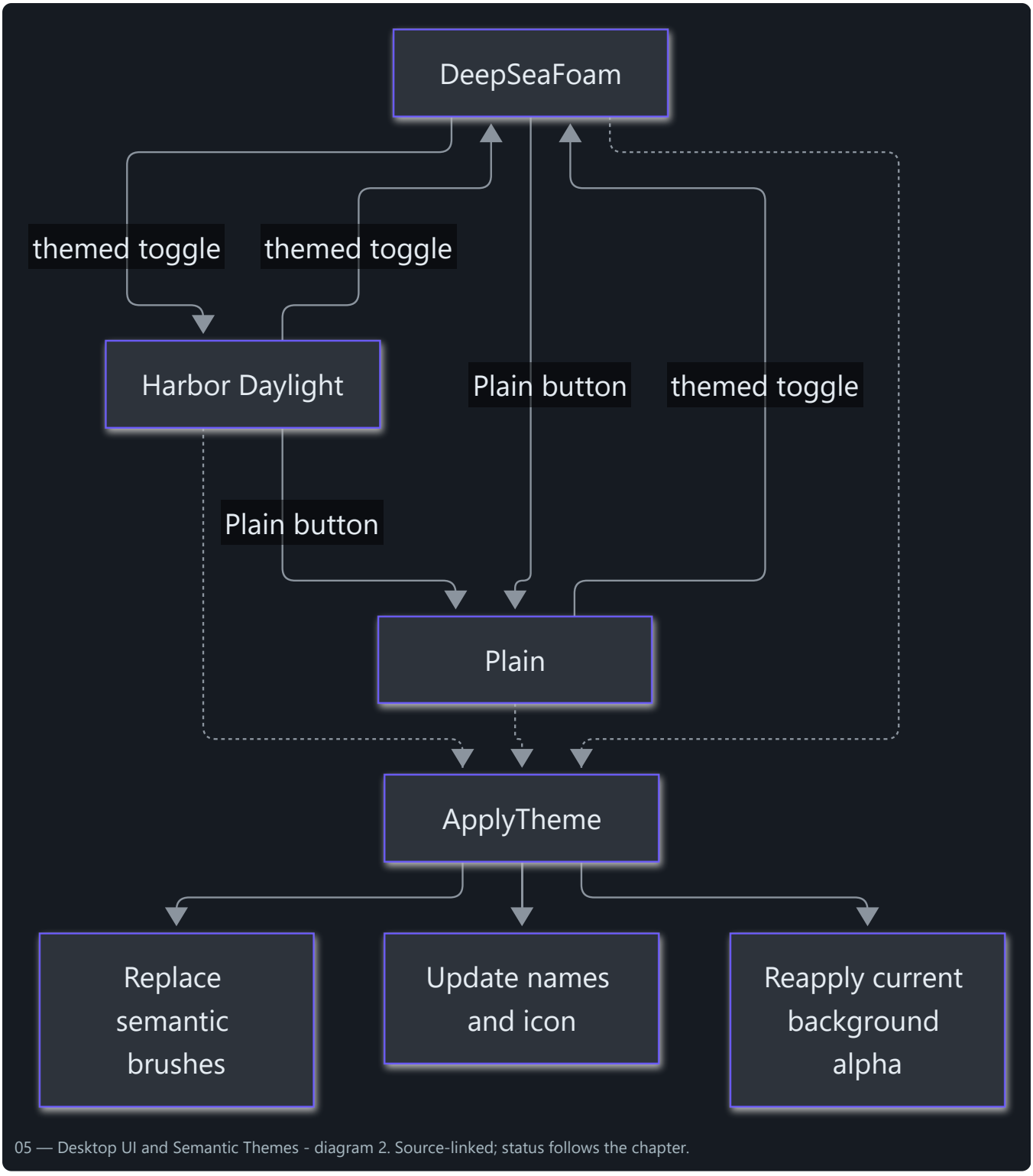
Layer	What it owns
`ThemePalette`	Immutable semantic color values and the three theme identities. (src\PhantomFudge.Core\ThemePalette.cs:3-28)
`ThemeBrushes`	Conversion to frozen WPF brushes and resource-key replacement. (src\PhantomFudge.App\ThemeBrushes.cs:9-40)
Application styles	Templates for button, text, list, scrollbar, slider, and progress states. (src\PhantomFudge.App\App.xaml:6-239)
`ApplyTheme`	Current selection, button labels, accessible name, and reapplication of root alpha. (src\PhantomFudge.App\MainWindow.xaml.cs:452-469)

Layer	What it owns
`UpdateInteractionControls`	Read-only/focus/input-method policy and file-operation availability. (src\PhantomFudge.App\MainWindow.xaml.cs:244-257)
`RunUiSmokeAsync`	Synthetic layout, style, preservation, and interaction assertions. (src\PhantomFudge.App\MainWindow.xaml.cs:818-1075)

A C# `record` is useful here because palette data is a value-shaped object rather than a control. Think of a frozen Python dataclass containing named color strings—not of a Python GUI implementation. The WPF-specific brush conversion stays in the App project, while Core tests can inspect the palette without constructing a window. (src\PhantomFudge.Core\ThemePalette.cs:5-25; src\PhantomFudge.App\ThemeBrushes.cs:9-40; tests\PhantomFudge.Core.Tests\ThemePaletteTests.cs:5-17)

Data flow: what a theme click actually changes

The themed button calls `NextThemed`. DeepSeaFoam advances to Harbor Daylight; every other current value advances to DeepSeaFoam. Plain has a separate button, so the themed toggle is a two-theme companion switch with an exit from Plain, not a three-item carousel. (src\PhantomFudge.Core\ThemePalette.cs:27-28; src\PhantomFudge.App\MainWindow.xaml.cs:467-469)



05 — Desktop UI and Semantic Themes - diagram 2. Source-linked; status follows the chapter.

`ThemeBrushes.Apply` replaces semantic resources such as `ThemeText`, `ThemeSurface`, `ThemeSelection`, and `ThemeWarning`; it also maps selected WPF `SystemColors` resource keys. Controls reference these through `DynamicResource`, allowing replacement to update existing controls. Each new brush is frozen, so later changes replace a resource rather than mutating a shared brush in place. (src\PhantomFudge.App\ThemeBrushes.cs:12-40; src\PhantomFudge.App\App.xaml:9-11,53-58)

`ApplyTheme` updates the displayed current name, the *next* theme's button label, a sun/moon glyph, and the button's automation name. It emphasizes the Plain button when selected, then rebuilds the root background using the existing slider value. It does not assign note text, clear the transcript, start audio, or recreate the

HWND. (src\PhantomFudge.App\MainWindow.xaml.cs:452-465)

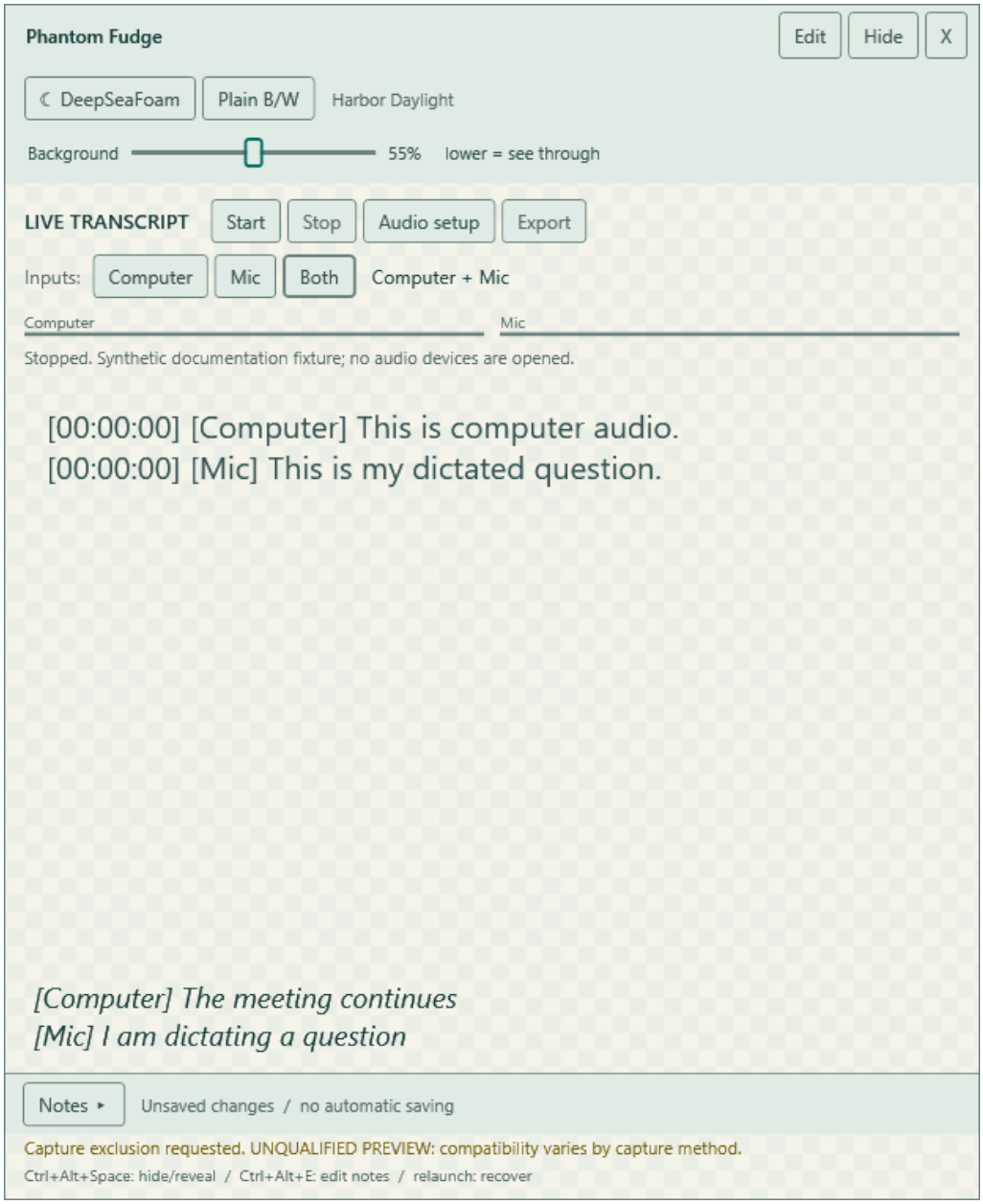
Implementation and tradeoffs

Semantic palettes include interaction states

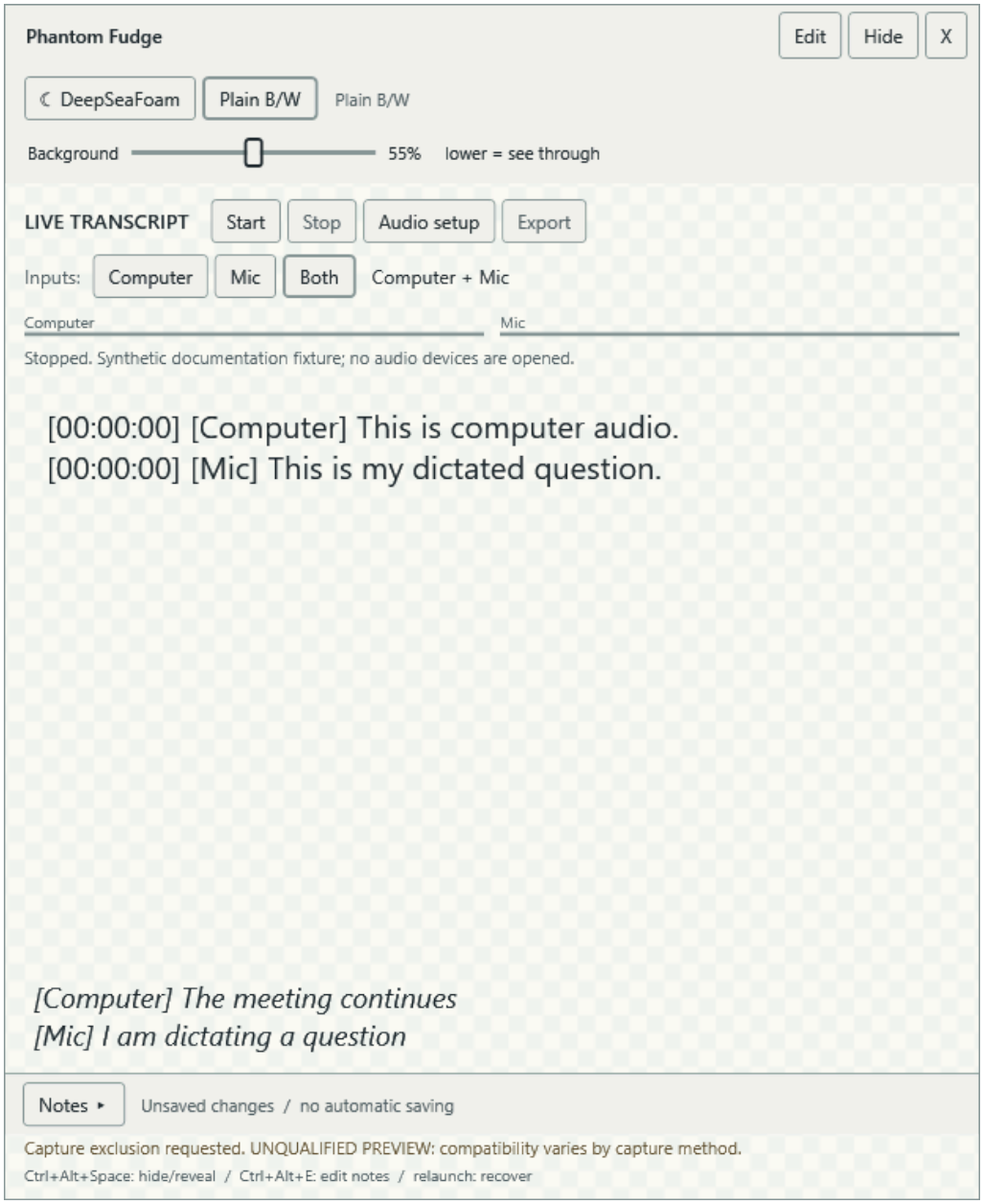
The following are selected roles, not the entire palette:

Role	DeepSeaFoam	Harbor Daylight	Plain B/W
Background	`#000F13`	`#F3F2E9`	`#FAF9F6`
Surface	`#001E26`	`#E3ECE7`	`#F0F0EC`
Reading text	`#93A1A1`	`#355451`	`#202B30`
Accent	`#00A591`	`#006F63`	`#202B30`
Selection	`#003748`	`#C7DDD7`	`#D7E1DC`

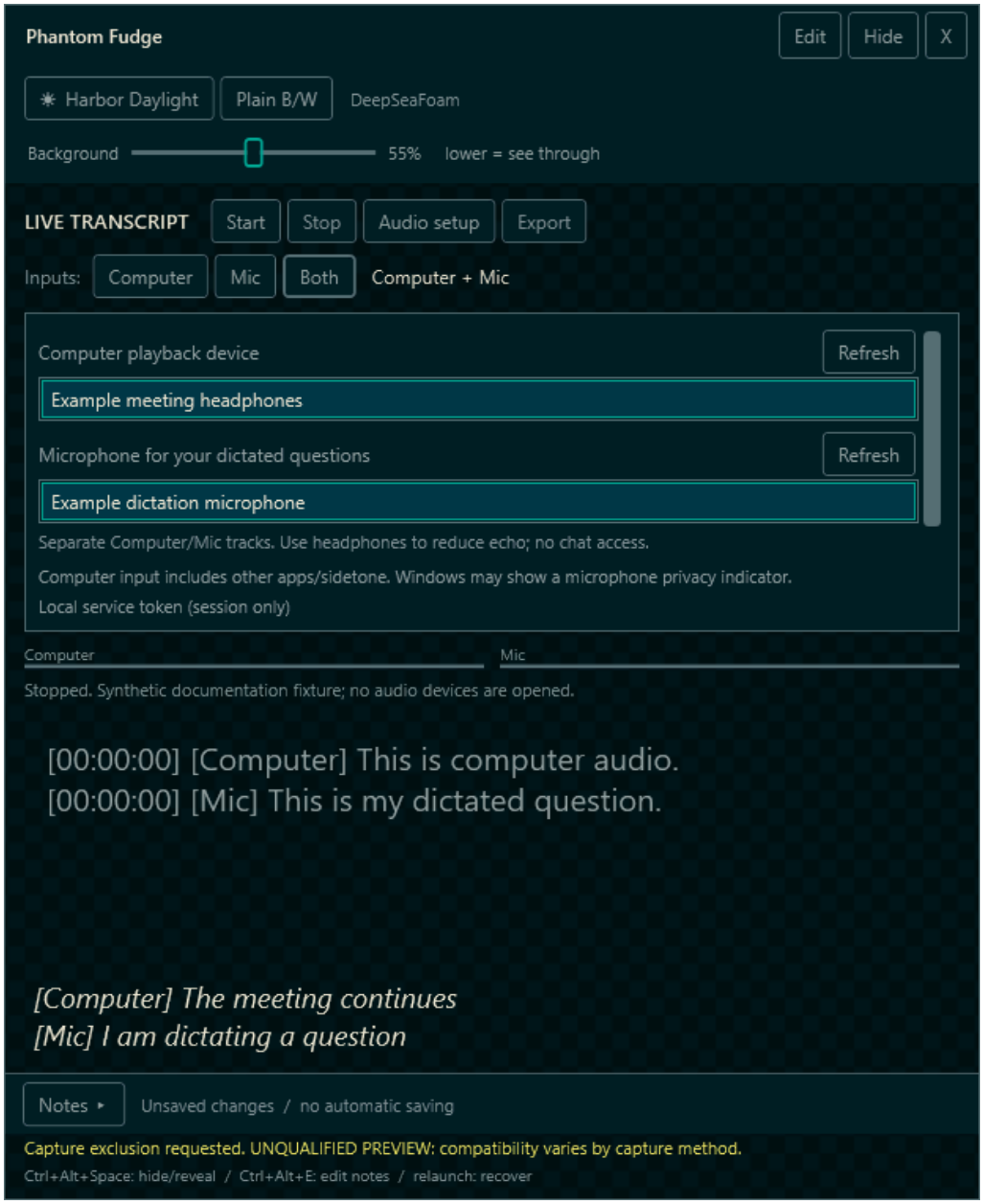
`ThemePalette.For` defines these values along with muted, heading, border, hover, selected text, action text, input, warning, and error roles. Plain B/W is a neutral interface name, not literally only two RGB values. The maintained provenance document records the canonical palette revision and composites; the WPF mapping is original to this app. (src\PhantomFudge.Core\ThemePalette.cs:5-24; docs\CURSOR-AND-THEMES.md:57-79)



Actual WPF visual-tree render with synthetic content, not a screen-sharing capture. `RunUiSmokeAsync` applies every theme and checks key resource and foreground colors before rendering the examples. (src\PhantomFudge.App\MainWindow.xaml.cs:872-884)



Actual WPF visual-tree render with synthetic content/device labels. Plain B/W is the neutral reset, not a claim that every pixel is exactly black or white. The stopped fixture status makes clear that choosing Both does not mean either source is recording. The corresponding DeepSeaFoam theme-only image is retained separately from the opening transcript fixture in the provenance manifest.



Actual WPF visual-tree render with synthetic content/device labels, not a desktop capture. Computer and microphone selectors are separate; the setup panel scrolls rather than creating a second window. The token is blank. No physical input was opened to produce this view. (src\PhantomFudge.App\MainWindow.xaml.cs:833-849,891-893)

Button templates distinguish hover, pressed, keyboard-focused, and disabled states. List items have separate hover and selected presentations. Scrollbars and sliders use app brushes rather than leaving conspicuous system-default fragments. Text boxes suppress their context menu and spell checking, reducing app-generated auxiliary surfaces without removing ordinary deliberate editing. (src\PhantomFudge.App\App.xaml:25-58,84-109,146-225)

Background alpha is not whole-window opacity

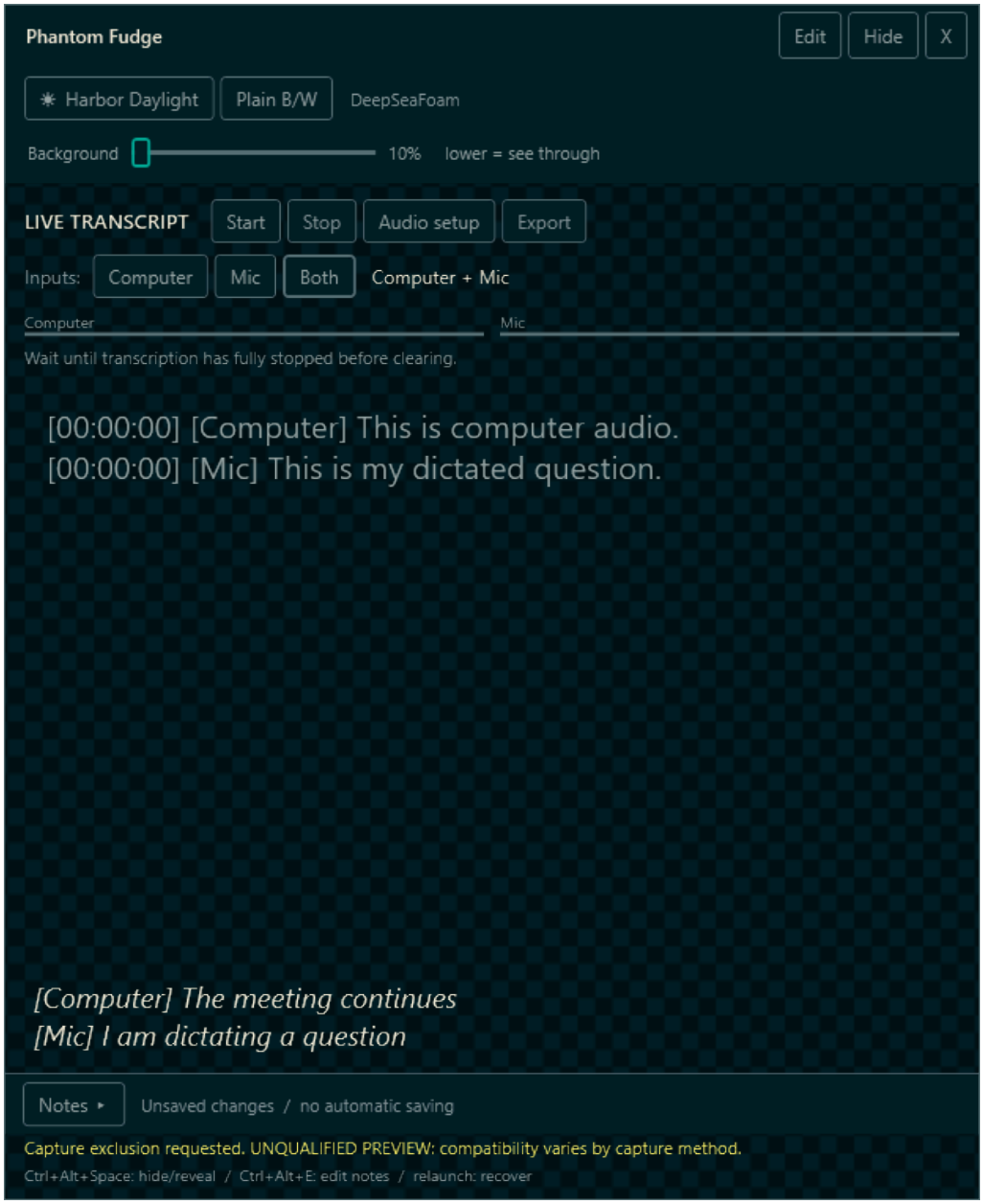
The root `WindowSurface` receives a new brush with:

```
alpha = round(clamp(percent, 10, 100) × 255 / 100)
```

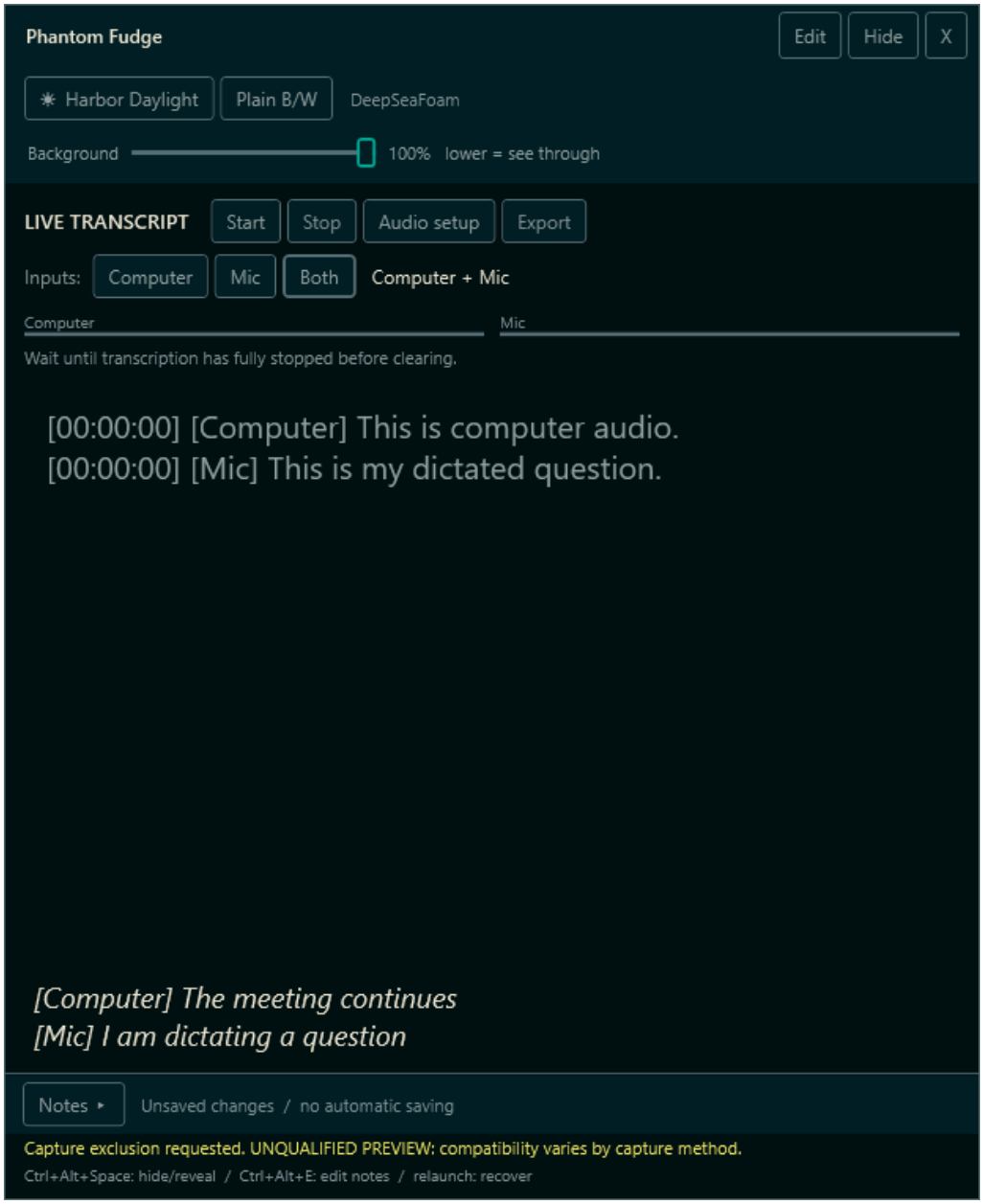
At 10%, 55%, and 100%, the byte alpha is respectively 26, 140, and 255. `MainWindow` remains transparent behind that tint; note and final-transcript control backgrounds are also transparent. Header, setup, and confirmation surfaces use opaque theme brushes. This avoids repeatedly applying translucent fills under the same text and avoids fading glyphs with the background. (src\PhantomFudge.App\MainWindow.xaml.cs:442-449; src\PhantomFudge.App\MainWindow.xaml:7,27,70-72,105-106,168-171,211-216)

The slider exposes 10–100%, defaults to 55%, and snaps to five-percent ticks. It remains available in Present. The implementation does not set `Window.Opacity`; the diagnostic explicitly checks window/editor/transcript opacity equals one and text brush alpha equals 255. (src\PhantomFudge.App\MainWindow.xaml:57-66; src\PhantomFudge.App\MainWindow.xaml.cs:977-981,1005-1008)

Why stop at 10% rather than zero? The maintained design calls out nonzero background alpha as avoiding entirely transparent mouse holes while keeping the surface usable. Why not promise readable text over every background? Composited contrast depends on the underlying desktop: approximately `alpha × tint + (1 − alpha) × background`. Solid glyphs alone cannot prevent visual interference from a busy slide. Increasing opacity or moving the window is a usability choice, not a capture-policy change. (docs\ARCHITECTURE.md:179-183; docs\CURSOR-AND-THEMES.md:51-55)



Actual WPF visual-tree render with synthetic content at10% background alpha. The reader's checkerboard is an illustrative backing for transparent PNG pixels, not a captured desktop. Header/control opacity stays unchanged.



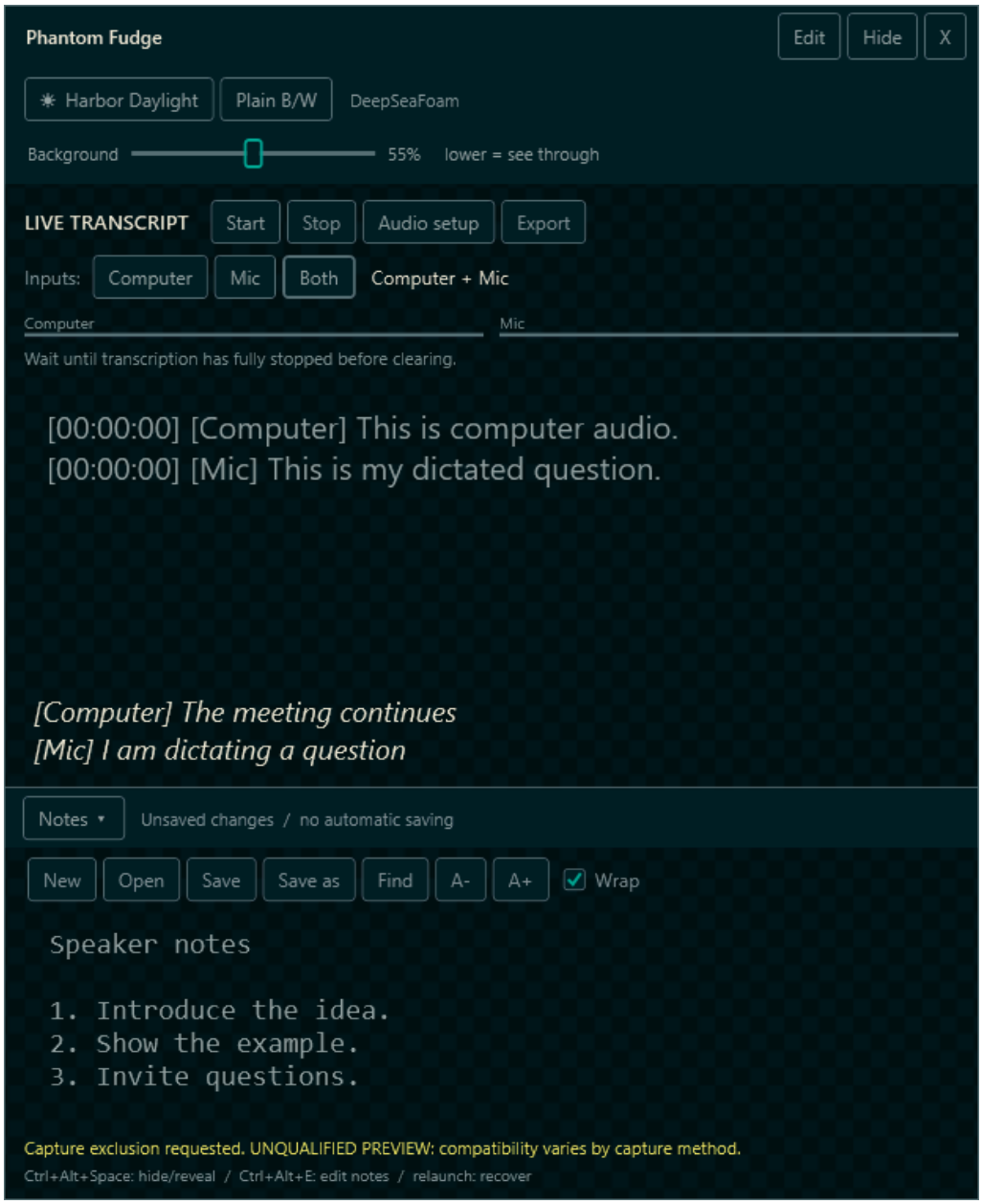
The same synthetic fixture at100% background alpha. Compare background coverage, not whether a recorder can capture the window. Both images come from the diagnostic alpha/render assertions; their retained stop-guard message is explained above. (src\PhantomFudge.App\MainWindow.xaml.cs:951-981)

Passive interaction requires more than read-only text

`UpdateInteractionControls` changes the shared `ControlsFocusable` resource, note editability, transcript focusability, and editor input-method availability. File operations and export require Edit; token entry additionally requires idle audio. Start is gated by technical readiness and explicit user action, not permission or rehearsal checkboxes. (src\PhantomFudge.App\MainWindow.xaml.cs:244-257,589-625,641-661)

The device selectors illustrate a subtle WPF issue: their ordinary click path requires focus. In Present, `OutputDevices_PreviewMouseLeftButtonDown` identifies the clicked `ListBoxItem`, assigns `SelectedItem` directly, and handles the event. Both playback and microphone lists use this handler. This preserves pointer selection without depending on keyboard focus transfer. (src\PhantomFudge.App\MainWindow.xaml.cs:568-578; src\PhantomFudge.App\MainWindow.xaml:114-125)

Opening Audio setup collapses Notes. Expanding Notes closes Audio setup; collapsing Notes also closes Find. These actions change visibility, not stored text. In contrast, switching Edit/Present hides and stops transcription before reconfiguring the native window. A theme click must not accidentally be implemented through that heavier mode-transition path. (src\PhantomFudge.App\MainWindow.xaml.cs:223-242,471-478,581-586)



Actual WPF visual-tree render with synthetic content, not a screen-sharing capture. Expansion is shown while still presenting; it does not itself enable editing. (src\PhantomFudge.App\MainWindow.xaml.cs:982-985)

The separate Present-after-mode-switch image preserves the same synthetic note/transcript content after the fixture explicitly enters Edit and returns to Present. It illustrates the stable surface, not keyboard-focus or recipient behavior by itself. (src\PhantomFudge.App\MainWindow.xaml.cs:1005-1009)

Worked example: changing appearance without changing the session

Imagine a reader using DeepSeaFoam at 55% with Notes collapsed. They click the themed button, lower Background to 10%, then expand Notes. The theme becomes Harbor Daylight, the root tint becomes alpha 26 in the new background color, and the existing notes become visible but remain read-only. None of these handlers starts or stops capture; an already-running session is not intentionally restarted. (src\PhantomFudge.App\MainWindow.xaml.cs:442-482)

This Python-shaped sketch explains the separation of values and presentation; it is **not production code**:

```
def apply_theme(theme):
    palette = palettes[theme]          # Like the C# record.
    resources.replace_colors(palette) # Like DynamicResource replacement.
    ui.current_name = palette.name
    ui.next_name = palettes[next_themed(theme)].name
    ui.root_brush = rgba(palette.background, opacity_percent)
    # No document assignment, audio command, or HWND reconstruction.
```

In the actual C# path, `ApplyTheme` calls `ThemeBrushes.Apply`, and brush construction calls `Freeze`. The slider handler only updates the root brush. That small separation makes it possible to test color selection independently of native capture, while still checking the integrated window for text preservation and stable handle identity. (src\PhantomFudge.App\ThemeBrushes.cs:9-40; src\PhantomFudge.App\MainWindow.xaml.cs:452-465,976-981,1010-1013)

Validation, exercises, and open questions

`ThemePaletteTests` checks canonical roles, toggle behavior, invalid theme rejection, and contrast of three opaque pairs: reading text/background, selection text/selection, and action text/accent. Its luminance calculation linearizes sRGB channels and requires a ratio of at least 4.5. It does not evaluate every control state, arbitrary backgrounds, or human comfort. (tests\PhantomFudge.Core.Tests\ThemePaletteTests.cs:5-58)

`RunUiSmokeAsync` checks passive device selection, theme resources, alpha samples, scrolling, caption/resize/editor hit tests, and preservation across visibility/mode operations. The documentation mode substitutes generic device names and synthetic text. PNGs use `RenderTargetBitmap` on the actual visual tree; they are not desktop screenshots with the underlying application composited behind them. No live capture is necessary to understand these fixtures. (src\PhantomFudge.App\MainWindow.xaml.cs:836-859,894-1008,1085-1098)

Exercise: Why not implement translucency as `Opacity = 0.1` on the window?
Answer: That would fade text and controls as well as the background, violating the implemented solid-text invariant.

Exercise: After Plain, which theme does the companion button select?
Answer: DeepSeaFoam; `NextThemed` has only one special case, DeepSeaFoam → Harbor Daylight.

Review question: What still needs people and hardware?
Answer: Readability over real backgrounds, accessibility and mixed-DPI behavior, presentation-focus acceptance, and recipient-side content/cursor qualification. Local tests do not supply owner approval. (src\PhantomFudge.App\MainWindow.xaml.cs:977-981; src\PhantomFudge.Core\ThemePalette.cs:27-28; docs\IMPLEMENTATION-STATUS.md:149-157)

References and maintenance checkpoints

- Main window layout and interaction code.
- Application templates: state styling and dynamic resources.
- Palette values, brush adapter, and contrast tests.
- Source labels and palette provenance.

Theme and opacity choices are session-only. Persisted preferences and broader accessibility acceptance must not be inferred from the current controls. Any future popup, template, or rendering change should revisit both local usability assertions and the capture qualification matrix. (docs\CURSOR-AND-THEMES.md:47-55; docs\QUALIFICATION.md:113-130)

06 — Notes, Transcripts, and Explicit Storage

Overview: persistence is a deliberate boundary

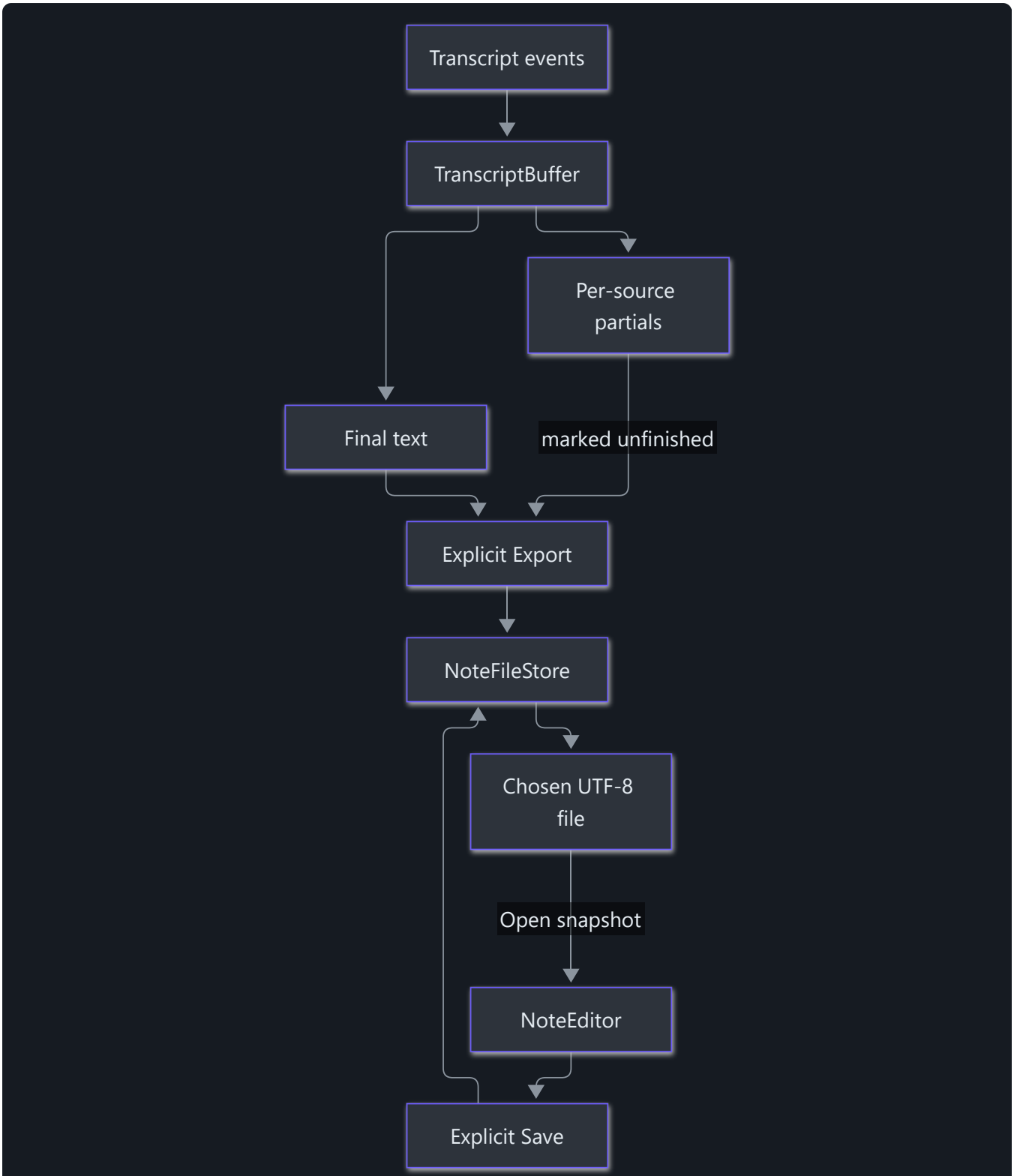
Seeing text, retaining it in memory, and saving it are three different events. Phantom Fudge keeps one editable note and an in-memory transcript; neither is automatically a file. The note tracks its last successful saved value. The transcript tracks its last explicitly exported value. These distinctions let the UI warn about unsaved work without silently persisting meeting content. They also mean a crash can lose unexported text: the preview does not implement automatic recovery. (src\PhantomFudge.App\MainWindow.xaml.cs:22-30,267-283,384-391,753-760; docs\IMPLEMENTATION-STATUS.md:95-100,156-157)

Capture exclusion does not extend to a `.txt` file, the clipboard, a native file dialog, or process memory. File storage is therefore designed for bounded input and accidental-overwrite resistance, not advertised as a secure vault. Saving to a user-selected location is an explicit act with the ordinary privacy properties of that location. (src\PhantomFudge.App\MainWindow.xaml.cs:350,391; docs\PRODUCT.md:125-140)

Newcomer path: start with the transcript example, then follow Save. **Principal-engineer path:** inspect session-local ID reset, snapshot comparisons, byte bounds, and the remaining check-to-replace race. Chapter 11 explains why session reset must wait for final draining.

Architecture: two content models, one file primitive

Notes use the WPF text editor as their current text surface. `TranscriptBuffer` separately owns accumulated final text and source-specific unfinished text. Both explicit note saves and transcript exports use `NoteFileStore`; no recognizer callback writes a document file. (src\PhantomFudge.App\MainWindow.xaml.cs:267-268,359-404,711-768)



06 — Notes, Transcripts, and Explicit Storage - diagram 1. Source-linked; status follows the chapter.

Components and units

Component or value	Contract
`NoteSnapshot`	Text, UTF-8 BOM presence, and SHA-256 fingerprint of original bytes. (src\PhantomFudge.Core\NoteFileStore.cs:6,41-44)

Component or value	Contract
`NoteFileStore.MaximumBytes`	1,048,576 encoded bytes, including a three-byte BOM when present. (src\PhantomFudge.Core\NoteFileStore.cs:12-19)
`TranscriptBuffer.MaximumCharacters`	250,000 .NET character units for accumulated final/session text; partials are separately bounded. (src\PhantomFudge.Core\TranscriptBuffer.cs:8-10,23-45,63-66)
`TranscriptEvent`	Type, text, optional ID/timestamps/source; an event, not a speaker identity. (src\PhantomFudge.App\Audio\TranscriptEvent.cs:9-15)
`AudioSourceNames`	Wire names `computer` / `microphone`; display names `Computer` / `Mic`. (src\PhantomFudge.Core\AudioSource.cs:16-30)
`savedText` / `exportedTranscript`	Successful persistence baselines used for equality-based dirty checks. (src\PhantomFudge.App\MainWindow.xaml.cs:26-27,267-268)

These units must not be conflated. A C# string length counts UTF-16 code units, not UTF-8 bytes or human-perceived characters. `Fits` asks the strict encoder for its actual byte count, including BOM allowance; unpaired surrogates fail rather than becoming replacement characters. The paste handler checks the *resulting note after selection replacement*, while saving checks again. This is not a universal keystroke-by-keystroke size limiter on the editor. (src\PhantomFudge.Core\NoteFileStore.cs:13-24; src\PhantomFudge.App\MainWindow.xaml.cs:428-436; src\PhantomFudge.App\MainWindow.xaml:211-217)

Data flow: recognition becomes a readable history

Transport events are dispatched to the UI thread and handled by `ReceiveTranscript`. A partial calls `SetPartial`; a final calls `AddFinal`. The UI then refreshes final text, scrolls to the end, updates the partial line, and hides the placeholder when content exists. Invalid data or a full transcript stops transcription and preserves the existing buffer for export. (src\PhantomFudge.App\MainWindow.xaml.cs:55,149-153,711-729)

Partials replace a dictionary entry rather than append to history. Keys distinguish Computer and Mic; a legacy unspecified source has its own key. Display ordering is by source key, not arrival time. A final removes only its source's partial. This prevents a Computer sentence from erasing a simultaneously unfinished dictated question. (src\PhantomFudge.Core\TranscriptBuffer.cs:14-19,41-46,69-71,87-96)

Final IDs are globally increasing within a session, not independently increasing per source. `AddFinal` ignores an ID no greater than the last accepted ID *before* removing any partial. An empty valid final advances the ID and clears its source's partial without appending an empty transcript line. The socket is stricter at the protocol boundary: non-increasing received IDs are invalid messages, rather than harmless deduplication. (src\PhantomFudge.Core\TranscriptBuffer.cs:48-72; src\PhantomFudge.App\Audio\TranscriptionSocket.cs:437-457)

Timestamps are optional in the buffer and rendered as hours/minutes/seconds. Hours do not wrap at 24. Source timestamps may differ because they describe separate input timelines; the display appends received finals, not a globally time-sorted speaker conversation. (src\PhantomFudge.Core\TranscriptBuffer.cs:58-71; tests\PhantomFudge.Core.Tests\TranscriptBufferTests.cs:85-90; tests\PhantomFudge.Core.Tests\SourceTranscriptTests.cs:13-19)

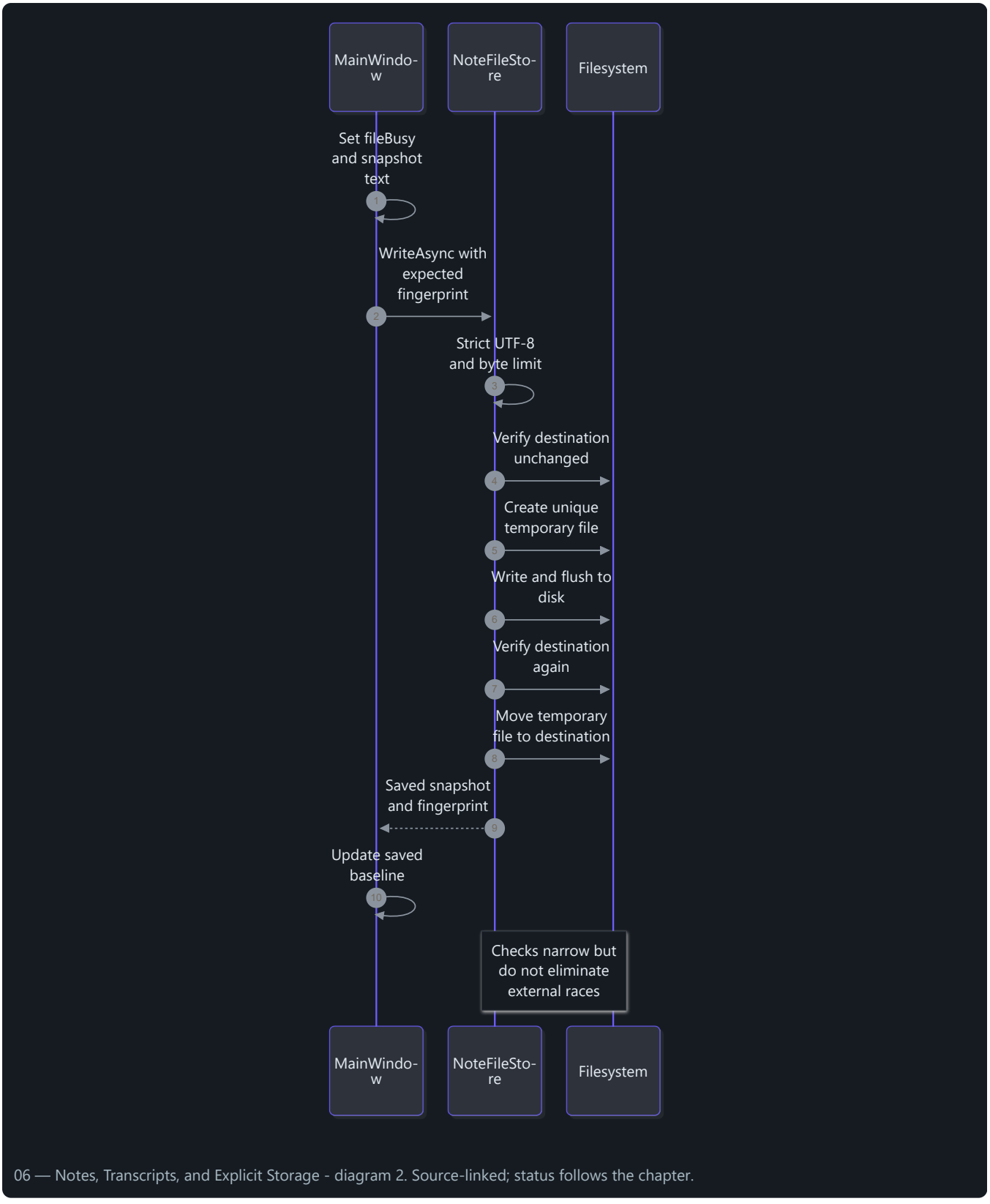
Session boundaries preserve unfinished work

`BeginSession` first calculates the space needed to preserve old partials as `[Unfinished recognition]` lines and append `--- New session ---` when appropriate. Only after the capacity check succeeds does it append, reset the last ID to zero, and clear partial entries. Failure leaves the previous content available for export. A new Start can therefore retain earlier transcript history without pretending unfinished words were finalized. (src\PhantomFudge.Core\TranscriptBuffer.cs:23-39; tests\PhantomFudge.Core.Tests\TranscriptBufferTests.cs:40-62)

Each partial/final text value is limited to 16,384 character units. Final/session accumulation is capped, but `ExportText` can additionally include the separately bounded partials and markers; 250,000 is not an exact total-memory budget. The buffer uses a `StringBuilder`, while properties materialize strings for the UI. This is bounded content management, not a zero-allocation streaming renderer. (src\PhantomFudge.Core\TranscriptBuffer.cs:8-19,41-66,75-77)

File implementation: validate, stage, verify, rename

`ReadAsync` reads in 16,384-byte chunks and enforces the cap as data arrives rather than trusting a pre-read file length. It recognizes an optional UTF-8 BOM, decodes strictly, and fingerprints the complete original byte sequence. It does not normalize line endings or guess UTF-16. The file is opened with `FileShare.ReadWrite`, so this is not a locked snapshot against a concurrent external writer. (src\PhantomFudge.Core\NoteFileStore.cs:27-44)



06 — Notes, Transcripts, and Explicit Storage - diagram 2. Source-linked; status follows the chapter.

`WriteAsync` stages bytes in `.phantom-<guid>.tmp` beside the destination, with exclusive sharing and `CreateNew`. It flushes asynchronously and then requests a disk flush. After a second fingerprint check and cancellation check, it calls `File.Move` with overwrite allowed only when an expected fingerprint was supplied. The `finally` block attempts to delete any remaining temporary file. This is a same-directory staged/atomic-rename pattern, not an in-place rewrite of the original. (src\PhantomFudge.Core\NoteFileStore.cs:47-80)

The expected fingerprint carries two meanings:

- **Null:** the caller expects a new path. An existing file is a conflict, and the move must not overwrite.
- **Non-null:** the caller expects particular existing bytes. Missing or changed bytes are a conflict.

For Save As over an existing file, the UI first obtains that file's current fingerprint after the user's dialog decision. A later conflict reports that edits remain in memory and recommends Save As, without silently choosing whose version wins. (src\PhantomFudge.Core\NoteFileStore.cs:83-95; src\PhantomFudge.App\MainWindow.xaml.cs:368-398)

The tradeoff is optimistic coordination. A writer can still change the destination between the final verification and move. The code provides no cross-process compare-and-swap primitive, secure erasure, encryption, directory durability guarantee, or universal network-filesystem atomicity. Temporary cleanup can itself fail. These limitations matter when interpreting “atomic save”: staged replacement avoids many partial-write failures but does not make every concurrent or catastrophic failure impossible. (src\PhantomFudge.Core\NoteFileStore.cs:60-95; TODO\08_LOCAL_STORAGE.todo:15-34)

UI snapshots and unsaved decisions

`SetFileBusy` disables the note editor and confirmation panel and updates command availability. `SaveNoteAsync` captures `NoteEditor.Text` into `content`; only a successful write changes `savedText`, path, and fingerprint. Open updates the editor only after a successful read and resets undo history. Failures retain the current note. (src\PhantomFudge.App\MainWindow.xaml.cs:333-416)

Export similarly snapshots `transcript.ExportText()` and records that exact value after success. Unlike the note editor, recognition callbacks can continue while an export is awaiting I/O in Edit mode. If new transcript content arrives, it differs from `exportedTranscript`, so it remains dirty rather than being falsely marked saved. Exports are BOM-free and include unfinished-recognition labels; they do not save raw audio. (src\PhantomFudge.App\MainWindow.xaml.cs:267-268,711-724,733-768)

Exit awaits Stop before checking transcript dirtiness. New/Open concern note dirtiness; Exit considers both. The inline confirmation offers save, export, discard, or cancel. Saving notes alone cannot finish Exit while the transcript remains dirty, and exporting alone cannot proceed while notes remain dirty. In Present, save/export are refused until deliberate Edit. Clear is also refused while running, starting, or stopping, and while transcript content is unexported. (src\PhantomFudge.App\MainWindow.xaml.cs:270-331,359-365,733-739,771-787)

Preserve content; reset the right metadata

New deliberately resets the note path, fingerprint, BOM flag, saved baseline, and editor text. It then toggles undo off and on so a new document does not inherit the previous document's undo history. Open performs an equivalent undo reset only after decoding succeeds. In contrast, collapse, theme, wrapping, and font-size changes do not represent document replacement. Keeping those categories separate prevents a visual operation from accidentally becoming a data-loss operation. (src\PhantomFudge.App\MainWindow.xaml.cs:301-310,341-349,440-441,452-484)

Dirty tracking is equality-based rather than a permanently latched “was edited” bit: returning the note to `savedText` makes it clean again. That answers whether the current in-memory text differs from the last successful save; it does **not** answer whether an external program has since edited the disk file. The fingerprint

comparison answers that second question when a save is attempted. This separation explains why the UI can show a locally clean note while a later Save reports an external conflict. (src\PhantomFudge.App\MainWindow.xaml.cs:267,384-398,424-426; src\PhantomFudge.Core\NoteFileStore.cs:83-95)

Worked examples and Python comparison

Consider this sequence, using conceptual timestamps:

```
partial Computer: "remote words"
partial Mic:      "my question"
final #1 Computer at 1.0 seconds: "remote sentence"
final #2 Mic      at 0.5 seconds: "my question"
```

The first final removes only the Computer partial. The Mic partial remains until final #2. Output order follows accepted IDs, even though the Mic start time is lower. Replaying final #1 cannot erase newer unfinished Mic text. These exact properties are exercised in `SourceTranscriptTests`. (tests\PhantomFudge.Core.Tests\SourceTranscriptTests.cs:5-19,37-55)

The following **explanatory pseudocode is not production code**. It omits encoding, cancellation, filesystem semantics, and native UI behavior:

```
def add_final(segment_id, source, words):
    if segment_id <= last_id:
        return False
    line = format_line(source, words) if words else ""
    check_capacity_before_mutation(line)
    last_id = segment_id
    partials.pop(source, None)
    history.append(line)

async def save_snapshot(text, expected_hash):
    verify_current_bytes(expected_hash)
    temporary = await write_and_flush_beside_destination(text)
    verify_current_bytes(expected_hash)
    rename_to_destination(temporary)
```

The C# `Dictionary` provides source-keyed replacement, `StringBuilder` accumulates history, and `await using` scopes asynchronous file disposal. Two fingerprint checks are an intentional narrowing of a race window, not proof that the window has vanished. (src\PhantomFudge.Core\TranscriptBuffer.cs:9-10,48-72; src\PhantomFudge.Core\NoteFileStore.cs:60-80)

Tests, exercises, and remaining limits

`NoteFileStoreTests` covers Unicode and mixed-newline/BOM round trips, changed/deleted destinations, invalid UTF-8, oversized reads/writes, byte accounting, and pre-cancelled writes. Transcript tests cover deduplication, unfinished preservation, long timestamps, full-buffer behavior, and independent sources. These

are deterministic policy checks, not exhaustive disk-full, permission, crash-recovery, or filesystem-race qualification. (tests\PhantomFudge.Core.Tests\NoteFileStoreTests.cs:13-109; tests\PhantomFudge.Core.Tests\TranscriptBufferTests.cs:5-115; tests\PhantomFudge.Core.Tests\SourceTranscriptTests.cs:5-87; TODO\08_LOCAL_STORAGE.todo:25-34)

Exercise: Why fingerprint bytes rather than only compare decoded text?
Answer: The byte fingerprint also distinguishes representation changes such as a BOM. The snapshot returns text and BOM separately.

Exercise: Is `[Mic]` a person identifier?
Answer: No; it is the selected microphone track's display label.

Exercise: Does collapsing Notes save or discard them?
Answer: Neither. The handler changes visibility; persistence remains explicit.

The answers follow `ReadAsync`, `AudioSourceNames.DisplayName`, and `SetNotesExpanded`, respectively. (src\PhantomFudge.Core\NoteFileStore.cs:41-44; src\PhantomFudge.Core\AudioSource.cs:25-30; src\PhantomFudge.App\MainWindow.xaml.cs:471-478)

References and maintenance checkpoints

- File store and file tests.
- Transcript buffer, source definitions, and event record.
- UI commands, buffer tests, and source-isolation tests.
- Storage workstream: open fault-injection and recovery gates.

Changing encoding, session IDs, export format, or automatic persistence would change the contracts in this chapter. In particular, planned recovery must remain labelled unimplemented until its retention, failure, and reveal behavior actually exists. (docs\ARCHITECTURE.md:227-241)

Source: docs/handbook/07-dual-input-capture.md

7. Dual-input capture: two sources, one owned session

Why two inputs must not become one waveform

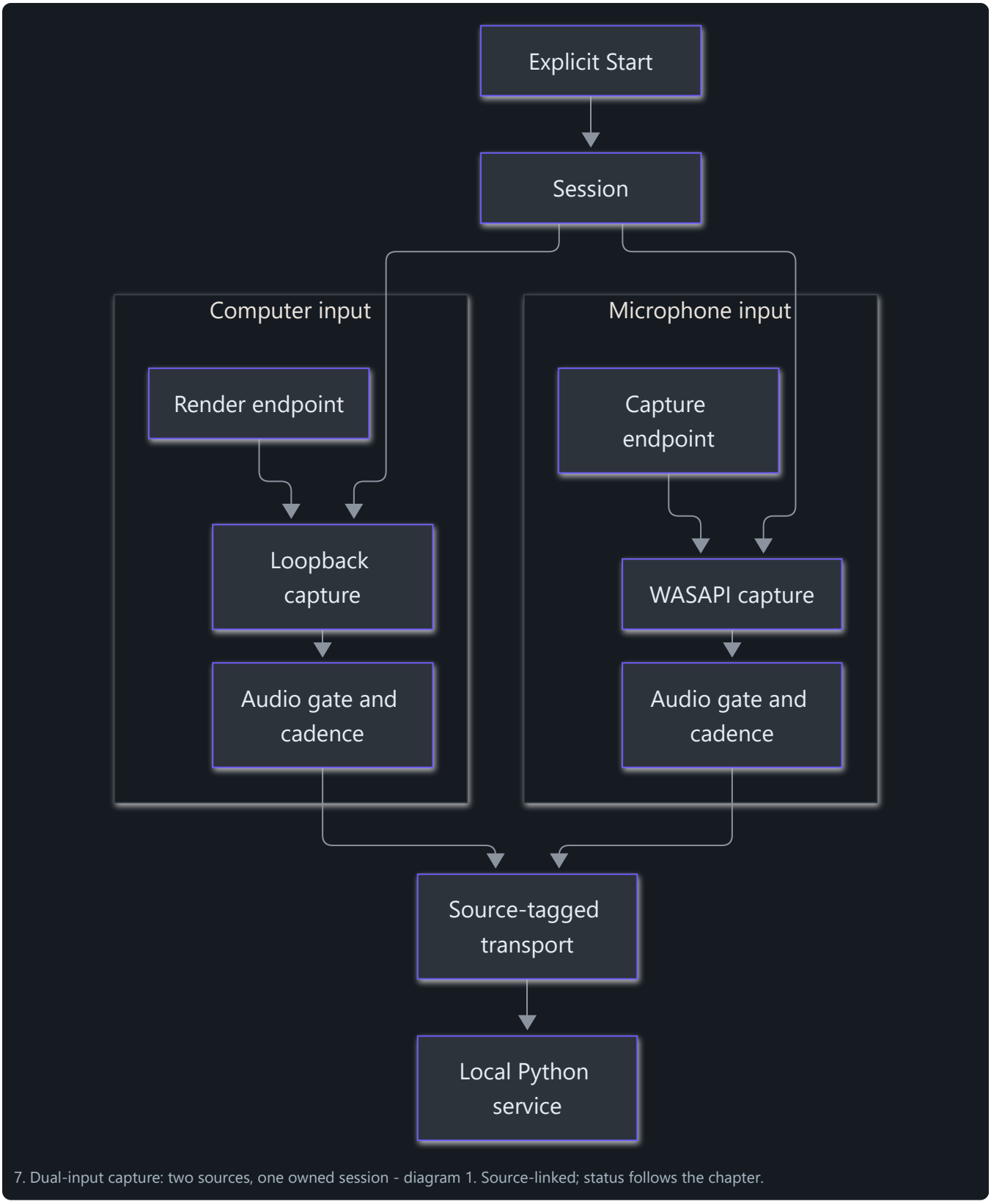
Imagine listening to computer playback while dictating a question into a microphone. Adding the samples together would discard the distinction the reader needs: which text came from playback, and which came from dictation? Phantom Fudge instead preserves two acquisition paths. `WasapiInput` selects render loopback for Computer and a capture endpoint for Microphone; each becomes a separate source-tagged stream. This is input attribution, **not speaker identification**. Several people can speak through Computer, and playback can acoustically reappear in Mic. The implementation contains no mixing or acoustic-echo-cancellation stage. (`src\PhantomFudge.App\Audio\WasapiInput.cs:20-55`, `WasapiInput`; `docs\DUAL-SOURCE-AUDIO.md:8-23`, "Design and boundaries".)

The equally important distinction is **selection versus capture**. `MainWindow` defaults `captureMode` to `Both`, populates device lists, and subscribes to notifications; its constructor does not start recording. `Start_Click` checks the requested selections, protection readiness, and token before calling the transcriber. Merely launching the idle Python service is not permission to acquire audio. (`src\PhantomFudge.App\MainWindow.xaml.cs:33-76`, `MainWindow`; `src\PhantomFudge.App\MainWindow.xaml.cs:589-625`, `Start_Click`; `scripts\Run-PhantomFudge.ps1:39-55`, launcher.)

This chapter follows that boundary from a deliberate Start to the last endpoint release. Chapter 8 continues at the socket; chapter 9 follows the independent recognizers.

The ownership map

The class name `LoopbackTranscriber` preserves its computer-only history, but its `Session` now owns an array of `Input` objects. Each input owns its endpoint ID, callback handlers, audio lock, silence cadence, startup task, recording-stopped completion, and latest level. The session owns one transport and shared cancellation/failure state. This divides mutable audio state without dividing the user's Start/Stop transaction. (`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:472-519`, `Input` and `Session`.)



7. Dual-input capture: two sources, one owned session - diagram 1. Source-linked; status follows the chapter.

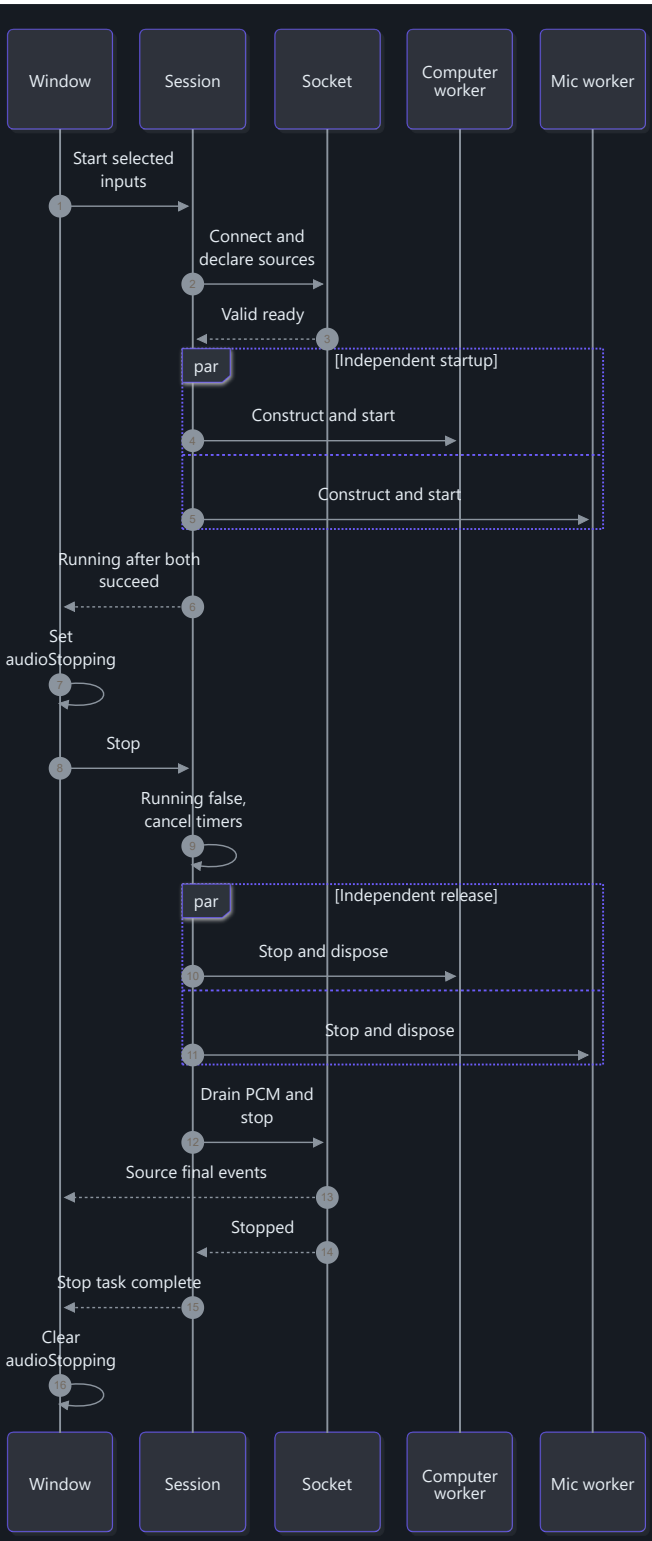
The diagram describes ownership and data flow, not synchronized acoustic clocks. The endpoint branches are constructed independently after transport readiness.
(`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:98-124` , `StartCoreAsync` .)

API or type	Exact role and consequence
`AudioSource`	Byte values `Computer = 1`, `Microphone = 2`; wire/display names are centralized. (`src\PhantomFudge.Core\AudioSource.cs:3-30`, `AudioSourceNames`.)
`AudioInputSelection`	Nullable Computer/Microphone IDs represent the requested subset. (`src\PhantomFudge.App\Audio\TranscriptEvent.cs:5-7`, record declarations.)
`StartAsync(string, Uri, string, CancellationToken)`	Legacy Computer-only, protocol-v1 entry point. (`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:64-65`, `StartAsync`.)
`StartAsync(AudioInputSelection, …)`	Protocol v2, including either single-source mode. (`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:67-69`, `StartAsync`.)
`SourceLevelChanged`	Reports a source and its latest normalized RMS value. (`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:264-287`, `PublishLevel`, `CalculateLevel`.)
`LevelChanged`	Compatibility aggregate: maximum of latest input levels, not a mixed waveform's RMS. (`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:264-274`, `PublishLevel`.)

The Start journey: readiness before native acquisition

`ValidateSelection` rejects an empty subset, whitespace IDs, and identical IDs for both requested inputs. `WasapiInput.MatchesSelection` then requires exact ordinal ID equality, the correct direction, and `DeviceState.Active`. Enumeration does not silently choose the current Windows default at capture time. The UI may initially select the first enumerated device, but the actual start uses that explicit ID.
(`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:138-153`, `ValidateSelection`;
`src\PhantomFudge.App\Audio\WasapiInput.cs:64-74`, `MatchesSelection`;
`src\PhantomFudge.App\MainWindow.xaml.cs:529-557`, `RefreshDevices`.)

Startup first reserves a session and connects its transport. Only after `ConnectAsync` accepts the service's `ready` response are input startup tasks scheduled. Native construction itself is inside `Task.Run`, not just `StartRecording`. The source comment identifies why: NAudio captures the construction-time synchronization context, so constructing on the WPF dispatcher could make a later native join depend on the very UI thread waiting for completion. (`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:84-111`, `StartCoreAsync`.)



7. Dual-input capture: two sources, one owned session - diagram 2. Source-linked; status follows the chapter.

`Running` becomes true only when every startup succeeded, no input already reported `RecordingStopped`, no fault exists, and Stop was not requested. This is not a promise that the two devices started at the same physical instant. Audio acceptance is enabled inside each worker before its `StartRecording` call, whereas the aggregate running state is established after all startup tasks complete.
(`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:108-121`, `StartCoreAsync`; `src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:191-215`, `StartCapture`.)

A callback, measured in bytes rather than intuition

Both WASAPI wrappers request shared-mode **16,000 Hz, 16-bit, mono PCM**. Computer uses `WasapiLoopbackCapture`; Mic uses `WasapiCapture`. The wrapper documents NAudio's shared-capture conversion flags rather than implementing a custom application resampler. (`src\PhantomFudge.App\Audio\WasapiInput.cs:48-54`, constructor.)

The format yields:

- `16,000 samples/second × 2 bytes/sample = 32,000 bytes/second` per source.
- A 100 ms silence frame contains `1,600 samples = 3,200 bytes`.
- A 48,000-byte callback represents 1.5 seconds of this PCM. `OnAudio` splits it into 32,000 and 16,000 bytes; the v2 transport adds a source byte to each, producing 32,001- and 16,001-byte binary messages.
- Two live sources produce 64,000 PCM bytes/second before framing overhead—not a 32 kHz mixed track.

These are calculations from `WaveFormat`, the socket constants, and the callback chunking loop, not measured hardware callback sizes. (`src\PhantomFudge.App\Audio\WasapiInput.cs:53-54`, constructor; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:13-18`, constants; `src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:230-254`, `OnAudio`; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:208-217`, `TryEnqueueAudio`.)

`OnAudio` rejects invalid lengths, records packet arrival under that input's `AudioGate`, and enqueues copied chunks. Copying matters because a capture callback's buffer is reusable: the sender must not later observe a buffer that the capture implementation or caller has overwritten. A full transport budget faults the session rather than pretending the missing chunk was silence. (`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:235-247`, `OnAudio`; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:182-229`, `TryEnqueueAudio`.)

Metering is throttled to approximately one update per 100 ms of callback time. Samples are read little-endian, divided by 32,768, squared, averaged, and square-rooted. Alternating `+16,384` and `-16,384` therefore yields RMS `0.5`, not zero; opposite signs cancel in an arithmetic mean but not in energy. `LevelAsync` tests that exact distinction. (`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:249-286`, `OnAudio`, `CalculateLevel`; `tests\PhantomFudge.Audio.Tests\Program.cs:202-209`, `LevelAsync`.)

Silence cadence is transport continuity, not endpoint tuning

An idle track has its own 100 ms periodic timer. `SilenceCadence.TrySchedule` permits one silence frame only after at least 250 ms without a real packet and at least 100 ms since the previous synthetic frame. A real packet resets both timestamps. The shared per-input lock prevents a recent packet and silence decision from racing within that track. (`src\PhantomFudge.App\Audio\SilenceCadence.cs:5-19`, `SilenceCadence`; `src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:289-314`, `SupplySilenceAsync`.)

For example, after a packet at 1,000 ms, a decision at 1,249 ms fails; one at 1,250 ms may succeed. A delayed decision at 10,000 ms still emits only one 3,200-byte frame—it does not manufacture every missing interval. Timer scheduling can place the actual decision later than the threshold. Computer activity does not reset Mic's cadence, and vice versa. The corresponding synthetic tests check both the threshold and cross-source independence. (`tests\PhantomFudge.Audio.Tests\Program.cs:212-225`, `SilenceCadenceAsync`; `tests\PhantomFudge.Audio.Tests\DualSourceTests.cs:498-519`, `DualInputPolicyAsync`.)

Consequently, these timestamps describe **callback arrival and supplied PCM**, not acoustic onset measurements. Cadence does not change Vosk endpoint settings; its implementation only creates zero PCM. Decoder-history rotation is a separate 30-second mechanism covered in chapter 9. (`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:14-14`, `SilenceFrame`; `src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:301-309`, `SupplySilenceAsync`; `services\transcription\recognizer.py:82-93`, `VoskRecognizer.accept`.)

Cancellation must invalidate tomorrow's queued Start

A semaphore alone would serialize requests but could allow a Start already queued behind another operation to run after the user's Stop. `_stopVersion` solves that ordering problem. Start snapshots it before waiting; Stop increments it before waiting for lifecycle ownership. A queued Start with an old version is rejected before allocating its session. (`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:71-91`, `StartCoreAsync`; `src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:155-188`, `StopAsync`, `RequestStop`.)

The following Python-shaped pseudocode translates that C# algorithm; it is not another implementation:

```
async def start(selection):
    observed_stop_version = atomic_read(stop_version)
    async with lifecycle:
        require_not_disposed()
        require_no_session_or_outstanding_release()
        inputs = validate(selection)
        with request_lock:
            if observed_stop_version != stop_version:
                raise StartupCanceled()
            session = reserve(inputs)
        await connect_and_validate_ready(session)
        await start_each_input_on_workers(session)

async def stop():
    with request_lock:
        increment_stop_version()
        request_current_session_stop()
    async with lifecycle:
        await finish_current_session()
```

`V2QueuedStartAsync` tests both Stop and Dispose against a pending handshake and a second queued Start. Its important assertion is not merely “eventually stopped”: only one handshake occurred, and no hardware seam was reached. (`tests\PhantomFudge.Audio.Tests\DualSourceTests.cs:471-496`, `V2QueuedStartAsync`.)

Stop is a drain, while failure is an abort

Boundary	Implemented behavior
Normal Stop	Clear `Running`, cancel startup/silence, release inputs, then dispose the transport through its drain path. (`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:363-423`, `StopSessionAsync`, `FinishSessionAsync`; `src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:504-518`, `Session.RequestStop`.)
Startup wait	Ten-second capture-start wait, cancellable; this cannot forcibly terminate a native call. (`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:12-13`, constants; `src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:108-111`, `StartCoreAsync`.)
Native release	Independent release workers; aggregate five-second wait. A hung input does not prevent the responsive input's worker from releasing it. (`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:384-402`, `FinishSessionAsync`.)
Late native return	Worker awaits its startup task, stops capture, waits for recording completion when applicable, disposes, and detaches handlers. (`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:425-469`, `ReleaseInputAsync`.)
Unsafe restart	Outstanding release blocks another Start; cleanup failure sets a sticky guard. (`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:79-82`, `StartCoreAsync`; `src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:455-459`, `ReleaseInputAsync`.)

Five seconds is **not an end-to-end Stop deadline**: transport final-drain has its own 15-second timeout. Nor does timing out prove the operating system released the endpoint. Ownership remains with the outstanding worker; the user-facing fault asks for app closure rather than silently opening another device. (`src\PhantomFudge.App\Audio\TranscriptionSocket.cs:20-21`, timeouts; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:255-280`, `StopAsync`; `src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:384-410`, `FinishSessionAsync`.)

The UI therefore maintains `audioStopping` independently of `IsRunning`. It disables Start and input changes until the stop task finishes. Without that distinction, `transcript.BeginSession()` could reset final-ID and partial state while old-session finals were still arriving. (`src\PhantomFudge.App\MainWindow.xaml.cs:589-620`, `Start_Click`; `src\PhantomFudge.App\MainWindow.xaml.cs:641-708`, audio controls and stop methods; `src\PhantomFudge.Core\TranscriptBuffer.cs:23-39`, `BeginSession`.)

For faults, `SignalFault` publishes only the first failure, disables both inputs, aborts transport, and queues cleanup. It deliberately avoids acquiring the other input's audio lock from a callback: simultaneous failures would otherwise risk lock inversion. Normal final-drain guarantees do not apply to this abort path. (`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:338-361`, `SignalFault`.)

Failure scenarios and evidence limits

Unplugging either selected endpoint stops the whole session; changing the Windows default does not switch it. `RenderEndpointNotification` filters by ID, invalidates once, and dispatches cleanup notification onto the thread pool so the Core Audio callback returns promptly. (`src\PhantomFudge.App\Audio\RenderEndpointNotification.cs:10-30`, notification methods.)

Synthetic tests exercise denied construction, failed startup, concurrent device loss, hung startup/stop, and failed disposal. `DualCaptureAsync` additionally checks readiness-before-opening, copied PCM, independent silence, and levels. These are executable descriptions of ownership guarantees—not evidence that every microphone driver behaves correctly. No tests were run for this documentation task. (`tests\PhantomFudge.Audio.Tests\DualSourceTests.cs:522-598`, `DualCaptureAsync`; `tests\PhantomFudge.Audio.Tests\DualSourceTests.cs:687-853`, failure scenarios.)

Actual microphone permission behavior, echo, recorder-visible OS privacy indicators, and hardware quality remain separate qualification questions. Source labels cannot remove echo, suppress Windows indicators, or establish who spoke. (`docs\DUAL-SOURCE-AUDIO.md:10-26`, boundaries; `docs\QUALIFICATION.md:1-7`, qualification status.)

Reader checkpoints

1. Trace a Stop arriving while a second Start waits on `_lifecycle`. Which comparison prevents reopening?
2. Compute the two wire-message lengths for a 40,000-byte Computer callback. Which byte is excluded from PCM duration?
3. Explain why a Mic stop hang must not block Computer release, yet must block a fresh session.
4. At 300 ms of callback silence, what can be inferred about supplied PCM—and what cannot be inferred about acoustic timing?

Source-reading route

Read source identities, endpoint ownership, session orchestration, silence decisions, then synthetic lifecycle cases. Following this order separates format, ownership, timing, and evidence before introducing the wire protocol.

8. Streaming protocol: preserving identity under backpressure

Why the protocol is more than a pipe

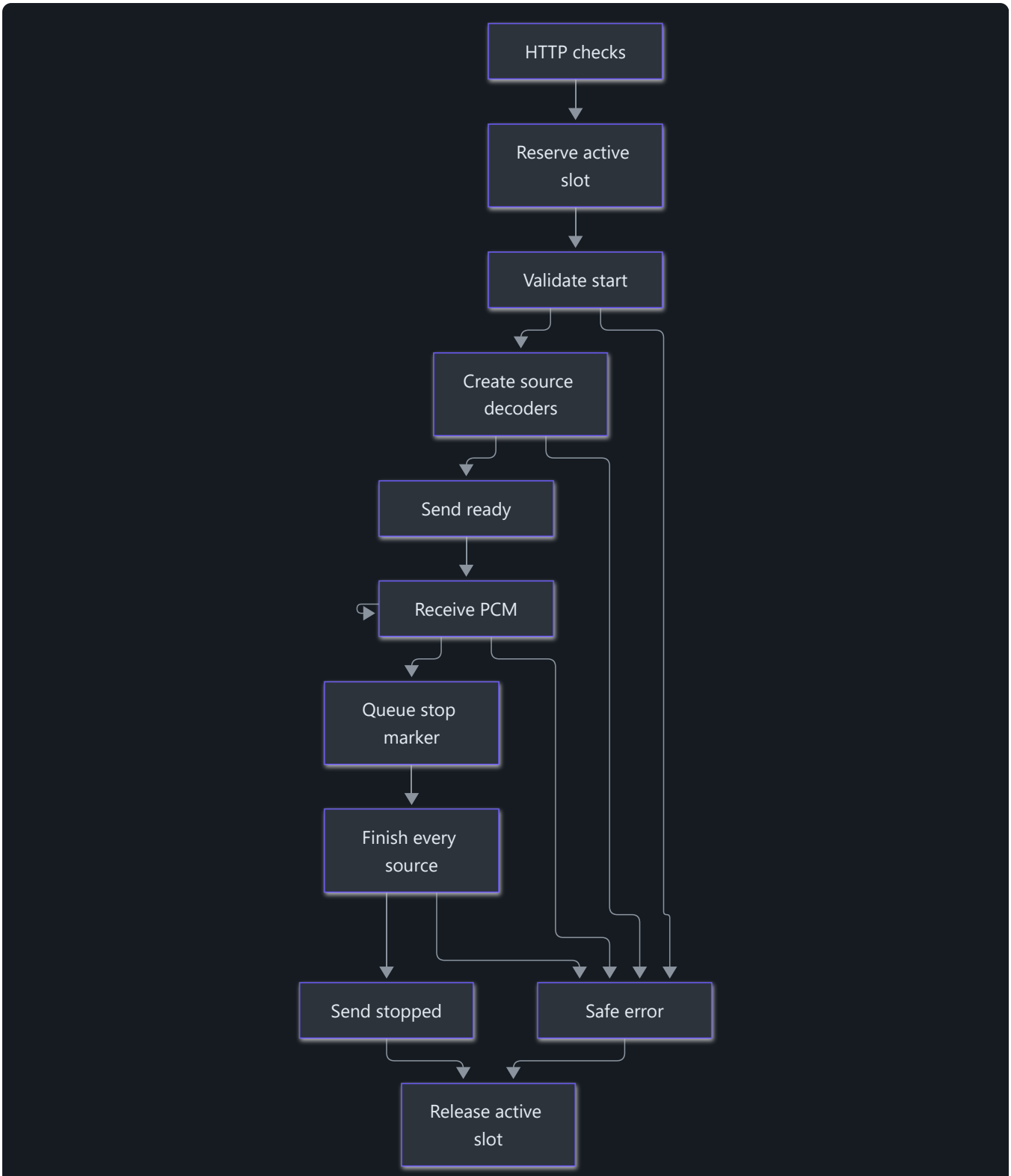
A socket that moves bytes successfully can still produce an incorrect transcript: it can mix sources, accumulate unbounded delay, accept replayed final IDs, or report completion before a decoder flushes. Phantom Fudge's protocol makes those boundaries explicit. The C# client owns admission and copied PCM; Python owns per-source decoding and publication; both reject invalid transitions rather than guessing how to recover. (`src\PhantomFudge.App\Audio\TranscriptionSocket.cs:185-229`, `TryEnqueueAudio`; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:389-468`, `ProcessMessage`; `services\transcription\server.py:225-348`, `Session`.)

Compatibility is deliberately additive. Both speech versions use `/v1/transcribe`; v1 has unprefixed Computer PCM, while v2 declares a source subset and prefixes each binary message. The URL's `v1` does not imply Computer-only operation. Management independently retains `protocolVersion: 1` and advertises `speechProtocolVersions: [1, 2]`. (`services\transcription\server.py:94-106`, `parse_start`; `services\transcription\server.py:179-193`, `runtime_info`; `services\transcription\server.py:255-262`, `Session.receive`; `services\transcription\server.py:446-449`, routes.)

Admission before audio

`TranscriptionSocket.ValidateEndpoint` accepts only literal IPv4 or IPv6 loopback WebSocket URLs with the exact path. It rejects hostname resolution, user information, queries, fragments, escaped paths, and alternate numeric IP spellings. Proxies and redirects are disabled. The production native launcher uses IPv4 `127.0.0.1`; accepting `[::1]` in the client does not mean the default server binds IPv6. (`src\PhantomFudge.App\Audio\TranscriptionSocket.cs:26-31`, HTTP handler; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:73-89`, `ValidateEndpoint`; `services\transcription\manage.py:167-172`, `start`.)

On the server, nonempty `Origin` headers are rejected, authentication requires exactly one Bearer header, and token comparison uses `hmac.compare_digest`. Query strings and request bodies are not accepted. Readiness and the single active-session gate are checked before upgrading. `runtime.active = True` occurs without an intervening `await`, so even a client stalled before its start message occupies the only admission slot. (`services\transcription\server.py:171-176`, `authorized`; `services\transcription\server.py:353-378`, `transcribe`.)



8. Streaming protocol: preserving identity under backpressure - diagram 1. Source-linked; status follows the chapter.

This state sketch includes the normal stop marker, not an automatic retry or reconnect. A `TranscriptionSocket` is single-use; a fresh transcription session creates a fresh transport. (`src\PhantomFudge.App\Audio\TranscriptionSocket.cs:119-133`, `ConnectCoreAsync`; `src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:490-496`, `Session`.)

The contract, field by field

These are complete start messages, with no token in JSON:

```
{"type":"start","sampleRate":16000,"channels":1,"encoding":"pcm_s16le"}
```

```
{"type":"start","protocolVersion":2,"sampleRate":16000,"channels":1,"encoding":"pcm_s16le","sources":["computer","microphone"]}
```

The server compares the parsed object against the exact expected shape. For v2, valid lists are `["computer"]`, `["microphone"]`, and `["computer","microphone"]`. Reversal, duplicates, unknown names, empty lists, extra fields, and noninteger format/version values fail. C# copies and sorts a caller's source list before sending, but the wire contract itself requires canonical order. (`services\transcription\server.py:94-106`, `parse_start`; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:95-105`, source-aware `ConnectAsync`; `services\transcription\tests\test_dual_protocol.py:57-92`, `StartSchemaTests`.)

Message	Direction	Required content
`start`	Client → server	Format fields above; v2 adds version and sources.
`ready`	Server → client	Accepted format; v2 echoes version and canonical sources.
Binary audio	Client → server	V1: PCM only. V2: one source byte followed by PCM.
`partial`	Server → client	`type`, `text`; v2 additionally requires `source`.
`final`	Server → client	`type`, `text`, `segmentId`, `startSeconds`, `endSeconds`; v2 adds `source`.
`stop`	Client → server	Exactly `{"type":"stop"}`; no source-specific stop exists.
`stopped`	Server → client	Exactly `{"type":"stopped"}`, after every source finalization.
`error`	Server → client	`type`, safe `code`, safe `message`; unchanged between versions.

The table is implemented by `parse_start`, `Session.receive`, `Session.publish`, `Session.recognize`, and `error`. (`services\transcription\server.py:65-66`, `error`; `services\transcription\server.py:94-106`, `parse_start`; `services\transcription\server.py:237-333`, `Session`; `services\transcription\server.py:378-385`, ready construction.)

A frame calculation

`AudioSource` assigns byte `1` to Computer and `2` to Microphone. For 100 ms of 16 kHz mono PCM16, the payload is `16,000 × 0.1 × 2 = 3,200` bytes. V2 sends `1 + 3,200 = 3,201` application-message bytes. The first byte is metadata, not a sample; duration, queue bytes, and byte-rate credit count only the remaining PCM. WebSocket/TCP overhead is additional and is not included in this calculation.
(`src\PhantomFudge.Core\AudioSource.cs:3-7`, `AudioSource`; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:13-17`, constants; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:208-217`, `TryEnqueueAudio`; `services\transcription\server.py:255-287`, `Session.receive`.)

The smallest legal v2 message is three bytes: a source plus one two-byte sample. The largest is 32,001 bytes: a source plus one second of PCM. Legal message lengths are therefore odd, while legal PCM lengths are even. A prefix without PCM, an undeclared source, or 32,002 PCM bytes is invalid.
(`services\transcription\server.py:257-263`, `Session.receive`; `tests\PhantomFudge.Audio.Tests\DualSourceTests.cs:75-88`, `DrainV2Async`.)

Backpressure: bytes, objects, and source fairness

Budget	Native client	Python service
Maximum PCM per message	32,000 bytes	32,000 bytes
Pending PCM per source	64,000 bytes	64,000 queued bytes
Dual-source PCM maximum	128,000 pending bytes	128,000 queued bytes
Source frame slots	No separate 32-frame source cap	32 queued frames per source
Aggregate queue slots	Channel capacity 64,000 frame objects	`32 × declared sources`, at most 64
Pending lifetime	Released after send completes	Released when worker dequeues, before recognition

These are different accounting boundaries. Native pending bytes include the frame currently sending; server queued bytes exclude the frame currently being decoded. The table is not a total-process-memory guarantee: message objects, socket buffers, copies, and decoder memory exist outside these PCM counters.
(`src\PhantomFudge.App\Audio\TranscriptionSocket.cs:13-43`, constants/channel; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:198-221`, admission; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:313-327`, sender; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:543-550`, `ReleaseFrame`; `services\transcription\server.py:225-235`, `Session.__init__`; `services\transcription\server.py:277-287`, receive budgets; `services\transcription\server.py:328-333`, dequeue.)

At 3,200 bytes per frame, twenty queued frames consume a source's full 64,000-byte budget: two seconds of audio. At two bytes per frame, the server's 32-frame cap wins after just 64 queued PCM bytes. Both limits are necessary: a byte-only rule permits many tiny objects, while a frame-only rule permits too much audio latency. A busy Computer stream cannot borrow Mic's unused source quota. The shared queue is FIFO, not a round-robin scheduler; "fairness" here means reserved admission budgets, not equal CPU time.
(`services\transcription\server.py:232-233`, `Session.__init__`; `services\transcription\server.py:277-287`, `Session.receive`; `services\transcription\tests\test_dual_protocol.py:461-497`, source-budget tests.)

Translated from the C# admission algorithm:

```
def admit(source, pcm):
    validate_declared_source_and_even_nonempty_pcm(pcm)
    with queue_lock:
        if not accepting:
            return False
        if len(pcm) > 64000 - source_pending[source]:
            overflow = True
        elif len(pcm) > 128000 - total_pending:
            overflow = True
```

```
        else:
            frame = prefix_if_v2(source) + copy_bytes(pcm)
            reserve_pcm_budget(source, len(pcm))
            if try_write_channel(frame):
                submitted[source] += len(pcm)
                return True
            release_budget_and_clear_copy(frame)
            overflow = True
    if overflow:
        fault_entire_session()
    return False
```

Overflow is visible failure, not packet dropping or disk spill. The real method also handles closed/disposed state and per-version framing. (`src\PhantomFudge.App\Audio\TranscriptionSocket.cs:185-229`, `TryEnqueueAudio`.)

Rate is not backlog

Each server source separately starts with 64,000 byte credits and 200 frame credits, replenished at 32,000 bytes/second and 100 frames/second using monotonic elapsed time. Credits are capped at their burst sizes. Thus a source can spend an initial two-second byte burst, but sending 32,000 additional bytes immediately needs replenishment even if the decoder has emptied its queue. Conversely, a slow decoder can trigger backlog at an otherwise valid live rate. (`services\transcription\server.py:42-56`, `Limits`; `services\transcription\server.py:212-222`, `SourceState`; `services\transcription\server.py:265-280`, `Session.receive`.)

Other defaults are a five-second start-message wait, fifteen-second receive-idle wait, five-second decoder-operation wait, two-second send wait, and eight hours of **submitted PCM duration per source**. These are not advertised latency targets. (`services\transcription\server.py:43-56`, `Limits`; `services\transcription\server.py:275-276`, duration enforcement.)

Transcript clocks: global order, local time

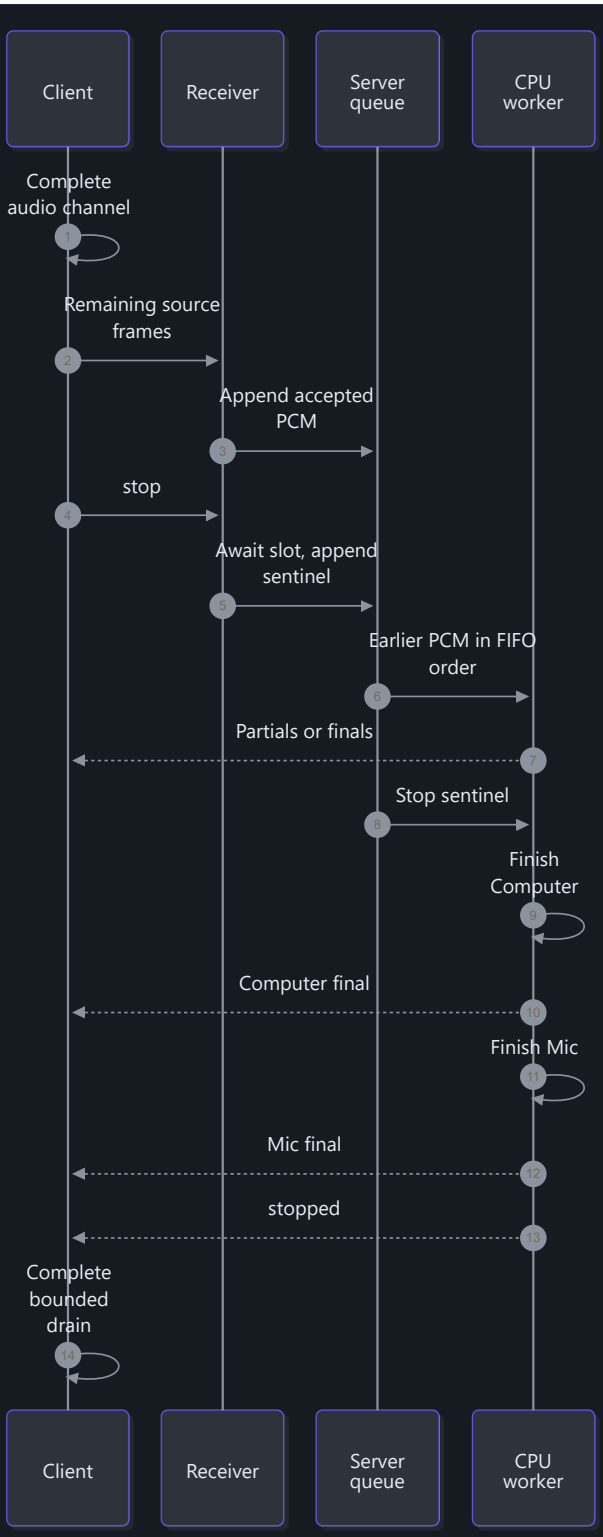
Consider three emitted finals: Computer ID 1 at `0.0–0.2`, Mic ID 2 at `0.0–0.1`, then Computer ID 3 at `0.2–0.3`. This is legal. IDs order session publication; timestamps order only the corresponding source's PCM. Comparing Mic's `0.1` against Computer's previous `0.2` would falsely reject valid interleaving. `Session.publish` keeps one global counter and a `last_end` per source. (`services\transcription\server.py:212-235`, state; `services\transcription\server.py:289-315`, `publish`; `services\transcription\tests\test_dual_protocol.py:182-200`, interleaved-final test.)

The server requires finite, nonnegative, nonregressing source times bounded by that source's received duration. V2 has no pre-rounding duration tolerance. Millisecond rounding is then clipped to whole elapsed milliseconds: 18 bytes represent `18 / 32,000 = 0.0005625` seconds, but cannot become a `0.001`-second wire endpoint, so the result is `0.000`. The native client independently checks finite times, increasing global IDs, and v2 source monotonicity, but its submitted-duration bound allows one extra second. Do not describe the two

validators as identical. (`services\transcription\server.py:294-308`, `publish`; `services\transcription\tests\test_dual_protocol.py:214-222`, rounding test; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:437-457`, final validation.)

Partials are also source-local. An unchanged Mic partial remains suppressed even after Computer finalizes. On the UI side, `TranscriptEvent.Source` carries the label, and `TranscriptBuffer.AddFinal` removes only that source's partial. A final can be empty while still advancing ordering and clearing pending text. (`services\transcription\server.py:301-315`, `publish`; `src\PhantomFudge.App\Audio\TranscriptEvent.cs:9-15`, `TranscriptEvent`; `src\PhantomFudge.Core\TranscriptBuffer.cs:41-72`, partial/final methods.)

Why Stop joins the queue



8. Streaming protocol: preserving identity under backpressure - diagram 2. Source-linked; status follows the chapter.

The client's sender emits Stop only after its channel drains. The server awaits space for a `None` sentinel instead of rejecting Stop because a valid audio queue is full. Its recognition loop reaches the sentinel after earlier PCM, finishes all declared decoders in canonical order, then sends `stopped`. Even an unused source emits an

empty final. (`src\PhantomFudge.App\Audio\TranscriptionSocket.cs:313-331`, `SendAudioAsync`; `services\transcription\server.py:247-252`, `receive`; `services\transcription\server.py:317-327`, `recognize`.)

The v2 client refuses `stopped` before Stop or without observing a final for every declared source after its stop-sent flag. Its fifteen-second flush deadline bounds the wait; failure warns that final text may be unavailable. This is a completion barrier, not recovery after a broken connection.
(`src\PhantomFudge.App\Audio\TranscriptionSocket.cs:232-280`, `StopAsync`; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:454-465`, `ProcessMessage`.)

Defensive parsing and testable limits

Server control messages are capped at 1,024 UTF-8 bytes. Decoder text is capped at 8,192 Python characters before publication, whereas native transcript text validation allows 16,384 .NET string characters inside its separate message-byte limit. Character budgets and encoded-message budgets are not interchangeable, especially for escaped Unicode. (`services\transcription\server.py:43-45`, `Limits`; `services\transcription\server.py:82-84`, `parse_control`; `services\transcription\server.py:289-292`, `Session.publish`; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:430-440`, `ProcessMessage`.)

Native inbound JSON is limited to 32,768 bytes and depth eight; only text messages are accepted. Duplicate properties are rejected, v2 ready requires an exact source echo, and transcript properties are exact. However, legacy ready validation is less restrictive about extra properties. Python uses ordinary `json.loads`, so duplicate-key rejection is not a symmetric server guarantee. A precise specification must preserve those implementation distinctions. (`src\PhantomFudge.App\Audio\TranscriptionSocket.cs:343-423`, receive/ready parsing; `services\transcription\server.py:82-91`, `parse_control`.)

Server protocol failures normally attempt a fixed error envelope and close code 1008; grossly oversized messages can instead hit aiohttp's transport ceiling and close with 1009. Native code never displays a service-provided error string directly. (`services\transcription\server.py:26-39`, error catalogue; `services\transcription\server.py:155-159`, `close_with_error`; `services\transcription\tests\test_dual_protocol.py:257-264`, ceiling test; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:397-405`, error handling.)

Read `V2FramingAsync`, `V2BackpressureAsync`, and `V2MissingFinalAsync` for copied bytes, independent native budgets, and incomplete stop rejection. Python's full-queue stop test proves sentinel ordering, while `RunDualSyntheticServiceCheckAsync` streams two in-memory fixtures through the actual C# transport and checks source-specific phrases. None of these test definitions constitutes a new passing run here, acoustic isolation, or cloud integration. Protocol v3 and question commit are not accepted by the current start parser.
(`tests\PhantomFudge.Audio.Tests\DualSourceTests.cs:96-140`, `V2FramingAsync`; `tests\PhantomFudge.Audio.Tests\DualSourceTests.cs:352-424`, stop/budget tests; `services\transcription\tests\test_dual_protocol.py:499-522`, full-queue stop test; `tests\PhantomFudge.Audio.Tests\DualSyntheticServiceCheck.cs:49-79`, synthetic runner; `services\transcription\server.py:94-106`, `parse_start`.)

Reader checkpoints

1. Why can a two-byte PCM flood hit the server's frame cap before its byte cap?
2. Which counters include the source prefix? Explain why the correct answer is none of the PCM counters.
3. Why is Computer ID 9 ending at 12 seconds compatible with Mic ID 10 ending at 3 seconds?
4. What changes if a decoder throws while finalizing the second source? Distinguish a received first final from a successful stopped barrier.

Source-reading route

Compare native transport with server session, then read dual-protocol tests and transcript state. Continue to recognition for the timing values the protocol validates.

Source: docs/handbook/09-python-recognition.md

9. Python recognition: shared model, independent histories

Why separate the model from its history?

Two sources need separate recognition histories, not two independently loaded copies of the same model. Computer playback must not become the acoustic context for a dictated Mic question. `VoskFactory` therefore retains one model and creates a new `VoskRecognizer` for each requested source. Each wrapper constructs its own `KaldiRecognizer` and maintains its own sample counter, offset, and last final endpoint. The shared model is treated as immutable by this application; mutable decoder state belongs to one source. (`services\transcription\recognizer.py:41-67`, `VoskFactory`, `VoskRecognizer`; `services\transcription\server.py:378-386`, `transcribe`.)

This distinction also explains the concurrency design. A single executor worker runs source-decoder operations away from aiohttp's event loop. The sources are logically independent but do not decode simultaneously on separate executor threads. That is a bounded-resource tradeoff, not a claim of equal per-source latency or proof that native libraries never use internal threads. The native manager additionally sets `OPENBLAS_NUM_THREADS` and `OMP_NUM_THREADS` to one. (`services\transcription\server.py:109-142`, `Runtime`; `services\transcription\manage.py:167-176`, `start`.)

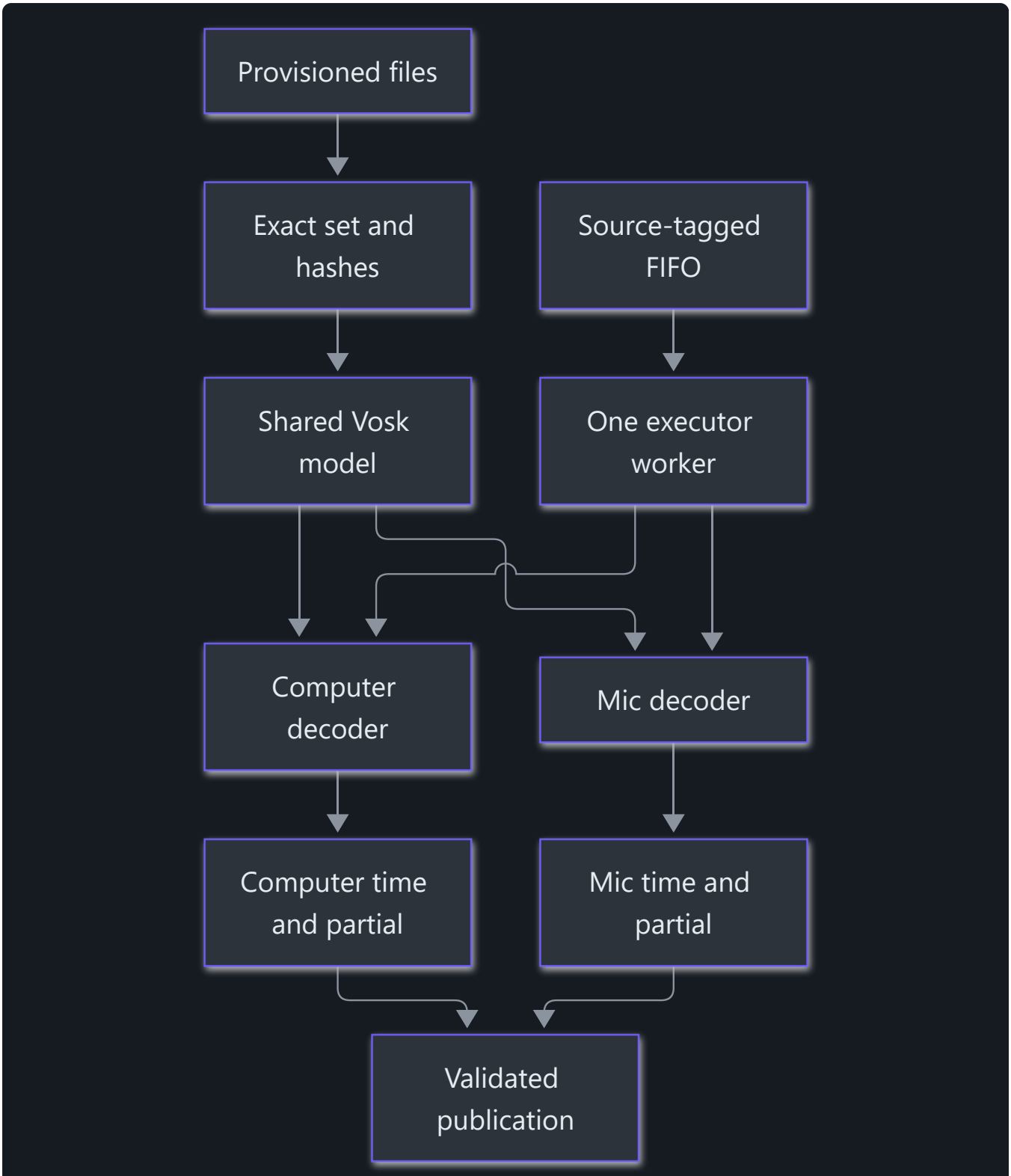
Provisioned identity, not a model chosen at request time

Pinned item	Repository value	Meaning
Engine package	`vosk==0.3.45`	Python dependency pin, not an accuracy guarantee.
HTTP/WebSocket package	`aiohttp==3.14.3`	Transport dependency pin.
Model	`vosk-model-small-en-us-0.15`	Specific English model directory and archive.
Model language/version	`en-US`, `0.15`	Manifest metadata; no automatic language selection.
Archive length	41,205,931 bytes	Approximately 41.21 MB decimal or 39.30 MiB binary.
License metadata	`Apache-2.0`	Manifest's model-license declaration.
Checksum provenance	Observed official HTTPS download	Explicitly not a publisher-signed checksum.

Package rows come from `requirements.txt`; model rows come from the maintained manifest. The “approximately 40 MB” setup description refers to archive size, not resident model memory. (`services\transcription\requirements.txt:1-20`, pinned dependencies; `services\transcription\model.json:2-10`, manifest metadata; `scripts\Initialize-Transcription.ps1:69-79`, model provisioning.)

`verify_model` requires a directory, rejects symlinks, enumerates the complete file set, compares it exactly with the manifest, and SHA-256-checks every expected file. Extra files are therefore a mismatch, not harmless metadata. Verification occurs before importing and constructing Vosk's model. The constructor explicitly supplies `model_path`; the source comment explains that language/name discovery could download a model, which is inappropriate in this request-serving path. (`services\transcription\recognizer.py:22-49`, `verify_model`, `VoskFactory.__init__`.)

The provisioning script is a separate consent and mutation boundary: it downloads only when the model/archive is absent and `-DownloadModel` is supplied, validates archive length and hash, refuses to overwrite an existing invalid model directory, and verifies extracted files. A cached archive can be reused without downloading. Neither the service's WebSocket handler nor the recognizer repairs or silently fetches missing weights. (`scripts\Initialize-Transcription.ps1:69-95`, provisioning; `services\transcription\recognizer.py:22-49`, verification and loading; `services\transcription\server.py:455-466`, `production_app`.)



9. Python recognition: shared model, independent histories - diagram 1. Source-linked; status follows the chapter.

The diagram combines `VoskFactory` ownership with `Session` source state. Loading the model does not start a decoder for an unrequested source, open a microphone, or create transcript text. (`services\transcription\recognizer.py:41-67`, model/decoder constructors; `services\transcription\server.py:212-235`, `SourceState`, `Session`; `services\transcription\server.py:380-385`, source-decoder construction.)

Following one 100 ms frame

At 16,000 Hz and two bytes per mono sample, 3,200 bytes represent 1,600 samples and `0.1` seconds. `Session.recognize` dequeues a `(source, pcm)` pair, releases its queue accounting, and invokes only that source's `decoder.accept` through `Runtime.cpu`. This separates backpressure accounting from recognition progress: total received bytes can include queued work, while `VoskRecognizer._samples` advances only as frames reach the decoder. (`services\transcription\server.py:274-287`, `Session.receive`; `services\transcription\server.py:317-333`, `Session.recognize`; `services\transcription\recognizer.py:82-84`, `VoskRecognizer.accept`.)

`accept` increments `_samples` by `len(pcm) // 2`, calls `AcceptWaveform`, and branches on endpoint completion and the 30-second history boundary. A normal incomplete result uses `PartialResult()["partial"]`; an endpoint-complete result uses `Result()` and `_final`. Every new decoder enables word results with `SetWords(True)` and disables partial-word results with `SetPartialWords(False)`. The application needs final word times, but exposes partial text rather than partial word alignments. (`services\transcription\recognizer.py:64-96`, decoder methods.)

`Recognition` is a small immutable result carrier: text, final flag, start, and end. It does not carry source identity; the enclosing `Session` already knows which source's decoder ran and adds the label during publication. Keeping these responsibilities separate lets recognizer tests focus on timing without needing a WebSocket. (`services\transcription\recognizer.py:14-19`, `Recognition`; `services\transcription\server.py:289-315`, `Session.publish`; `services\transcription\tests\test_recognizer.py:37-75`, `RecognizerTests`.)

Partials are replacements, not append-only sentences

The server stores `last_partial` independently for each source and sends only changed partial text. A final resets that source's partial memory to `None`; it does not reset the other track. An empty initial partial can be emitted because `""` differs from `None`. This is useful state information, not evidence of recognized speech. (`services\transcription\server.py:212-220`, `SourceState`; `services\transcription\server.py:301-315`, `Session.publish`.)

Final publication validates the result type, string type, 8,192-character service text limit, finite timestamps, and source-duration ordering before assigning the next global segment ID. These checks protect the wire contract even when a decoder adapter is replaced by a test double or returns invalid data. A final is committed recognition output, not a guarantee that the words are correct. (`services\transcription\server.py:289-315`, `Session.publish`; `services\transcription\tests\test_dual_protocol.py:349-377`, invalid-result and timestamp-regression tests.)

Bounding continuous-speech history

A stream need not pause conveniently. The wrapper therefore finalizes and recreates its mutable decoder once supplied audio since `_offset` reaches at least 30 seconds. This is a processed-audio threshold, not a wall-clock timer and not a configurable silence detector. The rollover check follows `AcceptWaveform`, so a frame that crosses the boundary is entirely consumed before rotation. (`services\transcription\recognizer.py:82-93`, `VoskRecognizer.accept`.)

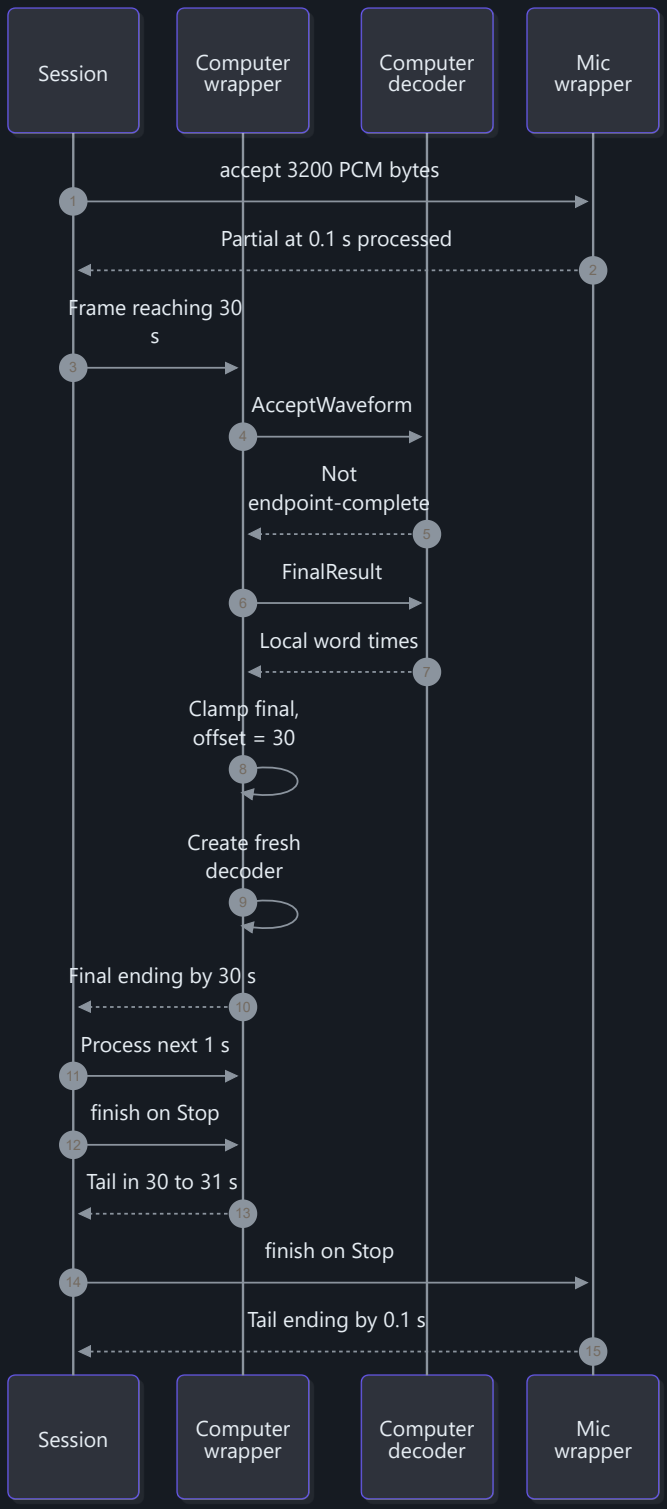
Thirty seconds contains `30 × 16,000 = 480,000` samples, or 960,000 PCM bytes per source. These are cumulative processed bytes, **not bytes retained in the server queue**. If the existing decoder ended an utterance, rotation uses `Result()`; otherwise it forces `FinalResult()`. The wrapper then sets `_offset` to cumulative elapsed audio and creates a fresh decoder against the same model. (`services\transcription\recognizer.py:83-90`, `accept`; `services\transcription\server.py:43-47`, queue limits.)

The following JavaScript-shaped pseudocode translates the Python recognition branch:

```
function accept(pcm) {
  samples += Math.floor(pcm.length / 2);
  const complete = decoder.AcceptWaveform(pcm);
  const elapsed = samples / 16000;

  if (elapsed - offset >= 30) {
    const result = toFinal(
      complete ? decoder.Result() : decoder.FinalResult()
    );
    offset = elapsed;
    decoder = newDecoder(sharedModel, 16000);
    return result;
  }
  if (complete) return toFinal(decoder.Result());
  return { text: JSON.parse(decoder.PartialResult()).partial ?? "",
           final: false };
}
```

This is explanatory pseudocode, not an additional runtime or language port. `toFinal` represents the timestamp normalization performed by Python's `_final`. (`services\transcription\recognizer.py:69-96`, `VoskRecognizer` methods.)



9. Python recognition: shared model, independent histories - diagram 2. Source-linked; status follows the chapter.

`test_shared_model_has_independent_decoders_rotation_and_timing` verifies the essential property: rotating Computer creates one new decoder without replacing Mic's decoder or advancing Mic's clock. The diagram's timing is based on synthetic fixture behavior, not measured acoustic recognition latency. (`services\transcription\tests\test_recognizer.py:61-75`, shared-model test.)

Worked absolute-time reconstruction

Suppose thirty one-second frames have reached Computer. The fixture produces a final ending at 30 seconds; `_offset` becomes 30. After one further second, a new decoder word starting locally at `0.05` becomes absolute `30.05`, and its one-second endpoint becomes `31`. The recognizer test asserts those exact values. No raw timestamp from the second decoder is allowed to send the source clock backwards to zero. (`services\transcription\tests\test_recognizer.py:38-51`, continuous-speech rotation test; `services\transcription\recognizer.py:69-89`, `_final`, `accept`.)

For a nonempty word list, `_final` takes the first word's start and last word's end, adding `_offset`. Without words, it uses `_last_end` and cumulative elapsed audio. It rejects nonfinite numbers, then clamps start to at least the previous endpoint and end to at least start, both no later than supplied audio. This bounds the timeline; it does not independently validate acoustic word alignment. (`services\transcription\recognizer.py:69-80`, `_final`.)

The 30-second policy bounds decoder history but introduces a tradeoff: forced finalization can divide continuous speech where the speaker did not pause. The code does not promise linguistically optimal boundaries. Nor should this reset be mistaken for the native 250 ms idle-silence threshold: one concerns decoder history; the other concerns callback-driven PCM supply. (`services\transcription\recognizer.py:85-90`, rotation branch; `src\PhantomFudge.App\Audio\SilenceCadence.cs:11-19`, cadence.)

Failure containment around native work

`Runtime.cpu` submits work to the one-thread executor and awaits a **shielded** future with a five-second default operation timeout. Shielding matters because cancelling an asyncio waiter does not safely terminate the native decoder. On an operation exception or timeout, the runtime becomes failed, the error becomes generic `decoder_error`, and later work is not admitted as a healthy session. If the await itself is cancelled, the method waits again for the outstanding call, retaining admission while that attempt completes or times out. (`services\transcription\server.py:43-51`, limits; `services\transcription\server.py:126-142`, `Runtime.cpu`; `services\transcription\server.py:387-408`, handler cleanup.)

Model initialization is a distinct case: `production_app` submits `VoskFactory` directly to the executor, not through `Runtime.cpu`. Therefore the five-second per-operation rule must not be attributed to initial model loading. The native manager separately limits its readiness observation to 45 seconds; it retains the receipt if startup exceeds that wait. (`services\transcription\server.py:455-466`, `production_app`; `services\transcription\manage.py:184-200`, `start`.)

Failure example	Observable policy
Missing or modified model	Factory cannot load; service readiness is unavailable, with no request-time download. (`services\transcription\recognizer.py:22-49`; `services\transcription\server.py:455-465`.)
Second decoder fails to initialize	No ready message for the partially constructed source set; runtime becomes unavailable. (`services\transcription\server.py:380-390`, `transcribe`; `services\transcription\tests\test_dual_protocol.py:282-294`, initialization test.)
Mic decoder raises or hangs	Entire session faults; Computer is not silently retained as the only source. (`services\transcription\tests\test_dual_protocol.py:296-305`, exception test; `services\transcription\tests\test_dual_protocol.py:325-347`, timeout test.)

Failure example	Observable policy
Mic finish fails after Computer final	The Computer final may already have arrived, but there is no successful `stopped` barrier. (`services\transcription\tests\test_dual_protocol.py:308-315`, finish-failure test.)
Result timestamp is NaN	Rejected rather than serialized as plausible time. (`services\transcription\recognizer.py:75-76`, `_final`; `services\transcription\tests\test_recognizer.py:53-59`, nonfinite test.)

The service disables logging, access logs, and native stderr retention in production because decoder diagnostics can contain model paths or content. This is a deliberate diagnostic tradeoff, not a universal secure-erasure claim. The client additionally replaces service error text with its own safe message. (`services\transcription\server.py:469-492`, `main`; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:397-405`, `ProcessMessage`.)

What the tests can establish

`DecoderFixture` deterministically checks history rotation and time offsets without loading real weights. `RecordingRecognizer` checks source routing, duplicate-partial suppression, and one executor thread. Together they distinguish algorithmic invariants from model quality. Their existence does not mean this documentation task reran them. (`services\transcription\tests\test_recognizer.py:10-75`, fixture/tests; `services\transcription\tests\test_dual_protocol.py:30-54`, `RecordingRecognizer`; `services\transcription\tests\test_dual_protocol.py:150-180`, independence test.)

The real-model synthetic check requires an explicit cached model and in-memory source fixtures. It verifies shared model/distinct decoders, source-specific expected keywords, absence of the other source's fixture keywords, global IDs, bounded source times, and the final-final-stopped tail. The C# helper independently exercises native v2 transport with source-specific expected phrases. These are stronger integration checks than fake decoders, but still do not qualify real microphone permissions, acoustics, echo cancellation, accents, or arbitrary meeting accuracy. (`services\transcription\tests\real_dual_model_check.py:34-67`, fixture/model setup; `services\transcription\tests\real_dual_model_check.py:84-118`, assertions; `tests\PhantomFudge.Audio.Tests\DualSyntheticServiceCheck.cs:49-79`, native helper.)

The current engine is local English streaming recognition. Source identity is not diarization, and no code here commits a selected question to Azure or generates an answer. Those are design-only extensions, not latent behavior unlocked by starting Python. (`services\transcription\model.json:2-4`, language metadata; `docs\DUAL-SOURCE-AUDIO.md:10-17`, boundaries; `docs\ARCHITECTURE.md:7-11`, proposed answer extension.)

Reader checkpoints

1. After 30 seconds of Computer PCM and 100 ms of Mic PCM, which state is shared and which counters differ?
2. Why is 960,000 processed bytes compatible with a 64,000-byte source queue?
3. Explain why `asyncio` cancellation cannot be treated as proof that a native decoder stopped executing.
4. Construct a final word timestamp after rotation and apply both `_offset` and `_last_end` clamping.

Source-reading route

Start with manifest identity and recognizer implementation, then server scheduling/publication, deterministic recognizer tests, and real synthetic integration. Chapter 10 explains who provisions and owns the process around this engine.

Source: docs/handbook/10-native-service-lifecycle.md

10. Native service lifecycle: readiness without accidental capture

Why process ownership is a protocol problem

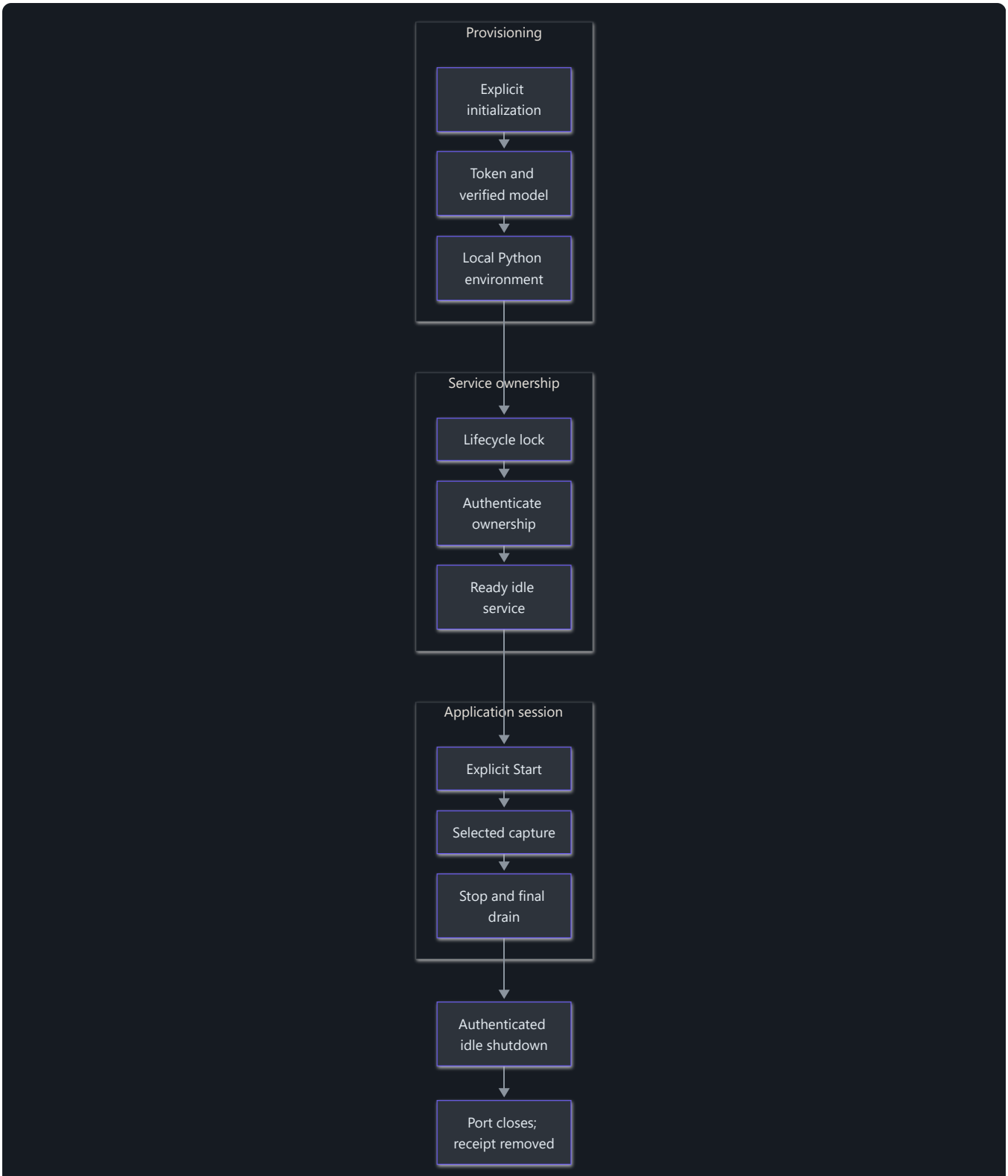
“Something is listening on port 8765” is insufficient evidence that Phantom Fudge may reuse or stop it. The listener could belong to another checkout, lack the installed client's speech version, or be transcribing an active session. `manage.py` treats a local service as owned only after combining a workspace receipt, authenticated runtime response, expected service identity, management version, and matching instance UUID. It does not convert a discovered PID into permission to terminate a process.
(`services\transcription\manage.py:93-122` , `read_record` , `owned_info` ; `services\transcription\manage.py:130-144` , `stop_owned` .)

The second boundary is **ready service versus recording application**. Initialization provisions dependencies and weights. Start makes an idle recognizer service available. The application still requires explicit Start before endpoint acquisition, and transport readiness precedes native capture. Keeping these operations distinct makes a launcher convenient without turning it into an automatic recorder. (`scripts\Initialize-Transcription.ps1:110-140` , Python setup; `scripts\Start-Transcription.ps1:8-15` , native start; `src\PhantomFudge.App\MainWindow.xaml.cs:589-625` , `Start_Click` ; `src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:98-111` , `StartCoreAsync` .)

The reader's journey: provision, launch, select, stop

Entry point	Supported switches	What it changes
`Initialize-Transcription.ps1`	`-DownloadModel` , `-SkipPythonSetup`	Token, verified model, setup receipts, and normally the service-local Python environment; no service or capture start.
`Start-Transcription.ps1`	`-Docker`	Default: invoke native `manage.py start` ; Docker is an explicit alternate branch.
`Run-PhantomFudge.ps1`	`-NoBuild` , `-NotesOnly` , `-PassThru`	Build when appropriate, start/reuse service unless notes-only, launch or reveal the app.
`Stop-Transcription.ps1`	`-Docker`	Default: authenticated native manager stop; optional Docker branch is separate.
`manage.py`	`start` , `stop` , `status`	Serialized native runtime lifecycle and status inspection.

These are source-supported interfaces, not commands executed while writing this chapter.
(`scripts\Initialize-Transcription.ps1:1-5` , parameters; `scripts\Start-Transcription.ps1:1-15` , native branch; `scripts\Run-PhantomFudge.ps1:1-55` , launcher; `scripts\Stop-Transcription.ps1:1-22` , stop branches; `services\transcription\manage.py:226-239` , `main` .)



10. Native service lifecycle: readiness without accidental capture - diagram 1. Source-linked; status follows the chapter.

The diagram's shutdown edge is deliberate: managed shutdown rejects an active speech session. Service readiness can also be reused without traversing provisioning again. (`services\transcription\server.py:196-209`, `managed_shutdown`; `services\transcription\manage.py:147-159`, `start`.)

Provisioning without hidden repair

Initialization creates a token only if one is absent, using 32 cryptographically random bytes rendered as lowercase hexadecimal. That is 256 generated bits represented by 64 ASCII characters—not a hard-coded credential. Exclusive file creation prevents an existing token from being overwritten through this path. On Windows, the script replaces the token file's DACL with a current-user full-control rule and disables inherited permissions, without changing ownership/audit sections that could require elevation. (`scripts\Initialize-Transcription.ps1:33-67`, token generation and ACL setup.)

Existing token content must match the accepted 32–512-character URL-safe pattern. The setup script does not silently rotate invalid credentials; it requests explicit repair. The manager reads and validates the token, while the server prefers the configured token file over an environment token. These are source-defined storage operations; this chapter neither reads nor cites a user's generated token. (`scripts\Initialize-Transcription.ps1:65-67`, validation; `services\transcription\manage.py:33-40`, `token`; `services\transcription\server.py:69-79`, `load_token`.)

Model provisioning is similarly explicit. The manifest specifies a 41,205,931-byte English archive and observed SHA-256 identity. Setup checks byte count and checksum before extraction, rejects unexpected extracted content, and refuses to overwrite a failed existing model directory. Runtime performs its own manifest verification; a setup receipt is not a substitute for checking the bytes currently present. (`services\transcription\model.json:2-10`, model metadata; `scripts\Initialize-Transcription.ps1:17-30`, `Test-Model`; `scripts\Initialize-Transcription.ps1:69-109`, archive/receipt path; `services\transcription\recognizer.py:22-43`, `verify_model`, `VoskFactory`.)

Python setup creates a virtual environment if necessary, hashes the maintained requirements file, and decides whether restore is needed from that receipt and an import check. Restore runs pinned installation followed by `pip check`; successful setup does not itself start the service. A requirements hash receipt is a restore optimization, not a claim that every installed wheel is cryptographically attested on each launch. (`scripts\Initialize-Transcription.ps1:110-140`, environment provisioning; `services\transcription\requirements.txt:1-20`, dependency pins.)

Serializing the launcher without trusting process names

`lifecycle_lock` is Windows-specific. It opens a workspace lock file, ensures at least one byte exists, and repeatedly attempts a nonblocking `msvcrt` lock on that byte. The acquisition deadline is 50 seconds with 100 ms polling. Start, Stop, and Status share this lock, so competing manager invocations cannot simultaneously rewrite the ownership receipt. This serializes cooperating managers for the workspace; it is not an operating-system-wide monopoly on the port. (`services\transcription\manage.py:65-90`, `lifecycle_lock`; `services\transcription\manage.py:147-150`, `start`; `services\transcription\manage.py:203-222`, `stop`, `status`.)

The receipt contains schema version 1, an instance UUID in canonical hexadecimal form, and a PID field. `write_record` writes a temporary sibling and replaces the receipt. `owned_info` authenticates the listener and compares service name, management version 1, and instance UUID. Crucially, it does **not** use the receipt PID as ownership proof or a kill target: process IDs can be reused, whereas the authenticated instance value

participates in the shutdown contract. (`services\transcription\manage.py:93-122`, receipt/ownership methods; `services\transcription\manage.py:164-166`, initial receipt; `services\transcription\manage.py:189-191`, PID update.)

Management requests use literal `http://127.0.0.1:8765`, a two-second timeout, no proxy, and a redirect handler that fails rather than forwarding credentials. Reading 4,097 bytes detects whether a response exceeds the 4,096-byte allowed payload. Parsing also requires a JSON object. These are bounded control-plane responses, not transcript downloads. (`services\transcription\manage.py:21-30`, base URL and `NoRedirects`; `services\transcription\manage.py:43-56`, `request`.)

Capability-aware reuse and stale-service replacement

Existing listener state	`start()` decision
No matching receipt	Refuse adoption.
Wrong token, service identity, management version, or UUID	Refuse reuse/change.
Owned but `ready` is not exactly true	Fail; require explicit stop/setup repair.
Owned, ready, advertises speech v2	Reuse; do not restart merely because it is active.
Owned, ready, lacks speech v2, explicitly idle	Request authenticated shutdown, then start updated service.
Owned, ready, lacks speech v2, active or ambiguous	Refuse restart; ask the user to stop transcription.

Ownership and capability are separate tests. `supports_current_audio` looks for the integer `2` in a list; the manager does not mistake management `protocolVersion: 1` for a stale speech protocol. Requiring `active is False` is intentionally stricter than treating a missing or false-like field as idle. (`services\transcription\manage.py:112-159`, `owned_info`, `supports_current_audio`, `stop_owned`, `start`; `services\transcription\server.py:179-193`, `runtime_info`.)

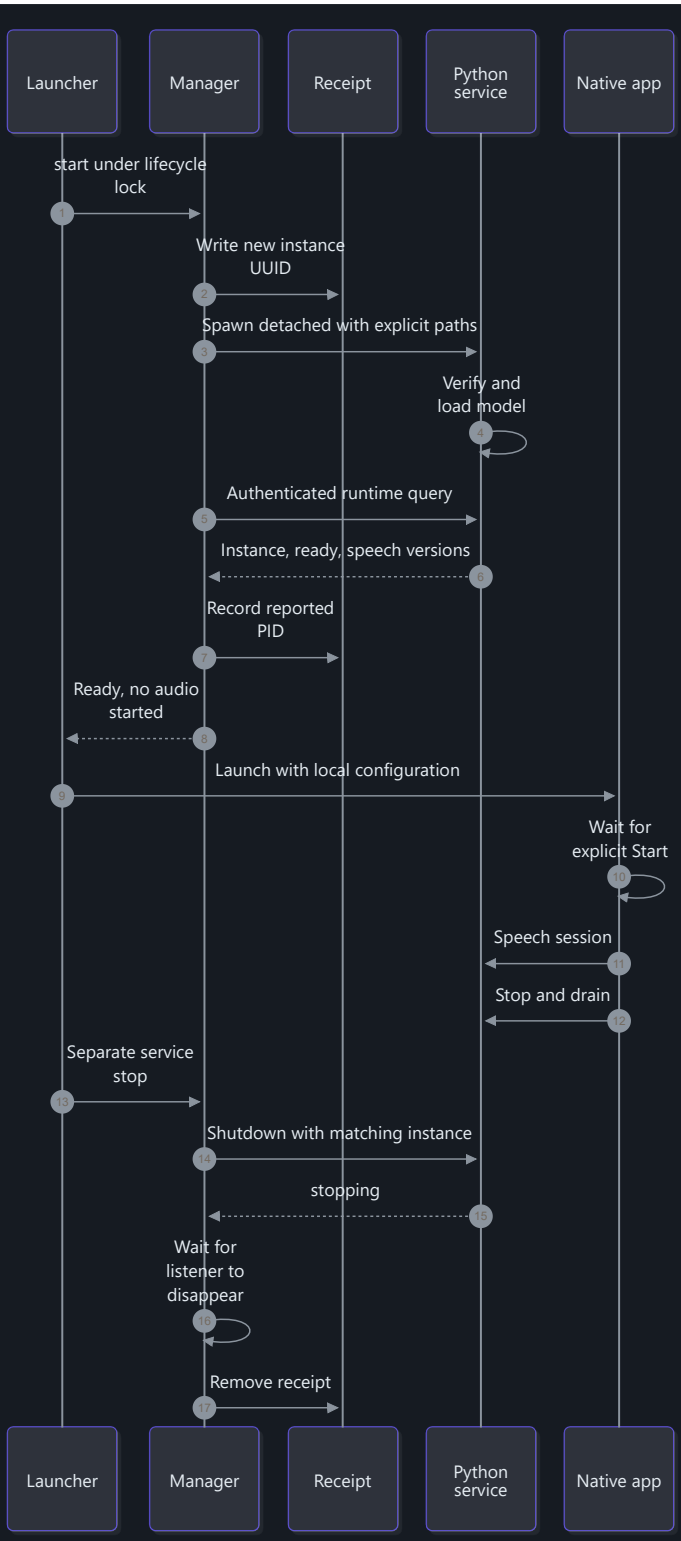
The following JavaScript-shaped pseudocode translates the manager's Python decisions:

```
function startNative() {
  const credential = readValidatedToken();
  withLifecycleLock(() => {
    const receipt = readValidatedReceipt();
    if (portInUse()) {
      const info = authenticateOwnedInstance(credential, receipt);
      if (info.ready !== true) throw new RepairRequired();
      if (supportsSpeechV2(info)) return; // Reuse, not capture.
      if (info.active !== false) throw new StopCaptureFirst();
      shutdownOwnedAndWaitForPort(credential, receipt);
    }
    requireProvisionedModel();
    const instance = newUuidHex();
    writeReceiptBeforeSpawn(instance);
    spawnDetachedService(instance);
    waitForAuthenticatedReadinessAndSpeechV2();
  });
}
```

The real manager keeps its receipt after a slow or failed startup instead of claiming success or killing a process by name. This favors conservative refusal over an aggressive “self-healing” action with uncertain ownership. (`services\transcription\manage.py:161-200`, `start`.)

The cold-start chain

After checking the expected model file exists, `start` generates a fresh instance UUID and writes the receipt before spawning. It launches the current Python interpreter with `server.py`, a detached process/new process group, no stdin, closed inherited descriptors, and a generic local log. It then polls for up to 45 seconds: process exit fails immediately; a listening port triggers authenticated instance validation, PID receipt update, readiness validation, and speech-v2 capability validation. (`services\transcription\manage.py:161-200`, `start`.)



10. Native service lifecycle: readiness without accidental capture - diagram 2. Source-linked; status follows the chapter.

`production_app` loads the model on the executor before returning its aiohttp application. A model error leaves the factory unavailable rather than downloading a replacement. `/health` then reports 503 when unready; runtime management can still describe that unavailable service after it starts listening. (`services\transcription\server.py:162-168`, `health`; `services\transcription\server.py:455-466`, `production_app`.)

Configuration is explicit, not discovered from a meeting

Configuration	Consumer and behavior
`PHANTOM_TRANSCRIPTION_TOKEN_FILE`	Server token source; manager supplies it.
`PHANTOM_TRANSCRIPTION_TOKEN`	Server fallback token source; launcher also passes a token to the app.
`PHANTOM_TRANSCRIPTION_MODEL_PATH`	Required explicit model directory for the production server.
`PHANTOM_TRANSCRIPTION_HOST`	Manager fixes `127.0.0.1`; server allows `127.0.0.1` or `0.0.0.0`, not arbitrary names.
`PHANTOM_TRANSCRIPTION_INSTANCE`	Managed service instance UUID; enables matching-instance shutdown.
`PHANTOM_TRANSCRIPTION_URI`	Launcher supplies the app's literal-loopback speech URI.
`OPENBLAS_NUM_THREADS`, `OMP_NUM_THREADS`	Manager sets both to `1`.

The server binds port 8765; there is no production port environment setting in `main`. Token values are intentionally omitted here. (`services\transcription\server.py:69-79`, `load_token`; `services\transcription\server.py:475-492`, `main`; `services\transcription\manage.py:167-176`, environment; `scripts\Run-PhantomFudge.ps1:43-55`, app environment.)

Shutdown: authority and idleness are rechecked

The manager first requires explicit idle state, then posts `/v1/shutdown` with Bearer authentication and `X-Phantom-Instance`. The server independently rejects browser origins, bad auth, queries/bodies, missing management ownership, mismatched UUIDs, and active transcription. That second active check closes the gap between the manager's earlier runtime query and the actual shutdown request.
(`services\transcription\manage.py:130-138`, `stop_owned`; `services\transcription\server.py:196-206`, `managed_shutdown`.)

On acceptance, the server marks itself stopping immediately, schedules managed exit after 0.2 seconds, and responds `{"status":"stopping"}`. The manager then waits up to 12 seconds for the port to become unused, removing the receipt only after that observation. If the port remains occupied, the receipt remains and the operation fails. No arbitrary OS termination is used as a timeout fallback.
(`services\transcription\server.py:207-209`, `managed_shutdown`; `services\transcription\manage.py:137-144`, `stop_owned`.)

The timeouts are per phase, not one promised shutdown duration: management HTTP has a two-second timeout, listener disappearance has a twelve-second polling budget, and lock acquisition has its own fifty-second budget. Likewise, a receipt with no reachable listener is not silently erased by `stop`; the manager refuses to infer which process, if any, still owns outstanding startup work.
(`services\transcription\manage.py:49-50`, `request`; `services\transcription\manage.py:76-85`, lock wait; `services\transcription\manage.py:139-144`, listener wait; `services\transcription\manage.py:203-214`, `stop`.)

Application launch and deployment tradeoffs

`Run-PhantomFudge.ps1` first recognizes an existing app at the expected executable path and requests reveal without building or restarting the service. Otherwise it builds a source checkout unless `-NoBuild`, skips build for a packaged executable, optionally starts the service, and launches the app with token/URI environment configuration. `-NotesOnly` skips service startup; it is not a separate microphone implementation. (`scripts\Run-PhantomFudge.ps1:10-55`, launcher.)

Native Python is the normal path. `-Docker` must be selected explicitly and requires available recipes and an existing engine; the start script does not install Docker. This retains a portable headless service interface without making containers a prerequisite for the native application. Azure answers, source-scoped question commit, and speech v3 remain design-only; the server advertises and the manager checks speech v2, not those planned extensions. (`scripts\Start-Transcription.ps1:8-36`, native/Docker branches; `docs\AI-ANSWER-ARCHITECTURE.md:124-158`, proposed source commit; `services\transcription\server.py:179-193`, `runtime_info`; `services\transcription\manage.py:125-127`, `supports_current_audio`.)

Evidence and operational checkpoints

Management tests cover unauthenticated queries, browser origins, wrong instance UUIDs, unmanaged shutdown, active-session refusal, and idle shutdown. A dual-source test confirms that two recognizers still occupy one active runtime and prevent management stop until final drain completes. Manager unit tests reject missing/mismatched receipts and redirects. These are maintained test definitions, not newly executed results. (`services\transcription\tests\test_management.py:38-90`, management cases; `services\transcription\tests\test_management.py:92-122`, dual/session controls; `services\transcription\tests\test_management.py:126-150`, `ManagerTests`.)

The native lifecycle script exercises blank-window reveal/minimize/hide/close and refuses to touch an existing app or service. It is a different evidence layer from service authentication, and neither layer qualifies live microphones, meeting capture exclusion, or acoustic quality. No lifecycle script, service, installation, or hardware probe was run for these chapters. (`scripts\Test-NativeLifecycle.ps1:16-22`, preconditions; `scripts\Test-NativeLifecycle.ps1:69-103`, lifecycle checks; `docs\QUALIFICATION.md:1-7`, qualification boundary.)

Reader checkpoints

1. Why is a matching PID weaker evidence than the authenticated instance contract used here?
2. A ready listener advertises only speech v1 but is active. Trace every reason the launcher must refuse replacement.
3. What remains on disk when startup exceeds 45 seconds, and why is that not a successful ready result?
4. Distinguish stopping capture, finishing recognition, stopping the Python service, and closing the native window.

Source-reading route

Follow initialization, native start wrapper, manager ownership, server management endpoints, application launcher, and management tests. Together they describe a local, explicitly owned service—not an automatic recorder or an external process supervisor.

Source: docs/handbook/11-state-machines-and-failure-containment.md

11 — State Machines and Failure Containment

Overview: a Boolean cannot describe a shutdown

“Running” answers whether a session is currently considered active. It does not answer whether a device has released its capture thread, whether queued audio has reached the recognizer, whether final text has returned, or whether a new session may safely reuse the UI's transcript IDs. Phantom Fudge's most important lifecycle rule is therefore:

> `IsRunning == false` **does not mean fully stopped, drained, and reusable.**

`Session.RequestStop` immediately sets `Running=false`; `StopSessionAsync` and transport disposal still have substantial work afterward. The UI separately holds `audioStopping` through its awaited Stop path. A documented review found and fixed a stop/restart race by preserving this distinction. (src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:29,363-423,504-518; src\PhantomFudge.App\MainWindow.xaml.cs:681-708; docs\IMPLEMENTATION-STATUS.md:23-25)

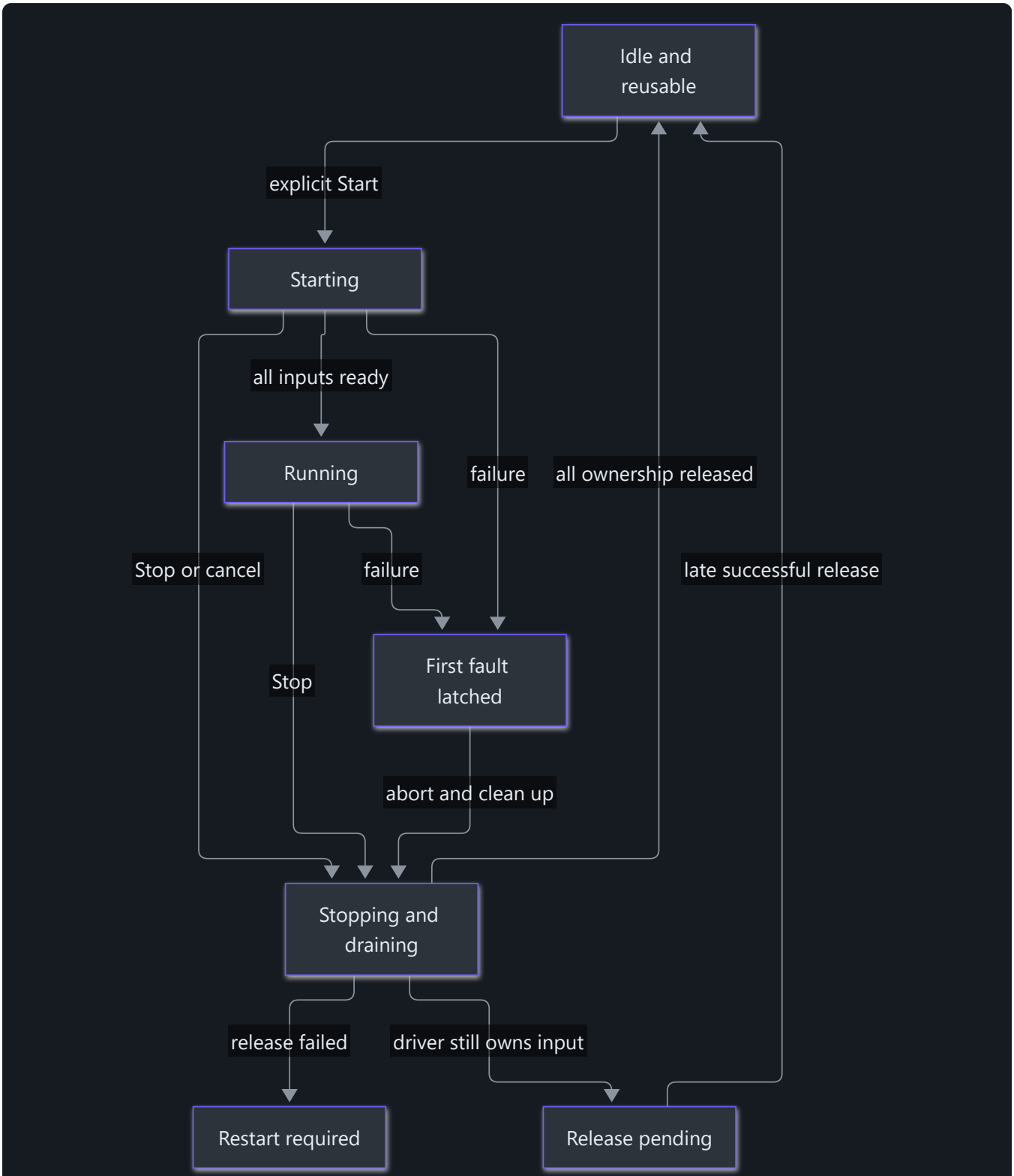
Newcomer path: follow the normal Stop sequence before reading the locks. **Principal-engineer path:** inspect queued-start invalidation, ownership after driver timeouts, and the difference between graceful drain and fault abort. The diagrams below name conceptual states; the production code implements them through fields, tasks, locks, and guards, not one central state enum.

Architecture: several coordinated state dimensions

Dimension	Actual representation	What it must not imply
Interaction	`presenting`, WPF visibility, `fileBusy`	A visible window need not be editable or recording.
Capture setup	`protectionReady`, `lastProtection`	Successful setup is not remote capture qualification.
UI audio operation	`audioBusy`, `audioStopping`, `audioStopTask`	`IsRunning` alone is insufficient for enabling commands.
Audio ownership	`_session`, `_pendingCaptureRelease`, `_captureReleaseFailed`	No active session does not necessarily mean no worker owns a device.
Transport	ready/fault/stop flags, sender/receiver tasks	Closing admission does not mean queued audio has drained.

These fields live at their relevant boundaries rather than in a generic workflow framework. (src\PhantomFudge.App\MainWindow.xaml.cs:31-46; src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:15-22;

src\PhantomFudge.App\Audio\TranscriptionSocket.cs:44-67)



11 — State Machines and Failure Containment - diagram 1. Source-linked; status follows the chapter.

The “release pending” distinction comes from a concrete Start guard: it rejects while the prior release task is incomplete or a release failure has been recorded. A timed-out operation can finish later; a recorded disposal failure remains a separate restart-blocking condition.
(src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:78-82,384-416,454-459)

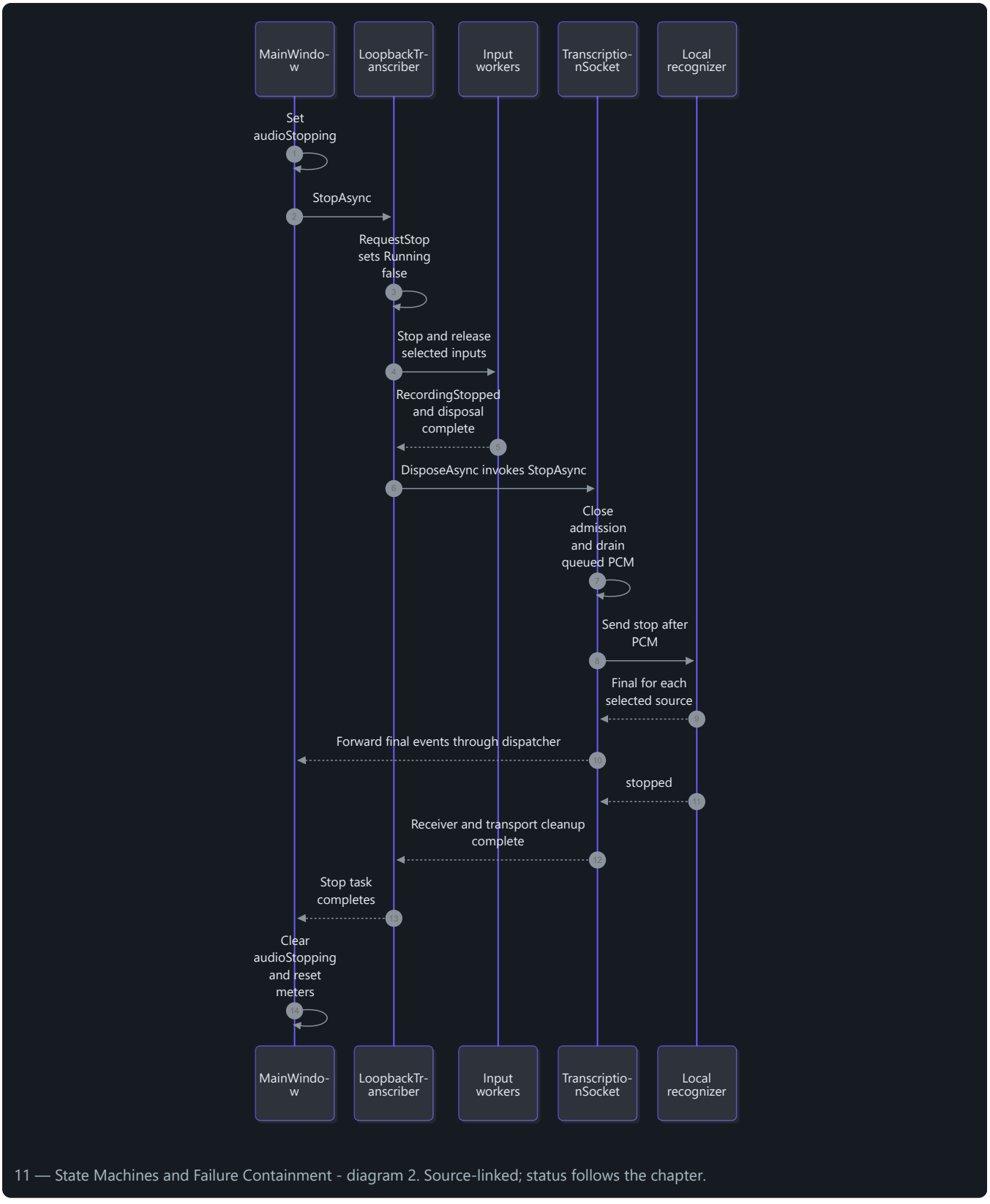
Components: who owns each decision?

`MainWindow` owns user intent and command availability. `LoopbackTranscriber` owns selected inputs and their lifecycle. `WasapiInput` owns an endpoint, capture object, and endpoint-notification registration. `TranscriptionSocket` owns one authenticated connection and bounded audio queue. `TranscriptBuffer` owns accumulated text and session-local ordering. No one layer can safely infer all the others' completion from a single flag. (src\PhantomFudge.App\MainWindow.xaml.cs:589-708; src\PhantomFudge.App\Audio\WasapiInput.cs:11-18,97-119; src\PhantomFudge.App\Audio\TranscriptionSocket.cs:10-47; src\PhantomFudge.Core\TranscriptBuffer.cs:8-11,23-39)

The UI invokes `BeginSession` only after its Start guards pass. It rejects Start during startup, stopping, or running; mode selection and transcript clearing apply equivalent busy/stopping restrictions. `UpdateAudioControls` mirrors these rules visually, but handlers also enforce them: disabling a button is not the only protection against reentrant or programmatic calls. (src\PhantomFudge.App\MainWindow.xaml.cs:589-625,641-668,771-776)

Data flow: a normal Stop is a protocol

`StopTranscriptionAsync` returns an already-pending stop task when one exists. Otherwise it sets `audioStopping` *before* cancelling startup and starting cleanup. This gives Hide, Exit, the Stop button, and other callers a shared completion operation rather than independently resetting session state. (src\PhantomFudge.App\MainWindow.xaml.cs:681-708)



11 — State Machines and Failure Containment - diagram 2. Source-linked; status follows the chapter.

This is the no-fault path. Input shutdown precedes transport drain so normal final capture callbacks can still contribute queued PCM. The sender consumes its channel to completion, then sends `stop`. The receiver accepts `stopped` only after Stop was sent and, for protocol v2, after a post-Stop final from every selected source. An empty final still satisfies that protocol requirement. (src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:377-418,425-468; src\PhantomFudge.App\Audio\TranscriptionSocket.cs:232-279,313-331,437-465)

The guarantee is conditional: a timeout, device failure, malformed response, or connection loss takes a fault path where final text can be unavailable. The status messages say so rather than reporting a clean complete transcript. (src\PhantomFudge.App\Audio\TranscriptionSocket.cs:271-280; src\PhantomFudge.App\MainWindow.xaml.cs:697-700)

Implementation: concurrency tools serve different purposes

Serialization does not cancel queued intent

`SemaphoreSlim _lifecycle` serializes asynchronous Start/Stop/Dispose bodies. But a semaphore alone would allow a Start already waiting behind another operation to run after Stop. The controller therefore captures `_stopVersion` before waiting, increments that version in `RequestStop`, and compares it under `_requestLock` before publishing a new session. A queued stale Start is rejected rather than resurrecting recording. (src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:71-91,155-189)

After connection acceptance, each selected input starts on a worker task. Running becomes true only when every requested input started, no stop/failure is recorded, and no input already reported `RecordingStopped`. Thus Both is all-requested-inputs behavior, not "start whichever device happened to succeed." (src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:98-132)

The first fault wins

`SignalFault` uses `Interlocked.CompareExchange` to store one safe failure message. Subsequent simultaneous faults return early. It disables audio admission flags, clears Running, requests cancellation, aborts transport, and schedules serialized cleanup. The cleanup checks that its session is still the controller's current session. (src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:338-360)

The code explicitly avoids acquiring another input's `AudioGate` while processing a callback failure. Otherwise two failing sources could each hold one gate and wait for the other. Per-input volatile admission flags and transport admission close the path without cross-track lock acquisition. This is a concrete deadlock-avoidance choice, not a reason to remove all synchronization. (src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:235-246,338-349)

Timeouts bound waiting, not driver execution

Deadline	Meaning in the implementation
Connection: 10 seconds	Deadline includes the service's ready response.
Capture startup: 10 seconds	Wait for selected input startup tasks.
Capture stop/release: 5 seconds	Bound waits around release and recording-stop completion.
Transport flush: 15 seconds	Bound sender drain, final acknowledgement, and receiver completion.

These are operation-specific waits, not a promised sum or end-to-end latency SLO. (src\PhantomFudge.App\Audio\TranscriptionSocket.cs:20-21,138-152,255-268; src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:12-13,108-111,395-402,437-444)

A hung driver is not forcibly made safe by cancelling a managed token. Release workers retain ownership and eventually finish startup/join/disposal if the driver returns. Inputs release independently, so a stuck microphone does not prevent the responsive Computer input from being released. The UI thread does not perform these joins. `WasapiInput.Dispose` also attempts notification and endpoint cleanup in nested `finally` blocks. (src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:106-109,384-418,425-468; src\PhantomFudge.App\Audio\WasapiInput.cs:97-119)

Fail closed without inventing a complete transcript

The socket budgets two seconds of PCM per source: 64,000 bytes at 16 kHz, mono, 16-bit; total pending PCM is capped at 128,000 bytes. A queue overflow faults the session rather than silently dropping frames and continuing a success-shaped transcript. Released frame arrays are cleared, but this is not a secure-erasure guarantee for every memory copy or crash dump. (src\PhantomFudge.App\Audio\TranscriptionSocket.cs:13-18,182-229,543-550; docs\QUALIFICATION.md:137-138)

Endpoint validation restricts the transport to literal loopback, and the HTTP handler disables proxies and redirects. Recognizer error bodies are schema-checked but replaced with fixed local messages rather than displayed verbatim. This limits accidental propagation of paths, secrets, or transcript text through an error surface. It is not a claim that every possible exception everywhere in the app is contained. (src\PhantomFudge.App\Audio\TranscriptionSocket.cs:26-31,73-89,389-405)

Device notifications invalidate only the explicitly selected endpoint, once, and dispatch work away from the notification callback. Default-device changes do not switch input identity. A lost selected source faults the entire session; the app does not quietly substitute a microphone or another playback device. (src\PhantomFudge.App\Audio\RenderEndpointNotification.cs:10-29; src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:195-200,338-349)

Worked example: the stop/reset race

Suppose session A has accepted final ID 20. Stop makes Running false, but final ID 21 is still returning. If the UI resets `TranscriptBuffer.lastSegmentId` for session B now, the late A final could advance B's ID baseline to 21. B's new final ID 1 would then be ignored by `AddFinal`. Text would disappear from the new session even though the user saw a plausible restart. This is an explanatory interleaving derived from the actual reset/deduplication logic and the recorded bug, not a claim of a newly executed reproduction. (src\PhantomFudge.Core\TranscriptBuffer.cs:23-38,48-56; docs\IMPLEMENTATION-STATUS.md:23-25)

The repaired UI keeps Start, source-mode changes, and Clear blocked through its awaited stop task. A synthetic regression explicitly sets `audioStopping`, attempts those actions, and verifies that the transcript/session selection remain unchanged. Controller-level ownership guards remain necessary underneath this UI protection. (src\PhantomFudge.App\MainWindow.xaml.cs:589-592,641-668,771-776,951-958; src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:78-88)

The following **Python-shaped pseudocode is explanatory, not production code**:

```
async def stop_from_ui():
    if stop_task_is_pending():
        await existing_stop_task()
        return
    state.stopping = True
    try:
        await controller.stop()  # Running may already be False here.
    finally:
        state.stopping = False

def may_start_or_reset():
    return not (state.starting or state.stopping or controller.running)
```

C# `Task` tracks completion; `CancellationToken` requests cooperation; `SemaphoreSlim` serializes async ownership transitions; `Interlocked` performs atomic state changes. They are complementary, not interchangeable spellings of "thread safe." `Dispatcher.BeginInvoke` supplies the separate requirement that WPF control updates run on the UI thread. (src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:74-91,155-188,338-360; src\PhantomFudge.App\MainWindow.xaml.cs:55-74,149-153)

Window and document failures remain separate

Protection invalidation clears readiness and hides before awaiting Stop. It preserves note/transcript memory. Mode changes similarly hide and stop before changing native interaction policy. Exit waits for Stop, then evaluates unsaved work; it does not confuse stopped audio with permission to discard text. Saving has its own `fileBusy` guard and only updates the saved baseline after successful I/O. (src\PhantomFudge.App\MainWindow.xaml.cs:204-241,270-283,384-415)

Protection errors use a generic local notice rather than an unprotected automatic message box. This is an application containment policy for detected failures—not a universal capture boundary, nor crash recovery. (src\PhantomFudge.App\LocalFailureNotice.cs:9-29; src\PhantomFudge.App\MainWindow.xaml.cs:139-143)

Tests, exercises, and qualification boundaries

Targeted synthetic tests verify:

- Missing Mic final before `stopped` is rejected: `V2MissingFinalAsync`. (tests\PhantomFudge.Audio.Tests\DualSourceTests.cs:352-367)
- Stop/Dispose invalidates a queued Start before opening hardware: `V2QueuedStartAsync`. (tests\PhantomFudge.Audio.Tests\DualSourceTests.cs:471-495)
- Repeated sessions detach callbacks; simultaneous source faults release both inputs. (tests\PhantomFudge.Audio.Tests\DualSourceTests.cs:652-743)
- Hung startup retains ownership, hung Stop releases the responsive input, and disposal failures prevent unsafe restart. (tests\PhantomFudge.Audio.Tests\DualSourceTests.cs:747-853)

The maintained evidence records 41 Core tests, 46 native synthetic groups, 71 backend tests, and four manager-upgrade guards. Those are existing results, not tests rerun for this chapter. They do not establish actual microphone permissions, hardware echo behavior, OS microphone-indicator visibility, or remote cursor inclusion. (docs\IMPLEMENTATION-STATUS.md:17-25; docs\QUALIFICATION.md:127-138)

Exercise: Why retain `_pendingCaptureRelease` after the visible session stops?
Answer: A worker may still own a device; allowing another Start would obscure ownership.

Exercise: Why not silently drop PCM when the queue fills?
Answer: Continuing would hide a gap in the audio-to-text history. The implemented policy faults instead.

Exercise: Does cancelling startup prove its driver worker has terminated?
Answer: No. Cancellation is cooperative; ownership remains until release completes. These answers follow the guards and cleanup paths described above. (src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:81-82,384-418; src\PhantomFudge.App\Audio\TranscriptionSocket.cs:227-229)

References and maintenance checkpoints

- UI lifecycle and audio controller.
- Transport state, endpoint owner, and device notifications.
- Transcript state, failure notice, and synthetic lifecycle tests.
- Recorded implementation evidence and qualification gaps.

Any future per-source commit or Azure Ask implementation must establish its own request/session ownership rules. Those features remain design-only; current Stop drains a selected-input session, not an implemented “commit question and ask cloud model” operation. (docs\IMPLEMENTATION-STATUS.md:5-8; src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:155-164)

12. Performance and Benchmarks

Why a fast decoder is not necessarily a responsive application

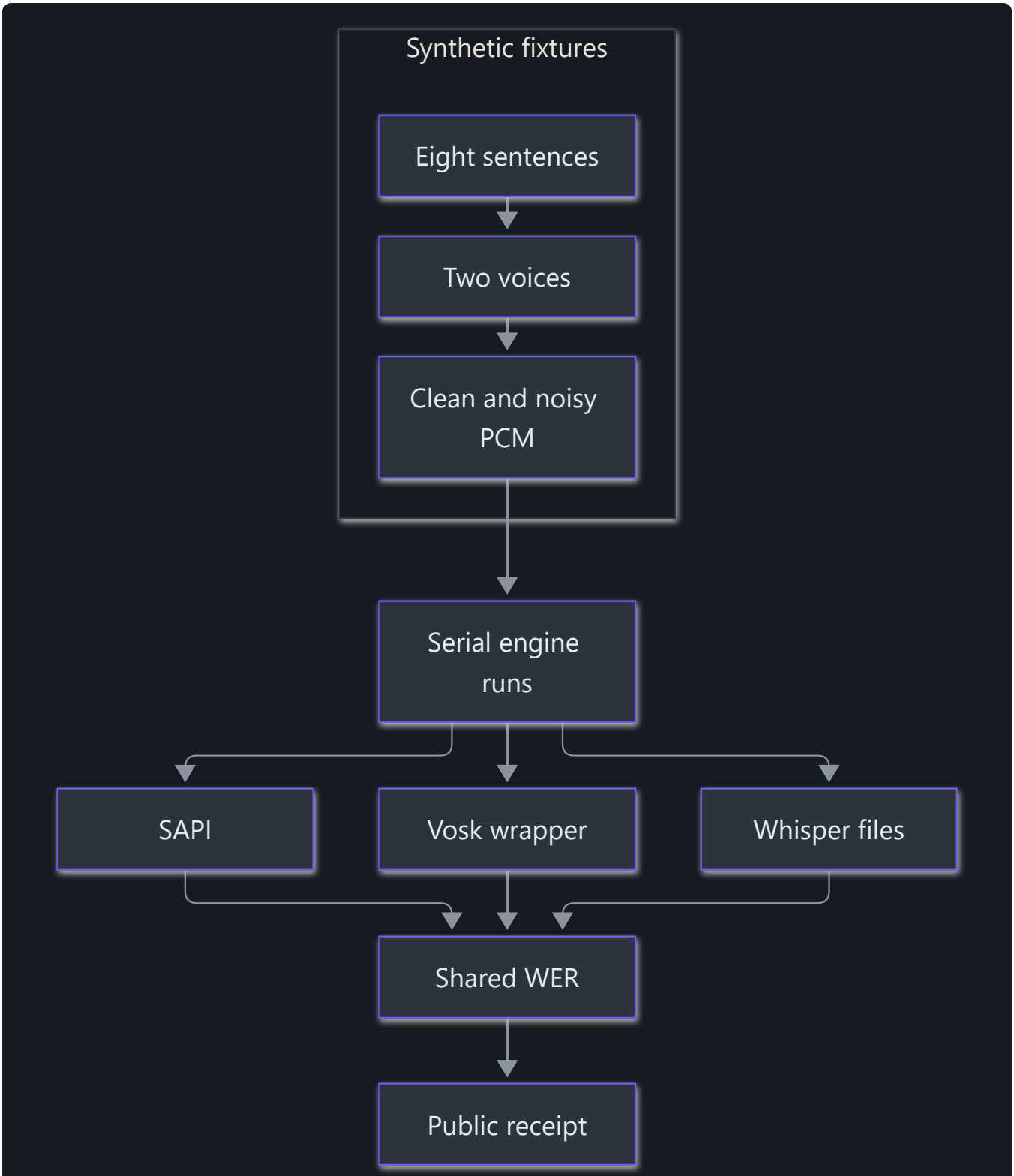
Performance starts with a question, not a stopwatch. “How quickly can this engine decode a complete recording?” differs from “When does the presenter see a useful partial?” An engine can process a finished file faster than its duration yet wait too long before showing live text. The repository deliberately separates unpaced file decoding from a small paced-delivery experiment; neither measures the complete capture-to-WPF path. (tools\SpeechBench\bench.py:207-230, `worker.decode`; docs\SPEECH-BENCHMARK.md:88-118, “Speed and live-update limits”.)

This chapter analyzes existing evidence, not a newly executed benchmark. The public synthetic receipt records **32 cases, 164.070 seconds of audio, two serial warm passes, and 292 normalized reference words per pass**. Repeating a recording improves timing observations; it does not create another independent speaker or double the accuracy sample. (tools\SpeechBench\results\synthetic-20261008.json:13-22, experiment metadata; tools\SpeechBench\results\synthetic-20261008.json:65-109, `summary`; tools\SpeechBench\bench.py:354-369, `run`.)

The comparison is also not C# versus Python. SAPI, Vosk, and Whisper are different recognition engines with different models and exposed APIs. C# hosts the installed Windows dictation engine; Python hosts Vosk and faster-whisper. Language-host differences are confounded with engine differences, so the experiment cannot isolate language overhead. (tools\SpeechBench\SapiBridge.cs:113-145, `Recognize`; tools\SpeechBench\bench.py:188-255, `worker`; docs\SPEECH-BENCHMARK.md:10-15, engine-versus-language distinction.)

Architecture of the experiment

The harness gives each engine the same completed PCM fixtures, runs engines serially, and scores their materialized transcripts with one normalization function. The diagram describes that file-only path, not device capture. (tools\SpeechBench\bench.py:106-152, `make_fixtures`; tools\SpeechBench\bench.py:319-369, `run`.)



12. Performance and Benchmarks - diagram 1. Source-linked; status follows the chapter.

Components and controlled inputs

Component	Responsibility and experimental consequence
`corpus.json`	Eight original sentences cover ordinary speech, technical terms, and numbers; coverage is intentionally small.

Component	Responsibility and experimental consequence
`SapiBridge.Synthesize`	Writes synthesized PCM to a memory stream and then files, never a playback device.
`make_fixtures`	Adds identical endpoint padding and a reproducible noisy condition.
`worker`	Loads one model, excludes one warm-up decode, then times full passes.
`errors` / `aggregate`	Uses common tokenization and corpus-weighted edit counts.
`verify_results`	Checks hashes, row coverage, scores, and timing totals in a local run receipt.

Sources: (tools\SpeechBench\corpus.json:1-10, corpus entries); (tools\SpeechBench\SapiBridge.cs:53-73, `Synthesize`); (tools\SpeechBench\bench.py:125-150, `make_fixtures`); (tools\SpeechBench\bench.py:257-279, `worker`); (tools\SpeechBench\metrics.py:28-62, `errors`, `aggregate`); (tools\SpeechBench\bench.py:282-316, `verify_results`).

Fixture construction matters because endpoint behavior can dominate short utterances. Every case receives 250 ms of leading silence and 1,000 ms of trailing silence. Noise uses a deterministic Gaussian generator and a target 15 dB ratio against active synthesized samples whose absolute amplitude is at least 100. Noise also covers padding. This is a precise laboratory transformation, **not recorded office noise** or a full-file SNR definition. (tools\SpeechBench\bench.py:125-150, `make_fixtures`.)

The two voices are Microsoft David Desktop and Microsoft Zira Desktop. Combined with clean/noisy variants, eight sentences become 32 cases, but the conditions remain correlated. Technical vocabulary examples include Kubernetes and authentication; they do not establish accent, overlap, distant-microphone, or real-meeting coverage. (tools\SpeechBench\corpus.json:2-9, sentence categories; docs\SPEECH-BENCHMARK.md:39-52, fixture description; docs\SPEECH-BENCHMARK.md:220-223, coverage limits.)

The scoring algorithm

Normalize before aligning

`normalize` applies Unicode NFKC, lowercases, expands `%`, converts numeric strings through English cardinal words, removes apostrophes, and extracts alphabetic tokens. It does not drop stop words or repair one engine's terminology after seeing its output. Consequently, the benchmark ignores capitalization and punctuation but retains many genuine lexical errors. (tools\SpeechBench\metrics.py:8-25, `NORMALIZATION`, `normalize`.)

Normalization is a measurement policy, not semantic interpretation. `9:30` becomes two cardinals, while `9.30` uses decimal “point” wording; these can describe a similar intended time but score differently. Conversely, “don't” and “dont” become identical tokens. The tests make several such choices executable rather than leaving readers to infer them. (tools\SpeechBench\test_metrics.py:7-17, `test_numbers_are_common`, `test_apostrophes`; docs\SPEECH-BENCHMARK.md:74-77, numeric-format limitation.)

Levenshtein alignment and ties

For reference prefix length `i` and hypothesis prefix length `j`, the dynamic-programming table stores `(total edits, substitutions, deletions, insertions)`. Equal tokens inherit the diagonal cell. Unequal tokens compare substitution, deletion, and insertion candidates, each costing one additional edit. When totals tie, candidate order prefers substitution, then deletion, then insertion. This stabilizes the reported S/D/I breakdown without making the alignment a semantic explanation of the recognizer's mistake. (tools\SpeechBench\metrics.py:28-55, `errors`.)

The table contains `(reference length + 1) × (hypothesis length + 1)` cells; thus this implementation's time and storage grow with their product. That is acceptable for the short fixture sentences, but a future long-recording evaluator should account for the matrix explicitly rather than assuming a streaming metric. This complexity follows directly from the allocation and nested loops. (tools\SpeechBench\metrics.py:31-48, `errors`.)

Corpus WER is:

```
WER = sum(substitutions + deletions + insertions)
      / sum(normalized reference words)
```

Do not average sentence percentages: one error in a one-word sentence should not outweigh one error in a much longer sentence simply because each sentence gets equal weight. An empty reference produces undefined WER; an empty hypothesis produces deletions. WER can exceed 100%, so `100 − WER` is not a general “accuracy percentage.” (tools\SpeechBench\metrics.py:49-62, `errors`, `aggregate`; tools\SpeechBench\test_metrics.py:24-37, empty-input and weighted-aggregation tests.)

Reading the measured result

These are historical results on the synthetic corpus, rounded for readability:

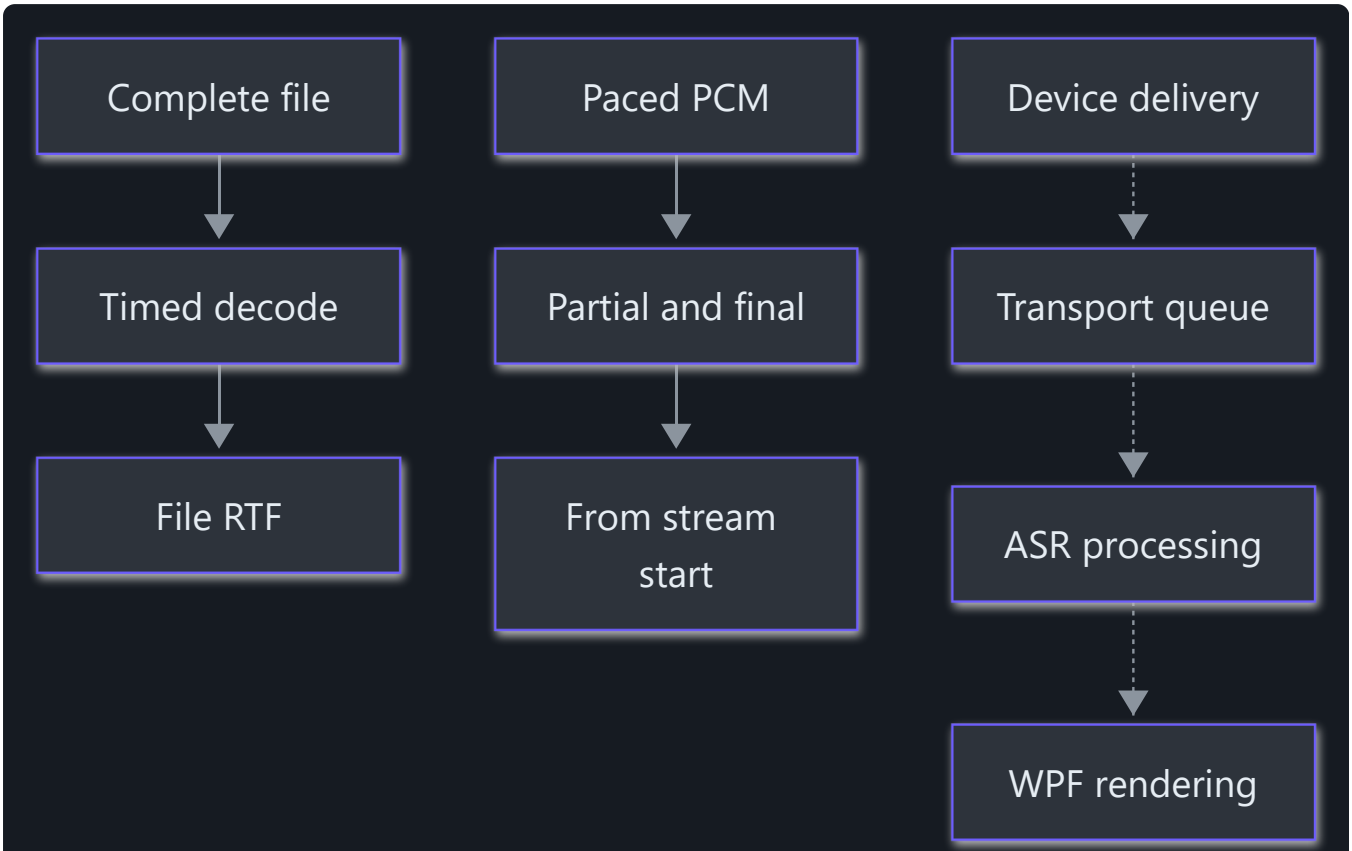
Engine/configuration	WER	S / D / I	Warm decode median	RTF
Installed SAPI en-US dictation	45.89%	99 / 2 / 33	10.139 s	0.062
Vosk small-en-us-0.15	8.22%	14 / 0 / 10	41.754 s	0.254
faster-whisper base.en, CPU int8, beam 5	4.79%	9 / 0 / 5	25.745 s	0.157

The receipt contains the unrounded numbers and the separate pass totals. Its `transcripts_changed_across_repeats` arrays are empty: the repeats did not change the scored text. SAPI's clean/noisy WER is 17.12%/74.66%; Vosk is 8.22%/8.22%; Whisper is 4.79%/4.79%. Equal percentages in two conditions are observations on these fixtures, not noise-invariance guarantees. (tools\SpeechBench\results\synthetic-20261008.json:65-173, `summary`; docs\SPEECH-BENCHMARK.md:17-27, configuration table.)

RTF divides median decoding seconds by audio seconds. A value below one says this completed corpus decoded faster than its playback duration. It says nothing by itself about the time required to collect an utterance, endpoint it, authenticate a service connection, or render a partial.

(tools\SpeechBench\bench.py:355-369, `run`; docs\SPEECH-BENCHMARK.md:97-118, timing exclusions.)

Which clock did the experiment actually observe?



12. Performance and Benchmarks - diagram 2. Source-linked; status follows the chapter.

Dashed edges identify application stages outside the file experiment. In the separate paced Vosk probe, first partials arrived at **0.976 and 0.960 seconds from stream start**, after 0.9 seconds of PCM had been supplied. Both natural finals arrived near **3.807 seconds from stream start**, not 3.807 seconds after speech ended. Without an annotated last spoken sample, those figures cannot be converted into end-of-speech lag. Two observations also cannot establish a p95 latency distribution. (tools\SpeechBench\results\synthetic-20261008.json:9027-9151, `paced_pcm`; docs\AI-ANSWER-EVALUATION.md:9-37, A1 and latency definitions.)

Timing boundaries, fairness, and provenance

The Python worker reads audio before timing, measures import/verification/load separately, and consumes Whisper's segment generator inside the decode call. PCM-to-float conversion is outside Whisper timing. SAPI times wave-stream binding through recognition completion, excluding initial file reads. These are exposed-API measurements, not isolated native-kernel timings. (tools\SpeechBench\bench.py:184-255, `worker`; tools\SpeechBench\SapiBridge.cs:75-110, `Decode`; tools\SpeechBench\SapiBridge.cs:123-157, `Recognize`.)

“Cold load” means a new model instance, **not flushed OS caches**. The fixed order is SAPI → Vosk → Whisper. The recorded Windows 11 machine uses a Ryzen 7 PRO 7840U; common affinity limits placement to four logical processors, while requested native thread counts do not make all engines schedule identically. Thermal

state, unrelated work, and same-vendor voices remain confounders. No confidence interval follows from two serial passes. (tools\SpeechBench\results\synthetic-20261008.json:4-22, environment/order; docs\SPEECH-BENCHMARK.md:90-95, cold-instance definition; docs\SPEECH-BENCHMARK.md:120-133, resource boundaries.)

There is also a living-document provenance gap: the checked-in receipt's recorded `bench.py` SHA-256 does not match the current harness file; the recorded corpus, metric, SAPI bridge, requirements, wrapper script, and metric-test hashes do match at this documentation inspection. Therefore, treat the receipt as **historical evidence**, not proof that today's entire harness was rerun. The stored hash identifies that distinction; this chapter neither reconstructs the older harness nor attributes the difference to an unverified cause. (tools\SpeechBench\results\synthetic-20261008.json:55-64, `source_sha256`; tools\SpeechBench\bench.py:378-387, receipt construction.)

Worked pseudocode: auditing without decoding audio

The following is explanatory pseudocode, not a benchmark invocation. It audits the public result's internal arithmetic and repeat coverage; unlike `verify_results`, it cannot establish unchanged raw WAVs from the public receipt alone. (tools\SpeechBench\bench.py:282-316, `verify_results`; tools\SpeechBench\results\synthetic-20261008.json:175-189, public case metadata.)

```
receipt = read_public_synthetic_receipt()
for name, engine in receipt["engines"].items():
    rows = engine["rows"]
    summary = receipt["summary"][name]
    require_unique_pairs(rows, keys=("id", "repeat"))
    require_complete_case_repeat_grid(rows, receipt["cases"],
                                      receipt["repeats"])
    first_pass = [row for row in rows if row["repeat"] == 0]
    expected_errors = sum(row["substitutions"] + row["deletions"]
                          + row["insertions"] for row in first_pass)
    expected_words = sum(row["reference_words"] for row in first_pass)
    assert summary["wer"] == expected_errors / expected_words
    pass_times = [sum(r["decode_seconds"] for r in rows if r["repeat"] == k)
                  for k in range(receipt["repeats"])]
    assert summary["median_decode_seconds"] == median(pass_times)
# No capture, synthesis, download, or inference is necessary for this audit.
```

The production decision remains conservative: retain Vosk's true streaming `accept` path and its bounded decoder rotation; investigate Whisper only as optional local refinement. A complete-file WER/throughput win does not justify replacing the live path without question-fidelity and latency evidence. (services\transcription\recognizer.py:82-96, `VoskRecognizer.accept`, `finish`; docs\SPEECH-BENCHMARK.md:29-35, recommendation.)

Failure outcomes belong in the experiment

An unavailable engine is not an infinitely slow engine, a zero-error engine, or a reason to install a new component silently. The SAPI bridge records unavailable status when no en-US recognizer exists; subprocess failures likewise produce an unavailable result rather than a fabricated timing row. A comparison report must

retain that outcome and explain which configurations actually ran. (tools\SpeechBench\SapiBridge.cs:113-121, `Recognize`; tools\SpeechBench\bench.py:328-353, `run`.)

Inputs also have a contract: the loader rejects duplicate IDs and empty corpora, and the WAV reader requires uncompressed 16 kHz mono signed 16-bit PCM. Silent resampling would move preprocessing work and change the experiment without an explicit decision. Empty reference strings are technically accepted, but their undefined WER makes them inappropriate for ordinary accuracy ranking. These are methodological limits to document, not inconveniences to bypass. (tools\SpeechBench\bench.py:99-103, `pcm_from_wav`; tools\SpeechBench\bench.py:155-176, `load_cases`; tools\SpeechBench\metrics.py:49-54, `errors`.)

Exercises

1. Hand-align reference “do not restart” with hypothesis “do restart.” Explain why a modest token error can reverse intent, motivating fidelity checks beyond WER. (tools\SpeechBench\metrics.py:28-55, `errors`; docs\AI-ANSWER-EVALUATION.md:81-83, question fidelity.)
2. Calculate WER for one erroneous one-word case and one correct four-word case. Compare corpus weighting with an unweighted mean; the existing test expects 0.2. (tools\SpeechBench\test_metrics.py:34-37, `test_weighted_aggregation_not_mean_of_case_wer`.)
3. Design, but do not run, a representative follow-up: independent speakers, annotated speech end, randomized engine order, and separate warm/cold strata. State which current limitation each addition addresses. (docs\SPEECH-BENCHMARK.md:127-133, run-order limits; docs\SPEECH-BENCHMARK.md:213-223, corpus limits; docs\AI-ANSWER-EVALUATION.md:30-40, clocks.)

References

- Existing comparison and methodology.
- Public synthetic result.
- Harness, metric implementation, and SAPI bridge.
- Metric tests and harness boundary tests; the latter explicitly reject escaped output paths, implicit resampling, duplicate case IDs, and empty corpora. (tools\SpeechBench\test_harness.py:8-38, `HarnessTests`.)
- Planned answer evaluation, which must not be read as an executed cloud benchmark.

13. Proposed Azure Answer Flow

Why question selection is the architectural center

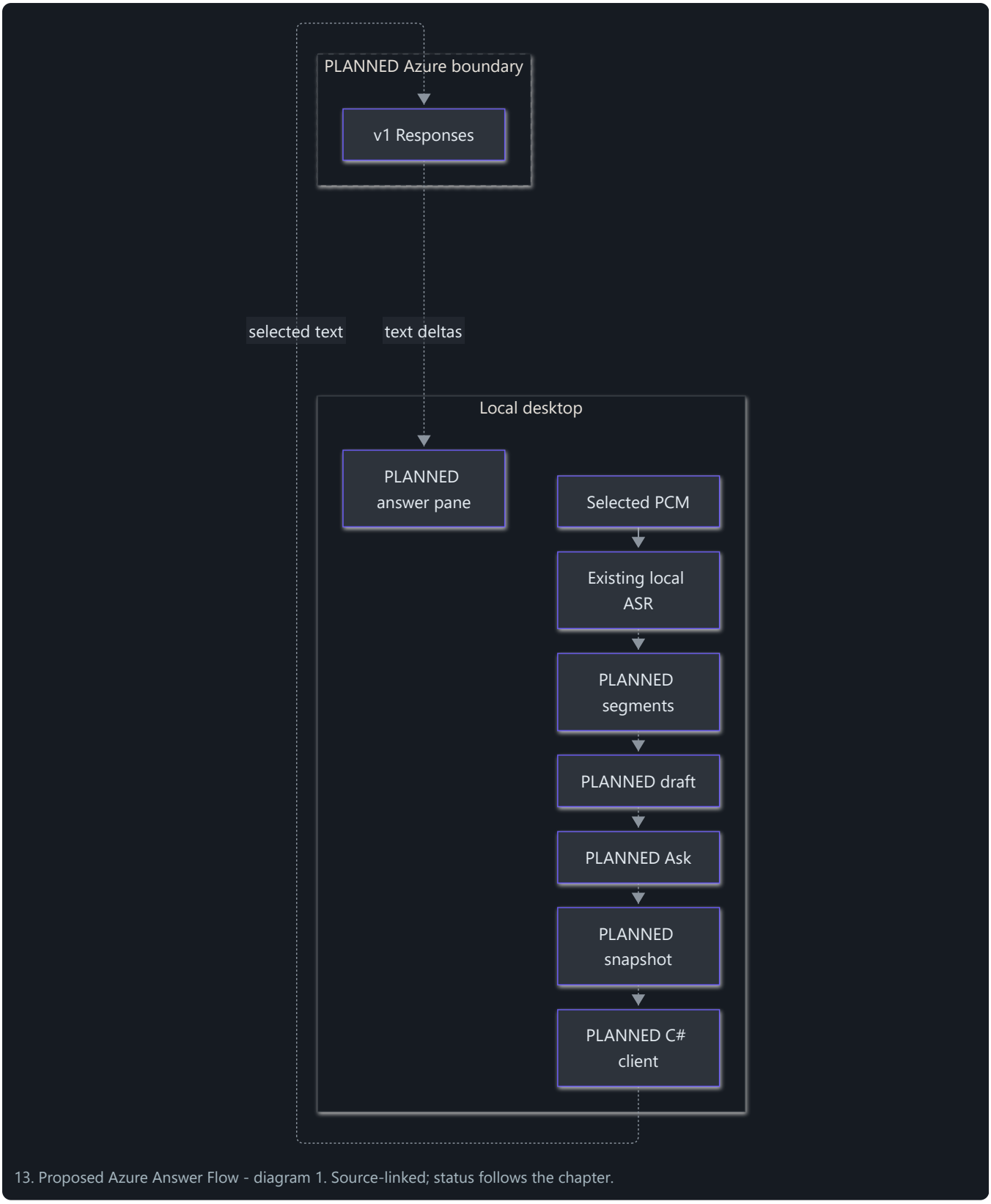
Everything concerning QuestionDraft, Ask, source commit, Azure inference, and answer rendering in this chapter is PLANNED, not implemented. The existing research is complete as a design deliverable; the draft, commit, connection, client, and UI workstreams remain unstarted. No cloud integration or Azure performance result is implied by the diagrams or pseudocode. (TODO\27_AI_ANSWER_RESEARCH.todo:29-36, completion evidence; TODO\28_QUESTION_DRAFTS.todo:3-5, status; TODO\31_STREAMING_ANSWER_CLIENT.todo:30-36, completion evidence; TODO\32_ANSWER_UI_LIFECYCLE.todo:30-36, completion evidence.)

The hard problem is not posting text to a model. It is knowing **which text the user intended to send**, keeping that text stable during asynchronous recognition, and associating every answer with the right question. A microphone may contain earlier conversation; Computer audio may continue while a question is dictated. Treating the entire transcript as a prompt silently changes the product's data boundary. The proposed solution is an explicit editable draft followed by an immutable snapshot at Ask. (docs\AI-ANSWER-ARCHITECTURE.md:44-68, deliberate submission paths; docs\AI-ANSWER-PRIVACY.md:6-20, proposed default.)

A useful mental model has three objects: the transcript is an evolving observation, the draft is the user's working selection, and the snapshot is the exact outbound request content. They are related, but none should substitute for another. In particular, a better late recognition result must not rewrite a question already submitted. (docs\AI-ANSWER-ARCHITECTURE.md:84-122, planned local records and revision rules.)

Architecture: local speech, explicit text egress

The baseline preserves native Python ASR and adds a direct asynchronous C# answer provider. Sending through the Python speech service merely because it exists would add coupling and another hop without a demonstrated requirement. Authentication, request identity, and WPF updates belong with the desktop coordinator; model network work does not belong in capture callbacks or the recognizer CPU worker. (docs\AI-ANSWER-ARCHITECTURE.md:8-38, recommended baseline.)



13. Proposed Azure Answer Flow - diagram 1. Source-linked; status follows the chapter.

Dashed paths represent proposed connections, including the cloud boundary. There is no automatic forwarding of partials, no chat reader, and no implicit full-meeting context. Bounded additional context would require explicit selection and a recorded context policy. Cloud ASR and an enterprise broker are separate optional workstreams; Docker remains final-stage packaging rather than a prerequisite. (docs\AI-ANSWER-

ARCHITECTURE.md:30-38, baseline; TODO\36_OPTIONAL_AZURE_SPEECH.todo:13-25, optionality; TODO\37_OPTIONAL_ANSWER_BROKER.todo:13-26, conditional broker; TODO\35_ANSWER_RELEASE_ACCEPTANCE.todo:23-26, Docker-last gate.)

What the existing interfaces provide—and omit

Existing component	Implemented contract	Consequence for the proposal
`AudioSource`	Stable byte values and separate wire/display names	Source identity must not be inferred from rendered labels.
`TranscriptEvent`	Type, text, optional final ID, times, source	No capture-session ID, source epoch, or draft revision exists here.
`TranscriptBuffer`	Per-source partials, flattened finals, bounded text	Display/export storage is not a segment-selection database.
`TranscriptionSocket`	Single-use session, v1/v2 negotiation, strict response fields	New controls and metadata require explicit version/capability negotiation.
`Session`	Independent decoders, one work queue, session-global final IDs	Received PCM and processed PCM must be distinguished for commit.

Sources: (src\PhantomFudge.Core\AudioSource.cs:3-30, `AudioSource`, `AudioSourceNames`); (src\PhantomFudge.App\Audio\TranscriptEvent.cs:9-15, `TranscriptEvent`); (src\PhantomFudge.Core\TranscriptBuffer.cs:6-19, `TranscriptBuffer`); (src\PhantomFudge.App\Audio\TranscriptionSocket.cs:108-160, `ConnectCoreAsync`; src\PhantomFudge.App\Audio\TranscriptionSocket.cs:482-506, property validation); (services\transcription\server.py:225-235, `Session`).

`TranscriptBuffer.AddFinal` appends a formatted line and advances one global final ID; `BeginSession` resets that ID while retaining existing text with a session separator. Therefore an ID such as `17` is not a durable cross-session reference. Likewise, `[Mic]` is presentation text, not an authenticated provenance field. A user can type that literal string. Planned selection must retain structured session/source/segment identity before formatting. (src\PhantomFudge.Core\TranscriptBuffer.cs:23-38, `BeginSession`; src\PhantomFudge.Core\TranscriptBuffer.cs:48-72, `AddFinal`; src\PhantomFudge.Core\TranscriptBuffer.cs:87-96, source formatting.)

Planned components and dataflow

The structured store would preserve segment references, source epochs, revisions, qualified sample bounds, text, and finality. A draft would record its selection, whether it remains provisional, and whether the user has edited it. A snapshot would copy exact displayed text plus provenance, creation time, and profile version. Retaining copied selected text prevents later transcript eviction from invalidating the submitted question. These are proposed records, not existing classes. (docs\AI-ANSWER-ARCHITECTURE.md:89-122, `TranscriptSegment`, `QuestionDraft`, `QuestionSnapshot`.)

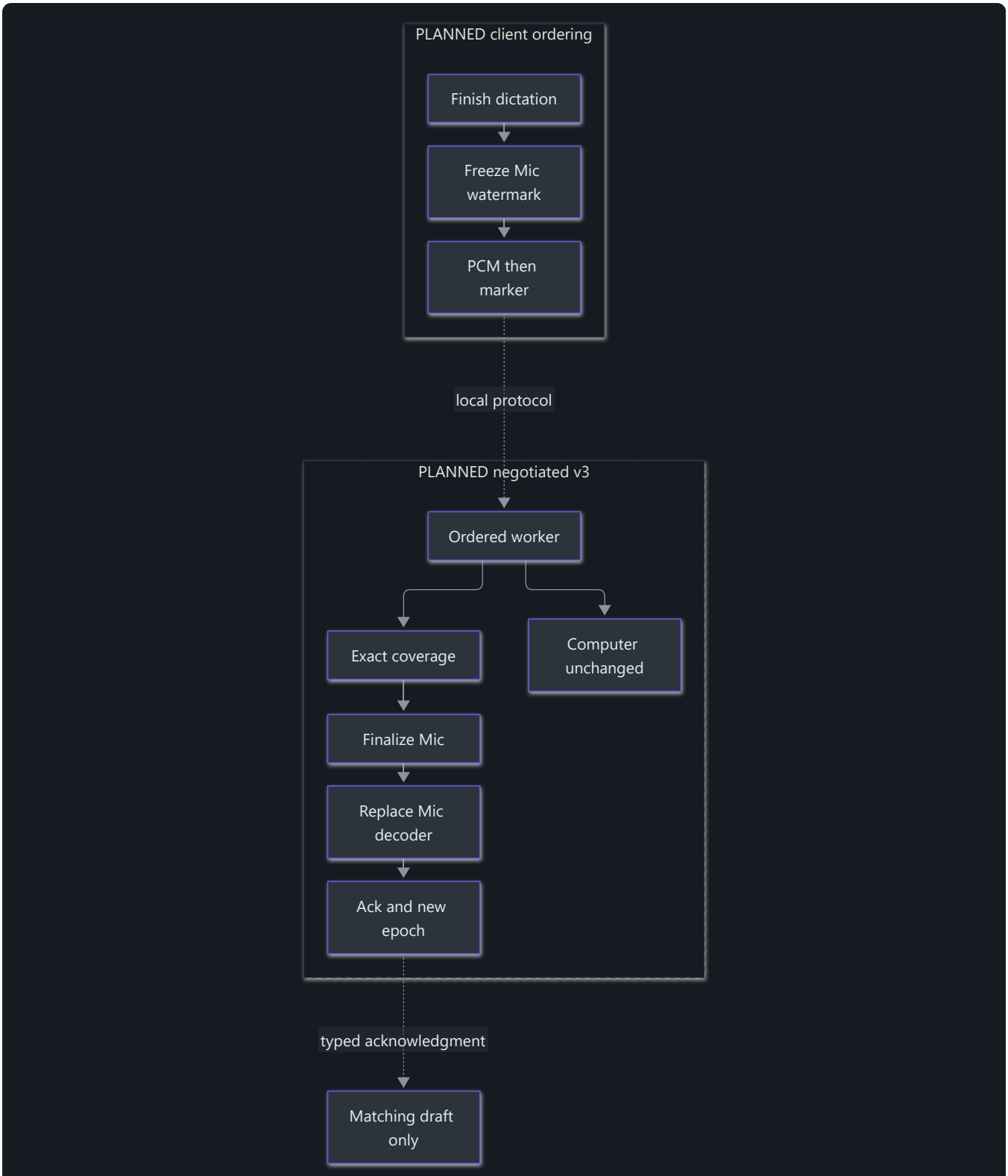
Consider “Explain Service Fabric like I am five.” The owner can dictate it, select complete Mic segments, or type and correct it. **Ask as shown** freezes that revision immediately, even when visibly provisional. **Finish dictation** instead finalizes the armed Mic span and returns it to review; Ask remains a separate action. A timeout must preserve the draft rather than silently sending an unreviewed alternative. No question mark or natural final automatically sends anything. (docs\AI-ANSWER-ARCHITECTURE.md:46-68, example and submission paths; TODO\28_QUESTION_DRAFTS.todo:24-27, explicit-send acceptance.)

The first implementation slice should use a fake answer stream. That allows snapshot, eviction, cancellation, and delayed-event tests before introducing authentication or paid inference. The provider abstraction is planned as `IAnswerProvider.StreamAsync`; a separate `IQuestionCommitter` keeps question finalization independent of the eventual recognizer backend. (docs\AI-ANSWER-ARCHITECTURE.md:211-222, proposed abstractions; docs\AI-ANSWER-ARCHITECTURE.md:262-270, delivery sequence.)

Planned algorithm: Mic-only commit as an ordered barrier

Today, `SendAudioAsync` drains queued PCM and sends a global Stop. The server accepts only the exact Stop control, then finalizes **every** selected decoder before acknowledging stopped. Calling that path to submit a question would terminate Computer transcription too. (src\PhantomFudge.App\Audio\TranscriptionSocket.cs:313-331, `SendAudioAsync`; services\transcription\server.py:247-252, `Session.receive`; services\transcription\server.py:317-327, `Session.recognize`.)

The proposed v3 capability inserts a Mic commit envelope into the **same ordered sender queue**, under the same synchronization boundary that freezes its delivered-sample watermark. A second concurrent WebSocket sender would break the intended ordering guarantee. Server processing must reach exactly that source watermark before finalization; bytes received into a queue are not evidence of decoded coverage. Current `SourceState.total_bytes` increments in receive, before the worker calls `accept`, so it cannot serve as the proposed processed counter. (docs\AI-ANSWER-ARCHITECTURE.md:124-158, commit design; services\transcription\server.py:274-287, `Session.receive`; services\transcription\server.py:328-333, `Session.recognize`.)



13. Proposed Azure Answer Flow - diagram 2. Source-linked; status follows the chapter.

The wrapper already provides a useful implementation pattern: after 30 audio seconds it retrieves a final, advances its cumulative offset, and constructs a new decoder. `finish()` alone retrieves a final but does not prepare continued recognition. The proposed commit must adapt finalize-and-replace for Mic while preserving Computer state and session-global final numbering. This is feasibility evidence, not an existing commit API. (services\transcription\recognizer.py:55-96, `VoskRecognizer`; TODO\29_QUESTION_COMMIT_LATENCY.todo:13-22, planned tasks.)

Worked pseudocode: retries without duplicate finalization

This sketch specifies invariants for a future implementation, not methods currently callable in the service:

```
def process_commit(marker, session):
    key = (session.capture_id, marker.source, marker.request_id)
    prior = session.completed_commits.get(key)
    if prior is not None:
        require(prior.parameters == marker.parameters)
        return prior.ack             # no second final or epoch increment

    track = session.sources[marker.source]
    require(marker.epoch == track.epoch)
    require(track.processed_samples == marker.through_sample)
    final = track.finalize_and_replace_preserving_offset()
    final_ref = session.record_final(final)   # including an empty final
    old_epoch = track.epoch
    track.epoch += 1
    ack = Ack(key, old_epoch, track.epoch,
              marker.through_sample, final_ref)
    session.remember_bounded_commit(key, marker.parameters, ack)
    return ack
```

Lookup precedes current-epoch validation so a genuine retry can replay its original acknowledgment after the epoch has advanced. Changed parameters under the same request ID must fail. The eventual implementation also needs a defined publication/failure boundary and bounded deduplication lifetime; the sketch does not promise durable exactly-once delivery across process crashes. (docs\AI-ANSWER-ARCHITECTURE.md:146-158, commit ordering/idempotency; TODO\29_QUESTION_COMMIT_LATENCY.todo:26-28, duplicate/empty-commit acceptance.)

An armed question needs a start boundary as well as an end. Otherwise earlier Mic conversation enters the question. Current partials disable word-level timing, so arbitrary retrospective acoustic cuts cannot be reconstructed exactly from displayed text. A click watermark also covers delivered samples, not audio still inside a device buffer. A bounded tail-drain policy must be specified and measured rather than disguised as sample-exact knowledge of when the user finished speaking. (services\transcription\recognizer.py:64-67, `_new_decoder`; docs\AI-ANSWER-ARCHITECTURE.md:160-166, start and device-buffer boundaries.)

Planned request, answer states, and UI

The initial provider would call Azure OpenAI **v1 Responses** with `stream:true`, `store:false`, `background:false`, no tools, no conversation ID, and no `previous_response_id`. Its model field identifies the approved deployment alias. “Foreground” describes API execution, not activating the desktop window. The exact SDK must be pinned and compile-checked; research examples are not an installed client. (docs\AI-ANSWER-RESEARCH.md:160-201, inference API; TODO\31_STREAMING_ANSWER_CLIENT.todo:13-21, adapter tasks.)

The research shortlist is `gpt-5.4-mini`, `gpt-5.4-nano`, and `gpt-5-mini`, with documented versions and lifecycle dates as of the existing research. None is established as fastest; subscription entitlement, quota, capacity, processing policy, and actual deployment availability remain unknown. (docs\AI-ANSWER-

RESEARCH.md:136-158, shortlist and availability intersection; docs\AI-ANSWER-RESEARCH.md:254-269, uncertainty summary.)

Question, answer, connection, audio, and capture-protection states must remain separate. Every streamed event would carry a request identity and sequence; stale events from A must never append to B. HTTP 200 or stream closure is not completion. Interrupted output stays visibly incomplete, and cancellation does not guarantee stopped remote billing. (docs\AI-ANSWER-ARCHITECTURE.md:168-207, state machines; docs\AI-ANSWER-ARCHITECTURE.md:238-252, streaming failures.)

The pane would remain inside the protected WPF surface. However, the existing `TogglePresentationAsync` hides the window and stops transcription before changing modes. Reusing it for draft correction would violate continuous Computer transcription. The future editor therefore needs a deliberate, separately tested focus transition—not an undocumented change to global mode semantics. (src\PhantomFudge.App\MainWindow.xaml.cs:223-241, `TogglePresentationAsync`; TODO\32_ANSWER_UI_LIFECYCLE.todo:13-21, planned UI requirements.)

Tradeoffs and adversarial event ordering

Consider an ordinary scheduling race rather than a network attack: the user asks A, cancels it, and asks B; a previously queued A delta then reaches the dispatcher. Disposing the HTTP stream is insufficient because the callback may already exist. The UI must compare the delta's request identity with the current answer generation before appending it. A cancellation acknowledgment and a stale-output guard solve different problems. (docs\AI-ANSWER-ARCHITECTURE.md:185-207, answer state and event rules; TODO\32_ANSWER_UI_LIFECYCLE.todo:19-25, delayed callbacks.)

Likewise, user edits during Mic finalization must remain authoritative for their own draft revision. The final acknowledgment can make a finalized alternative available, but cannot overwrite intervening edits or secretly submit it. Immediate Ask favors speed and visible user control; Finish dictation favors acknowledged recognition coverage. Neither path universally dominates, which is why the plan exposes both instead of hiding a heuristic choice behind one button. (docs\AI-ANSWER-ARCHITECTURE.md:58-68, submission paths; docs\AI-ANSWER-ARCHITECTURE.md:155-166, epochs and boundaries.)

Finally, bounded storage is a correctness decision: transcript eviction, commit-cache eviction, and answer truncation cannot be treated identically. Selected question text needs a retained copy; commit deduplication needs an explicit lifetime; oversized input or output needs a visible policy rather than silent alteration. Each limit should be exercised with synthetic boundary cases before the first authorized model request. (docs\AI-ANSWER-ARCHITECTURE.md:119-122, memory/input limits; docs\AI-ANSWER-ARCHITECTURE.md:155-156, bounded deduplication; TODO\31_STREAMING_ANSWER_CLIENT.todo:19-28, adapter bounds and tests.)

Exercises

1. Trace Ask on draft revision 4, then a final creating revision 5. Specify what the answer header and editor show without mutating snapshot 4. (docs\AI-ANSWER-ARCHITECTURE.md:114-120, revision rules.)
2. Place a Mic marker behind two Mic frames and an interleaved Computer frame. Explain why acknowledging reception is insufficient and why global Stop is incorrect. (services\transcription\server.py:237-333, `Session.receive`, `recognize`.)

3. Retry a commit after its epoch changes, then retry the same ID with a different watermark. Write expected acknowledgment/rejection assertions without opening audio devices. (TODO\29_QUESTION_COMMIT_LATENCY.todo:16-28, retry and boundary requirements.)

References

- Detailed proposal, research and source register, and identity/privacy design.
- Draft workstream, commit workstream, and streaming workstream.
- Existing transport, server, and recognizer.
- Azure Responses primary documentation, recorded in the existing research; not newly retrieved or account-verified for this chapter.

14. Answer Latency, Identity, and Privacy

Why these concerns belong together

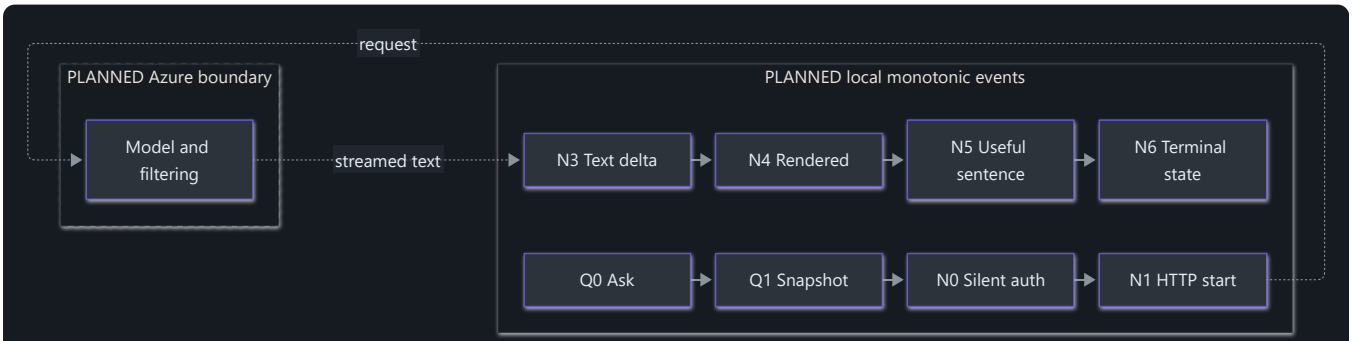
The answer pipeline is PLANNED. No Azure endpoint, sign-in, model entitlement, retention configuration, or cloud latency was verified while writing this chapter. Its service contracts come from the repository's existing research, not fresh retrieval. Connection and latency workstreams explicitly report no implementation or execution evidence. (TODO\30_AZURE_ANSWER_CONNECTION.todo:31-37, completion evidence; TODO\33_ANSWER_LATENCY_QUALITY.todo:30-36, completion evidence; docs\AI-ANSWER-PRIVACY.md:133-151, primary-reference scope.)

Latency, identity, and privacy interact in practical ways. A surprise sign-in window delays the answer and introduces an external surface not covered by the application's capture policy. Sending the whole transcript makes request content harder to review and violates selected-text scope. Retrying a partial answer may duplicate disclosure and spending while making the UI look continuous. The proposed design therefore optimizes **a useful answer to an explicitly frozen question**, not merely an early network byte. (docs\AI-ANSWER-PRIVACY.md:27-40, authentication flow; docs\AI-ANSWER-PRIVACY.md:112-131, application safeguards; docs\AI-ANSWER-EVALUATION.md:30-40, user-visible metrics.)

The baseline keeps ASR local and adds text egress only at explicit Ask. Optional cloud audio requires a separate decision; good text-model performance cannot authorize uploading microphone or Computer PCM. (docs\AI-ANSWER-PRIVACY.md:6-20, proposed default; TODO\36_OPTIONAL_AZURE_SPEECH.todo:13-25, separate cloud-audio gate.)

Measurement architecture: name every boundary

A duration is meaningful only when both endpoints are defined. The evaluation plan uses one local monotonic clock for client observations, retaining source-relative sample positions as a separate coordinate system. Subtracting a server wall clock from a desktop clock is not a defensible latency measurement without a synchronization model. (docs\AI-ANSWER-EVALUATION.md:7-28, stage-clock definitions.)



14. Answer Latency, Identity, and Privacy - diagram 1. Source-linked; status follows the chapter.

The diagram intentionally omits some instrumentation details for readability: N2 is headers, and Q2 is an optional commit acknowledgment. Neither is answer text. The source plan defines these separately from N3, the first nonempty answer-text delta, and N4, its actual WPF rendering. (docs\AI-ANSWER-EVALUATION.md:19-28, Q/N events.)

Metric	Definition	What it answers
First visible partial	A4 − A0	How soon a controlled utterance becomes visible
Natural-final lag	Final A4 − annotated A1	How much delay follows the last spoken sample
Commit latency	Q2 − Q0	How long explicit finalization takes
API TTFT	N3 − N1	Request-start to actual answer text
Ask-to-first-text	N4 − Q0	Delay the user experiences after Ask
First useful sentence	N5 − Q0	When a task-relevant answer becomes usable

These definitions come from the proposed evaluation, not implemented telemetry. The original paced Vosk numbers start at stream onset and cannot replace an A1 annotation. (docs\AI-ANSWER-EVALUATION.md:30-40, metrics; docs\SPEECH-BENCHMARK.md:106-118, paced-test limits.)

Current local mechanisms that affect future measurements

The production callback splits delivered packets into bounded frames under the input audio lock. Silence scheduling shares that lock and follows per-source activity, so device delivery and inserted continuity samples must remain distinguishable from acoustic speech boundaries. `SilenceCadence` waits for 250 ms without a packet and spaces silence scheduling by at least 100 ms; that is **not** a recognizer endpoint-silence setting. (src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:230-254, `OnAudio`; src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:289-309, `SupplySilenceAsync`; src\PhantomFudge.App\Audio\SilenceCadence.cs:11-19, `TrySchedule`.)

The Vosk wrapper's sample counter advances when `accept` processes PCM. It supports partials, natural finals, and periodic decoder replacement. Meanwhile, the transport separately maintains submitted-byte counts and bounded pending queues. Combining these different counters into one generic “audio latency” field would conceal where time was spent. (services\transcription\recognizer.py:82-96, `VoskRecognizer.accept`, `finish`; src\PhantomFudge.App\Audio\TranscriptionSocket.cs:13-18, queue limits; src\PhantomFudge.App\Audio\TranscriptionSocket.cs:185-228, `TryEnqueueAudio`.)

Targets are hypotheses, not results

The proposed warm-path budgets are starting points for authorization and measurement:

Planned metric	Proposed target
Freeze reviewed draft	At most 50 ms
Source commit to frozen text	p95 at most 500 ms
First visible partial	p95 at most 1.5 s

Planned metric	Proposed target
Ask to first rendered answer	p50 at most 1 s; p95 at most 2.5 s
Ask to useful short sentence	p50 at most 2 s; p95 at most 4 s
Local cancellation acknowledgment	At most 100 ms
Stream UI batching	16–50 ms

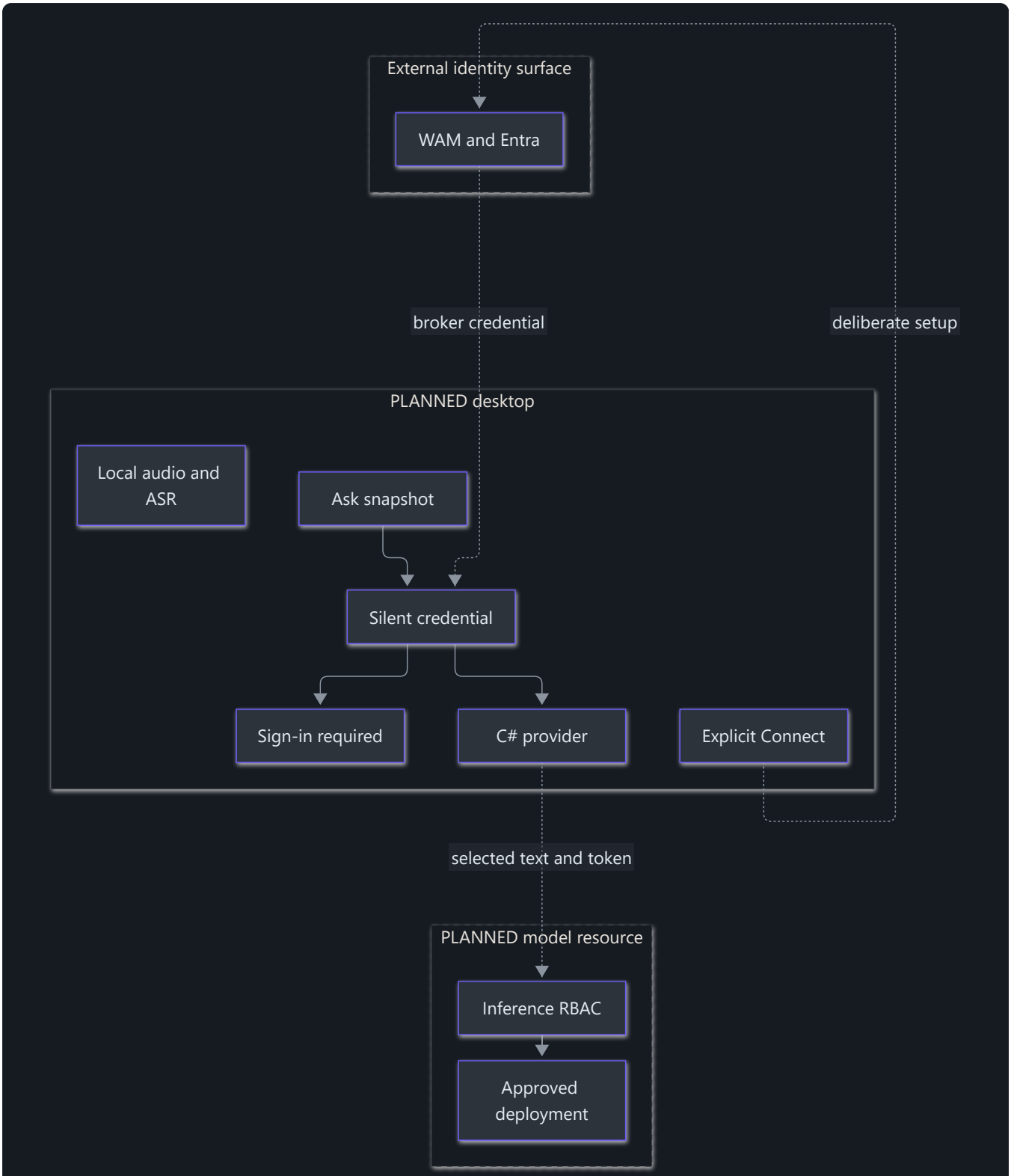
None is a provider SLA or a measured application result. End-to-end p95 must be computed from complete request observations; adding individual stage p95s does not establish the overall p95. Warm/cold authentication and connection cases must remain separate strata. (docs\AI-ANSWER-EVALUATION.md:42-59, proposed targets; docs\AI-ANSWER-EVALUATION.md:98-112, controlled comparison.)

A fair model comparison fixes question content, output requirements, deployment/version, processing policy, and filtering. It preserves errors, filtered outcomes, and incomplete answers rather than dropping slow or unsuccessful samples. A ten-word non-answer must not win against a correct explanation simply by finishing sooner. (docs\AI-ANSWER-EVALUATION.md:94-116, Azure comparison.)

The research shortlist—5.4-mini, 5.4-nano, and 5-mini—is an evaluation set, not a speed ranking or entitlement receipt. Hidden reasoning can consume an output budget before visible text appears; model-specific parameters must be verified rather than copied between families. Default filtering also has different semantics from asynchronous filtering, which can expose text before delayed moderation signals. Optimization must preserve the approved quality and filtering policy. (docs\AI-ANSWER-RESEARCH.md:136-158, candidates; docs\AI-ANSWER-RESEARCH.md:219-233, reasoning budget and filtering.)

Planned identity flow: explicit setup, silent Ask

The desktop is a **public client**. It cannot safely protect a distributed application secret, so the plan uses a tenant-bound Entra user identity through supported MSAL/WAM or broker integration. The registration's client ID is not a secret; API keys, private keys, and refresh tokens must not be bundled into the EXE, Python service, configuration, or ZIP. (docs\AI-ANSWER-PRIVACY.md:22-31, desktop identity.)



14. Answer Latency, Identity, and Privacy - diagram 2. Source-linked; status follows the chapter.

The isolated audio node is intentional: this text-inference design has no raw-audio cloud edge. Interactive authentication happens only through explicit setup, preferably outside a presentation. Ask tries cached/silent acquisition; a UI-required result becomes an inline sign-in-required state, not automatic browser or broker fallback. Credential/client reuse avoids adding a new authentication construction path to every question. (docs\AI-ANSWER-PRIVACY.md:30-40, silent/interactive separation.)

Resource-scoped **Cognitive Services OpenAI User** is the researched inference role for the documented Azure OpenAI path. Management Contributor is not a substitute for data-plane permission, and this feature needs no Graph chat/mail permissions. The exact token audience depends on the pinned adapter: the research records v1's `https://ai.azure.com/.default`, while ARM uses a different audience. Those strings are integration inputs to verify, not evidence that any local account has access. A laptop does not acquire an Azure managed identity merely by using Azure libraries. (docs\AI-ANSWER-PRIVACY.md:41-62, RBAC and token audiences.)

Privacy is several contracts, not one switch

`store:false` avoids the requested Responses storage behavior; it is **not universal zero retention**. Existing research distinguishes model-training restrictions, optional service state, abuse monitoring, and processing geography. Flagged content can enter automated or human review; modified abuse monitoring requires eligibility and approval that have not been established here. (docs\AI-ANSWER-PRIVACY.md:83-107, data statements and limits.)

Global deployments may process prompts in supported geographies; a DataZone constrains processing to its zone, not necessarily one resource region. Therefore a nearby resource name proves neither locality nor low latency. Destination, deployment type, retention policy, and account configuration require explicit approval before a cloud test. (docs\AI-ANSWER-PRIVACY.md:97-110, geography; TODO\30_AZURE_ANSWER_CONNECTION.todo:13-22, connection prerequisites.)

Local behavior needs equally precise language:

- The current transport rejects nonliteral-loopback ASR endpoints and disables proxying and redirects. The proposed HTTPS model client is a **separate** boundary, not a reason to relax the local speech validator. (src\PhantomFudge.App\Audio\TranscriptionSocket.cs:26-31, HTTP handler; src\PhantomFudge.App\Audio\TranscriptionSocket.cs:73-89, `ValidateEndpoint`.)
- Remote speech errors are replaced with a generic local message because server text may contain paths, secrets, or transcript content. The answer adapter should preserve that principle while distinguishing useful safe error codes. (src\PhantomFudge.App\Audio\TranscriptionSocket.cs:397-405, `ProcessMessage`; docs\AI-ANSWER-PRIVACY.md:123-129, diagnostics.)
- The existing transcript is memory-bounded, and export is explicit. Planned Q/A history should likewise remain bounded and memory-only unless deliberately exported; logs should contain random IDs, durations, counts, and result codes—not content or content hashes. (src\PhantomFudge.Core\TranscriptBuffer.cs:6-8, `MaximumCharacters`; src\PhantomFudge.App\MainWindow.xaml.cs:733-760, `ExportTranscriptAsync`; docs\AI-ANSWER-PRIVACY.md:125-131, logging/persistence.)
- The current `ShowProtected` requests protection before showing the window, but neither it nor the planned same-HWND answer pane guarantees exclusion from every recorder. Authentication UI, OS indicators, accessibility, clipboard, and process memory have separate boundaries. (src\PhantomFudge.App\MainWindow.xaml.cs:79-105, `ShowProtected`; docs\PRODUCT.md:125-140, privacy boundaries.)

Worked pseudocode: useful timing without content telemetry

This is a proposed coordinator sketch. `renderer` must report an actual presented update, not merely a queued dispatcher operation; first-useful-sentence scoring belongs to the controlled evaluation rubric.

```
async def ask_reviewed_text(draft, profile):
    q0 = monotonic()
    snapshot = freeze_exact_displayed_revision(draft)
    request_id = random_id()
    credential = await silent_token_or_inline_signin_required(profile)
    request = bind_request(request_id, snapshot, profile)
    n1 = monotonic()
    first_delta = None
    first_render = None
    terminal = "incomplete"
    async for event in provider.stream(request, credential):
        if not active_request_matches(request_id, event.request_id):
            continue
        require_monotonic_sequence(event)
        if event.is_nonempty_text:
            first_delta = first_delta or monotonic()
            await renderer.append_bounded_batch(event.text)
            first_render = first_render or monotonic()
        if event.is_terminal:
            terminal = event.safe_terminal_code
    emit_metadata(request_id, terminal,
                  api_ttft=elapsed_or_missing(n1, first_delta),
                  ask_to_render=elapsed_or_missing(q0, first_render))
```

The real implementation must also bound deadlines, cancellation, buffering, and pre-output retries. It must not interpret a closed stream as completion or silently replay after partial output. Cancellation stops local consumption, not necessarily server compute or billing. (docs\AI-ANSWER-ARCHITECTURE.md:205-207, event identity/completion; docs\AI-ANSWER-ARCHITECTURE.md:238-255, failure policy; TODO\31_STREAMING_ANSWER_CLIENT.todo:18-20, adapter requirements.)

For a purely hypothetical trace, let Ask occur at 0 ms, HTTP start at 70 ms, first text at 480 ms, and first rendering at 510 ms. API TTFT would be 410 ms, while user-visible delay would be 510 ms. These invented teaching numbers are **not measurements**; the arithmetic illustrates the evaluation plan's distinct endpoints. (docs\AI-ANSWER-EVALUATION.md:34-37, metric definitions.)

Tradeoffs: optimize the critical path without hiding failures

UI batching trades a small bounded rendering delay for avoiding a dispatcher operation per tiny delta. The proposed 16–50 ms interval is therefore a tuning range, not permission to accumulate an unbounded queue. Both bytes waiting and time without progress need limits; HTTP activity that never yields useful text should not keep an answer apparently alive forever. (docs\AI-ANSWER-ARCHITECTURE.md:240-249, batching and deadlines; docs\AI-ANSWER-EVALUATION.md:54-59, no-progress policy.)

A broker can enforce central quotas or private routing, but adds another authentication and streaming hop. It should be chosen for a demonstrated governance requirement, not assumed faster. If APIM enters that path, response buffering and body diagnostics become explicit review items because they can defeat streaming or export content. The direct path remains the initial proposal; no infrastructure was provisioned to test either option here. (docs\AI-ANSWER-PRIVACY.md:64-81, optional broker and APIM; TODO\37_OPTIONAL_ANSWER_BROKER.todo:28-34, completion evidence.)

Exercises

1. Design a fake-stream case with headers, metadata, one text delta, and a post-200 error. Identify the timing fields and incomplete UI state. (docs\AI-ANSWER-EVALUATION.md:125-131, failure matrix.)
2. Explain why a 403 must not trigger another region or public-network fallback, even if that alternative appears faster. (docs\AI-ANSWER-EVALUATION.md:127-129, network/auth failure policy.)
3. Review a proposed diagnostic payload containing a prompt hash. Replace it with fields that support correlation without identifying question content. (docs\AI-ANSWER-PRIVACY.md:125-129, metadata-only diagnostics.)

References

- Evaluation plan, privacy/identity design, and research.
- Connection workstream and privacy/resilience gate.
- Existing research primary URLs: Azure data privacy, abuse monitoring, and MSAL.NET WAM. These were not freshly retrieved or verified against an account for this chapter.

Development and testing

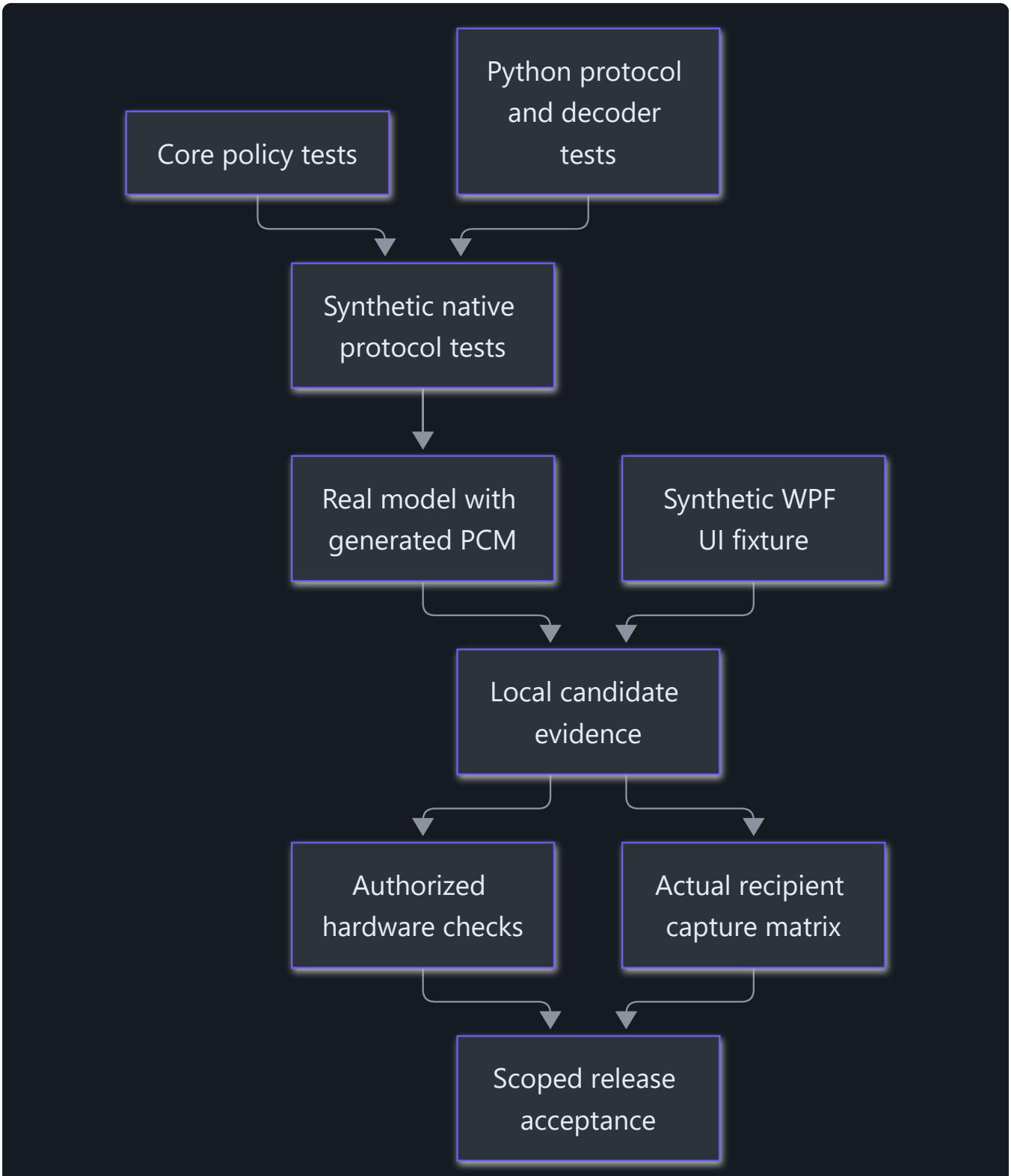
> **TL;DR:** There is no single command that proves the whole product. Build/unit tests, transport tests, real-model synthetic tests, UI fixtures, hardware tests and recipient-side capture qualification each establish different evidence. Run the smallest relevant layer, then cross the next boundary deliberately.

Overview: reproducibility starts before a test assertion

The repository pins .NET SDK10.0.112 with patch roll-forward and prerelease disabled (`global.json:1-7`). Common build properties enable nullable references, implicit usings, deterministic builds, warnings as errors and dependency lock files (`Directory.Build.props:1-9`). These choices make local differences easier to diagnose; they do not eliminate differences in Windows, drivers or external capture applications.

Python dependencies are pinned separately from the .NET graph (`services\transcription\requirements.txt:1-20`). The service uses a local virtual environment. Do not install packages globally because a test needs them, and do not add Docker as a shortcut around understanding the native setup.

Architecture of the test portfolio



15 - Development and Testing - diagram 1. Source-linked; status follows the chapter.

This is not a strict test-pyramid claim. Some layers run independently. The diagram shows how evidence accumulates before a broad release claim, not a requirement to rerun every test for every prose edit.

The real runners

Layer	Command	What it establishes
Native build	`dotnet build .\PhantomFudge.slnx -c Release`	Compiler/project consistency
Core xUnit	`dotnet test .\tests\PhantomFudge.Core.Tests -c Release`	Text, source, palette and persistence policies
Audio console harness	`dotnet run --project .\tests\PhantomFudge.Audio.Tests -c Release`	Synthetic transport/native-lifecycle cases
Python unittest	`.\services\transcription\.venv\Scripts\python.exe -m unittest discover -s .\services\transcription\tests`	Server/decoder/management cases
Dual real-model fixture	`.\scripts\Test-DualSourceSynthetic.ps1 -NativeClient`	Generated PCM through C# v2 and actual local recognition
UI fixture	`PhantomFudge.exe --ui-smoke <private-output>`	Local synthetic rendering/state assertions
Documentation views	`.\scripts\Export-HandbookScreenshots.ps1`	Actual UI images with synthetic labels/content and source hashes

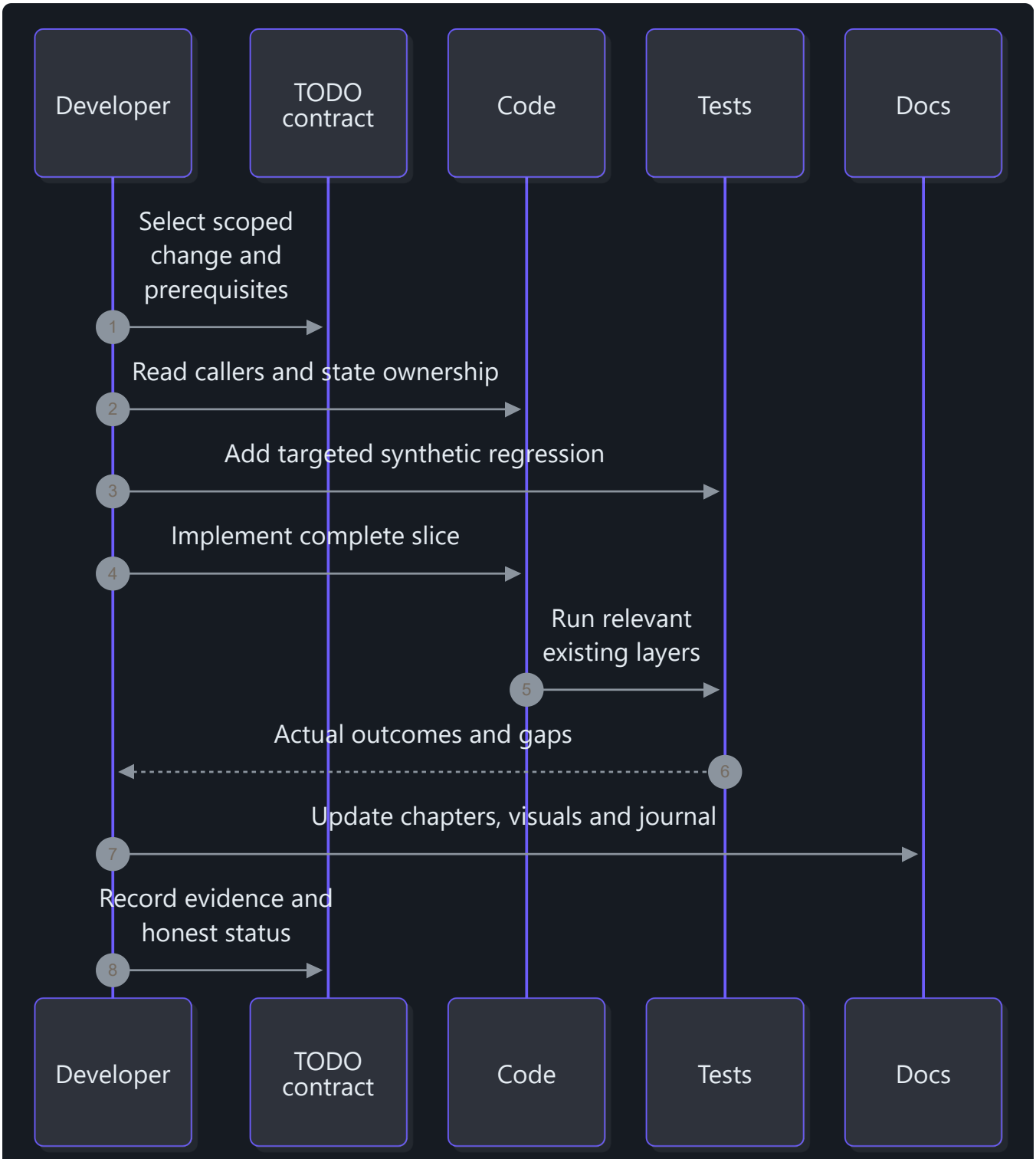
The Core project is an xUnit test project with explicit test SDK/runner references (`tests\PhantomFudge.Core.Tests\PhantomFudge.Core.Tests.csproj:1-24`). The audio project is an **executable harness**, not another xUnit assembly; it links the actual audio sources and references ASP.NET Core for local test servers (`tests\PhantomFudge.Audio.Tests\PhantomFudge.Audio.Tests.csproj:1-13`). Do not assume `dotnet test` alone executes its scenarios.

Safe setup

The initialization script verifies the model manifest, creates a random local token if absent, restricts its Windows ACL, and creates/repairs the service-local Python environment as needed. Model download is explicit; it is not performed silently while handling live audio (`scripts\Initialize-Transcription.ps1:12-44`, `:65-140`).

Use the documented setup command once, then ordinary builds/runs. Existing lock files should be respected. If a restore fails, inspect the actual failure rather than changing TLS settings or switching to an unapproved feed. A shell HTTP probe and the .NET restore stack can behave differently; the development record already contains such a distinction.

The developer loop



15 - Development and Testing - diagram 2. Source-linked; status follows the chapter.

A useful test is tied to an invariant. "The app launches" would not have caught a late final poisoning the next session's IDs. A source-partial test that proves one track's final leaves the other pending text intact is much stronger for that specific requirement (`tests\PhantomFudge.Core.Tests\SourceTranscriptTests.cs:1-80`).

Working on a shared Windows desktop

An agent or test must not kill a running user's app to overwrite a locked executable. Build to an isolated output directory or create a new candidate. The native lifecycle script refuses to touch a preexisting PhantomFudge process and tracks only the processes it starts (`scripts\Test-NativeLifecycle.ps1:16-22`, `:69-113`).

Avoid parallel .NET commands that write the same project `obj` tree. Parallel research or independent Python checks are fine; two builds of the same Core project can race on generated cache files even when they use different final output directories. Compile once, then use `--no-build` where appropriate for follow-up checks.

The documentation fixture must never use real notes or chat content. It has its own command line path and validates a synthetic/no-recording result before copying images (`src\PhantomFudge.App\App.xaml.cs:24-36`, `scripts\Export-HandbookScreenshots.ps1:1-76`). A screenshot request is not authorization to capture a meeting or disable display affinity.

Failure tests worth keeping

The native harness covers authentication, framing, cancellation, bounded queues and delayed shutdown. Its entry point reports named groups and returns failure on a failing group (`tests\PhantomFudge.Audio.Tests\Program.cs:18-71`). Tests must also cover the interaction between layers: a fake device can fail after another input has started, or a final can arrive while UI Stop is still awaiting cleanup.

Python tests use injected recognizers for deterministic failure cases, while separate real-model fixtures prove that the production decoder can recognize generated speech. A fake recognizer is appropriate for protocol determinism, not evidence of speech accuracy. Conversely, one recognized synthetic sentence does not prove every cancellation path.

Interpreting results

Record source identity, environment, exact command, pass/failure, sample size and limitations. The implementation-status document is a bounded receipt, not a badge that everything is safe (`docs\IMPLEMENTATION-STATUS.md:1-70`).

For capture, an in-process `RenderTargetBitmap` image proves UI appearance. A cropped GDI fixture proves only the sampled local path. Neither proves what a remote participant sees. Run an intentionally unprotected positive control only outside a live presentation; it is supposed to be capturable.

Contributor exercises

1. Locate the normal audio harness dispatch and explain why it does not open hardware.
2. Identify which tests should change when a source label is added.
3. Explain why a successful model fixture cannot replace a malformed-frame test.
4. Write the expected sequence for Start, immediate Stop, then another Start.

5. Choose the smallest checks for a theme-only change versus a protocol change.

References

- `global.json:1-7` — SDK pin.
- `Directory.Build.props:1-9` — common build rules.
- `tests\PhantomFudge.Core.Tests\PhantomFudge.Core.Tests.csproj:1-24` — xUnit runner.
- `tests\PhantomFudge.Audio.Tests\PhantomFudge.Audio.Tests.csproj:1-13` — executable harness.
- `tests\PhantomFudge.Audio.Tests\Program.cs:18-71` — synthetic group execution.
- `scripts\Initialize-Transcription.ps1:12-140` — environment/model/token setup.
- `scripts\Test-NativeLifecycle.ps1:16-113` — owned-process testing.
- `docs\QUALIFICATION.md:1-26` — evidence separation.

Distribution and troubleshooting

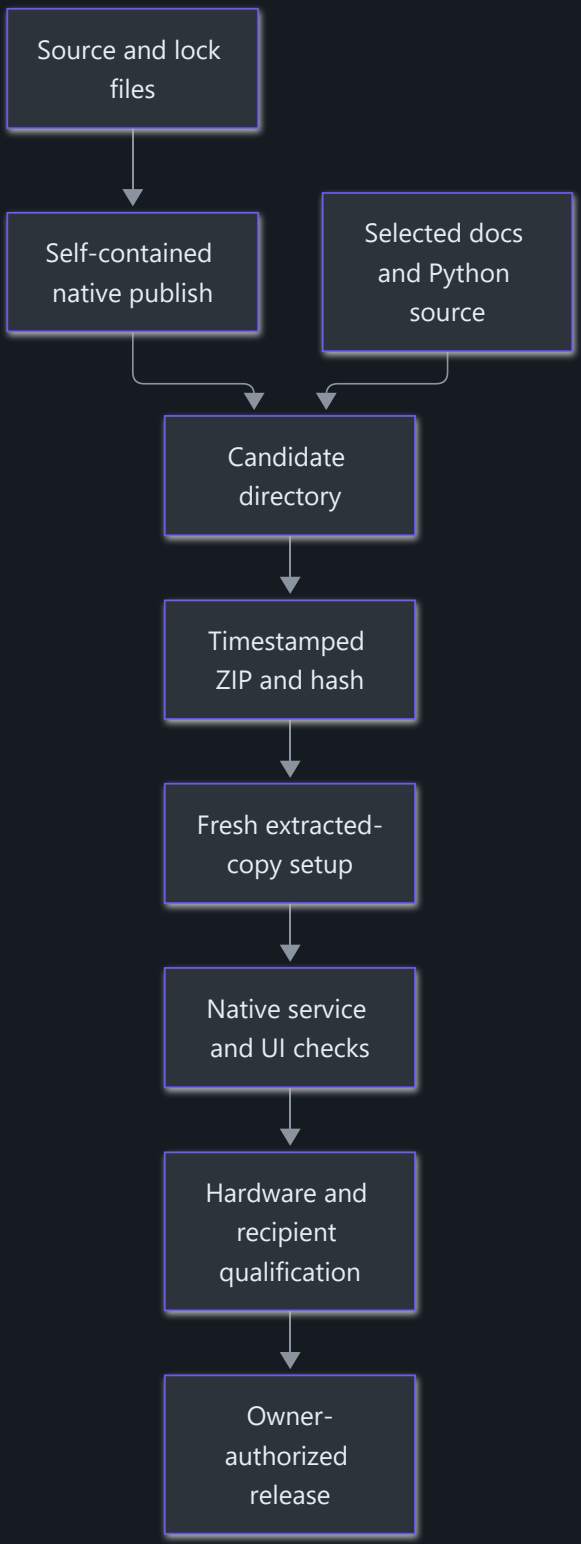
> **TL;DR:** A ZIP is a candidate, not a release claim. The native bundle carries the C# app and the Python service's source/setup contract; it does not carry a secret, a user's notes, a virtual environment, or an invisible promise that every machine is supported.

Overview: what gets shipped

`Publish-Preview.ps1` builds a self-contained win-x64 application into a timestamped directory, copies a selected documentation/script/service allowlist, and creates a ZIP plus SHA256 output (`scripts\Publish-Preview.ps1:1-41`). The exact list matters: copying the repository wholesale would also risk models, token files, captures, temporary data and unrelated development tooling.

The native runtime and Python runtime are different dependencies. Self-contained .NET publishing includes the .NET runtime needed by the executable. It does not install Python, create the service's venv or embed the ASR model. Those operations are owned by explicit initialization (`scripts\Initialize-Transcription.ps1:110-140`).

Artifact flow



16 - Distribution and Troubleshooting - diagram 1. Source-linked; status follows the chapter.

The final arrows are deliberately not automatic. A passing local smoke test is not signing, distribution authorization, enterprise-policy acceptance or four-product capture qualification. The implementation-status document identifies the actual local candidate and its remaining boundaries (`docs\IMPLEMENTATION-`

`STATUS.md:1-100`).

Source checkout versus extracted bundle

The launcher detects whether `PhantomFudge.exe` sits at the root of its working package. If it does, it uses that executable without building. Otherwise it targets the source-build path. An already-running source instance is revealed rather than rebuilt underneath the user (`scripts\Run-PhantomFudge.ps1:8-36`).

Situation	Correct action	Avoid
First source setup	Run explicit initialization with model-download permission	Expecting the app to fetch weights during a meeting
First extracted bundle	Keep the full folder, then run its setup/launcher	Copying only the EXE away from dependent files
Existing app running	Save/close when an update requires replacement, or reveal it	Killing an unknown process to release a file lock
Notes-only use	Use the explicit NotesOnly launcher option	Pretending a missing ASR service is healthy
Speech service idle	Use its managed Stop command	Broad process-name termination
Service active	Stop transcription in the UI first	Discarding queued audio by terminating the backend

Setup is intentionally explicit

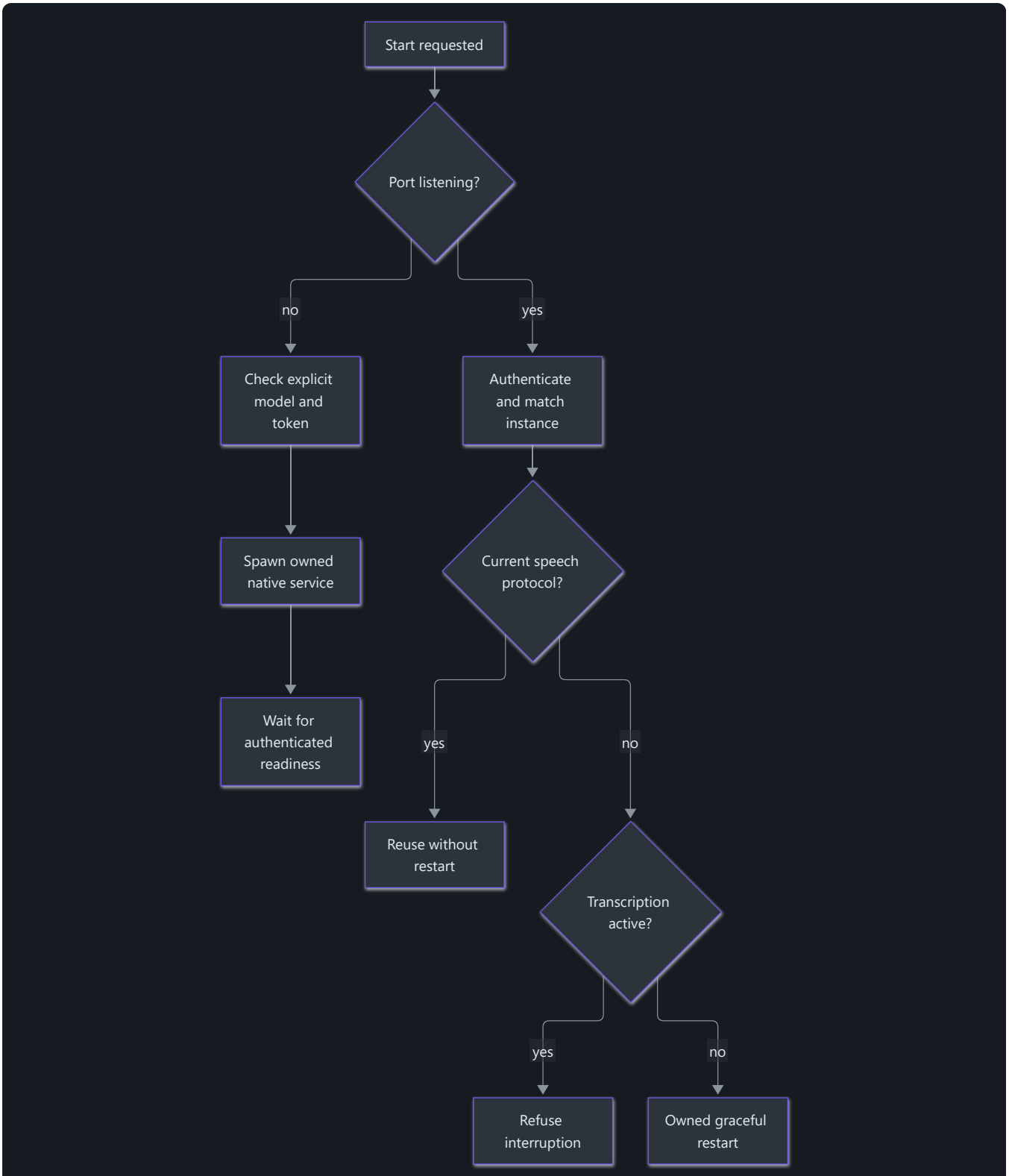
The initializer checks a pinned archive size/hash and verifies the extracted model files. An existing damaged model is not silently overwritten. Token creation uses random bytes, does not rotate an existing valid token automatically, and restricts the Windows DACL (`scripts\Initialize-Transcription.ps1:17-67`, `:69-109`).

The recorded archive hash is an observed integrity value from a known public download, not a publisher-signed guarantee. That distinction is explained in the service guide (`services\transcription\README.md:120-170`).

The model can be reused from the local cache. Dependency repair is controlled by the requirements hash/import check and stays in the service's venv. Setup does not mean recording; launching the service still leaves audio idle until the user presses Start.

Service ownership is more than a port

A listening port may belong to another workspace or process. The manager authenticates `/v1/runtime`, checks the service identifier/management version and matches the saved instance ID before reuse or shutdown (`services\transcription\manage.py:112-144`). A stale owned idle service can be restarted for the supported speech protocol, but active transcription is refused.



16 - Distribution and Troubleshooting - diagram 2. Source-linked; status follows the chapter.

This design refuses ambiguity instead of adopting an arbitrary process. The manager's status output intentionally reports only readiness/activity/PID, not token or transcript content (`services\transcription\manage.py:217-239`).

Troubleshooting by symptom

The window is missing

First distinguish Hidden, minimized, failed initialization and a different running instance. Ctrl+Alt+Space and relaunch recovery are intended paths. The single-instance pipe accepts one fixed recovery command under current-user restrictions, not arbitrary scripts or filenames (`src\PhantomFudge.App\Native\SingleInstance.cs:18-75`).

If protection setup fails, the app withholds content and records a generic local notice rather than displaying an unprotected branded error window (`src\PhantomFudge.App\LocalFailureNotice.cs:1-30`). That notice is diagnostic data, not saved notes.

Start fails

Check the selected input mode, every requested device, local token, model and service version. In Both mode, a missing microphone must not silently become Computer-only. The native adapter validates selection and waits for protocol readiness before opening inputs (`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:71-152`).

Nothing is transcribed

A selected playback endpoint may not be the meeting application's actual output. A microphone may be muted or denied by Windows policy. A zero meter and an empty transcript are not enough to diagnose the same problem. Do not "fix" either by opening a different device automatically. Actual hardware diagnosis needs explicit user authorization and the named device configuration.

Stop seems slow

The application may be releasing native inputs, draining accepted audio or waiting for final recognition. IsRunning is not a complete shutdown contract. A hung native call cannot be safely treated as disposed merely because a UI timeout expired. The source retains ownership and blocks unsafe restart rather than using a success-shaped fallback.

Docker is missing

Normal native operation does not require it. Docker recipes are optional final-stage packaging. Only an explicit Docker mode should require a Docker engine (`scripts\Start-Transcription.ps1:1-35`). Do not turn a portability task back into a mandatory infrastructure dependency.

Release discipline

Keep candidate names/hashes immutable. If source, runtime, rendering or protocol changes, create a new candidate and rerun the affected evidence. Signed versus unsigned status must be explicit; signing does not guarantee SmartScreen or organizational acceptance.

The handbook and journal are now part of that discipline. A release that changes what the user sees needs current screenshots; a protocol change needs a current sequence diagram and source references. Freshness tooling detects changed inputs, but semantic review remains the author's job.

References

- `scripts\Publish-Preview.ps1:1-41` — package recipe.
- `scripts\Initialize-Transcription.ps1:17-140` — model/token/runtime setup.
- `scripts\Run-PhantomFudge.ps1:8-57` — source/package launch selection.
- `services\transcription\manage.py:112-239` — instance ownership, reuse and stop.
- `src\PhantomFudge.App\Native\SingleInstance.cs:18-75` — window recovery channel.
- `src\PhantomFudge.App\LocalFailureNotice.cs:1-30` — non-content failure notice.
- `docs\IMPLEMENTATION-STATUS.md:1-100` — candidate evidence and limits.

Privacy and assurance

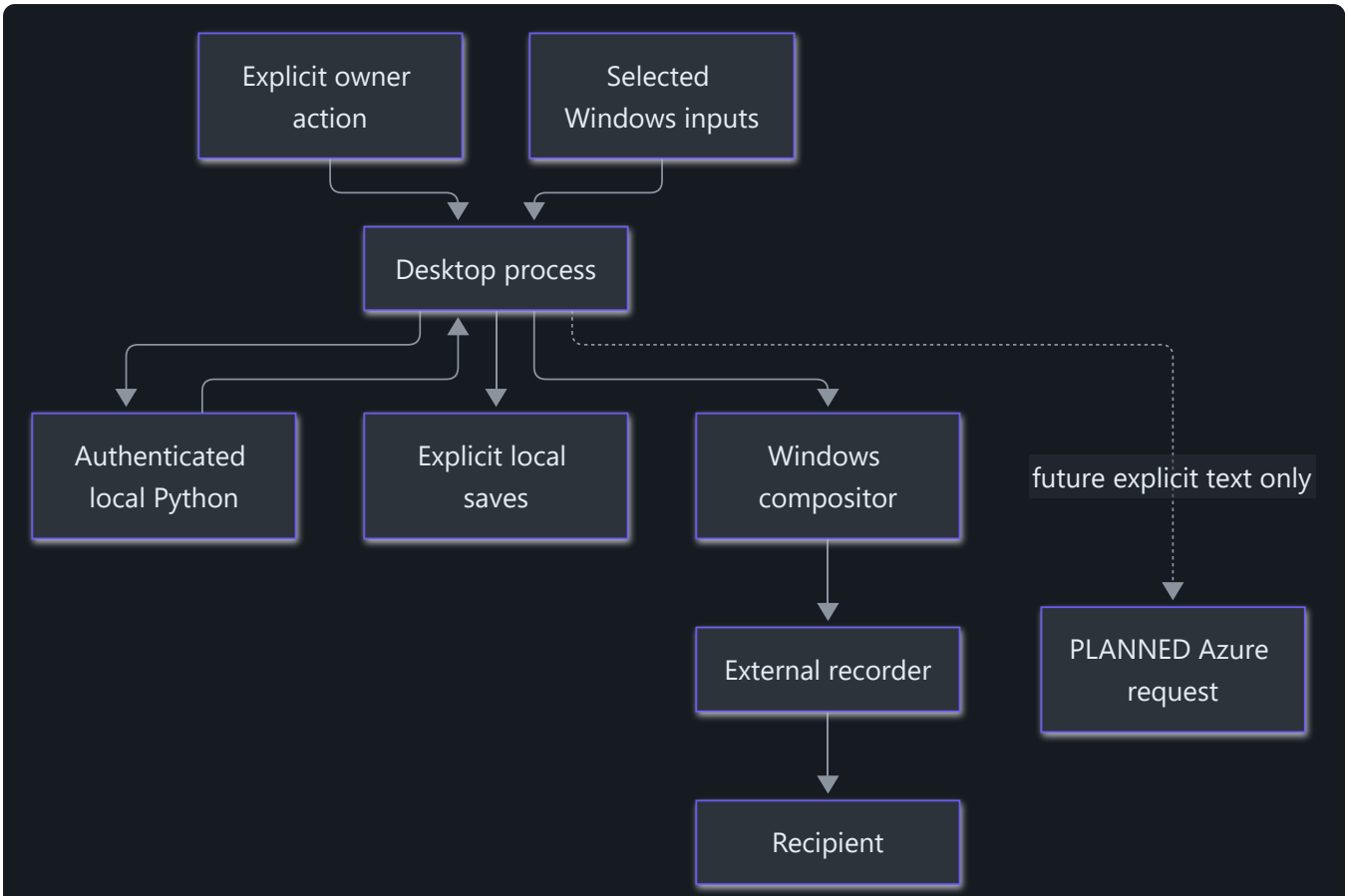
> **TL;DR:** The application can control its own inputs, storage, windows and requests. It cannot turn those controls into a universal guarantee about another recorder, the operating system, an administrator, a camera or a person. Assurance requires both a mechanism and the right experiment for the claim.

Overview: separate the questions

"Private" can mean that notes are absent from a recipient's frames, audio stays local, a token is not logged, a file is encrypted, or a process is not listed. Those are different properties. The current product explicitly does not promise process invisibility or a security/DRM boundary (`docs\PRODUCT.md:125-140`).

The first useful threat-model step is therefore to state what is controlled. The app owns its HWND and can request display affinity. It owns the selected capture objects and can require explicit Start. It owns the local protocol and can authenticate it. It does not own a meeting application's cursor-inclusion setting, the Windows microphone indicator or a camera aimed at the display.

Trust boundaries



17 - Privacy and Assurance - diagram 1. Source-linked; status follows the chapter.

The compositor/recorder boundary is deliberately outside the local socket boundary. Securing localhost traffic does not prove pixel exclusion; setting display affinity does not authenticate the local service.

Window and cursor controls

`WindowProtection.Configure` checks the platform/composition path, updates owned-window styles, requests exclusion and records readback. It does not inject into a recorder or change a global system privacy setting (`src\PhantomFudge.App\Native\WindowProtection.cs:18-45`).

The cursor policy chooses the standard Arrow for the app, while the default Present mode avoids unnecessary activation. The owner still reported a recipient-side disappearing pointer, and that exact sharing configuration remains unqualified. Local shape/foreground observations are useful but not substitutes for recipient frames (`docs\CURSOR-AND-THEMES.md:1-45`, `src\PhantomFudge.App\Native\CursorPolicy.cs:1-25`).

Automatic protection failures should not reveal the application through a new unprotected message box. A generic, bounded local notice contains no note/transcript payload (`src\PhantomFudge.App\LocalFailureNotice.cs:1-30`). Explicit OS file/auth surfaces are still separate from the main protected window.

Audio authority and retention

The later microphone request changed product authority: Computer, Mic or Both may be selected, but no endpoint opens before Start. Source labels are not people. Computer can include other app audio and sidetone; Mic can include speaker echo. Headphones reduce one practical risk but do not create an acoustic-echo-cancellation guarantee (`docs\DUAL-SOURCE-AUDIO.md:1-26`).

Raw audio is handled as bounded in-memory frames. The test harness uses synthetic PCM and must not implicitly open real devices. This is a meaningful policy, but it is not a claim of secure memory erasure, immunity to dumps, or control over host swap/administrators.

Local service authentication

The server uses a bounded local token, constant-time comparison, a one-active-session admission rule and explicit frame limits. Management operations additionally match an instance identity; they are not arbitrary process-control APIs (`services\transcription\server.py:65-176`, `services\transcription\manage.py:112-144`).

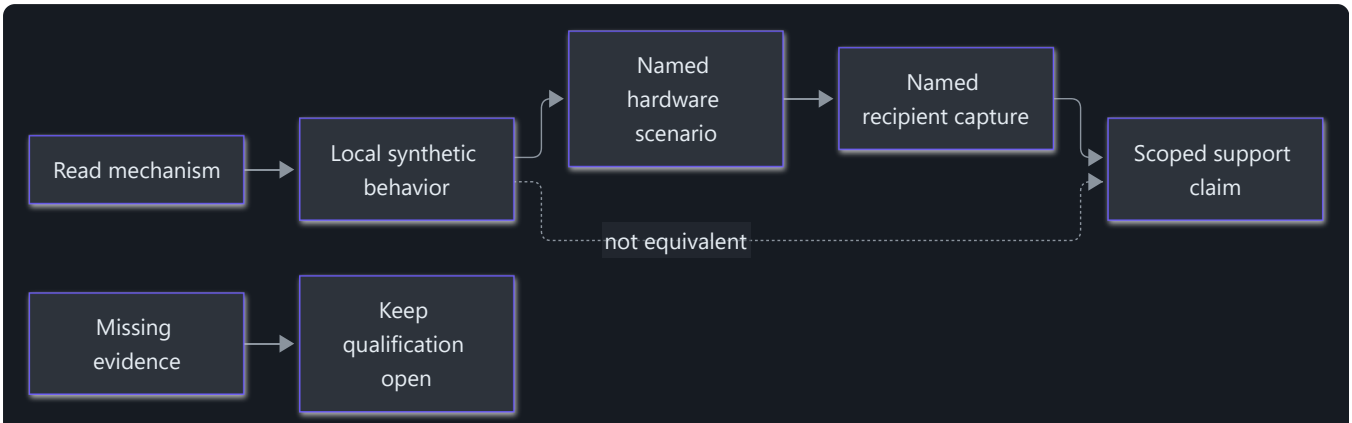
A malicious process running as the same user or an administrator is not contained by calling the service "localhost." The token and current-user boundaries reduce accidental/cross-context access; they do not transform the workstation into a trusted enclave. The service itself does not open microphones or read chats; the C# client owns the selected device operations.

Storage boundaries

Notes and transcripts are not automatically persisted in the current preview. Explicit file saves are bounded UTF-8 operations with conflict checks and same-directory replacement (`src\PhantomFudge.Core\NoteFileStore.cs:16-91`). Ordinary file deletion does not establish secure erasure from SSDs, backups or snapshots. An unsaved warning prevents one kind of data loss, not every crash.

Do not log note hashes as a supposedly harmless substitute for text. A short predictable question can be dictionary-tested against a hash. Use random correlation IDs and counts where diagnostics need identity without content.

Evidence ladder



17 - Privacy and Assurance - diagram 2. Source-linked; status follows the chapter.

A screenshot in this book is an actual WPF visual-tree render with generated content. It proves that the controls can render, not that a recorder did or did not capture them. A GDI positive/negative fixture is a different experiment. A remote meeting capture is different again. The qualification matrix names these dimensions and failure cases (`docs\QUALIFICATION.md:1-40`).

The same separation applies to speech: fake decoder tests prove deterministic protocol handling; a real recognizer with generated audio proves integration; real-speaker/device/echo tests establish a different scope. No single green test count can replace the others.

The planned Azure boundary

The future Ask design intentionally sends a chosen immutable question, not the whole transcript. Cloud audio is optional and separately configured. Model text is displayed as data, not executed as code or used to choose tools/URLs. The desktop is a public client and must not contain a distributed application secret (`docs\AI-ANSWER-PRIVACY.md:1-88`).

`store:false` is not universal zero retention, and a resource's nominal region does not necessarily pin all Global deployment processing to that region. Those statements belong in the product contract before cloud execution, not after an incident. At this edition, Azure Ask is design only; no deployed identity/profile or cloud latency result exists.

Review questions

1. What evidence would establish that an actual Teams screen-share mode excludes the new answer pane?
2. Which data could still be visible through an OS indicator or an explicit file dialog?
3. Why is the source label Mic not proof that the owner spoke?
4. Why does local authentication not defend against every same-user process?
5. What must an Azure Ask implementation avoid logging even when diagnostics are enabled?

References

- `docs\PRODUCT.md:125-140` — privacy scope.
- `src\PhantomFudge.App\Native\WindowProtection.cs:18-45` — owned-window policy.
- `src\PhantomFudge.App\Native\CursorPolicy.cs:1-25` — standard cursor behavior.
- `src\PhantomFudge.App\LocalFailureNotice.cs:1-30` — generic local error notice.
- `services\transcription\server.py:65-176` — protocol/authentication boundaries.
- `services\transcription\manage.py:112-144` — owned shutdown.
- `src\PhantomFudge.Core\NoteFileStore.cs:16-91` — local file semantics.
- `docs\AI-ANSWER-PRIVACY.md:1-88` — proposed cloud boundary.

18. Extending the System

Why a small feature can require a wide review

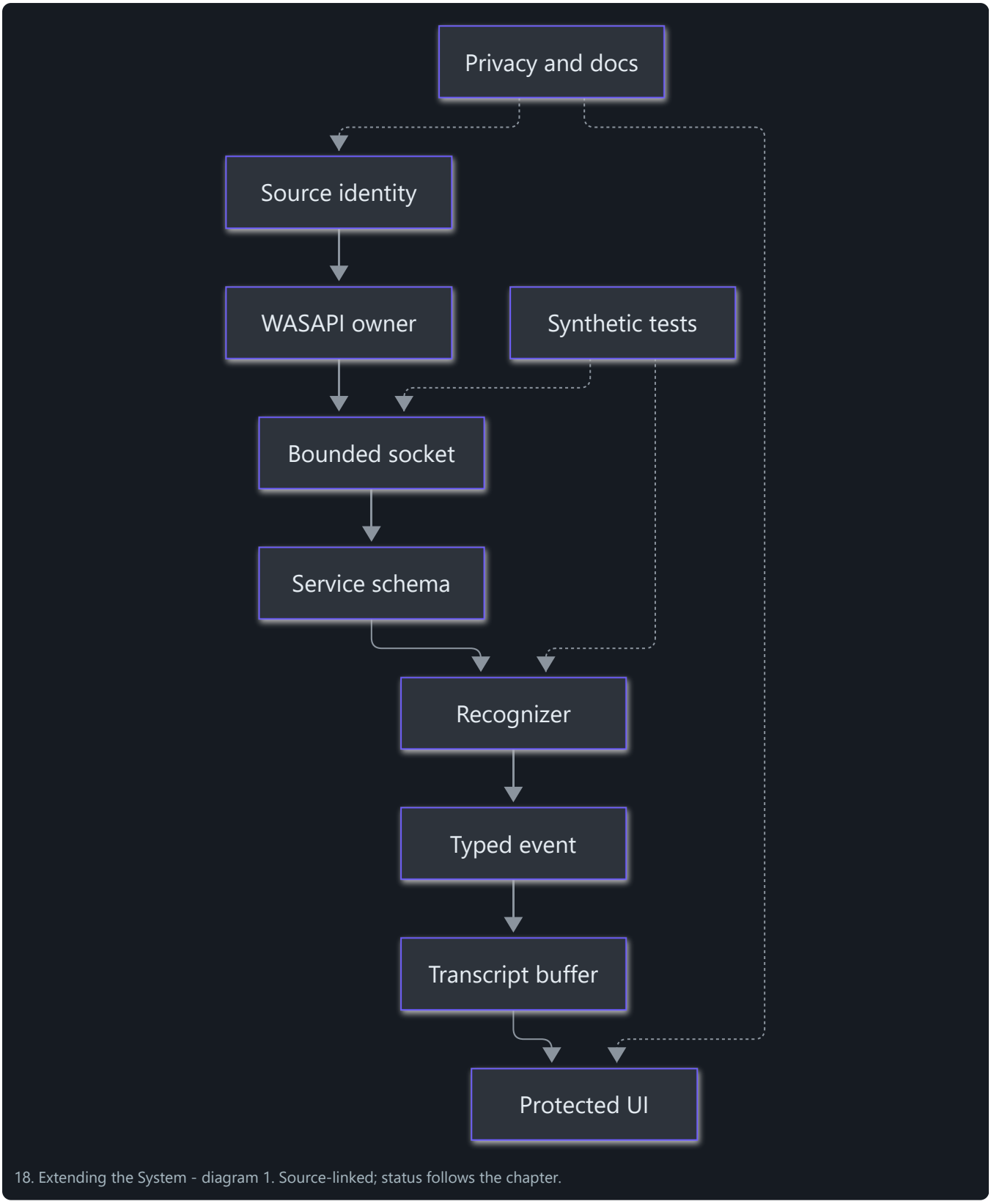
An extension is safe when its responsibilities are narrow **and its consequences are traced completely**. Adding a source is not just another selector; it affects byte framing, queue budgets, timestamp validation, stop behavior, display provenance, tests, and privacy wording. Adding a theme is not just changing a background; selection, disabled controls, alpha composition, and capture qualification also matter. The architecture explicitly prefers small testable interfaces over a generic plugin framework. (docs\ARCHITECTURE.md:35-54, component responsibilities; src\PhantomFudge.Core\AudioSource.cs:3-30, source identity; src\PhantomFudge.App\ThemeBrushes.cs:9-31, semantic resources.)

This chapter teaches change analysis. It does **not** add a source, backend, provider, state field, or runtime capability. The Azure answer extension remains planned; its workstreams are useful examples of how to separate local state, protocol evolution, authorized integration, and final qualification. (TODO\28_QUESTION_DRAFTS.todo:13-20, local state tasks; TODO\29_QUESTION_COMMIT_LATENCY.todo:13-22, protocol tasks; TODO\30_AZURE_ANSWER_CONNECTION.todo:13-22, integration prerequisites.)

The key discipline is to write the invariant before the edit. “A selected source cannot borrow another source's queue allowance” is testable. “Support more audio” is not. Existing tests already express this distinction through canonical schemas, bounded per-source resources, independent timing, and explicit rejection of undeclared input. (services\transcription\tests\test_dual_protocol.py:57-92, `StartSchemaTests`; services\transcription\tests\test_dual_protocol.py:150-200, independent tracks; services\transcription\tests\test_dual_protocol.py:250-255, undeclared-source rejection.)

Architecture: trace both directions across the boundary

For audio changes, follow data downstream from selected device to rendered transcript, then follow failures upstream to cancellation and cleanup. A happy-path trace alone misses ownership leaks and stale callbacks. The implemented path connects typed source selection, native input, a bounded transport, service decoding, typed events, and the buffer/UI. (src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:83-132, `StartCoreAsync`; src\PhantomFudge.App\Audio\TranscriptionSocket.cs:185-228, `TryEnqueueAudio`; services\transcription\server.py:317-333, `Session.recognize`; src\PhantomFudge.App\MainWindow.xaml.cs:711-729, `ReceiveTranscript`.)



18. Extending the System - diagram 1. Source-linked; status follows the chapter.

This map is intentionally not a claim that every box is a formal interface. For example, Python expects recognizer objects with `accept` and `finish`, returning `Recognition`; C# injects a capture-construction delegate for tests. Understanding the actual seam prevents introducing abstraction machinery that the code does not need. (services\transcription\recognizer.py:14-19, `Recognition`; services\transcription\server.py:317-333, recognizer calls; src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:17-27, capture injection.)

Checklist 1: a new audio source

Start with product meaning. Define what the source captures, how the user chooses it, and what its label does not mean. Current `Computer` and `Mic` labels identify inputs, not speakers; no source addition should silently become diarization or automatic meeting discovery. (src\PhantomFudge.Core\AudioSource.cs:16-30, `AudioSourceNames`; tests\PhantomFudge.Core.Tests\SourceTranscriptTests.cs:81-87, `LabelsDescribeInputNotSpeaker`.)

Then inspect these concrete surfaces:

1. **Identity and selection:** extend the enum and explicit selection model deliberately. `AudioInputSelection` currently has only two device fields; `ValidateSelection` constructs at most two inputs and rejects duplicate device IDs. A new enum value alone is insufficient. (src\PhantomFudge.App\Audio\TranscriptEvent.cs:5-7, selection records; src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:138-152, `ValidateSelection`.)
2. **Native ownership:** specify endpoint direction and exact identity matching, construction failure, disconnect, stop, and disposal. `WasapiInput` distinguishes Render from Capture and releases both endpoint notifications and native capture ownership. Do not substitute a default device after selection fails. (src\PhantomFudge.App\Audio\WasapiInput.cs:20-74, constructor and `MatchesSelection`; src\PhantomFudge.App\Audio\WasapiInput.cs:97-119, `Dispose`.)
3. **Transport capacity:** audit fixed arrays of length three, allowed source counts, byte prefixes, and total pending-byte limits. Those structures encode the present two-source contract. Update resource arithmetic explicitly rather than increasing a limit until tests stop failing. (src\PhantomFudge.App\Audio\TranscriptionSocket.cs:13-18, limits; src\PhantomFudge.App\Audio\TranscriptionSocket.cs:51-56, source-indexed fields; src\PhantomFudge.App\Audio\TranscriptionSocket.cs:95-105, source validation.)
4. **Server symmetry:** align `SOURCES`, canonical source ordering, declared-source validation, per-source rate credits, and queue accounting. The existing worker must still publish every selected source's final before stopped. (services\transcription\server.py:23-25, source map; services\transcription\server.py:94-106, `parse_start`; services\transcription\server.py:255-287, `Session.receive`; services\transcription\server.py:317-327, stop drain.)
5. **UI, tests, privacy, documentation:** update selectors, meters, source visibility, export labels, denied-device behavior, and user-facing capture scope. Existing UI branches explicitly on Computer/Microphone/Both; do not let a new value fall into an accidental “Both” default. (src\PhantomFudge.App\MainWindow.xaml.cs:641-678, `UpdateAudioControls`, `SetCaptureMode`; tests\PhantomFudge.Audio.Tests\DualSourceTests.cs:17-44, synthetic regression catalogue.)

Checklist 2: a new recognition backend

Begin at the recognition contract, not the model download. The service requires a `Recognition` object, bounded text, finite ordered times, and final results at stop. Its runtime serializes CPU calls through one executor and marks readiness failed after a decoder failure; a replacement must respect those ownership and timeout semantics. (services\transcription\server.py:109-142, `Runtime.cpu`; services\transcription\server.py:289-333, `Session.publish`, `recognize`.)

Specify the backend's answers to five questions:

- Does `accept` produce useful revisable partials or only completed chunks?
- Does `finish` flush every processed sample, and does it permit continuation?
- Are timestamps cumulative, and how are reset offsets preserved?
- Can independent source decoders share immutable model weights without sharing recognition history?
- Are inference and setup strictly separated so recognition cannot trigger an implicit download?

These questions follow directly from `VoskFactory`'s verified explicit model path and `VoskRecognizer`'s independent decoder, 30-second rotation, and finalization behavior. Synthetic decoder tests check absolute timing and source independence. (services\transcription\recognizer.py:22-52, `verify_model`, `VoskFactory`; services\transcription\recognizer.py:55-96, `VoskRecognizer`; services\transcription\tests\test_recognizer.py:37-75, `RecognizerTests`.)

Do not advertise batch throughput as streaming latency. The existing Whisper result supports an **optional question-only refinement experiment**, not replacement of production Vosk. Refinement would create a new draft revision, never silently alter an already submitted question. Backend changes also need protocol, UI status, privacy, model provenance, dependency, and benchmark documentation reviews even if their wire shape stays unchanged. (docs\SPEECH-BENCHMARK.md:29-35, recommendation; docs\AI-ANSWER-RESEARCH.md:87-96, optional refinement.)

Checklist 3: a new theme or semantic color role

Theme changes have a narrower code surface but a wider visual evidence surface. `ThemePalette.For` provides semantic colors; `ThemeBrushes.Apply` maps those roles into both application resources and standard WPF control resources. Adding a role requires checking each palette and its consumers, not inserting an isolated literal into one control. (src\PhantomFudge.Core\ThemePalette.cs:5-25, `ThemePalette.For`; src\PhantomFudge.App\ThemeBrushes.cs:9-31, `Apply`.)

Review text, muted text, headings, selection foreground/background, action text, warning/error, input, hover, and border states. Preserve frozen brushes and dynamic-resource bindings. Also decide whether a new theme participates in the two-theme toggle: `NextThemed` currently alternates the companions and exits Plain toward DeepSeaFoam. That behavior is tested and should not change accidentally. (src\PhantomFudge.App\ThemeBrushes.cs:34-40, `Brush`; src\PhantomFudge.App\MainWindow.xaml:15-55, dynamic resources; src\PhantomFudge.Core\ThemePalette.cs:27-28, `NextThemed`; tests\PhantomFudge.Core.Tests\ThemePaletteTests.cs:19-25, toggle test.)

The background slider changes only the root brush alpha. Do not “simplify” it into whole-window opacity, which would fade text and change the product contract. Existing contrast tests cover **opaque** reading/selection pairs; they do not certify readability over arbitrary content behind a translucent window. Recheck actual UI, focus/hit regions, and applicable capture scenarios without promoting local rendering evidence into meeting compatibility. (src\PhantomFudge.App\MainWindow.xaml.cs:442-465, opacity and `ApplyTheme`; tests\PhantomFudge.Core.Tests\ThemePaletteTests.cs:31-40, opaque contrast test; docs\QUALIFICATION.md:111-135, transition matrix.)

Checklist 4: a provider is a new trust boundary

The proposed answer provider must stay separate from local speech transport. Do not loosen `ValidateEndpoint` to accept internet ASR URLs simply to accommodate model inference; it deliberately accepts only literal loopback addresses and the exact transcription path. (src\PhantomFudge.App\Audio\TranscriptionSocket.cs:73-89, `ValidateEndpoint`.)

For a **PLANNED** provider, review immutable input, explicit Ask, destination/model allowlists, silent identity, finite buffering/deadlines, cancellation, terminal reasons, output filtering, and metadata-only diagnostics. UI consumers must reject stale request events and display text as data, not execute model-supplied XAML, HTML, code, or links. Add fake-stream tests before an authorized cloud call. (TODO\31_STREAMING_ANSWER_CLIENT.todo:13-28, provider contract; TODO\34_ANSWER_PRIVACY_RESILIENCE.todo:13-28, failure/privacy gates; docs\AI-ANSWER-PRIVACY.md:112-129, output and logging boundaries.)

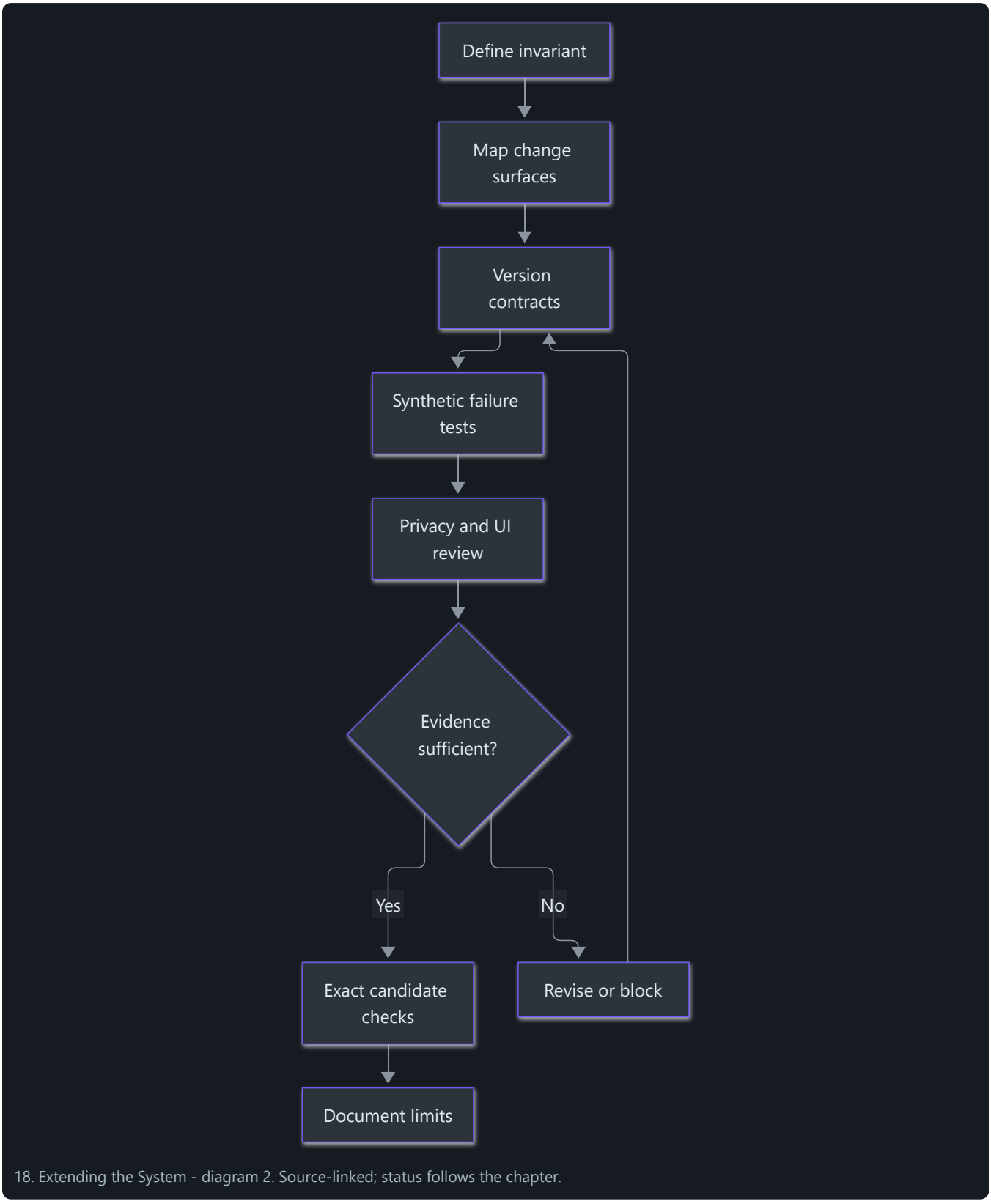
Cloud ASR changes audio egress; an enterprise broker changes authentication, routing, hosting, and streaming behavior. Neither is a routine interchangeable implementation detail. Both need separately justified policy and evaluation decisions, and neither makes Docker an ordinary runtime requirement. (TODO\36_OPTIONAL_AZURE_SPEECH.todo:13-25, optional Speech; TODO\37_OPTIONAL_ANSWER_BROKER.todo:13-26, optional broker.)

Checklist 5: a new state field is a lifecycle change

For every proposed field, write down its owner, creation boundary, mutation authority, reset rule, serialization version, memory bound, and logging/export policy. Distinguish source-relative timing, session-global IDs, draft revisions, and request IDs. They answer different questions and reset at different boundaries. Existing `BeginSession` resets final numbering; `AddFinal` deduplicates globally within that session. Adding a field without deciding its lifetime creates ambiguity during restart. (src\PhantomFudge.Core\TranscriptBuffer.cs:23-38, `BeginSession`; src\PhantomFudge.Core\TranscriptBuffer.cs:48-72, `AddFinal`; docs\AI-ANSWER-ARCHITECTURE.md:89-120, proposed identity records.)

Wire evolution is especially surgical. The current service requires exact start objects, accepts only Stop after start, and the C# client checks exact response property sets. Even an apparently optional extra JSON field can break compatibility. Preserve v1/v2 behavior and negotiate a new version or capability rather than silently weakening parsers. (services\transcription\server.py:94-106, `parse_start`; services\transcription\server.py:247-252, `Session.receive`; src\PhantomFudge.App\Audio\TranscriptionSocket.cs:482-506, `RequireTranscriptProperties`, `RequireProperties`.)

A repeatable change algorithm



18. Extending the System - diagram 2. Source-linked; status follows the chapter.

This is a recommended engineering workflow, not an executed release pipeline. Qualification must identify the exact packaged candidate and distinguish source tests, UI observations, recipient capture evidence, and owner approval. A successful build cannot fill an untested sharing-product row. (docs\QUALIFICATION.md:3-7,

current matrix; docs\QUALIFICATION.md:140-166, result record; TODO\35_ANSWER_RELEASE_ACCEPTANCE.todo:13-26, candidate acceptance.)

Worked pseudocode: planning a future epoch field

Suppose a future Mic commit needs `sourceEpoch`. Before implementing it, create a synthetic test matrix with the following semantics. The functions below are illustrative, not existing APIs:

```
cases = [
    ("same_session_same_epoch", "accept"),
    ("old_session_same_segment_id", "reject_stale"),
    ("old_epoch_after_commit", "reject_stale"),
    ("duplicate_commit_same_parameters", "replay_ack"),
    ("duplicate_commit_changed_watermark", "reject"),
    ("late_final_after_user_edit", "preserve_user_revision"),
    ("new_field_sent_to_v2_peer", "reject_protocol"),
]
for scenario, expected in cases:
    state, event = synthetic_fixture(scenario)
    before = snapshot_unrelated_source(state)
    actual = proposed_transition(state, event)
    assert actual == expected
    assert unrelated_source(state) == before
    assert no_request_was_sent_without_explicit_ask(state)
```

This matrix links protocol changes to UI and privacy consequences rather than testing the field in isolation. It extends the intent of existing independent-partial, empty-final, and duplicate-global-ID tests, while adding the proposed session/epoch/request rules. Tests should use generated data and injected capture objects, not silently open hardware. (tests\PhantomFudge.Core.Tests\SourceTranscriptTests.cs:5-56, source isolation tests; src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:24-27, synthetic capture seam; TODO\29_QUESTION_COMMIT_LATENCY.todo:13-28, proposed commit requirements.)

Tradeoffs and the final change-surface review

Before calling a surgical change complete, review six surfaces even when some require no edit:

Surface	Question to answer in the change record
UI	Did focus, protection lifecycle, source visibility, or status meaning change?
Protocol	Can an older peer reject, misinterpret, or accept an unsupported message?
Tests	Which existing invariant survives, and which new failure ordering is exercised?
Privacy	Did captured, transmitted, retained, logged, or exported data expand?
Resources	Are per-source limits, total limits, deadlines, and cleanup still bounded?
Documentation	Are implemented behavior, proposed behavior, and qualification gaps still distinguishable?

This checklist synthesizes the existing strict transport, resource controls, and qualification record rather than introducing a new runtime framework. “No change required” is useful only when accompanied by the reason: a palette-only edit may leave framing unchanged, while a provider edit necessarily revisits egress and credential handling. (src\PhantomFudge.App\Audio\TranscriptionSocket.cs:13-18, limits; src\PhantomFudge.App\Audio\TranscriptionSocket.cs:503-506, strict properties; docs\QUALIFICATION.md:140-166, evidence record; docs\AI-ANSWER-PRIVACY.md:112-131, provider safeguards.)

Prefer a smaller supported contract over a permissive parser or automatic fallback that conceals unsupported behavior. Preserve failed and untested outcomes in the handoff: a new backend passing synthetic tests does not qualify a new window surface, a real microphone, or a cloud data policy. (docs\QUALIFICATION.md:24-26, outcome vocabulary; TODO\35_ANSWER_RELEASE_ACCEPTANCE.todo:23-26, acceptance boundaries.)

Exercises

1. Find every fixed two-source assumption in the checklist. Explain why changing only the enum can compile in one layer while violating another layer's runtime schema. (src\PhantomFudge.App\Audio\TranscriptionSocket.cs:51-56, arrays; services\transcription\server.py:94-106, `parse_start`.)
2. Propose a warning color with acceptable opaque contrast, then explain why translucent-background readability still needs separate observation. (tests\PhantomFudge.Core.Tests\ThemePaletteTests.cs:31-57, contrast calculation; docs\PRODUCT.md:100-107, translucency limits.)
3. Draft a provider change receipt that records capability and privacy differences without including a real question, token, or account secret. (docs\AI-ANSWER-EVALUATION.md:137-146, receipt requirements.)

References

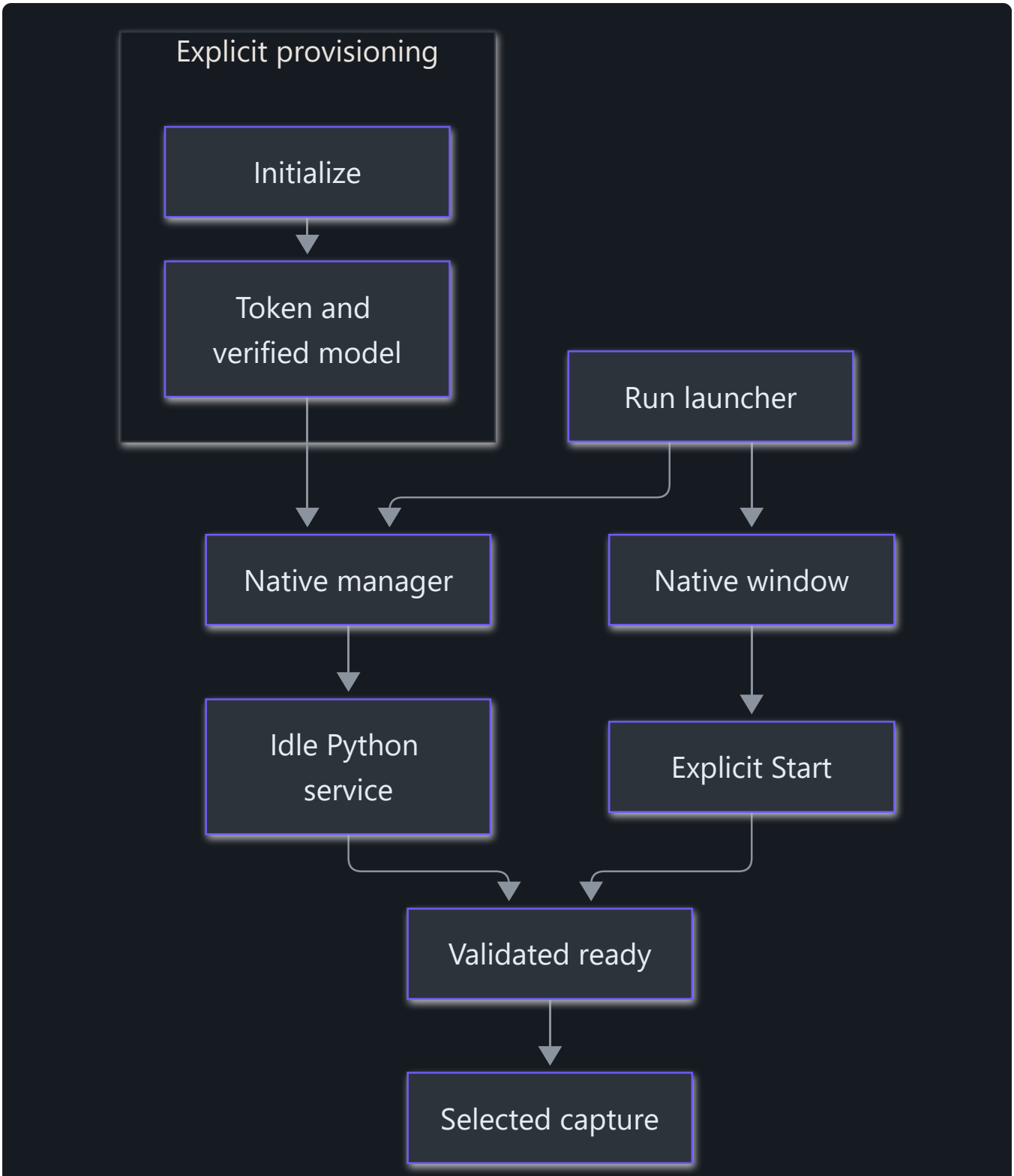
- Architecture responsibilities and qualification contract.
- Source model, transport, and recognizer.
- Theme roles and brush mapping.
- Source-state tests and dual-protocol tests.
- Planned answer architecture and release acceptance.

19. API and configuration reference

Readiness is not recording

This reference distinguishes three operations that must never be conflated: **provisioning**, **starting an idle service**, and **starting selected audio capture**. The launcher starts/reuses Python, but the window separately requires explicit Start, valid selections, protection readiness, and a token. Both is the initial input-mode selection, not automatic recording. (`scripts\Run-PhantomFudge.ps1:39-55`, launcher; `src\PhantomFudge.App\MainWindow.xaml.cs:33-54`, initial state; `src\PhantomFudge.App\MainWindow.xaml.cs:589-625`, `Start_Click`.)

Commands below are reference examples from the repository root, not actions performed to produce this chapter. No credential value is reproduced. Follow capture, protocol, recognition, and service ownership for the implementation narratives.



19. API and configuration reference - diagram 1. Source-linked; status follows the chapter.

The final gate is implemented by `StartCoreAsync`: native construction follows successful transport connection, not merely process launch. (`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:98-111`, `StartCoreAsync`.)

1. Startup commands and argument semantics

The normal operational sequence is:

```
# Explicit provisioning; may download dependencies and the pinned model.
.\scripts\Initialize-Transcription.ps1 -DownloadModel

# Start/reuse native Python and launch/reveal the application.
.\scripts\Run-PhantomFudge.ps1

# After stopping capture in the app, stop the owned idle service.
.\scripts\Stop-Transcription.ps1
```

Command/switch	Exact implemented meaning
`Initialize-Transcription.ps1`	Verify/provision token and model; normally create/restore the service-local Python environment.
`-DownloadModel`	Permit downloading an absent model archive. A valid cached archive does not require another download.
`-SkipPythonSetup`	Skip environment/dependency setup, **not** token/model work.
`Start-Transcription.ps1`	Invoke the existing service-local interpreter with `manage.py start`. Missing setup fails; it is not installed here.
`Run-PhantomFudge.ps1 -NoBuild`	Skip source-build work; an executable must already exist. Packaged launches skip build regardless.
`-NotesOnly`	Skip service startup; it does not remove transcription controls or disable a separately running service.
`-PassThru`	Also return the launched process object, including the recovery process on the reveal path.
Start/Stop `-Docker`	Explicit optional container branch; never selected automatically. Start requires existing recipes/engine.

Definitions: `Initialize-Transcription` parameters/provisioning (`scripts\Initialize-Transcription.ps1:1-15`, `scripts\Initialize-Transcription.ps1:69-140`); native/Docker branches (`scripts\Start-Transcription.ps1:8-36`, `scripts\Stop-Transcription.ps1:8-22`); launcher branches (`scripts\Run-PhantomFudge.ps1:1-57`). The launcher requires PowerShell 7.2 or later, prefers the packaged executable when present, and reveals an existing app at the expected executable path **before** any build/service operation. (`scripts\Run-PhantomFudge.ps1:1-37`, launcher.)

The application itself recognizes three exact two-argument diagnostic patterns: `--capture-proof outputDirectory`, `--ui-smoke outputDirectory`, and `--documentation-screenshots outputDirectory`. These dispatch capture/UI diagnostics, not ordinary headless unit tests. Unmatched argument shapes fall through to normal single-instance startup; there is no implemented `--ask`, input-selection, or automatic-capture CLI switch. (`src\PhantomFudge.App\App.xaml.cs:13-52`, `App.OnStartup`.)

2. Environment, defaults, and secret ownership

Variable	Consumer, default, and precedence
`PHANTOM_TRANSCRIPTION_URI`	App speech endpoint; defaults to `ws://127.0.0.1:8765/v1/transcribe`. Launcher explicitly sets this value.
`PHANTOM_TRANSCRIPTION_TOKEN`	App initial token field, empty when absent. Server uses it only when a token-file setting is not supplied.
`PHANTOM_TRANSCRIPTION_TOKEN_FILE`	Server reads this file first; native manager supplies the workspace token-file path. It is not an app token-file setting.
`PHANTOM_TRANSCRIPTION_MODEL_PATH`	Required by production server; no implicit model download or default model directory in `server.main`. Manager supplies the pinned directory.
`PHANTOM_TRANSCRIPTION_HOST`	Server default `127.0.0.1`; only `127.0.0.1` and `0.0.0.0` accepted. Native manager overrides to loopback.
`PHANTOM_TRANSCRIPTION_INSTANCE`	Optional canonical UUID-hex identity; manager generates it. Without managed identity/callback, authenticated shutdown is unavailable.
`PYTHONDONTWRITEBYTECODE`	Manager sets `1` for its child.
`OPENBLAS_NUM_THREADS`, `OMP_NUM_THREADS`	Manager sets each to `1`; these are child environment settings, not WebSocket controls.
`PHANTOM_AUDIO_TEST_TOKEN`	Native fixture/playback harness credential; distinct from the application's token-variable name.

Source: `MainWindow` initialization and `Start_Click` (`src\PhantomFudge.App\MainWindow.xaml.cs:48-54`, `src\PhantomFudge.App\MainWindow.xaml.cs:606-625`); launcher environment (`scripts\Run-PhantomFudge.ps1:43-55`); `load_token`, `main`, and `production_app` (`services\transcription\server.py:69-79`, `services\transcription\server.py:455-485`); manager child environment (`services\transcription\manage.py:167-176`, `start`); harness lookup (`tests\PhantomFudge.Audio.Tests\DualSyntheticServiceCheck.cs:24-27`, `RunDualSyntheticServiceCheckAsync`).

Do not infer a port setting: production `server.main` binds port 8765. The C# validator allows other valid ports on literal `127.0.0.1` or `[::1]`, but that does not reconfigure the server. It rejects `localhost`, remote hosts, `wss`, query credentials, fragments, user information, and normalized/escaped alternate paths. The default native server is IPv4 despite client IPv6-loopback acceptance. Redirects and proxies are disabled. (`services\transcription\server.py:475-492`, `main`; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:26-31`, HTTP handler; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:73-89`, `ValidateEndpoint`.)

Initialization generates 32 random bytes as 64 hexadecimal characters and restricts the Windows token DACL to the current user. Existing invalid tokens require explicit repair, not silent rotation. Manager/server validation requires `[A-Za-z0-9_-]{32,512}`; the native socket's broader lexical check is not the server's credential contract. Never put credentials in URLs or command arguments. (`scripts\Initialize-Transcription.ps1:33-67`, token setup; `services\transcription\manage.py:33-40`, `token`; `services\transcription\server.py:69-79`, `load_token`; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:108-135`, `ConnectCoreAsync`.)

Engine/model pins are `vosk==0.3.45`, `aiohttp==3.14.3`, and English `vosk-model-small-en-us-0.15`. The archive is 41,205,931 bytes; its recorded hash is observed download identity, not a publisher-signed checksum. Runtime verifies the exact manifest file set and hashes before explicit-path model loading. (`services\transcription\requirements.txt:1-2`, dependency pins; `services\transcription\model.json:2-10`, manifest; `services\transcription\recognizer.py:22-49`, `verify_model`, `VoskFactory`.)

3. Native management contract

```
.\services\transcription\.venv\Scripts\python.exe .\services\transcription\manage.py status
```

`manage.py` accepts exactly one verb: `start`, `stop`, or `status`; exit codes are `0` success, `1` runtime failure, and `64` invalid usage. Status reports `ready`, `active`, and `pid` when it authenticates a listener; no listener produces a textual status instead. (`services\transcription\manage.py:217-239`, `status`, `main`.)

Start/Stop/Status share a workspace byte-range lock. Ownership requires the receipt's UUID plus authenticated service name and management version, not merely a PID or occupied port. A ready v2-capable instance is reused, including an active one. Automatic replacement is limited to an owned, ready, explicitly idle instance lacking speech v2. Active, ambiguous, foreign, or unready instances are not silently repaired. (`services\transcription\manage.py:65-90`, `lifecycle_lock`; `services\transcription\manage.py:112-159`, ownership/capability/start decisions.)

The Python decision can be read as this JavaScript pseudocode:

```
if (listenerExists()) {
  const info = authenticateReceiptInstance();
  require(info.ready === true);
  if (supportsSpeechV2(info)) return; // Reuse, not record.
  require(info.active === false);
  await authenticatedShutdownAndPortClosure();
}
await spawnWithNewReceiptAndVerifyReadiness();
```

Management waits are phase-specific: lock acquisition 50 seconds, HTTP request 2 seconds, startup observation 45 seconds, shutdown-port wait 12 seconds. Failed shutdown/startup preserves ownership evidence rather than killing an arbitrary OS process. The request reader caps JSON at 4,096 bytes and forbids redirects/proxies. (`services\transcription\manage.py:43-90`, request/lock; `services\transcription\manage.py:130-200`, stop/start.)

4. HTTP routes

Route	Authentication and successful response
`GET /health`	No authentication; `status` (`ready` / `unavailable`) and Boolean `modelAvailable`; HTTP 200/503. This is readiness, not ownership proof.
`GET /v1/runtime`	Bearer authentication; `service`, management `protocolVersion: 1`, `speechProtocolVersions: [1,2]`, `instanceId`, `pid`, `ready`, `active`.

Route	Authentication and successful response
`POST /v1/shutdown`	Bearer plus matching `X-Phantom-Instance`; no body; idle managed instance returns `{"status":"stopping"}`.
`GET /v1/transcribe`	Authenticated WebSocket upgrade; one active session, reserved before the start handshake.

These routes are registered by `create_app`. Runtime/shutdown/transcription reject query strings and bodies. Authentication requires exactly one Bearer header; nonempty browser Origin is rejected—403 on transcription, 401 on management. An identity mismatch or active shutdown attempt yields 409; unavailable speech yields 503. (`services\transcription\server.py:162-209`, `health`, `authorized`, management handlers; `services\transcription\server.py:353-378`, `transcribe`; `services\transcription\server.py:435-452`, `create_app`.)

5. Speech v1/v2 messages and bytes

Both versions use `/v1/transcribe`. Send one of these complete text messages after upgrade:

```
{"type":"start","sampleRate":16000,"channels":1,"encoding":"pcm_s16le"}
```

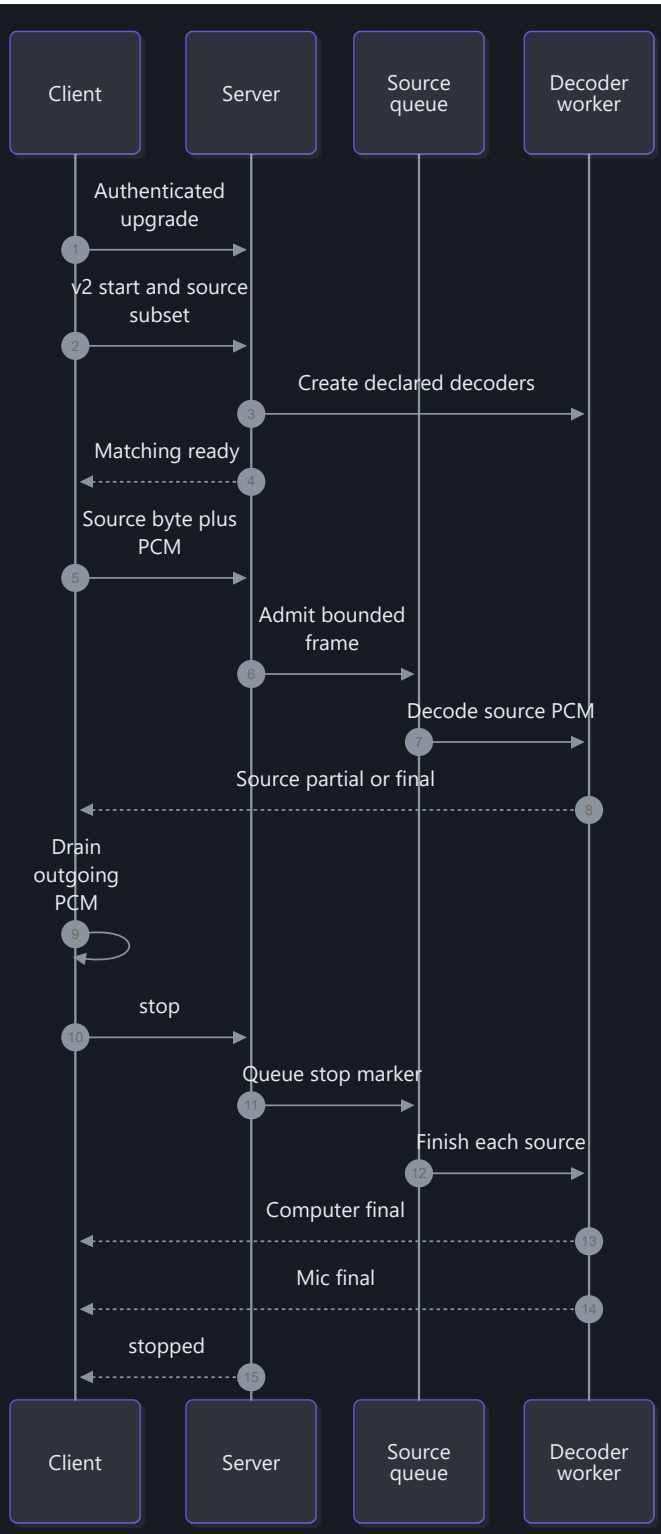
```
{"type":"start","protocolVersion":2,"sampleRate":16000,"channels":1,"encoding":"pcm_s16le","sources":["computer","microphone"]}
```

V1 is Computer-only. V2 source subsets must be nonempty, unique, and canonically ordered: Computer alone, Mic alone, or Computer then Mic. Version/rate/channel values must be integers; extra fields or unknown sources fail. `ready` echoes the accepted start fields with a different `type`; v2 clients reject downgrade. (`services\transcription\server.py:94-106`, `parse_start`; `services\transcription\server.py:378-385`, ready construction; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:407-426`, `ProcessMessage`.)

Item	V1	V2
Binary message	Unprefixed PCM	Source byte, then PCM
Source byte	Absent	`1` Computer; `2` Microphone; must be declared
PCM	Signed 16-bit little-endian, mono, 16 kHz	Same, separate tracks—not mixed
PCM length	Even, 2–32,000 bytes	Same
Application-message length	2–32,000 bytes	Odd, 3–32,001 bytes
Partial fields	`type`, `text`	Also `source`
Final fields	`type`, `text`, `segmentId`, `startSeconds`, `endSeconds`	Also `source`

The server strips the prefix before validation/accounting. At 32,000 PCM bytes/second, 100 ms means 3,200 PCM bytes or 3,201 v2 message bytes; neither duration nor queue credit includes the prefix. (`services\transcription\server.py:23-25`, source/format constants; `services\transcription\server.py:255-287`, `Session.receive`; `services\transcription\server.py:289-315`, `Session.publish`; `src\PhantomFudge.Core\AudioSource.cs:3-23`, wire identities.)

Final IDs increase globally across the session, while final times and replacement partials are source-local. Thus Computer ending at 5 seconds followed by Mic ending at 2 seconds is valid. Labels identify inputs, not speakers. `{"type":"stop"}` stops the entire session; source-scoped commit, Ask, and speech v3 are **not implemented**. (`services\transcription\server.py:247-252`, control handling; `services\transcription\server.py:289-327`, publication/drain; `docs\DUAL-SOURCE-AUDIO.md:10-17`, identity boundary; `docs\AI-ANSWER-ARCHITECTURE.md:124-158`, proposed commit.)



19. API and configuration reference - diagram 2. Source-linked; status follows the chapter.

Stop waits for a queue slot rather than discarding accepted PCM. Each declared source receives a final, even if empty, before `stopped`. Native drain has its own timeout and requires v2 stop-final evidence from every source; disconnection is not equivalent to completed drain. (`services\transcription\server.py:247-252`, `Session.receive`; `services\transcription\server.py:317-327`, `Session.recognize`; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:232-280`, `StopAsync`; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:454-465`, `ProcessMessage`.)

6. Resource defaults are code limits, not CLI knobs

Boundary	Current default
Server control text	1,024 UTF-8 bytes
Server queue per source	64,000 PCM bytes **and** 32 frames; Both totals 128,000 bytes/64 frames
Native pending audio	64,000 PCM bytes/source, 128,000 total; no server-style 32-frame/source cap
Server source rate credit	64,000-byte burst, 32,000 bytes/s refill; 200-frame burst, 100 frames/s refill
Server start / idle / decoder / send waits	5 / 15 / 5 / 2 seconds
Source duration ceiling	28,800 seconds of received PCM, not wall time
Native connection / final-drain waits	10 / 15 seconds
Native capture startup / release waits	10 / 5 seconds; separate from socket waits
Native idle-silence cadence	At least 250 ms without real packets, at least 100 ms between synthetic frames
Recognizer history rotation	At least 30 seconds of processed source audio since the previous rotation
Native response limit	32,768 bytes, JSON depth 8
Text limits	Server 8,192 Python characters; native 16,384 .NET string characters

Defaults are declared in `Limits` and native constants/parser, not environment overrides. Per-source queue quotas cannot borrow another source's spare capacity. For 3,200-byte frames, the byte quota is twenty queued frames; tiny two-byte frames instead reach the server's frame quota first. (`services\transcription\server.py:42-56`, `Limits`; `services\transcription\server.py:225-287`, `Session`; `services\transcription\server.py:289-292`, text validation; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:13-43`, constants/channel; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:343-366`, `ReceiveAsync`; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:430-440`, transcript parser.)

Capture waits, silence scheduling, and decoder rotation are distinct mechanisms, not one end-to-end deadline or acoustic endpoint setting. A timed-out native release retains worker ownership and blocks unsafe restart; a delayed silence tick does not backfill elapsed time. (`src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:12-14`, constants; `src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:79-82`, restart gate; `src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:289-309`, `SupplySilenceAsync`; `src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:384-402`, `FinishSessionAsync`; `src\PhantomFudge.App\Audio\SilenceCadence.cs:11-19`, scheduling; `services\transcription\recognizer.py:82-90`, rotation.)

7. Testing commands: choose the evidence layer

```
dotnet run --project .\tests\PhantomFudge.Audio.Tests -c Release
.\services\transcription\.venv\Scripts\python.exe -m unittest discover -s .\services\transcription\tests
```

The native harness is an executable, not an xUnit project; ordinary arguments filter test names case-insensitively. A filter matching nothing still returns success with zero checks, so inspect the reported count. Python protocol tests create isolated test servers with fake recognizers; they do not require a running production listener. (`tests\PhantomFudge.Audio.Tests\PhantomFudge.Audio.Tests.csproj:1-11`, executable definition; `tests\PhantomFudge.Audio.Tests\Program.cs:26-75`, `RunAsync`; `services\transcription\tests\test_protocol.py:23-62`, fixtures/setup.)

Specialized entry	Inputs and side effects
`scripts\Test-DualSourceSynthetic.ps1 [-NativeClient]`	Generates in-memory speech; existing cached model and ephemeral loopback server. Switch builds/uses C# transport.
`services\transcription\tests\Test-DualSyntheticTranscription.ps1 [-ModelPath path]`	Cached-model Python integration; no playback, real token, or persistent service.
Harness `--service URI --synthetic-pcm - --expect phrase`	Raw synthetic PCM stdin, at most 60 seconds; connects to the supplied service using the test-token environment variable.
Harness `--service URI --synthetic-wave path --expect phrase`	Same v1 check, but an explicitly supplied synthetic PCM16 WAV.
Harness `--service-v2 URI --synthetic-tracks-stdin --expect-computer phrase --expect-microphone phrase`	JSON stdin containing two base64 PCM tracks, at most 60 seconds each; checks source-specific finals.
`scripts\Test-PlaybackTranscription.ps1 -AllowPlayback`	**Audible/hardware test:** plays synthetic speech and captures all default render-device output. Explicit approval and paused unrelated audio are required.

These distinctions follow the actual wrappers and runners: `Test-DualSourceSynthetic` (`scripts\Test-DualSourceSynthetic.ps1:1-43`), probe `run` (`tools\DualSourceProbe\probe.py:18-57`), cached-model wrapper (`services\transcription\tests\Test-DualSyntheticTranscription.ps1:1-45`), v1 runner (`tests\PhantomFudge.Audio.Tests\Program.cs:78-136`), v2 runner (`tests\PhantomFudge.Audio.Tests\DualSyntheticServiceCheck.cs:10-79`), and playback wrapper/runner (`scripts\Test-PlaybackTranscription.ps1:1-14`; `tests\PhantomFudge.Audio.Tests\PlaybackCheck.cs:39-78`, `RunAsync`). The `--playback` executable branch itself has no `-AllowPlayback` switch: do not bypass the consent-bearing wrapper.

The repository's testing contract explicitly requires consent and paused unrelated audio for playback; ordinary development tests must not open the live microphone. (`AGENTS.md:65-67`, unattended-test boundaries.)

8. Errors and recovery interpretation

Service errors have exactly `type: "error"`, `code`, and a fixed safe `message`. The client replaces remote error text with its own generic message. Protocol errors normally attempt close code 1008; oversized transport messages can close with 1009 instead. (`services\transcription\server.py:26-66`, catalogue/`error`; `services\transcription\server.py:155-159`, `close_with_error`; `src\PhantomFudge.App\Audio\TranscriptionSocket.cs:397-405`, error handling; `services\transcription\tests\test_dual_protocol.py:257-264`, transport-ceiling test.)

Code	Meaning and first distinction to check
`auth`, `origin`	Credential/header failure versus unsupported browser-origin access.
`busy`	Another session owns admission; do not kill its process.
`bad_format`	Invalid start schema, version, source subset, or PCM format.
`bad_frame`	Invalid binary/control message, undeclared source, or disallowed request shape.
`backlog`, `rate_limit`	Queue/send pressure versus input arriving faster than permitted live rate.
`timeout`, `session_limit`	Expected-message wait expired versus source PCM-duration ceiling.
`decoder_error`, `model_unavailable`	Recognition failure versus unavailable readiness/model; neither invokes cloud fallback.
`server_stopping`	Service shutdown notification, not a successful transcription-completion acknowledgement.

The meanings come from `ERRORS` and their enforcing paths. A decoder-operation failure marks runtime readiness failed; an idle-only managed restart cannot safely terminate a stuck native call. Repair and restart are deliberate operational steps, not promises of automatic recovery. (`services\transcription\server.py:26-39`, `ERRORS`; `services\transcription\server.py:126-142`, `Runtime.cpu`; `services\transcription\server.py:237-287`, receive limits; `services\transcription\server.py:422-428`, shutdown; `services\transcription\manage.py:130-144`, `stop_owned`.)

Checkpoints and reference boundary

Before integrating, answer: Which component reads a token **file**? Does `modelAvailable` prove ownership? Why does a 3,201-byte message represent only 3,200 PCM bytes? Why must empty Mic finals precede `stopped`? The tables and cited functions above resolve these without inspecting generated private files.

This chapter validates source contracts, not running-service health, installed dependencies, microphone quality, or meeting compatibility. No listed command was executed for documentation. Ask/Azure answers and v3 remain proposals, not configuration options. (`docs\AI-ANSWER-ARCHITECTURE.md:1-6`, design status; `docs\QUALIFICATION.md:1-7`, qualification boundary.)

For direct lookup, use the manager, HTTP/session implementation, native transport, initialization script, and fixture entry points. All references resolve to maintained source, not generated receipts or local credentials.

20 — Glossary and Repository Map

Overview: vocabulary is part of the architecture

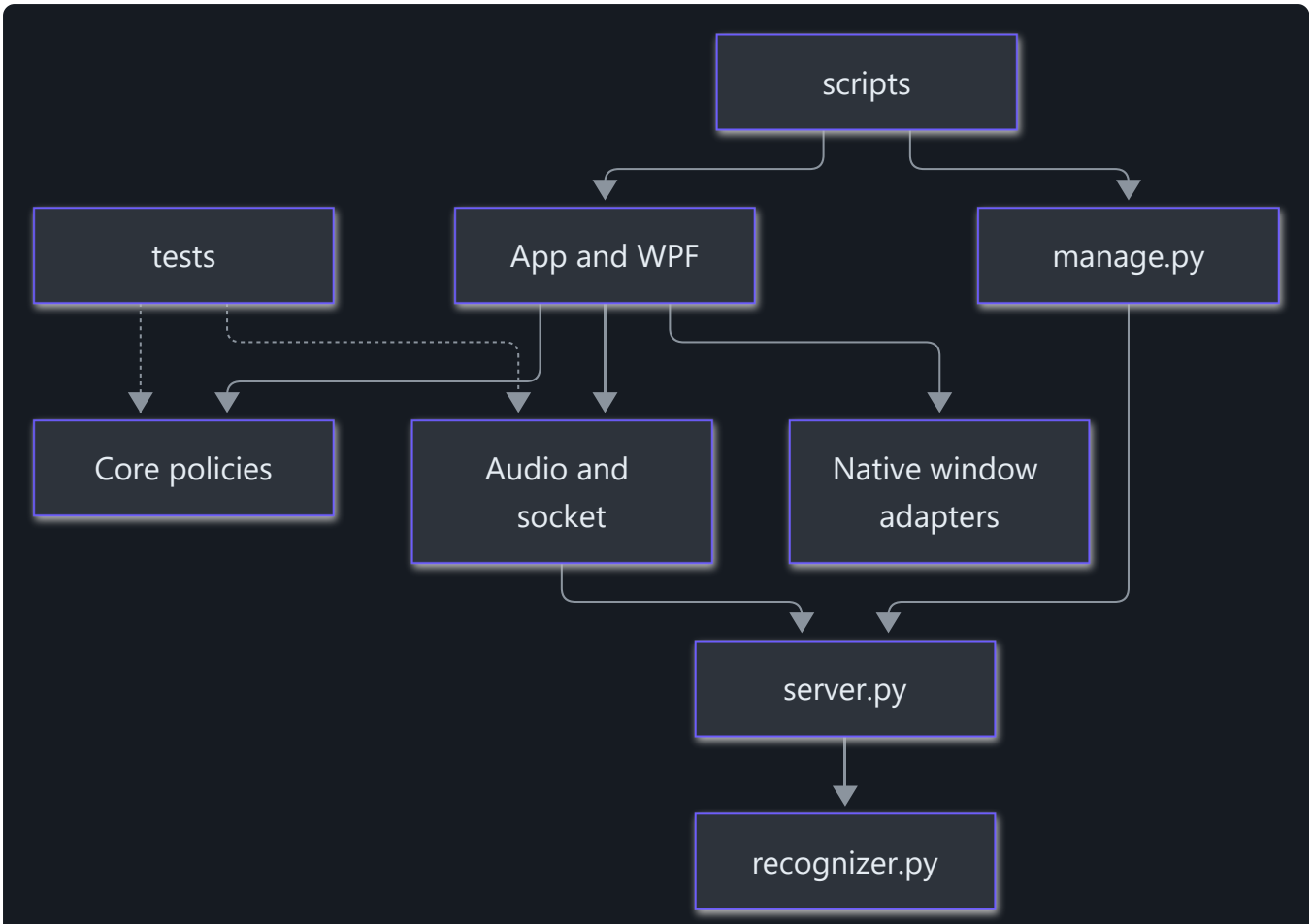
Many dangerous misunderstandings are substitutions between nearly related words: *source* becomes *speaker*, *configured* becomes *qualified*, or *stopped* becomes *fully drained*. This chapter makes those substitutions visible. A term is useful when it identifies an owner, a unit, or a completion condition—not merely when it expands an acronym.

For example, `AudioSource` names Computer and Microphone, while `Session.RequestStop` clears Running before cleanup finishes. Those are executable distinctions. “Ask” and source-scoped “commit,” by contrast, currently name proposed operations, not buttons or protocol messages supported by the runnable application. (src\PhantomFudge.Core\AudioSource.cs:3-30; src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:504-518; docs\AI-ANSWER-ARCHITECTURE.md:3-6,124-158)

Use this chapter as a junction rather than a replacement for the detailed chapters. Start with a question, locate the responsible boundary, then follow the cited function. Source establishes implementation; a test establishes its asserted conditions; qualification requires separate environmental evidence. (docs\QUALIFICATION.md:9-26)

Architecture and repository map

The main dependency direction is native application → Core policies and local speech service. The service manager owns process lifecycle, not microphone capture. Recognition remains in Python; controls, selected WASAPI inputs, and transcript presentation remain in C#. (src\PhantomFudge.App\PhantomFudge.App.csproj:3-13; src\PhantomFudge.App\Audio\WasapiInput.cs:20-54; services\transcription\manage.py:147-197; services\transcription\server.py:353-386)



20 — Glossary and Repository Map - diagram 1. Source-linked; status follows the chapter.

This is an ownership map, not a claim that the manager processes PCM. `Run-PhantomFudge.ps1` starts the service unless NotesOnly is selected, then launches the app with a local endpoint; audio still requires `Start_Click`. (scripts\Run-PhantomFudge.ps1:39-55; src\PhantomFudge.App\MainWindow.xaml.cs:589-625)

Repository area	Open this first; why
`src\PhantomFudge.App`	App entry, then `MainWindow`: startup, recovery, UI orchestration. `OnStartup` explicitly calls protected reveal. (src\PhantomFudge.App\App.xaml.cs:39-59)
`src\PhantomFudge.Core`	TranscriptBuffer and NoteFileStore: text/encoding policies without WPF controls. (src\PhantomFudge.Core\TranscriptBuffer.cs:6-39; src\PhantomFudge.Core\NoteFileStore.cs:10-44)
`services\transcription`	server.py owns protocol; recognizer.py owns Vosk adaptation; manage.py owns authenticated lifecycle. (services\transcription\server.py:435-451; services\transcription\recognizer.py:41-96; services\transcription\manage.py:112-144)
`scripts`	Run-PhantomFudge.ps1: source/package launch and existing-window recovery, not automatic recording. (scripts\Run-PhantomFudge.ps1:9-41)
`tests`	Core file tests and native harness: deterministic policies and synthetic protocol checks. (tests\PhantomFudge.Core.Tests\NoteFileStoreTests.cs:13-109; tests\PhantomFudge.Audio.Tests\Program.cs:26-75)
`spikes` and `tools`	CaptureProof project links diagnostic code; SpeechBench and DualSourceProbe serve separate experiments. (spikes\CaptureProof\CaptureProof.csproj:9-12; tools\SpeechBench\bench.py:1-25; tools\DualSourceProbe\probe.py:18-33)

Repository area	Open this first; why
`docs` and `TODO`	Qualification defines evidence; TODO index records dependencies and status, not executable behavior. (docs\QUALIFICATION.md:140-166; TODO\00_INDEX.todo:38-47)
Repository root	Solution lists projects; global.json pins SDK policy. Compose is optional, last-stage packaging—not a native prerequisite. (PhantomFudge.slnx:1-13; global.json:2-6; docs\IMPLEMENTATION-STATUS.md:54-57)

Navigation by question

Question	Read next
“Where do I start?”	Reading map, then newcomer guide
“Why isn't capture affinity a security boundary?”	Windows and cursors, assurance
“Which device produced these words?”	Dual-input capture, protocol
“Why does Stop take time?”	State machines
“Why is the service ready but nothing recorded?”	Native lifecycle, recognition
“What is saved, and what survives collapse?”	Storage, UI/themes
“Does the app already send questions to Azure?”	Proposed answer flow, identity/privacy
“Which experiment or check should I inspect?”	Testing, benchmarks, troubleshooting

Grounded glossary

Audio: units before labels

Term	Meaning here
Sample	One amplitude value. `VoskRecognizer.accept` counts one sample per two PCM bytes—not one sentence or callback. (services\transcription\recognizer.py:82-84)
PCM16	Signed 16-bit, little-endian pulse-code modulation. `StartMessage` requires this encoding; compressed audio is not interchangeable. (src\PhantomFudge.App\Audio\TranscriptionSocket.cs:13-24)
Sample rate	Samples per second per channel: 16,000 in this contract, giving 32,000 PCM bytes/second for mono PCM16. (src\PhantomFudge.App\Audio\TranscriptionSocket.cs:13-23)
Channel	A waveform dimension, not a person. `WasapiInput` requests one channel for each independently selected input. (src\PhantomFudge.App\Audio\WasapiInput.cs:48-54)
Audio frame	Here, a bounded transport payload: at most 32,000 PCM bytes. It is not a video frame or necessarily one sample frame. (src\PhantomFudge.App\Audio\TranscriptionSocket.cs:15-18,182-195)
Chunk	A convenient processing slice, not a fixed-duration semantic unit. `OnAudio` splits a callback's bytes into bounded transport pieces. (src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:230-246)
Source	Computer or Microphone, explicitly selected; `DisplayName` renders the latter as Mic. Identity describes origin, not speaker recognition. (src\PhantomFudge.Core\AudioSource.cs:3-30)
Stream / track	One source's ordered PCM and decoder state. Both tracks share a WebSocket without becoming one mixed waveform. (services\transcription\server.py:225-235,255-282,378-386)

Term	Meaning here
Session	One negotiated transcription conversation: selected sources, decoders, queue, and global final-ID counter. It is shorter-lived than the service process. (services\transcription\server.py:225-235,378-408)
Speaker	A human identity the current source labels do not establish. Two voices on Computer remain Computer, not automatically separate people. (tests\PhantomFudge.Core.Tests\SourceTranscriptTests.cs:81-87)
Loopback	Capture of a render endpoint's output through `WasapiLoopbackCapture`, not capture limited to a particular meeting application. (src\PhantomFudge.App\Audio\WasapiInput.cs:48-54; docs\PRODUCT.md:76-87)
Endpoint	The selected Windows audio device identified by ID, direction, and active state—not the WebSocket network address. (src\PhantomFudge.App\Audio\WasapiInput.cs:64-74)
ASR	Automatic speech recognition: `VoskRecognizer.accept` turns PCM into recognition results. It does not answer the semantic question spoken aloud. (services\transcription\recognizer.py:82-96)

Transport and lifecycle: readiness is not recording

Term	Meaning here
WebSocket	The persistent local message connection carrying source-prefixed binary PCM and text controls/results. `ConnectCoreAsync` negotiates the session first. (src\PhantomFudge.App\Audio\TranscriptionSocket.cs:135-160,208-218)
Ready	Scope matters: `Runtime.ready` describes service availability; a session `ready` response confirms negotiated format/sources. Neither means hardware capture already began. (services\transcription\server.py:122-124,378-386; src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:98-109)
Backpressure	More incoming work than bounded processing capacity permits. `TryEnqueueAudio` faults on overflow rather than silently dropping PCM and continuing. (src\PhantomFudge.App\Audio\TranscriptionSocket.cs:198-229)
Stop	A session-wide request: close audio admission, drain queued PCM, send `stop`, and await terminal recognition—not a source-only question commit. (src\PhantomFudge.App\Audio\TranscriptionSocket.cs:232-280,313-331)
Drain	Completion of previously admitted work. `Session.recognize` consumes queued audio before finishing every decoder and sending `stopped`. (services\transcription\server.py:247-252,317-333)
IsRunning	A current-activity flag, not proof of drain or released devices. `RequestStop` clears Running before asynchronous cleanup finishes. (src\PhantomFudge.App\Audio\LoopbackTranscriber.cs:29-29,377-423,504-518)
Service instance	The manager's process-lifecycle identity. `start` generates an instance ID; it is not a transcript session ID or speaker ID. (services\transcription\manage.py:164-172)
Receipt	Lifecycle metadata checked against authenticated runtime identity by `owned_info`; it is not a transcript or authorization to kill unrelated processes. (services\transcription\manage.py:93-122)
Authentication token	The local shared credential checked by `authorized`, not an Azure identity or a model token-count unit. (services\transcription\server.py:171-176)

Text and storage: provisional, durable, and dirty differ

Term	Meaning here
Partial	Replaceable unfinished recognition, held separately per source by `SetPartial`; appending every revision would duplicate words. (src\PhantomFudge.Core\TranscriptBuffer.cs:10-19,41-46)
Final	A recognizer-finalized segment, possibly empty. `AddFinal` clears only its source's partial; “final” does not certify transcription accuracy. (src\PhantomFudge.Core\TranscriptBuffer.cs:48-72)

Term	Meaning here
Segment ID	Session-global ordering across sources, not a durable cross-session identifier. `publish` increments one counter; `BeginSession` resets the UI baseline. (services\transcription\server.py:301-311; src\PhantomFudge.Core\TranscriptBuffer.cs:35-38)
Audio-relative timestamp	Position in a source's audio timeline, not wall-clock meeting time. `publish` checks bounds against that source's received PCM duration. (services\transcription\server.py:289-311)
Session reset	`BeginSession` retains earlier history and marks unfinished text before resetting IDs/partials. It is not `Clear`. (src\PhantomFudge.Core\TranscriptBuffer.cs:23-39,79-85)
Export	Explicit file persistence of a text snapshot, including labelled unfinished recognition. `ExportTranscriptAsync` writes text, not raw audio. (src\PhantomFudge.App\MainWindow.xaml.cs:733-768; src\PhantomFudge.Core\TranscriptBuffer.cs:75-77)
Dirty	Current text differs from the last successful save/export baseline. It does not mean the disk file was checked recently. (src\PhantomFudge.App\MainWindow.xaml.cs:267-268,384-398)
UTF-8 byte cap	`Fits` permits at most 1 MiB encoded bytes; character count alone is insufficient for multibyte Unicode. (src\PhantomFudge.Core\NoteFileStore.cs:12-24)
BOM	Optional three-byte UTF-8 signature, preserved in `NoteSnapshot` and included in the byte budget—not visible document text. (src\PhantomFudge.Core\NoteFileStore.cs:19-19,41-44,56-58)
Fingerprint	SHA-256 of file bytes used by `VerifyUnchangedAsync` for optimistic conflict detection, not encryption or a tamper-proof vault. (src\PhantomFudge.Core\NoteFileStore.cs:41-44,83-95)
Staged rename	`WriteAsync` writes and flushes bytes beside the destination, rechecks, then moves. This narrows failure exposure without eliminating check-to-move races. (src\PhantomFudge.Core\NoteFileStore.cs:53-80)

Desktop: pixels, focus, and discovery are separate

Term	Meaning here
HWND	Native window handle obtained by `EnsureHandle`; capture affinity is configured on this object, not on every surface in the process. (src\PhantomFudge.App\Native\WindowProtection.cs:23-36)
Visual tree	WPF's nested content/control structure. Inline setup and confirmation panels belong to `MainWindow`; an external dialog is a different boundary. (src\PhantomFudge.App\MainWindow.xaml:15-27,70-85,105-141)
DWM	Desktop Window Manager composition, checked and flushed by `Configure`; a flush is not independent recipient observation. (src\PhantomFudge.App\Native\WindowProtection.cs:24-27,43-45)
Affinity	Here, display affinity `0x11`: a Windows capture-exclusion request. It is unrelated to CPU affinity and not universal invisibility. (src\PhantomFudge.App\Native\WindowProtection.cs:16-45; docs\QUALIFICATION.md:9-26)
Opacity	The root background brush's alpha, 10–100%; `BackgroundOpacity_ValueChanged` does not fade the whole window or glyphs. (src\PhantomFudge.App\MainWindow.xaml.cs:442-449,977-981)
Taskbar presence	Shell presentation controlled by `ShowInTaskbar` and tool-window styles, not proof of picker omission or hidden processes. (src\PhantomFudge.App\MainWindow.xaml:5-8; src\PhantomFudge.App\Native\WindowProtection.cs:29-31; docs\QUALIFICATION.md:13-18)
Focus	Keyboard/foreground interaction state, distinct from visibility and cursor shape. `UpdateInteractionControls` changes focusability separately from content. (src\PhantomFudge.App\MainWindow.xaml.cs:244-257)
Present	Default passive, read-only interaction mode; it neither starts capture automatically nor certifies the sharing configuration. (src\PhantomFudge.App\MainWindow.xaml.cs:31-34,244-257,589-625)

Term	Meaning here
Edit	Deliberate focusable editing. `TogglePresentationAsync` currently hides and stops transcription before switching—not a purely cosmetic toggle. (src\PhantomFudge.App\MainWindow.xaml.cs:223-241)
Arrow	Standard local cursor shape set by `CursorPolicy`, not an instruction controlling a recorder's separate pointer stream. (src\PhantomFudge.App\Native\CursorPolicy.cs:8-24; docs\CURSOR-AND-THEMES.md:23-37)
Semantic theme	Named color roles converted into WPF resources by `ThemeBrushes.Apply`; appearance changes do not constitute content or session replacement. (src\PhantomFudge.Core\ThemePalette.cs:5-28; src\PhantomFudge.App\ThemeBrushes.cs:9-40)

Plans and evidence: naming is not implementation

Term	Meaning here
Commit	Planned source-scoped finalization through a negotiated watermark/acknowledgement—not current global Stop, and not a Git commit. (docs\AI-ANSWER-ARCHITECTURE.md:124-158)
Ask	Planned explicit submission of selected question text to Azure. Current v2 accepts Stop, not an Ask/commit control. (docs\AI-ANSWER-ARCHITECTURE.md:48-68; services\transcription\server.py:247-252)
QuestionSnapshot	Proposed immutable submitted question revision, insulated from later ASR updates; not a current Core record. (docs\AI-ANSWER-ARCHITECTURE.md:84-118)
Synthetic fixture	Deliberately generated test content. `RunUiSmokeAsync` uses synthetic text/devices; visual-tree renders are not screen-sharing captures. (src\PhantomFudge.App\MainWindow.xaml.cs:818-859,1085-1098)
Qualification	Configuration-specific recipient/recording evidence with controls and explicit outcomes; neither build success nor API readback substitutes for it. (docs\QUALIFICATION.md:9-26,79-107)
Preview	Implemented development software with open release gates, not “planned only” and not a universally supported release. (docs\IMPLEMENTATION-STATUS.md:1-3,149-158)

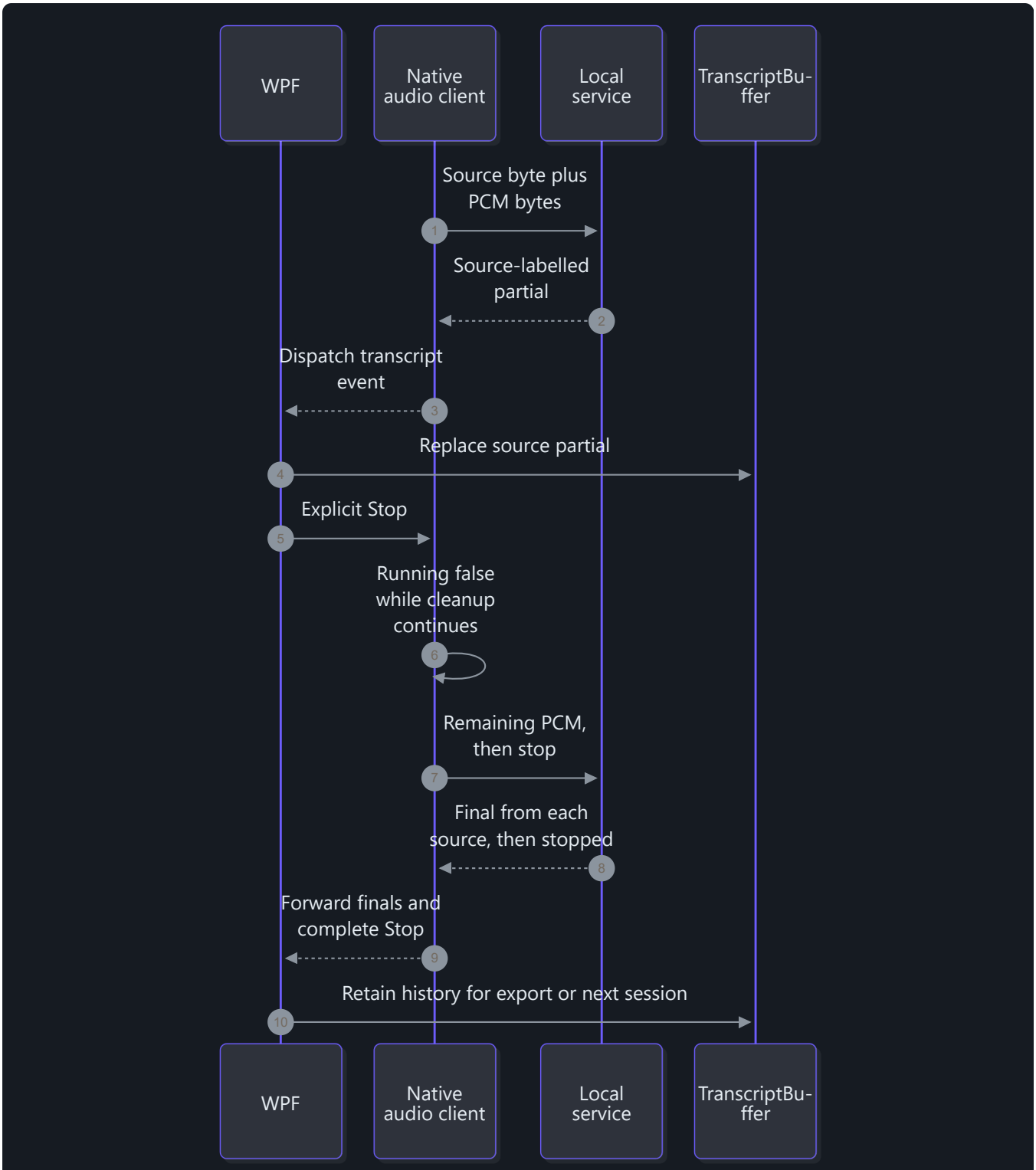
Data flow: one worked boundary trace

A 100 ms mono PCM16 block at 16 kHz contains 1,600 samples and 3,200 PCM bytes. Protocol v2 adds one source byte, so the binary message contains 3,201 bytes. It still represents one source, not 3,201 samples or one recognized sentence. `TryEnqueueAudio` explicitly keeps PCM length separate from prefixed wire bytes. (src\PhantomFudge.App\Audio\TranscriptionSocket.cs:13-17,208-218)

C#'s `ReadOnlySpan<byte>` and Python's `bytes` describe byte containers, not automatic sample counters. `VoskRecognizer.accept` performs the byte-to-sample conversion explicitly; changing languages does not change the units. (src\PhantomFudge.App\Audio\TranscriptionSocket.cs:185-195; services\transcription\recognizer.py:82-84)

This Python-shaped arithmetic is explanatory, not production capture code:

```
samples = 16_000 // 10
pcm_bytes = samples * 2
wire_bytes = 1 + pcm_bytes
assert (samples, pcm_bytes, wire_bytes) == (1600, 3200, 3201)
```



20 — Glossary and Repository Map - diagram 2. Source-linked; status follows the chapter.

The diagram shows a graceful path, not a guarantee after transport or device failure. Python's stop sentinel follows queued PCM; the client validates finals before terminal acknowledgement; the UI holds `audioStopping` until its awaited operation finishes. (services\transcription\server.py:247-252,317-333; src\PhantomFudge.App\Audio\TranscriptionSocket.cs:437-465; src\PhantomFudge.App\MainWindow.xaml.cs:681-708)

Glossary pitfalls and review exercises

“Two streams means two speakers.” No: independent source tracks prevent state mixing, not acoustic echo or multiple voices sharing one endpoint. Inspect source labels before inferring people. (docs\PRODUCT.md:76-87)

“Final means saved.” No: final concerns recognizer state; export concerns filesystem persistence. The buffer can contain finals while `TranscriptDirty` remains true. (src\PhantomFudge.App\MainWindow.xaml.cs:267-268,733-760)

“Hidden means stopped and exited.” No: Hide awaits Stop but retains the process and content; Exit separately evaluates unsaved work. (src\PhantomFudge.App\MainWindow.xaml.cs:212-220,270-300)

Exercise: Is 100% background opacity stronger capture exclusion than 10%?
Answer: No. Brush alpha and native affinity are independent mechanisms. (src\PhantomFudge.App\MainWindow.xaml.cs:442-449; src\PhantomFudge.App\Native\WindowProtection.cs:35-45)

Exercise: Can an idle ready Python service contain several successive sessions?
Answer: Yes. Session admission is released after cleanup; service identity need not change. (services\transcription\server.py:364-386,403-408)

References and maintenance rule

Follow the repository links above for implementation; use qualification for evidence and answer architecture for proposed contracts. Update a glossary entry when its owner, units, or completion condition changes—not merely when a class is renamed. Preserve the implemented/planned distinction until the corresponding call path exists.

21. Evidence and Roadmap

Why evidence needs its own architecture

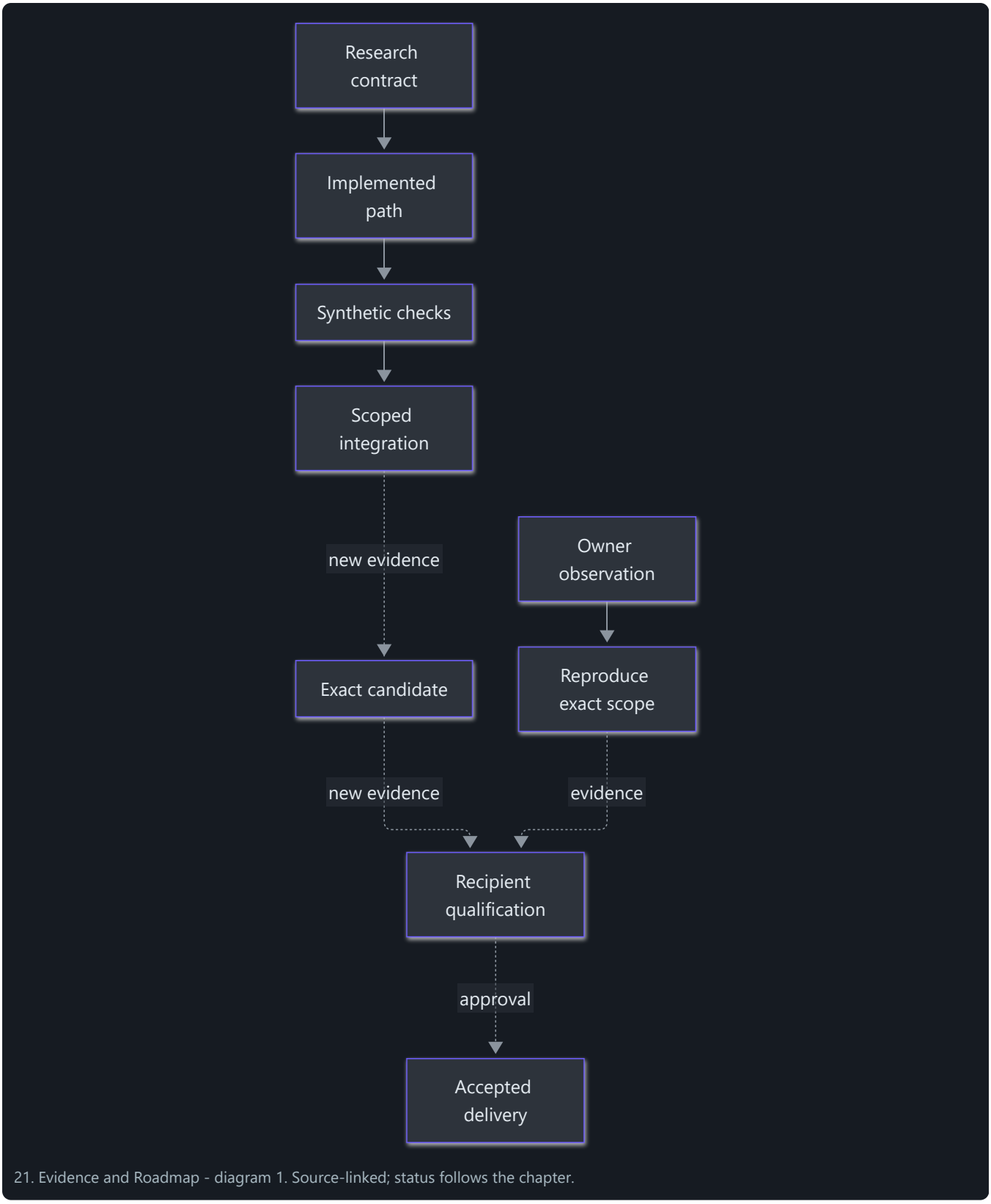
A living engineering book must distinguish **what exists**, **what was observed**, and **what is justified for users**. Phantom Fudge has an implemented native preview and recorded local results; it does not have a qualified public release or an implemented Azure answer feature. A plausible design, passing synthetic test, and owner acceptance answer different questions. Combining them into one “done” badge would discard the most important information. (docs\IMPLEMENTATION-STATUS.md:3-21, current scope; docs\TODO-CONVENTIONS.md:58-70, evidence classes.)

This chapter reads the frozen application source and current public evidence summaries. It does not rerun application tests, inspect private capture receipts, requalify a package, or perform cloud/audio operations. “Recorded result” below means the cited repository reports that observation, not that this documentation session independently repeated it. The qualification contract explicitly rejects substituting local previews, API return codes, or missing recordings for recipient evidence. (TODO\13_COMPATIBILITY_QUALIFICATION.todo:24-35, acceptance and evidence.)

The engineering unit of assurance is therefore a **bounded claim**: a behavior, an exact subject, an environment, a procedure, an observation, and its limits. The repository's result template already asks for source identity, package hash, operating system, GPU, sharing backend, positive controls, frame coverage, and reviewer approval. A result without those boundaries may help investigation, but should not silently widen support. (docs\QUALIFICATION.md:140-166, result record.)

The evidence ladder: stronger claims require additional proof

The ladder is not permission to infer the next rung. Each transition requires evidence appropriate to its scope; an owner report remains a separate observation until reproduced under a known configuration. (docs\TODO-CONVENTIONS.md:64-70, noninterchangeable evidence; docs\CURSOR-AND-THEMES.md:21-38, unresolved owner report.)



21. Evidence and Roadmap - diagram 1. Source-linked; status follows the chapter.

Evidence class	What it establishes	What it cannot establish alone
Primary research	A documented API capability or proposed design	This account's permissions or a working integration
Source inspection	An implemented branch and its invariants	Successful execution on a user's machine

Evidence class	What it establishes	What it cannot establish alone
Synthetic checks	Behavior under controlled inputs and injected failures	Real microphone quality or recorder compatibility
Real engine, synthetic PCM	Model/protocol integration without live capture	Real-speaker, room, echo, or meeting accuracy
Scoped hardware observation	A particular device path actually worked	Other devices, capture backends, or environments
Exact-package qualification	Observed behavior for an identified candidate/configuration	All future versions or unsampled frames
Owner/release acceptance	Deliberate approval of delivered scope	A universal privacy or invisibility guarantee

These distinctions are instantiated by the native-runtime evidence, dual-source integration limits, qualification procedure, and final acceptance gates. (TODO\22_NATIVE_PYTHON_RUNTIME.todo:30-38, native results; TODO\26_MICROPHONE_AND_COMPUTER.todo:34-41, synthetic integration; docs\QUALIFICATION.md:79-107, capture procedure; TODO\17_RELEASE_ACCEPTANCE.todo:13-29, acceptance.)

Configuration is not observation of captured pixels

`WindowProtection.Configure` checks Windows/DWM conditions, sets window styles and affinity, inspects available readback, and flushes composition. None of those calls observes another participant's screen. `ShowProtected` configures before showing and reports an unqualified-preview status. This is implemented failure-prevention logic, not recipient certification. (src\PhantomFudge.App\Native\WindowProtection.cs:18-45, `Configure`; src\PhantomFudge.App\MainWindow.xaml.cs:79-105, `ShowProtected`.)

Similarly, `CursorPolicy.SetArrow` sets the local standard cursor. It does not choose the recorder's cursor-capture option. The owner's reported disappearing shared cursor is credible as a reported symptom, while its exact product/backend reproduction remains missing. Local Arrow and nonactivation evidence cannot close that gap. (src\PhantomFudge.App\Native\CursorPolicy.cs:8-24, `SetArrow`; docs\CURSOR-AND-THEMES.md:23-37, recorder boundary.)

What the current evidence actually says

The latest dual-source update records **41 Core tests, 46 native synthetic groups, 71 backend tests, and four manager-upgrade guards**. These are differently scoped counts, not one interchangeable test total. Their historical pass reports are useful evidence, but no fresh pass is asserted here. (docs\IMPLEMENTATION-STATUS.md:17-25, current source evidence.)

The recorded real Python recognizer processed generated dual-source PCM with Computer/Mic partial counts of **11/8** and three finals each. The actual C# v2-to-Python path produced **13/10** partials and two finals each, with expected phrases and tail drain. “Real” describes the engine and transport; the input was still synthetic. No physical microphone, device-permission, echo, or OS-indicator qualification follows. (docs\IMPLEMENTATION-STATUS.md:17-21, integration results.)

Two earlier observations have different boundaries:

- An explicitly approved playback-loopback check recorded 11 partials, three finals, 37 nonzero-meter updates, expected phrase matching, and zero faults. That establishes one actual render-endpoint path,

not the later dual-input hardware matrix. (docs\IMPLEMENTATION-STATUS.md:119-131, playback evidence.)

- A cropped GDI proof reported a 100,000-differing-pixel positive control and 16 protected samples matching the underlying baseline. That is specific BitBlt evidence, not continuous startup-frame coverage or Teams/Zoom/Meet/OBS qualification. (docs\IMPLEMENTATION-STATUS.md:120-127, synthetic capture evidence.)

The public speech benchmark covers **32 synthetic cases, 164.070 seconds, and 292 normalized reference words**, with two serial warm passes. Its recorded results are:

Engine	WER	Median warm decode
SAPI en-US dictation	45.89%	10.139 s
Vosk small-en-us-0.15	8.22%	41.754 s
Whisper base.en, CPU int8	4.79%	25.745 s

Those values are present in the public receipt, not extrapolated from model names. WER is normalized edit count divided by reference words, not a general accuracy percentage. File RTF is throughput, not live latency. Two paced Vosk first partials near 0.96 seconds and finals near 3.807 seconds are measured **from stream start**, not end-of-speech lag. Vosk remains the streaming implementation; Whisper refinement remains an experiment. (tools\SpeechBench\results\synthetic-20261008.json:13-22, run metadata; tools\SpeechBench\results\synthetic-20261008.json:65-173, `summary`; docs\SPEECH-BENCHMARK.md:61-118, metrics and timing; TODO\23_SPEECH_ENGINE_BENCHMARKS.todo:31-35, disposition.)

The current preview artifact is an identity, not a release claim

The latest dual-source bundle recorded by the public status and packaging workstream is:

```
artifacts\phantom-fudge-0.1.0-preview-win-x64-20261008-145231.zip
Size: 76,184,066 bytes
Contents: 485 files
Recorded SHA-256:
BC89867277F3E8C8BAA179BE74B7F517A98D6E09CBE35534AFB21E12E127C846
```

The hash is reproduced from the source record, not freshly computed. Exact-package UI checks are reported with `recordingStarted:false`; that makes them deliberately unsuitable as proof of hardware recording. Earlier `123245`, `035959`, and other bundles remain historical artifacts, not alternative evidence for the latest package by default. (docs\IMPLEMENTATION-STATUS.md:27-33, current bundle; TODO\14_PACKAGING_RELEASE.todo:34-46, package evidence.)

The publication script creates a timestamped self-contained x64 preview, refuses overwrite, copies selected documentation/scripts/service source, archives the result, and prints its hash as a **local unqualified preview**. Running that script would not satisfy clean-machine, signing, recipient, or release gates. No package was generated for this chapter. (scripts\Publish-Preview.ps1:10-41, publication flow; TODO\14_PACKAGING_RELEASE.todo:13-30, remaining acceptance.)

What still needs qualification

Recipient pixels: the required meeting/OBS candidates remain `NOT_TESTED`. Record product/version, exact sharing source and backend, local visibility, remote live view, applicable recording, positive controls, and transitions. A window/tab result cannot justify whole-screen support; a missing licensed recording capability is not a pass. A black rectangle can fail the intended experience even without readable text. (docs\QUALIFICATION.md:3-7, status; docs\QUALIFICATION.md:28-64, matrix and artifact criteria; docs\QUALIFICATION.md:79-107, procedure.)

Cursor: reproduce the owner's report in the actual sharing mode and record hover/click/drag/type, foreground changes, and recipient cursor behavior independently. Do not substitute a fake cursor, recorder modification, or the local value `MA_NOACTIVATE=3` for that observation. (TODO\24_CURSOR_AND_FOCUS.todo:14-26, open gates; docs\QUALIFICATION.md:127-135, C15.)

Hardware and usability: microphone access/denial, selected-device loss, echo, privacy indicators, headsets, alternative GPU/display/DPI/HDR combinations, startup transitions, accessibility, and long-session behavior remain scoped work. The code selects render versus capture endpoints explicitly; source inspection proves that branch exists, not that every physical configuration works. Owner readability/focus approval remains separate from automated success. (src\PhantomFudge.App\Audio\WasapiInput.cs:20-74, selection and `Direction`; TODO\26_MICROPHONE_AND_COMPUTER.todo:34-41, hardware gaps; docs\QUALIFICATION.md:66-77, environments; docs\IMPLEMENTATION-STATUS.md:149-158, remaining work.)

The owner handles permission and recipient rehearsal externally; those preferences removed repeated runtime confirmations. They are not an engineering pass, and this roadmap does not reintroduce those dialogs. (docs\PREVIEW-GUIDE.md:51-56, interaction contract; TODO\16_DEVELOPMENT_LEDGER.todo:63-65, decision record.)

Roadmap: statuses describe scope, dependencies describe order

The current index covers workstreams **01–38**; documentation work adds an ongoing maintenance obligation without changing the implemented-versus-planned application boundary:

IDs	Current disposition
01–12	02 research is DONE; 11 accessibility is NOT_STARTED; the remaining product/core workstreams are IN_PROGRESS.
13–18	13 qualification, 17 acceptance, and optional 18 future scope are NOT_STARTED; 14 packaging, 15 guide, and 16 ledger are IN_PROGRESS.
19–26	22 native runtime and 23 benchmark are DONE; 20 Docker is NOT_STARTED; 19, 21, and 24–26 remain IN_PROGRESS.
27	DONE for research/design only.
28–35	NOT_STARTED: drafts, commit, Azure connection, client/UI, evaluation, resilience, and answer acceptance.

IDs	Current disposition
36–37	NOT_STARTED optional cloud-ASR and conditional broker extensions.
38	DONE for the initial illustrated book and public journal; workflow06 continues maintenance with later changes.

"DONE" is scoped: research completion does not mean Ask exists. `NOT_STARTED` does not automatically mean `BLOCKED`; the latter requires a concrete attempted-work blocker and needed action. Stable numbers are not execution order. (TODO\00_INDEX.todo:38-90, statuses; docs\TODO-CONVENTIONS.md:40-56, dependency semantics.)

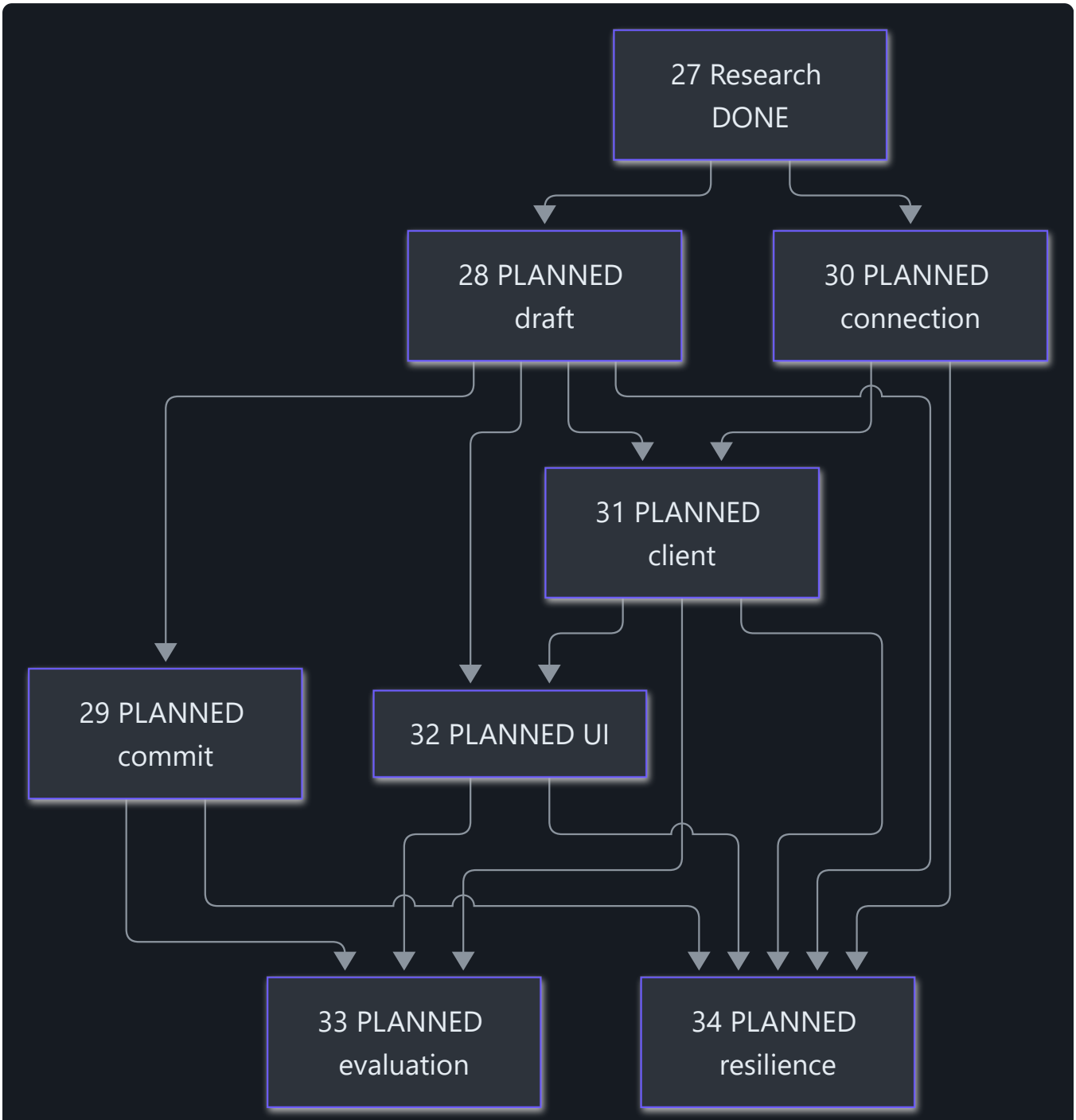
Candidate, qualification, and release



21. Evidence and Roadmap - diagram 2. Source-linked; status follows the chapter.

The diagram abbreviates native prerequisites, not their completion requirements. Packaging **14 precedes qualification 13**; the current expanded release graph also includes planned answer client/UI/resilience work. Final acceptance 17 requires 13, 14, 15, and 35. Docker **20 is LAST, optional, and depends on 17**—never a prerequisite for ordinary transcription or native release. Recipe files do not prove a container ran. (TODO\00_INDEX.todo:95-127, dependency map; TODO\14_PACKAGING_RELEASE.todo:3-5, prerequisites; TODO\20_DOCKER_LOCAL_SERVICE.todo:3-10, final-stage contract; TODO\20_DOCKER_LOCAL_SERVICE.todo:29-36, execution limits.)

The answer path remains PLANNED



21. Evidence and Roadmap - diagram 3. Source-linked; status follows the chapter.

Start with structured local drafts and fake streams, not a cloud deployment. The baseline remains local ASR, explicit immutable Ask, negotiated Mic-only commit, and direct C# streamed text into the same protected UI. Azure identity/resource/geography approvals, actual model availability, and latency measurements remain future gates. Optional 36 depends on 27 and 33; conditional 37 depends on 27 and 30, neither silently becoming mandatory. (TODO\00_INDEX.todo:108-121, answer dependencies; TODO\27_AI_ANSWER_RESEARCH.todo:13-32, research-only scope; docs\AI-ANSWER-ARCHITECTURE.md:262-283, delivery and unresolved decisions.)

Worked review algorithm: promote claims, not enthusiasm

The following is explanatory pseudocode for reviewing evidence, not an implemented tool or a test executed here. Its checks derive from the exact-candidate and positive-control requirements. (docs\QUALIFICATION.md:79-107, procedure; TODO\14_PACKAGING_RELEASE.todo:22-30, candidate freeze.)

```
def assess_support_claim(claim, evidence):
    matching = [e for e in evidence
                if e.artifact_hash == claim.artifact_hash
                and e.configuration == claim.configuration]
    if not matching:
        return "NOT_TESTED"
    if any(e.observed_readable_content for e in matching):
        return "FAIL_CONTENT_LEAK"
    if any(e.observed_unacceptable_artifact for e in matching):
        return "FAIL_ARTIFACT"
    if not has_required_controls_and_recipient_frames(matching):
        return "INSUFFICIENT_EVIDENCE"  # review result, not a matrix status
    return "SCOPED_PASS_CANDIDATE"      # still needs review and acceptance
```

For example, a claim that the `145231` package supports Teams desktop sharing cannot be promoted using an older cropped GDI result. Both artifact and capture scope differ. Likewise, an owner's rehearsal preference cannot replace the missing recipient record. Retain failed and unavailable cases; changing a binary or backend makes prior results historical rather than transferable. (docs\QUALIFICATION.md:24-26, version changes; docs\QUALIFICATION.md:164-166, exact artifact; docs\IMPLEMENTATION-STATUS.md:27-33, latest package.)

Keeping the book honest as the project changes

Reviewing the book exposed historical wording that no longer matched the implementation. The summary now describes Computer/Microphone/Both and labels its initial-preview evidence historical; workstream17 distinguishes the existing preview from unstarted final acceptance; workstream13 follows the current C01-C18 scenario set. These corrections preserve the earlier journey without erasing implemented work or promoting a local package to a qualified release. (docs\IMPLEMENTATION-STATUS.md:101-110, corrected summary and historical heading; TODO\17_RELEASE_ACCEPTANCE.todo:31-37, final acceptance scope; TODO\14_PACKAGING_RELEASE.todo:34-46, local artifacts; TODO\13_COMPATIBILITY_QUALIFICATION.todo:18-21, current scenario list; docs\QUALIFICATION.md:127-135, added scenarios.)

Current documentation coordination: workstream **38** is **DONE for the initial local edition**. Its receipt records the generated HTML/PDF, reviewed illustrations and freshness result. Workflow **06**, `06_LIVING_DOCUMENTATION.todo`, is **DONE as a procedure definition**, with ongoing execution **IN_PROGRESS**: future feature changes still need matching book updates. This is documentation completion, not application release acceptance. The documentation update did not replace the current preview ZIP. (TODO\38_LIVING_DOCUMENTATION.todo:3-22, status and tasks; TODO\WORKFLOWS\06_LIVING_DOCUMENTATION.todo:3-11, definition/execution distinction; docs\IMPLEMENTATION-STATUS.md:167-173, documentation edition boundary.)

The public journal now supplies the narrative counterpart to the operational ledger. Future behavior changes require corresponding chapter, citation, diagram, image, and journal maintenance. Mechanical freshness detects drift in recorded inputs, but cannot prove semantic correctness or discover every uncited feature; human source review remains necessary. This extends the evidence discipline instead of relaxing application qualification. (JOURNAL.md:8-16, journal scope; docs\DOCUMENTATION-MAINTENANCE.md:7-22, completion requirements; docs\DOCUMENTATION-MAINTENANCE.md:58-66, freshness limitations; TODO\00_INDEX.todo:162-168, documentation definition of done.)

Exercises and references

1. Classify the dual-source PCM result, owner cursor report, and Azure latency target. For each, write one supported claim and one prohibited inference. (docs\IMPLEMENTATION-STATUS.md:17-21; docs\CURSOR-AND-THEMES.md:23-37; docs\AI-ANSWER-EVALUATION.md:42-59.)
2. Given a rebuilt package with unchanged UI, identify which previous qualification claims must become historical and which observations remain useful diagnostics. (docs\QUALIFICATION.md:24-26, version semantics.)
3. Trace why completing optional Docker or cloud-ASR work cannot close native release 17. (TODO\00_INDEX.todo:114-122, release dependencies.)

Continue with the implementation evidence, qualification matrix, TODO index, ledger, public journal, and maintenance contract. The benchmark and answer architecture provide detailed measured-versus-planned boundaries.

Source: JOURNAL.md

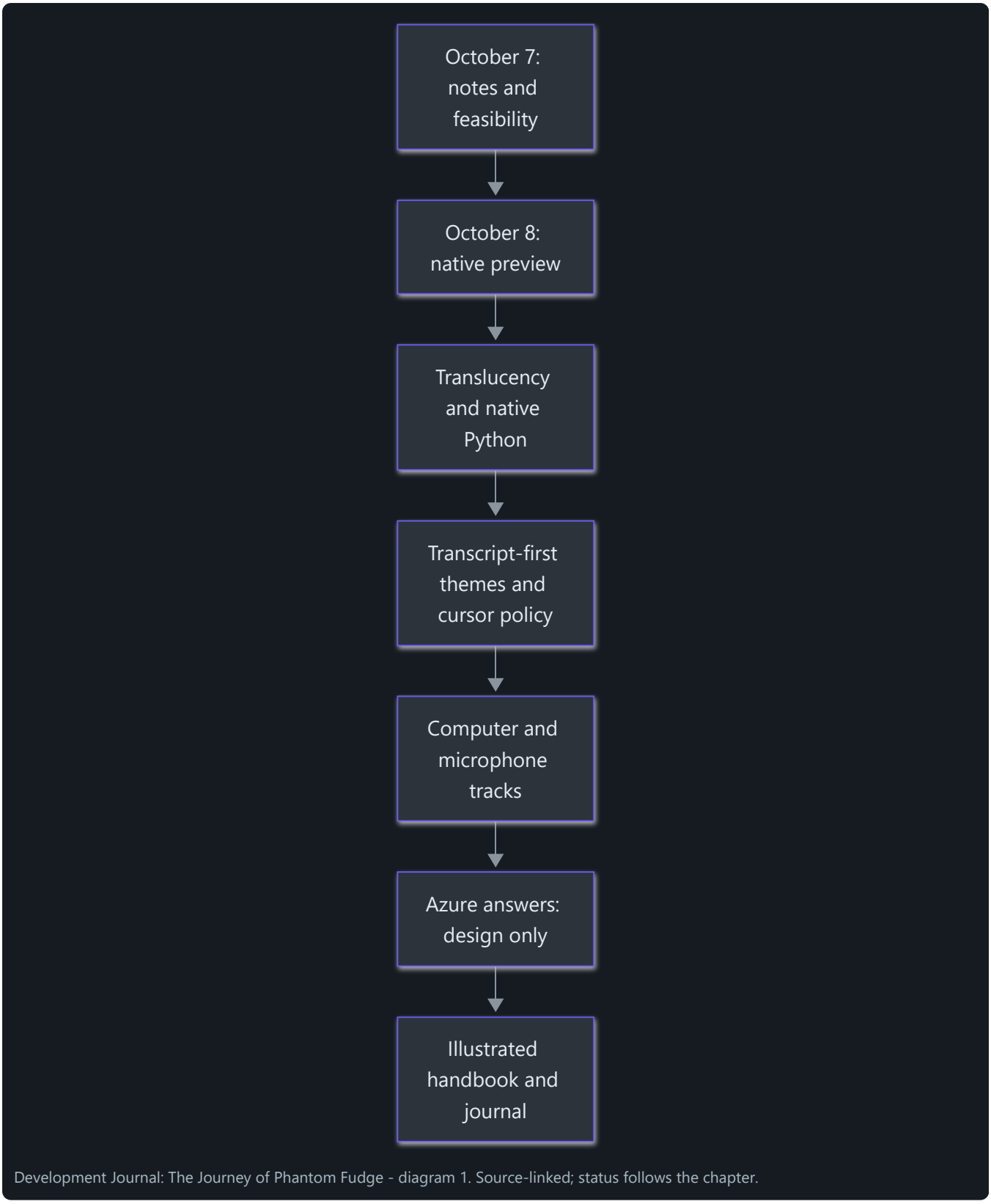
Development Journal: The Journey of Phantom Fudge

This is the public engineering history of Phantom Fudge. It explains not only what changed, but why the change mattered and what the evidence did **not** establish. It is not a transcript of private conversations and contains no meeting content, personal notes or credentials.

Dates below are project-session dates in **Asia/Kolkata (UTC+05:30)**. Times identify recorded requests or decisions, not invented release timestamps. The project moved quickly across one night and the following day; that speed did not turn an experimental preview into a qualified release. The TODO ledger remains the concise operational history; this journal is the narrative counterpart. The handbook explains the system.

The through-line

The original idea was a quiet place for presentation notes. The central challenge was not drawing text: it was preserving useful interaction while making an honest, bounded request to the Windows capture system. Speech then became the primary content source, and explicit AI assistance became the next proposed extension. Throughout those changes, three decisions persisted: keep control with the user, keep audio local by default, and never substitute a successful local test for a universal privacy claim.



Development Journal: The Journey of Phantom Fudge - diagram 1. Source-linked; status follows the chapter.

2026-10-07, 23:30 - An idea becomes a bounded product

The opening request was a draggable, notepad-like surface for speaker notes: absent from the normal taskbar and omitted from screen sharing. The owner first asked for project continuity, a survey of their existing TODO conventions, deep research and an implementation workflow rather than immediate coding.

The repository began without an application. Planning adopted indexed, numbered `.todo` workstreams with explicit status, dependencies, acceptance criteria, evidence and completion logs. The survey found this useful recurring pattern, not a universal convention across every project or remote tracker. Stable IDs made later changes possible without rewriting earlier work.

The most important early correction was conceptual. A tool window is still a real window owned by a real process. Omitting a taskbar entry does not hide it from every application picker. Windows display affinity can request exclusion on supported capture paths; it cannot promise that no recorder, camera, privileged program or future OS change will reveal content. Cursor behavior is another observable channel, not automatically solved by excluding window pixels.

Windows11 became the first platform, with Teams, Zoom, Google Meet and OBS treated as separate qualification targets. The plan deliberately separated desk research, a local proof, usable implementation and release acceptance. That separation would become increasingly important as features arrived faster than real sharing environments could be qualified.

Public record: initial decisions and scope appear in `TODO/16_DEVELOPMENT_LEDGER.todo:75-85`; the convention's bounded survey and evidence semantics are in `docs/TODO-CONVENTIONS.md:1-73`. The current capture-claim limits are stated in `README.md:41-50`.

2026-10-08, 00:32 onward - Implementation, and a new primary signal

The owner authorized implementation and requested eventual Docker support. Live transcription of the **computer's meeting playback**, initially not the microphone, quickly became mandatory. This changed the architecture from an editor with special window behavior into a small real-time system.

C# and WPF took responsibility for the native surface, Windows interaction and WASAPI audio capture. Python and Vosk handled local speech recognition. A narrow authenticated WebSocket boundary connected the two. Notes and transcripts remained local; normal capture required an explicit Start. A hidden service process being available was not permission to begin recording.

The first preview included plain-text editing, capture-affinity setup, recovery hotkeys, a single-instance reveal path, Edit/Present modes, bounded in-memory transcripts and explicit export. Several failure paths proved as important as the happy path: repeated close/save actions, malformed endpoints, unfinished recognition at restart and restoring a minimized hidden utility. Those corrections made the preview less likely to lose data or leave the user unable to recover it.

One explicitly approved synthetic playback test exercised actual default-output loopback through the real recognizer. It did not use a microphone or establish performance on a real meeting. Subsequent sleep instructions ruled out further speaker playback, questions, uploads and risky system changes. Later testing therefore relied on generated PCM and isolated synthetic fixtures. No public release or push followed from creating a local ZIP.

Public record: the historical implementation/evidence table and approved playback boundary are in `docs/IMPLEMENTATION-STATUS.md:93-147`; the authorization and subsequent restrictions are preserved in `TODO/16_DEVELOPMENT_LEDGER.todo:66-74`.

03:01 - A hidden window must still be recoverable

The first operational question was simple: after clicking Hide, how does the presenter get the window back? The recovery path was already part of the product: `Ctrl+Alt+Space` reveals it, and relaunching the executable signals the existing instance rather than creating competing surfaces. The app also recenters on relaunch recovery.

This episode exposed a general design lesson. Removing an ordinary taskbar presence takes away a familiar recovery mechanism. The product must replace that affordance deliberately. Hiding also stops transcription; showing the surface again does not silently resume audio capture.

The UI should feel quiet without making the owner guess whether it is still listening or how to reach it. The native recovery design is explained in the state-machine chapter.

03:04 - Remove repetitive acknowledgments, not technical safeguards

The owner explicitly stated that meeting permission and recipient preflight would be handled outside the app, and asked for the associated checkboxes and repeat confirmations to disappear. The change removed those gates from Start, Present and reset paths.

That instruction did not mean ignoring all failures. Explicit Start/Stop, token/input validation, protection setup and unsaved-content safeguards remained. Nor did it turn an external responsibility into recorded compatibility evidence. The lasting requirement is **not to reintroduce permission or rehearsal prompts** under a different name during future feature work.

Public record: `docs/IMPLEMENTATION-STATUS.md:86-91` records the change and local artifact; `docs/DEVELOPMENT-WORKFLOW.md:95-102` preserves the durable behavioral requirement.

03:12 - Seeing through the surface without washing out the text

The next request was practical: the opaque window blocked the material behind it. Instead of reducing the opacity of the whole window, the implementation made the content background translucent while preserving solid text. The Background slider spans10-100%, defaulting to55%. Header and setup controls remain opaque; the surface does not become click-through.

This is more than a color change. Layered WPF rendering changes the conditions under which earlier capture observations were made. Local affinity/readback, alpha, hit-region and synthetic GDI observations were repeated for the changed rendering path. Their success remained bounded to that machine and those samples. It did not qualify four meeting products or prove an uninterrupted first-frame guarantee.

The lesson was to treat rendering changes as changes to the capture-evidence boundary, not just visual polish.

Public record: `docs/IMPLEMENTATION-STATUS.md:75-84` identifies the layered result and limitations; the implementation is explored in chapter05.

03:25 - Correcting Docker's place in the architecture

The owner clarified an important mismatch: Docker should come at the **absolute end**, as a future packaging option, not as the architecture's normal operating prerequisite. The implementation was corrected around native Python setup, startup, ownership and shutdown.

This was not a late discovery that Python might help C#. The system already used C# for Windows integration and Python for recognition. The correction was about deployment responsibility. Normal launch now starts or reuses an authenticated local Python service. It will not kill an unrelated process merely because that process holds the expected port. Model/interpreter/token setup remains explicit, and an idle running service still does not capture audio.

Docker recipes could remain useful as a future adapter around the headless boundary, but workstream20 was moved after native release17. Runtime22 became the required operational path. This distinction prevented infrastructure work from blocking the thing the owner actually wanted to run.

Public record: native operation and the Docker correction are recorded in `docs/IMPLEMENTATION-STATUS.md:54-73`; the current dependency graph is in `TODO/00_INDEX.todo`.

The muted-laptop experiment - Compare engines, not language stereotypes

The owner also questioned whether C# transcription would be good enough and asked for a real comparison, while planning to mute the laptop. A mute-independent experiment sent the same generated PCM to Windows SAPI, Python Vosk and Python faster-whisper. No speakers, microphone or cloud ASR were needed.

The corpus contained32 cases,164.07seconds of audio and292 reference words. Across two serial warm passes, the recorded comparison was:

Engine	Word error rate	Warm decode time
Windows SAPI	45.89%	10.139seconds
Python Vosk	8.22%	41.754seconds
faster-whisper base.en, CPU int8	4.79%	25.745seconds

These measurements challenged an easy answer. SAPI was quickest on this batch but substantially less accurate. Whisper looked promising on quality and warm throughput. Neither result proved that one programming language was inherently better at speech, or that batch completion predicted the first useful live partial. Vosk remained the production streaming recognizer; swapping it out requires a real streaming contract and suitable latency/quality evidence.

Generated English speech also does not represent every accent, interruption, name or noisy meeting. The historical receipt must be read with its exact harness/model provenance; the handbook notes that today's harness hash differs from the historical receipt rather than rewriting that receipt.

Public record: the full benchmark report and chapter12. Summary measurements and their limit are recorded in `docs/IMPLEMENTATION-STATUS.md:60-63`.

11:31 - Transcription moves to the foreground

The owner reported two interaction cues: the recipient-side cursor could disappear when clicking the app, and the local pointer became an I-beam over text. They also asked for transcription to become the main view, notes to collapse by default, and DeepSeaFoam/Harbor Daylight plus Plain B/W.

The response separated what the app controls from what the recorder controls. The app now defaults to non-activating Present mode, requests a normal Arrow and handles its own native cursor messages. Edit remains the deliberate route for keyboard input and accessibility. These changes reduce local focus/cursor cues; they do not establish control over a recipient's capture pipeline.

Moving to passive interaction exposed a real WPF nuance: default ListBox row selection expects focus to succeed. The device selector needed explicit passive selection rather than pretending that a non-activating window could use the ordinary focus path unchanged.

The transcript became primary, notes collapsed, and canonical theme tokens were used instead of approximate colors. Automatic unprotected protection-failure message boxes were replaced by bounded generic local notices. The already-running owner app was preserved while new candidates were packaged separately.

Still unresolved: recipient-side cursor disappearance needs qualification in the actual recorder/sharing mode. No implementation change justified the requested absolute invisibility claim.

Public record: `docs/IMPLEMENTATION-STATUS.md:35-52`, cursor/theme notes and workstreams24/25.

13:47-14:01 - Small UI questions clarify the evidence

The owner asked where the theme toggle was and later confirmed that themes worked. They also asked about the footer reading "Capture exclusion requested. UNQUALIFIED PREVIEW: compatibility varies by capture method." That text was an informational boundary, not a failed setup error.

The owner reported that Snipping Tool omitted the window. That is useful **owner observation**, not independent proof of every recorder or sharing path. Documentation retains the distinction. This episode also explains why this book's UI images use internal synthetic visual-tree rendering: they can illustrate the actual controls without claiming that a desktop screenshot should succeed or asking the owner to expose private content.

14:01 - Dictated questions require a microphone track

The owner wanted to read a chat question aloud and have it transcribed, while explicitly avoiding chat-reading integration. The new requirement was to listen to both computer output and microphone dictation. It superseded the original no-microphone product scope; it did not authorize development tests to record a real microphone.

Computer, Mic and Both modes were added. Both is the default selection, but capture remains stopped until Start. The sources use separate selected WASAPI inputs, bounded queues and independent Vosk recognizers. A v2 protocol prefixes PCM with a source byte while preserving v1 compatibility. Source labels identify inputs, not people; they do not provide diarization or guaranteed acoustic echo cancellation.

Service management also learned to distinguish a stale, owned, idle service from an active or foreign one. Only the first is eligible for the capability-aware restart path.

Review found a subtle stop/restart race: an audio component could report not-running before final events and cleanup were finished. A new Start in that interval could reset transcript IDs and mix the lifecycle of two sessions. A coalesced UI stopping gate now prevents Start/mode/clear changes until final draining and cleanup settle.

The recorded evidence included41 Core tests,46 native synthetic groups,71 backend tests and four manager-upgrade guards, plus real generated dual PCM and C#-to-Python v2 recognition. No actual microphone/render endpoint or chat content was opened for this work. Hardware permissions, echo, OS microphone indicators and recipient behavior remain to be qualified.

Public record: `docs/IMPLEMENTATION-STATUS.md:11-33`, `docs/DUAL-SOURCE-AUDIO.md:1-30` and chapters07-11.

15:05 - An answer feature is researched before it is built

The next idea was to turn a chosen transcript question into a prompt and display the model's answer. The owner requested deep architecture research, Azure where useful, and low latency. The requested deliverable was design and backlog, not immediate cloud execution.

The proposed path is deliberately narrow: local ASR, an editable source-scoped question draft, an immutable snapshot on explicit Ask, direct C# streaming from Azure OpenAI v1 Responses, then rendering inside the same protected native surface. Audio stays local by default. The selected question text crosses a new, deliberate cloud boundary; the entire meeting does not automatically become prompt context.

A proposed Mic-only commit/ack avoids stopping Computer transcription to finalize a dictated question. It needs protocol evolution; current v2 stop-all is not that feature. Planned public-client Entra/WAM authentication must avoid embedded secrets and surprise sign-in windows during presenting. Request cancellation must prevent stale tokens from updating a newer answer.

Research considered current model lifecycle and capabilities rather than choosing a familiar but retiring default. No candidate was declared fastest without measuring it in the intended region and configuration. Likewise, `store:false` is not a universal zero-retention promise.

Four design documents, workstreams27-37 and workflows04/05 capture the plan. Only research27 is DONE. There is no implemented Ask button, composer, Azure client, source commit or answer pane. No Azure account/resource discovery, sign-in, inference or upload was performed.

Public record: `docs/IMPLEMENTATION-STATUS.md:5-9`, `docs/AI-ANSWER-ARCHITECTURE.md:1-30` and the four linked AI design documents.

16:11 - Turn the implementation into a teachable engineering record

The owner requested documentation detailed enough to become a book, using another project's documentation as a depth/style reference, and asked that screenshots, diagrams and a proper journey journal remain current thereafter.

The resulting handbook is organized into22 chapters plus this journal appendix: first-principles onboarding, a principal engineer's guide, native Windows behavior, storage, the complete audio pipeline, state machines, measured performance, planned Azure architecture, development/operations and grounded reference material. The other project's prose and artwork were not copied.

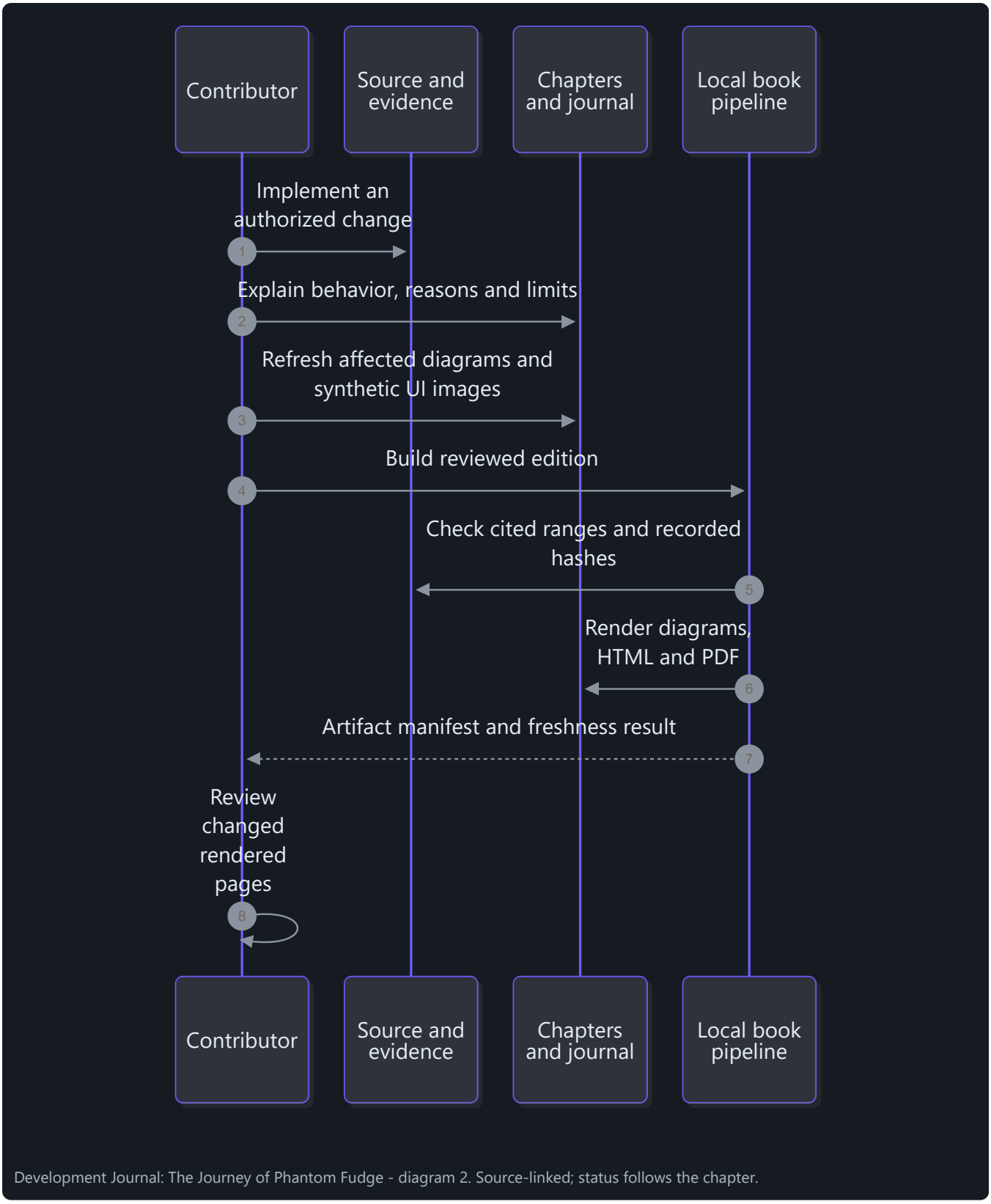
Illustrations have a defined provenance. Nine images render the actual WPF visual tree with synthetic text and device labels. They do not record audio, read chats, expose notes or capture the desktop. They illustrate appearance, not capture exclusion. Mermaid diagrams are rendered locally into SVG while retaining their editable sources.

The book pipeline produces offline HTML and a printable PDF with a generated manifest. It checks source ranges, local links/images, diagram rendering, recorded source freshness and printable layout. Those mechanical gates cannot prove a paragraph's interpretation; chapters still require source review. Exact generated counts and artifact hashes belong in the edition manifest, not a hand-maintained competing receipt.

This milestone adds a durable workflow obligation, not merely more files. Any later behavior change must update relevant chapters and diagrams; visible UI changes must refresh synthetic images; and significant decisions/evidence must extend this journal. Workstream38 establishes the first edition. Workflow06 and the repository working contract carry the requirement forward.

The initial edition was completed locally. Print review exposed undersized diagrams, so their layout was enlarged and sequence labels wrapped; narrow-screen review exposed long source paths, so the reader gained responsive wrapping. Those corrections improved the actual artifacts rather than treating a successful export as sufficient. Nothing was uploaded or published.

The documentation-only fixture does not replace the most recent user preview ZIP. That preview remains the145231 dual-source bundle identified in `docs/IMPLEMENTATION-STATUS.md:27-33`. The book does not implement Azure Ask, qualify remote cursors, or move Docker earlier.



Development Journal: The Journey of Phantom Fudge - diagram 2. Source-linked; status follows the chapter.

What remains open at this edition

The application is still a local unqualified preview. Full Teams/Zoom/Meet/OBS qualification, recipient cursor behavior, hardware microphone/echo/privacy-indicator observations, long-session quality, accessibility and owner acceptance have not been replaced by this book.

The planned answer feature needs implementation and measured quality/latency. Optional cloud ASR and a broker are separate decisions. Docker remains a final-stage packaging option after native release, not a prerequisite for transcription.

The next journal entry should say what actually happened, identify its public evidence and preserve these boundaries until new evidence changes them. A useful engineering history is allowed to say "not yet," "the earlier approach was wrong," and "this result covers only this case."